

Scientific Data Analysis with Python

From absolute beginner to advanced practitioner

A complete, hands-on guide written for Google Colab

NumPy · pandas · Matplotlib · SciPy · scikit-learn · Statistics · Machine Learning

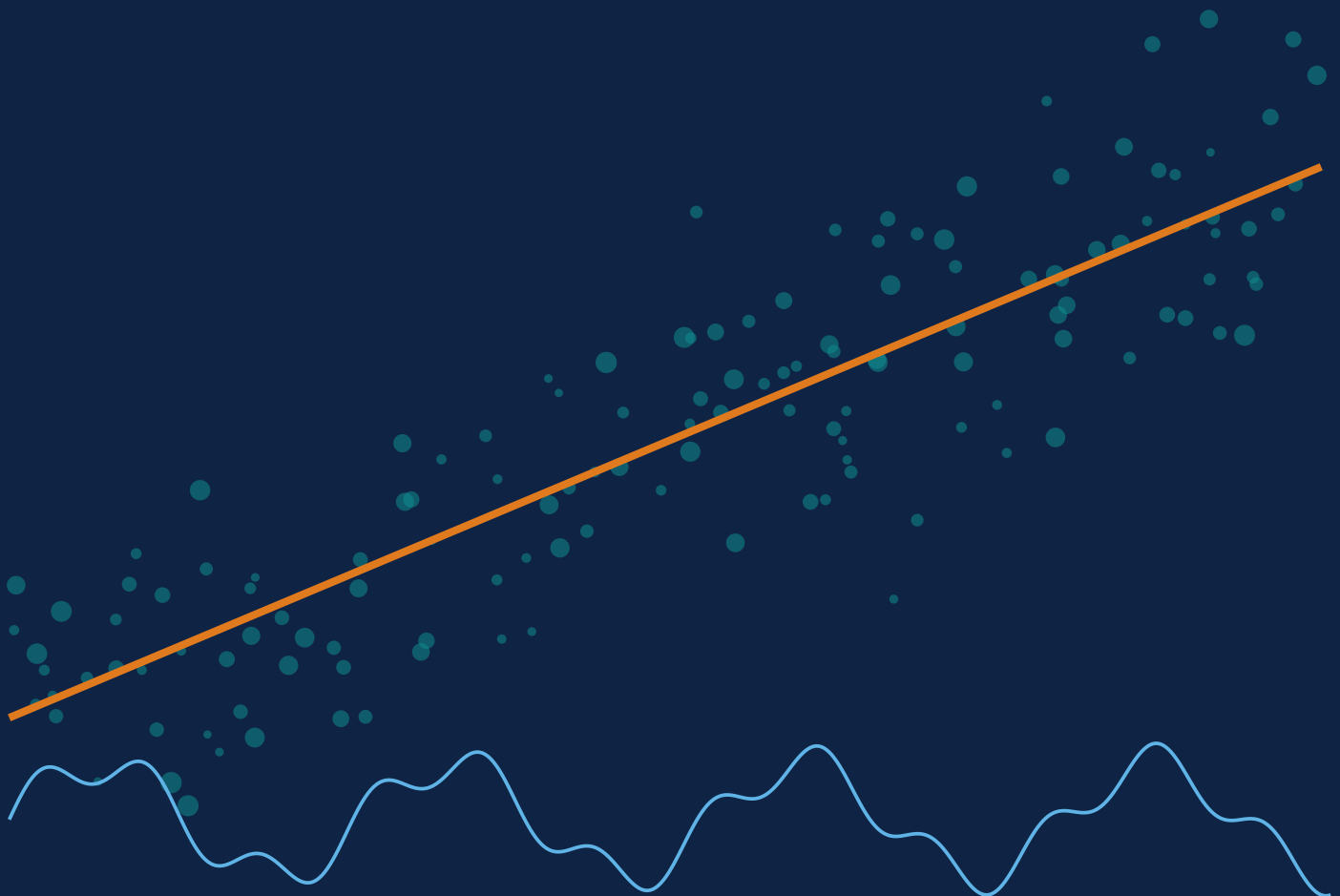


Table of Contents

Foundations 11

1. Welcome: Data Analysis, Python and Google Colab 12

1.1 What is scientific data analysis?	12
1.2 Why Python?	12
1.3 Google Colab: your laboratory	13
1.4 How to use this book	15
1.5 Chapter summary	15
1.6 Practice exercises	16

2. Python Essentials for Data Analysis 17

2.1 Variables and basic data types	17
2.2 Strings: working with text	18
2.3 Collections: lists, tuples, dictionaries, sets	19
2.4 Making decisions with if	21
2.5 Repeating work with loops	21
2.6 Functions	22
2.7 Modules and imports	23
2.8 Handling errors	23
2.9 A first mini analysis with plain Python	24
2.10 Chapter summary	24
2.11 Practice exercises	24

3. The Math and Statistics Toolkit 25

3.1 Numbers, units and scientific notation	25
3.2 The summation symbol Σ	25
3.3 Exponents and logarithms	26
3.4 Functions and graphs	27
3.5 Rate of change: the derivative as an idea	27
3.6 Area under a curve: the integral as an idea	28
3.7 Vectors and matrices	28
3.8 Probability basics	29
3.9 Measuring error	30
3.10 Greek letters you will meet	31
3.11 Chapter summary	31
3.12 Practice exercises	31

Core Libraries 32

4. NumPy Part 1: Arrays 33

4.1 Why NumPy?	33
--------------------------	----

4.2 Creating arrays	33
4.3 Data types	35
4.4 Indexing and slicing	35
4.5 Changing the shape	36
4.6 Copies versus views	37
4.7 Useful array methods at a glance	38
4.8 Chapter summary	38
4.9 Practice exercises	38
5. NumPy Part 2: Vectorisation, Broadcasting and Linear Algebra	39
5.1 Vectorised arithmetic	39
5.2 Broadcasting	39
5.3 Aggregations and the axis argument	40
5.4 Conditional selection with where	41
5.5 Sorting, searching and unique values	42
5.6 Cumulative and difference operations	42
5.7 Linear algebra with NumPy	42
5.8 Random numbers and simulation	43
5.9 Grids of coordinates: meshgrid	43
5.10 Saving and loading arrays	44
5.11 Performance tips	44
5.12 Chapter summary	44
5.13 Practice exercises	45
6. pandas Part 1: Series and DataFrames	46
6.1 What is pandas?	46
6.2 Series: one labelled column	46
6.3 DataFrame: the table	47
6.4 Selecting columns	48
6.5 Selecting rows: loc and iloc	49
6.6 Filtering rows with conditions	49
6.7 Creating and changing columns	50
6.8 The index	50
6.9 Quick summaries of columns	51
6.10 Applying your own function	51
6.11 Chapter summary	51
6.12 Practice exercises	52
7. pandas Part 2: Cleaning and Preparing Data	53
7.1 A messy dataset to practise on	53
7.2 Missing values	54
7.3 Duplicates	55
7.4 Fixing data types	56
7.5 Cleaning text	56

7.6 Dates and times	57
7.7 Outliers: unusual values	57
7.8 Validating with simple rules	58
7.9 A reusable cleaning function	59
7.10 Chapter summary	59
7.11 Practice exercises	59
8. pandas Part 3: Grouping, Combining and Reshaping	61
8.1 A dataset for this chapter	61
8.2 groupby: split, apply, combine	61
8.3 Pivot tables and crosstabs	63
8.4 Combining tables	63
8.5 Wide versus long format	64
8.6 Working with time series	65
8.7 Categorical data	66
8.8 Method chaining	66
8.9 Performance tips	67
8.10 Chapter summary	67
8.11 Practice exercises	67
9. Data Visualisation: Matplotlib and Seaborn	69
9.1 Matplotlib: the anatomy of a figure	69
9.2 Line plots with several series	70
9.3 Scatter plots: relationships between two variables	71
9.4 Bar charts: comparing categories	71
9.5 Histograms: the shape of one variable	72
9.6 Box plots and violin plots: comparing distributions	73
9.7 Subplots: several panels	73
9.8 Heatmaps and images	74
9.9 Saving figures	75
9.10 Seaborn: statistical plots in one line	75
9.11 Plotting directly from pandas	77
9.12 Error bars and confidence bands	78
9.13 Annotations and text	78
9.14 Choosing the right chart	79
9.15 Honest and readable figures	79
9.16 Chapter summary	79
9.17 Practice exercises	80
10. Getting Data In and Out	81
10.1 Files in Colab: where things live	81
10.2 CSV files	81
10.3 Excel files	82
10.4 JSON files	83

10.5 Other formats worth knowing	83
10.6 Reading directly from the web	83
10.7 Databases and SQL	84
10.8 Saving your analysis results	85
10.9 Chapter summary	85
10.10 Practice exercises	85

Exploration and Statistics 86

11. Exploratory Data Analysis (EDA) 87

11.1 The EDA checklist	87
11.2 A worked example: the wine dataset	87
11.3 Questions to ask of every dataset	91
11.4 Simple automated overview	92
11.5 Common EDA pitfalls	92
11.6 Chapter summary	93
11.7 Practice exercises	93

12. Descriptive Statistics 94

12.1 Measures of centre	94
12.2 Measures of spread	95
12.3 Percentiles and quantiles	96
12.4 Shape: skewness and kurtosis	97
12.5 Covariance and correlation	97
12.6 Standardising: z-scores	98
12.7 Frequencies and proportions for categories	99
12.8 Weighted and grouped statistics in one table	99
12.9 Chapter summary	99
12.10 Practice exercises	100

13. Probability Distributions 101

13.1 Discrete versus continuous	101
13.2 scipy.stats: one interface for everything	101
13.3 Discrete distributions	102
13.4 Continuous distributions	103
13.5 Which distribution fits my data?	106
13.6 The Law of Large Numbers	106
13.7 The Central Limit Theorem (CLT)	107
13.8 Fitting and sampling recipe	108
13.9 Chapter summary	108
13.10 Practice exercises	108

14. Sampling, Estimation and Confidence Intervals 109

14.1 Population and sample	109
--------------------------------------	-----

14.2 The sampling distribution and the standard error	109
14.3 Point estimates and bias	110
14.4 Confidence intervals	110
14.5 Confidence interval for a proportion	112
14.6 The bootstrap: intervals without formulas	113
14.7 How large a sample do I need?	114
14.8 Maximum likelihood: how estimates are "derived"	114
14.9 Chapter summary	115
14.10 Practice exercises	115
15. Hypothesis Testing	116
15.1 The logic of hypothesis testing	116
15.2 A simulation to build intuition	116
15.3 The t-tests	117
15.4 Checking assumptions	119
15.5 Non-parametric tests	119
15.6 Comparing more than two groups: ANOVA	120
15.7 Chi-square tests for counts	120
15.8 Permutation tests: a test without formulas	121
15.9 The multiple-comparisons problem	122
15.10 Estimating power by simulation	122
15.11 Choosing a test	123
15.12 Reporting results well	123
15.13 Chapter summary	123
15.14 Practice exercises	123
16. Regression: Modelling Relationships	125
16.1 Simple linear regression	125
16.2 How good is the fit?	127
16.3 Confidence versus prediction bands	128
16.4 Assumptions and residual diagnostics	128
16.5 Multiple linear regression	130
16.6 Categorical predictors: dummy variables	132
16.7 Interactions	132
16.8 Multicollinearity	133
16.9 Polynomial regression and non-linear terms	133
16.10 Regularisation: Ridge and Lasso	134
16.11 Logistic regression: predicting yes or no	135
16.12 Choosing and comparing models	136
16.13 Chapter summary	136
16.14 Practice exercises	136
Advanced Scientific Analysis	138

17. Preprocessing and Feature Engineering	139
17.1 The golden rule: fit on training data only	139
17.2 Scaling numeric features	139
17.3 Transforming skewed data	140
17.4 Encoding categorical variables	141
17.5 Binning and discretising	142
17.6 Creating new features	142
17.7 Handling missing values for models	143
17.8 Feature selection	143
17.9 Pipelines: one object for the whole workflow	144
17.10 A preprocessing checklist	144
17.11 Chapter summary	145
17.12 Practice exercises	145
18. Machine Learning for Scientists	146
18.1 Vocabulary	146
18.2 Overfitting and the bias-variance trade-off	146
18.3 Train, validation, test and cross-validation	147
18.4 Classification metrics	148
18.5 Regression metrics	149
18.6 Key algorithms	150
18.7 Hyperparameter tuning	151
18.8 Learning curves: do I need more data?	152
18.9 Interpreting models	152
18.10 Good ML practice for science	153
18.11 Chapter summary	153
18.12 Practice exercises	153
19. Unsupervised Learning: Clustering and Dimensionality Reduction	154
19.1 Distance: the basis of clustering	154
19.2 k-means clustering	154
19.3 Hierarchical clustering	156
19.4 DBSCAN: density-based clustering	157
19.5 Gaussian mixture models	158
19.6 Principal Component Analysis (PCA)	158
19.7 t-SNE and UMAP for visualisation	161
19.8 Anomaly detection	161
19.9 Evaluating clusters	162
19.10 Chapter summary	162
19.11 Practice exercises	163
20. Time Series Analysis	164
20.1 A synthetic monthly series	164
20.2 Components of a time series	165

20.3 Smoothing with moving averages	165
20.4 Decomposition	166
20.5 Stationarity	167
20.6 Autocorrelation	167
20.7 Autoregressive models	168
20.8 Exponential smoothing	169
20.9 Forecasting and validating properly	169
20.10 Practical tasks	171
20.11 Chapter summary	172
20.12 Practice exercises	172
21. Scientific Computing with SciPy	173
21.1 Physical constants and units	173
21.2 Non-linear curve fitting	173
21.3 General optimisation: minimise	175
21.4 Root finding	176
21.5 Interpolation	176
21.6 Numerical integration	177
21.7 Ordinary differential equations (ODEs)	177
21.8 Linear algebra with scipy.linalg	179
21.9 Special functions and sparse matrices	180
21.10 A general workflow for model fitting	180
21.11 Chapter summary	181
21.12 Practice exercises	181
22. Signals and Images	182
22.1 Sampling	182
22.2 The Fourier transform: from time to frequency	183
22.3 Spectrograms: frequency changing over time	184
22.4 Filtering	185
22.5 Peak finding	187
22.6 Convolution and correlation	187
22.7 Images as arrays	187
22.8 Filtering images	188
22.9 Counting objects: labelling	189
22.10 Image data for machine learning	190
22.11 Chapter summary	190
22.12 Practice exercises	190
23. Uncertainty, Error Analysis and Bayesian Thinking	192
23.1 Types of error	192
23.2 Reporting a measurement	193
23.3 Propagation of uncertainty	193
23.4 Combining measurements: the weighted mean	195

23.5 Bayesian inference	195
23.6 MCMC: when there is no closed form	197
23.7 Frequentist or Bayesian?	199
23.8 Chapter summary	199
23.9 Practice exercises	199
24. Special Topics: Survival, Experiments, Text, Networks and Maps	200
24.1 Survival analysis: time until an event	200
24.2 Design of experiments (DOE)	202
24.3 Text as data	203
24.4 Networks and graphs	203
24.5 Geospatial data	204
24.6 Scaling up: bigger data	206
24.7 Chapter summary	206
24.8 Practice exercises	206
Putting It All Together	207
25. Reproducible Analysis and Good Scientific Practice	208
25.1 The structure of a good notebook	208
25.2 Randomness and seeds	209
25.3 Write functions, not copy-pasted code	209
25.4 Organise your project files	210
25.5 Data provenance and a data dictionary	210
25.6 Automating a whole analysis: a pipeline function	211
25.7 Common statistical mistakes (and fixes)	211
25.8 Communicating results	212
25.9 Ethics and responsibility	213
25.10 Practising the workflow: a checklist before sharing	213
25.11 Chapter summary	213
25.12 Practice exercises	214
26. Capstone Projects	215
26.1 Project 1: Climate station analysis	215
26.2 Project 2: Dose-response experiment	218
26.3 Project 3: Predicting a measured property with machine learning	221
26.4 What to do next	223
26.5 Chapter summary	223
26.6 Practice exercises	224
Appendices	225
A. Cheat Sheets	226
A.1 Python and Colab essentials	226

A.2 NumPy	226
A.3 pandas	227
A.4 Matplotlib and Seaborn	227
A.5 scipy.stats quick reference	228
A.6 scikit-learn pattern	228
B. Formula Sheet and Symbols	229
B.1 Descriptive statistics	229
B.2 Probability	229
B.3 Inference	229
B.4 Regression	230
B.5 Machine learning metrics	230
B.6 Error propagation	230
B.7 Greek letters and notation	231
C. Glossary	232
D. Google Colab Tips and Troubleshooting	234
D.1 Everyday tips	234
D.2 Library versions	234
D.3 Common errors and what to do	235
D.4 A debugging routine that works	235
E. Solutions to Selected Exercises	236
E.1 Chapter 2: Python essentials	236
E.2 Chapter 3: Math toolkit	236
E.3 Chapter 4 and 5: NumPy	237
E.4 Chapter 12 and 13: statistics and distributions	237
E.5 Chapter 14 and 15: intervals and tests	237
E.6 Chapter 16: regression	238
E.7 Chapter 18: machine learning	238
E.8 Chapter 19: PCA	239
E.9 Chapter 21: curve fitting	239
E.10 Chapter 22: signals	239
E.11 Chapter 23: uncertainty	239
F. A Learning Roadmap and Further Reading	241
F.1 If you want to go deeper	241
F.2 Suggested books and resources (search by title)	241
F.3 Final advice	242

PART I

Foundations

Set up your workspace, learn the Python you need, and understand how scientific data analysis works from start to finish.

CHAPTER 1

1. Welcome: Data Analysis, Python and Google Colab

IN THIS CHAPTER YOU WILL LEARN

- What scientific data analysis is, and the workflow every analysis follows
- Why Python is the tool of choice, and which libraries do which job
- How to use Google Colab: cells, running code, installing packages, uploading files and saving your work
- How to read this book and practise effectively

1.1 What is scientific data analysis?

Data is recorded information: temperatures from a weather station, the heights of plants in an experiment, the brightness of a star every night, the results of a clinical trial. **Analysis** is the process of turning that raw information into knowledge: What is typical? What changed? Is the difference real or just luck? Can we predict what happens next?

Scientific data analysis adds discipline. A scientist must be able to say how sure they are, must be able to repeat the analysis, and must be able to defend each step. That is why this book teaches you not only *how* to write the code, but *why* each method works, the simple math behind it, and how to check that the answer is trustworthy.

The analysis workflow

Almost every analysis, from a school lab report to a research paper, follows the same loop:

Step	Question you ask	Typical tools
1. Ask	What exactly do I want to know?	Notebook text, pen and paper
2. Collect / load	Where is the data and what format is it in?	pandas, files, databases
3. Clean	Are there missing values, typos, impossible numbers?	pandas, NumPy
4. Explore	What does the data look like?	Statistics, plots
5. Model / test	Is a pattern real? Can I predict?	SciPy, statsmodels, scikit-learn
6. Communicate	How do I show the result clearly and honestly?	Matplotlib, Seaborn, text

Roughly 60 to 80 percent of real work sits in steps 2 to 4. Beginners often rush to step 5; experienced analysts know that a model built on messy data gives confident nonsense. This book gives step 2 to 4 the space they deserve.

1.2 Why Python?

Python is free, readable (it looks close to plain English) and has a huge ecosystem of scientific libraries. A **library** (or **package**) is a collection of ready-made tools written by other people that you can use with one line of code. The ones you will meet in this book:

Library	What it is for	Chapter
NumPy	Fast arrays of numbers, math on whole datasets at once	4, 5
pandas	Tables (like Excel, but programmable): loading, cleaning, grouping	6 to 8
Matplotlib	Drawing plots with full control	9
Seaborn	Beautiful statistical plots with very little code	9
SciPy	Statistics, optimisation, integration, signal processing	13 to 15, 21, 22
statsmodels	Classical statistical models with detailed reports	16, 20
scikit-learn	Machine learning and preprocessing	17 to 19

1.3 Google Colab: your laboratory

Google Colab (short for Colaboratory) is a free service that lets you write and run Python in your web browser. You do not install anything on your computer. Google gives you a temporary Linux computer in the cloud with Python and all the scientific libraries above already installed.

Opening your first notebook

1. Go to colab.research.google.com and sign in with a Google account.
2. Click **New notebook**. You get a document called a **notebook** (file ending in `.ipynb`).
3. Click the title at the top-left ("Untitled0.ipynb") and rename it, for example `chapter01.ipynb`.

Cells: the building blocks

A notebook is made of **cells**. There are two kinds:

- **Code cells** contain Python. Press the round play button on the left, or press **Shift + Enter**, to run the cell. The result appears right below it.
- **Text cells** contain notes written in a simple format called Markdown. Use them to explain your work. Add one with the **+ Text** button.

Here is a tiny code cell and its output. Everything in this book appears in this style: the shaded box is the code you type, and the white box under it is what Colab prints.

```
print("Hello, data analysis!")
2 + 3 * 4
```

OUTPUT

```
Hello, data analysis!
14
```

Notice two things. First, `print(...)` displays whatever you give it. Second, when the **last line** of a cell is an expression (like `2 + 3 * 4`) Colab automatically shows its value. Both ways of seeing results are used throughout the book.

NOTE

The output box that follows each example in this book was produced by actually running the code. Numbers coming from random generators use a fixed seed, so you will see exactly the same values when you run the same code. Small differences in the last digits can appear between library versions.

Essential keyboard shortcuts

Action	Shortcut
Run the cell and move to the next	Shift + Enter
Run the cell and stay	Ctrl + Enter
Insert a code cell below	Ctrl + M then B
Insert a code cell above	Ctrl + M then A
Delete the selected cell	Ctrl + M then D
Convert a cell to text / code	Ctrl + M then M / Y
Undo cell deletion	Ctrl + M then Z
Comment or uncomment lines	Ctrl + /
Show the arguments of a function	Shift + Tab (inside the brackets)
Autocomplete a name	Tab (or Ctrl + Space)

On a Mac, use Cmd instead of Ctrl.

The runtime, memory and the order of cells

When you run code, it runs on a **runtime**: the temporary computer Google lends you. The runtime remembers every variable you created, in the order in which you **ran** the cells (not the order in which they appear on screen). This causes the most common beginner confusion:

WATCH OUT

If you change a cell but forget to re-run it, or run cells out of order, a variable may hold an older value than you think. When something looks wrong, use **Runtime, Restart session and run all** to execute the notebook cleanly from top to bottom. A good notebook always works when run in order.

Colab disconnects the runtime after a period of inactivity, and all variables and files stored on the temporary machine are then erased. Your **notebook file** (the code) is saved to your Google Drive, but the **variables** are lost. This is normal: just run the cells again.

Installing and importing libraries

The common libraries are pre-installed. To use one you **import** it, usually giving it a short nickname (an *alias*) by convention:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print(np.__version__)
print(pd.__version__)
```

OUTPUT

```
2.4.4
3.0.2
```

If a library is not installed, use **pip**, Python's package installer. In Colab any line starting with an exclamation mark **!** is sent to the operating system's command line:

```
!pip install lifelines
```

TIP

After installing a package, if Python still cannot import it, restart the runtime with **Runtime, Restart session** and run the import again.

Getting your own files into Colab

There are three common ways. You will use them all in Chapter 10.

1. **Upload directly.** Click the folder icon on the left sidebar, then the upload icon. Files appear in `/content/` but disappear when the runtime resets.
2. **Mount Google Drive.** Files stay permanently in your Drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

3. **Download from a web address.** `pd.read_csv("https://...")` reads a CSV straight from a URL.

Saving your work

Colab autosaves the notebook to Drive. You can also use **File, Download, Download .ipynb** or **File, Save a copy in Drive**. Because a notebook is also a report, use text cells to add headings and explanations. Future you will be grateful.

1.4 How to use this book

- **Type the code yourself.** Copy-pasting teaches your fingers nothing. Typing forces you to notice every symbol.
- **One notebook per chapter.** Start each chapter with a fresh notebook and run the setup cell (the imports) first.
- **Change the numbers.** After each example, modify a value and predict what will happen before running it. This is the fastest way to learn.
- **Read error messages.** They are the most helpful teacher. The last line tells you *what* went wrong; the lines above tell you *where*.
- **Do the exercises.** Every chapter ends with practice questions. Attempt them before moving on.
- **Grey boxes.** "Math behind it" boxes explain formulas in plain words. If you find them hard at first, read the code and the pictures, then come back. You do not need to memorise formulas, you need to understand what they *do*.

TIP

Learning data analysis is like learning to drive. You cannot get there by reading. Expect to feel slow for the first few chapters; the speed comes from repetition.

1.5 Chapter summary

CHAPTER SUMMARY

- Scientific data analysis follows a loop: ask, load, clean, explore, model, communicate.
- Python plus NumPy, pandas, Matplotlib, SciPy and scikit-learn covers the whole workflow.
- Colab gives you Python in the browser. Code cells run with Shift + Enter, and the order in which you run cells matters.

- Install extra packages with `!pip install`, and bring in data by upload, Drive or URL.

1.6 Practice exercises

1. Create a new Colab notebook named `chapter01`. Add a text cell with a heading (start a line with `#`) and a code cell that prints your name.
2. In a code cell, compute the number of seconds in one week using multiplication only.
3. Deliberately write `pritrn("hi")`, run it and read the error message. What does the last line say?
4. Run a cell that sets `x = 5`, then in a *second* cell run `x * 2`. Now restart the session and run only the second cell. What error appears and why?
5. Find the Colab menu that shows how much RAM and disk your runtime has.

CHAPTER 2

2. Python Essentials for Data Analysis

IN THIS CHAPTER YOU WILL LEARN

- Variables, numbers, text and Booleans
- Lists, tuples, dictionaries and sets
- Making decisions (`if`) and repeating work (`for`, `while`)
- Functions, list comprehensions, modules and error handling

You do not need to be a programmer to analyse data, but you need a small core of Python. This chapter teaches exactly that core. Every idea is shown with a tiny example.

2.1 Variables and basic data types

A **variable** is a name attached to a value, like a label on a box. You create it with the equals sign `=`, which here means "store this value under this name", not "is mathematically equal to".

```
temperature = 23.5      # a decimal number (float)
n_samples = 40          # a whole number (int)
sensor = "thermometer-A" # text (str)
is_valid = True         # yes/no value (bool)

print(temperature, n_samples, sensor, is_valid)
print(type(temperature), type(n_samples), type(sensor), type(is_valid))
```

OUTPUT

```
23.5 40 thermometer-A True
<class 'float'> <class 'int'> <class 'str'> <class 'bool'>
```

The four basic **types**:

Type	Meaning	Example
<code>int</code>	whole numbers	<code>7</code> , <code>-3</code> , <code>1000</code>
<code>float</code>	numbers with a decimal point	<code>3.14</code> , <code>-0.5</code> , <code>2e-3</code> (means 0.002)
<code>str</code>	text, written in quotes	<code>"hello"</code> , <code>'H2O'</code>
<code>bool</code>	truth values	<code>True</code> , <code>False</code>

NOTE

Variable names may contain letters, digits and underscores, cannot start with a digit, and are case-sensitive (`Mass` and `mass` are different). Use descriptive names such as `mean_height`, not `x1`, whenever you can. Readable code is correct code that is easier to check.

Arithmetic

```
a, b = 17, 5
print("add", a + b)
print("subtract", a - b)
print("multiply", a * b)
print("divide", a / b)           # always gives a float
print("floor divide", a // b)    # whole part of the division
print("remainder", a % b)       # what is left over (modulo)
print("power", a ** 2)          # 17 squared
```

OUTPUT

```
add 22
subtract 12
multiply 85
divide 3.4
floor divide 3
remainder 2
power 289
```

Python follows the usual order of operations: brackets first, then powers, then multiplication and division, then addition and subtraction.

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(2 ** 3 ** 2)  # powers group from the right: 2 ** (3 ** 2)
```

OUTPUT

```
14
20
512
```

Comparison and logic

Comparisons produce **True** or **False**. Combine them with **and**, **or**, **not**.

```
x = 12
print(x > 10, x == 12, x != 5)    # bigger, equal, not equal
print(x > 10 and x < 20)          # both must be true
print(x < 5 or x > 11)            # at least one true
print(not (x > 10))
```

OUTPUT

```
True True True
True
True
False
```

WATCH OUT

`=` stores a value; `==` asks "are these equal?". Writing `if x = 5` is an error. Also beware of floats: `0.1 + 0.2 == 0.3` is **False** because computers store decimals in binary with tiny rounding. Compare floats with a tolerance, for example `abs(a - b) < 1e-9` or `math.isclose(a, b)`.

2.2 Strings: working with text

Text matters in data analysis because column names, labels and categories are all strings.

```
name = " Sodium Chloride "
print(name.strip())           # remove spaces at both ends
print(name.strip().upper())
print(name.strip().lower())
print(name.replace("Sodium", "Potassium"))
print(len(name.strip()))      # number of characters
```

```
word = "spectrum"
print(word[0], word[-1])    # first and last character (counting starts at 0)
print(word[0:4])           # characters 0,1,2,3 (end is excluded)
print("spec" in word)      # is one string inside another?
```

OUTPUT

```
Sodium Chloride
SODIUM CHLORIDE
sodium chloride
Potassium Chloride
15
s m
spec
True
```

f-strings let you place variables inside text. Put an **f** before the quote and put the variable in curly braces. You can control the number of decimals with **:.2f**.

```
mean_value = 3.14159265
n = 25
print(f"The mean of {n} samples is {mean_value:.2f}")
print(f"Scientific notation: {123456.789:.3e}")
print(f"Percent: {0.4567:.1%}")
```

OUTPUT

```
The mean of 25 samples is 3.14
Scientific notation: 1.235e+05
Percent: 45.7%
```

"a,b,c".split(",") cuts text into a list, and **",".join(list)** glues a list back into text.

```
line = "Na,K,Ca,Mg"
elements = line.split(",")
print(elements)
print(" | ".join(elements))
```

OUTPUT

```
['Na', 'K', 'Ca', 'Mg']
Na | K | Ca | Mg
```

2.3 Collections: lists, tuples, dictionaries, sets

Lists

A **list** is an ordered collection you can change. It is written with square brackets. Positions are called **indices** and start at 0.

```
masses = [12.5, 14.1, 13.3, 12.9]
print(masses[0])    # first element
print(masses[-1])   # last element
print(masses[1:3])  # slice: index 1 and 2

masses.append(13.8)  # add to the end
masses[0] = 12.0     # change an element
print(masses)
print(len(masses), sum(masses), min(masses), max(masses))
print(sorted(masses)) # returns a new sorted list
```

OUTPUT

```
12.5
12.9
[14.1, 13.3]
[12.0, 14.1, 13.3, 12.9, 13.8]
5 66.1 12.0 14.1
[12.0, 12.9, 13.3, 13.8, 14.1]
```

THE MATH BEHIND IT

Why does counting start at zero? Think of the index as "how many steps from the start". The first item is 0 steps away. A slice `a[start:stop]` includes `start` but excludes `stop`, so `a[0:3]` has exactly $3 - 0 = 3$ items. This half-open rule makes slices join neatly: `a[0:3]` and `a[3:6]` never overlap and never leave gaps.

Tuples

A **tuple** is like a list but cannot be changed after creation. It is written with round brackets and is used for fixed groups such as coordinates or image sizes.

```
point = (3.0, 4.5)
x, y = point          # "unpacking" into two variables
print(x, y)
```

OUTPUT
3.0 4.5

Dictionaries

A **dictionary** stores **key: value** pairs, like a real dictionary stores word: meaning. You look up by key instead of by position.

```
sample = {"id": "S01", "species": "E. coli", "count": 240, "valid": True}
print(sample["species"])
sample["count"] = 260          # change a value
sample["temperature"] = 37.0   # add a new key
print(sample)
print(sample.keys())
print(sample.get("ph", "not measured")) # safe lookup with a default
```

OUTPUT
E. coli
{'id': 'S01', 'species': 'E. coli', 'count': 260, 'valid': True, 'temperature': 37.0}
dict_keys(['id', 'species', 'count', 'valid', 'temperature'])
not measured

Dictionaries are the natural way to describe one record. Later, pandas will turn a dictionary of columns into a table.

Sets

A **set** holds unique items with no order. It is handy for removing duplicates and for asking "which items are in both?".

```
labels = ["A", "B", "A", "C", "B", "A"]
unique = set(labels)
print(unique)
print({"A", "B"} & {"B", "C"}) # intersection: in both
print({"A", "B"} | {"B", "C"}) # union: in either
```

OUTPUT
{'A', 'C', 'B'}
{'B'}
{'A', 'C', 'B'}

Collection	Ordered	Changeable	Duplicates	Typical use
list <code>[]</code>	yes	yes	yes	a sequence of measurements
tuple <code>()</code>	yes	no	yes	fixed pairs, shapes
dict <code>{k: v}</code>	yes (insertion)	yes	keys unique	one record, lookups

Collection	Ordered	Changeable	Duplicates	Typical use
set { }	no	yes	no	unique labels

2.4 Making decisions with if

An **if** statement runs code only when a condition is true. Python uses **indentation** (4 spaces) to show which lines belong to the block, and a colon **:** at the end of the header line.

```
ph = 6.2

if ph < 7:
    print("acidic")
elif ph == 7:
    print("neutral")
else:
    print("basic")
```

OUTPUT
acidic

elif means "else if"; **else** catches everything left.

2.5 Repeating work with loops

for loops

A **for** loop repeats a block once for each item of a sequence.

```
readings = [2.1, 2.4, 1.9, 2.8]
total = 0
for value in readings:
    total = total + value
print("total =", total)
print("mean =", total / len(readings))
```

OUTPUT
total = 9.2
mean = 2.3

range(n) produces the numbers 0 to n-1. **enumerate** gives both position and value; **zip** walks through two lists together.

```
for i in range(3):
    print("step", i)

names = ["Na", "K", "Ca"]
weights = [22.99, 39.10, 40.08]
for i, name in enumerate(names):
    print(i, name)
for name, w in zip(names, weights):
    print(f"{name}: {w}")
```

OUTPUT
step 0
step 1
step 2
0 Na
1 K
2 Ca
Na: 22.99
K: 39.1
Ca: 40.08

while loops

A **while** loop repeats as long as a condition stays true. Make sure something inside the loop eventually makes it false.

```
population = 100
day = 0
while population < 400:
    population = population * 2    # doubles each day
    day += 1                      # same as day = day + 1
print(day, population)
```

OUTPUT
2 400

Use **break** to leave a loop early and **continue** to skip to the next round.

List comprehensions

A **list comprehension** is a compact way to build a list from another. Read it as: "give me **expression** for each **item** in **sequence** (if **condition**)".

```
values = [1, 2, 3, 4, 5, 6]
squares = [v ** 2 for v in values]
evens = [v for v in values if v % 2 == 0]
celsius = [0, 25, 100]
fahrenheit = [c * 9 / 5 + 32 for c in celsius]
print(squares, evens, fahrenheit)
```

OUTPUT
[1, 4, 9, 16, 25, 36] [2, 4, 6] [32.0, 77.0, 212.0]

The same idea works for dictionaries: `{k: v for ...}`.

2.6 Functions

A **function** is a named, reusable recipe. It takes **inputs** (called *parameters* or *arguments*), does work and **returns** a result. Writing functions avoids repeating yourself and gives a name to each step of your analysis.

```
def mean(numbers):
    """Return the arithmetic mean of a list of numbers."""
    return sum(numbers) / len(numbers)

def describe(numbers, decimals=2):
    """Return the mean and range. 'decimals' has a default value."""
    m = mean(numbers)
    spread = max(numbers) - min(numbers)
    return round(m, decimals), spread

print(mean([4, 8, 6]))
print(describe([4, 8, 6, 10]))
m, r = describe([4, 8, 6, 10], decimals=1)  # keyword argument
print(m, r)
```

OUTPUT
6.0
(7.0, 6)
7.0 6

The text in triple quotes just below **def** is a **docstring**: documentation that shows up when you type `help(mean)` or press Shift + Tab.

NOTE

A variable created inside a function exists only inside it (its **scope**). This protects your other variables from accidental changes. To get a value out, **return** it.

Lambda functions

A **lambda** is a tiny one-line unnamed function. You will see it often with pandas.

```
square = lambda x: x ** 2
print(square(7))
pairs = [("a", 3), ("b", 1), ("c", 2)]
print(sorted(pairs, key=lambda p: p[1])) # sort by the second item
```

OUTPUT

```
49
[('b', 1), ('c', 2), ('a', 3)]
```

2.7 Modules and imports

A **module** is a file of Python code, and a **package** is a folder of modules. `import math` gives you the `math` module; access its contents with a dot.

```
import math
from statistics import mean, median

print(math.sqrt(16), math.pi, math.log(math.e))
print(math.floor(2.7), math.ceil(2.1))
print(mean([1, 2, 3, 10]), median([1, 2, 3, 10]))
```

OUTPUT

```
4.0 3.141592653589793 1.0
2 3
4 2.5
```

Use `dir(math)` to list what a module contains and `help(math.sqrt)` to read about one function.

2.8 Handling errors

When Python cannot continue it raises an **exception**. You can catch it with `try` and `except` so your program stays alive, which is useful when reading messy files.

```
def safe_float(text):
    try:
        return float(text)
    except ValueError:
        return None # marker for "could not convert"

raw = ["3.5", "4.1", "n/a", "2.9"]
print([safe_float(t) for t in raw])
```

OUTPUT

```
[3.5, 4.1, None, 2.9]
```

Common error types you will meet: `NameError` (variable does not exist), `TypeError` (wrong kind of value), `ValueError` (right type, bad value), `KeyError` (missing dictionary key), `IndexError` (position out of range), `FileNotFoundError`.

2.9 A first mini analysis with plain Python

Let us use only what we know to analyse a small experiment: the times (in seconds) for a ball to fall from a fixed height.

```
times = [0.45, 0.47, 0.44, 0.46, 0.52, 0.45, 0.46]

n = len(times)
mean_t = sum(times) / n
# variance: average squared distance from the mean
variance = sum((t - mean_t) ** 2 for t in times) / (n - 1)
std = variance ** 0.5

print(f"n = {n}")
print(f"mean = {mean_t:.4f} s")
print(f"standard deviation = {std:.4f} s")

# estimate gravity from h = 0.5 * g * t^2 -> g = 2h / t^2
h = 1.0 # metres
g = 2 * h / mean_t ** 2
print(f"estimated g = {g:.2f} m/s^2")
```

OUTPUT

```
n = 7
mean = 0.4643 s
standard deviation = 0.0264 s
estimated g = 9.28 m/s^2
```

In the next chapters NumPy and pandas will do this in one line each. Doing it by hand once shows you exactly what those functions do.

2.10 Chapter summary

CHAPTER SUMMARY

- Variables hold values of type `int`, `float`, `str`, `bool`. Use f-strings to display results.
- Lists (ordered, changeable), tuples (fixed), dictionaries (key: value), sets (unique items).
- `if/elif/else` decides, `for` and `while` repeat, comprehensions build lists compactly.
- Functions package a step of work with `def` and `return`; modules add tools via `import`.
- `try/except` prevents crashes from bad values. Indentation defines blocks.

2.11 Practice exercises

1. Write a function `to_kelvin(c)` and use a list comprehension to convert `[0, 37, 100]` degrees Celsius to Kelvin ($K = C + 273.15$).
2. Given `data = [3, 8, 1, 9, 4]`, use a loop to find the maximum without using `max()`.
3. Create a dictionary of three planets and their number of moons. Print the planet with the largest value.
4. Write a function `count_above(values, limit)` that returns how many values exceed the limit.
5. Use `try/except` to convert `["1", "two", "3"]` to numbers, replacing failures with `0`.

CHAPTER 3

3. The Math and Statistics Toolkit

IN THIS CHAPTER YOU WILL LEARN

- Reading mathematical notation: Σ , Π , exponents, logarithms, Greek letters
- Functions, rates of change (derivatives) and areas (integrals) as *ideas*
- Vectors and matrices: what they are and the few operations you need
- Probability basics: counting, conditional probability and Bayes' rule
- Error, relative error, significant figures and scientific notation

Many people close a data analysis book when the first formula appears. Please do not. Every formula in this book describes something you can already do with a calculator, written in a shorter form. This chapter is a friendly toolkit. You can skim it now and return whenever a symbol looks unfamiliar (Appendix B lists all symbols).

NOTE

This chapter uses a little NumPy and Matplotlib ahead of their full introduction in Chapters 4 to 9. Read those lines as "make a list of numbers" and "draw a picture"; you will not need the details yet.

3.1 Numbers, units and scientific notation

Scientists work with numbers from the size of an atom to the size of the universe, so we write them in **scientific notation**: a number between 1 and 10 times a power of ten. For example, 6.02×10^{23} is written `6.02e23` in Python.

```
avogadro = 6.02e23
electron_charge = 1.602e-19
print(avogadro, electron_charge)
print(f"{avogadro:.3e}")
print(avogadro * electron_charge) # the Faraday constant, about 96,000 C/mol
```

OUTPUT

```
6.02e+23 1.602e-19
6.020e+23
96440.4
```

Significant figures tell how many digits of a number are trustworthy. A length recorded as 12.3 cm has three significant figures; recording it as 12.3000 cm claims far more precision than a ruler can give. When you present results, round to a sensible number of digits and keep the full precision only inside the computer.

TIP

Always attach units to your numbers in column names or labels, such as `mass_kg` or `time_s`. Many famous failures in science and engineering came from mixing units.

3.2 The summation symbol Σ

The Greek capital sigma means "add up". In data analysis you will see it constantly:

$$\sum_{i=1}^n x_i = x_1 + x_2 + \dots + x_n$$

Read it as: "for i going from 1 up to n, add the values x_i ". The small letter below is where the counter starts; the one above is where it stops. It is exactly a **for** loop that keeps a running total.

```
x = [2, 4, 6, 8]

total = 0
for value in x:      # the loop version of the sigma
    total += value
print(total, sum(x))

# Sigma of squares: x1^2 + x2^2 + ...
print(sum(v ** 2 for v in x))
# Sigma of (x - 3): each value minus 3, then added
print(sum(v - 3 for v in x))
```

OUTPUT

```
20 20
120
8
```

The mean (average) is therefore

$$\text{mean} = (1 / n) \times \sum x_i$$

Its cousin Π (capital pi) means "multiply everything": $\Pi x_i = x_1 \times x_2 \times \dots \times x_n$. It appears in probability and likelihood formulas.

3.3 Exponents and logarithms

An **exponent** says "multiply by itself": $2^5 = 2 \times 2 \times 2 \times 2 \times 2 = 32$. A **logarithm** is the reverse question: "which power do I need?" Since $2^5 = 32$, we say $\log_2(32) = 5$.

$$b^y = x \Leftrightarrow \log_b(x) = y$$

```
import math

print(2 ** 5, math.log2(32))      # log base 2
print(10 ** 3, math.log10(1000))  # log base 10
print(math.exp(1), math.log(math.e)) # e = 2.718..., natural log ln
```

OUTPUT

```
32 5.0
1000 3.0
2.718281828459045 1.0
```

The special number $e \approx 2.71828$ is the base of natural growth. The natural logarithm **ln** (Python's **math.log**) undoes e^x .

Why do scientists love logarithms?

1. They turn multiplication into addition: $\log(a \times b) = \log(a) + \log(b)$. Multiplying huge numbers becomes adding small ones.
2. They compress huge ranges. $\text{pH} = -\log_{10}[\text{H}^+]$, so a change of one pH unit is a tenfold change in acidity. Earthquake magnitudes and decibels work the same way.
3. They straighten exponential growth: a curve that doubles every day becomes a straight line on a log axis.

```

h_plus = 1e-7 # concentration in mol/L (pure water)
print("pH =", -math.log10(h_plus))

h_plus = 1e-3
print("pH =", -math.log10(h_plus))

# growth: bacteria doubling every hour, starting at 100
for hours in [0, 1, 2, 3, 10]:
    print(hours, 100 * 2 ** hours)

```

OUTPUT
pH = 7.0
pH = 3.0
0 100
1 200
2 400
3 800
10 102400

WATCH OUT

You can only take the logarithm of a **positive** number. `math.log(0)` and `math.log(-2)` raise errors. In datasets, zeros are common, so people often use `log(x + 1)`.

3.4 Functions and graphs

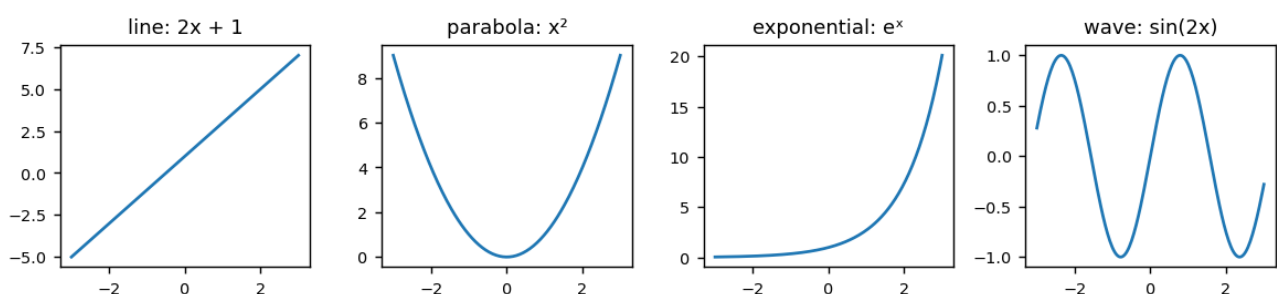
A **function** takes an input x and gives an output $y = f(x)$. The graph plots those pairs. Here are four shapes that appear everywhere in science: a straight line, a parabola, an exponential and a wave.

```

import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 200) # 200 evenly spaced points from -3 to 3
fig, axes = plt.subplots(1, 4, figsize=(9, 2.2))
axes[0].plot(x, 2 * x + 1); axes[0].set_title("line: 2x + 1")
axes[1].plot(x, x ** 2); axes[1].set_title("parabola: x^2")
axes[2].plot(x, np.exp(x)); axes[2].set_title("exponential: e^x")
axes[3].plot(x, np.sin(2 * x)); axes[3].set_title("wave: sin(2x)")
plt.tight_layout()
plt.show()

```



The line $y = m \cdot x + b$ has **slope** m (how much y rises when x increases by 1) and **intercept** b (the value of y at $x = 0$). Fitting a line to data is the core of regression (Chapter 16).

3.5 Rate of change: the derivative as an idea

If a car's position is $f(t)$, its speed is *how fast f changes*. The **derivative** is the exact rate of change at one instant. We estimate it by looking at a tiny step h :

$$f'(x) \approx (f(x + h) - f(x)) / h \quad (h \text{ very small})$$

```
def f(x):
    return x ** 2

x0 = 3
for h in [1, 0.1, 0.01, 0.001, 0.0001]:
    slope = (f(x0 + h) - f(x0)) / h
    print(f"h = {h:<7} slope = {slope:.5f}")
# calculus says the exact slope of x^2 at x = 3 is 2*3 = 6
```

OUTPUT

```
h = 1      slope = 7.00000
h = 0.1    slope = 6.10000
h = 0.01   slope = 6.01000
h = 0.001  slope = 6.00100
h = 0.0001 slope = 6.00010
```

As h shrinks, the estimate settles on 6. You will not need to differentiate by hand in this book, but the idea explains **optimisation**: to find the lowest point of a curve, look for where the slope is zero. Least-squares fitting, machine-learning training and curve fitting all "walk downhill" until the slope is zero.

3.6 Area under a curve: the integral as an idea

The **integral** adds up tiny slices to give the total area under a curve. Cut the range into thin rectangles, add their areas, and take thinner and thinner rectangles:

$$\int_a^b f(x) dx \approx \sum f(x_i) \times \Delta x$$

```
def f(x):
    return x ** 2

a, b = 0, 3
for n in [3, 30, 300, 3000]:
    dx = (b - a) / n
    area = sum(f(a + i * dx) * dx for i in range(n))
    print(f"{n:>5} rectangles -> area = {area:.4f}")
# exact answer: x^3/3 evaluated from 0 to 3 = 9
```

OUTPUT

```
3 rectangles -> area = 5.0000
30 rectangles -> area = 8.5550
300 rectangles -> area = 8.9550
3000 rectangles -> area = 8.9955
```

Integrals turn speed into distance, a probability density into a probability, and a spectrum into total energy.

3.7 Vectors and matrices

A **vector** is an ordered list of numbers, such as a measurement made of three values (temperature, pressure, humidity). Geometrically it is an arrow. A **matrix** is a rectangular table of numbers with rows and columns. A dataset with 100 samples and 5 measured variables is naturally a 100×5 matrix.

The two key operations:

- **Dot product** of two vectors: multiply matching entries and add. $a \cdot b = \sum a_i b_i$. It measures how much two vectors point the same way.
- **Matrix multiplication**: each entry of the result is the dot product of a row of the first matrix with a column of the second.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print("dot:", np.dot(a, b))          # 1*4 + 2*5 + 3*6
print("length of a:", np.linalg.norm(a)) # sqrt(1 + 4 + 9)

A = np.array([[1, 2],
              [3, 4]])
B = np.array([[0, 1],
              [1, 0]])
print(A @ B)          # matrix product (rows of A times columns of B)
print(A.T)            # transpose: swap rows and columns
```

OUTPUT
dot: 32
length of a: 3.7416573867739413
[[2 1]
 [4 3]]
[[1 3]
 [2 4]]

Solving equations. A system such as $2x + y = 5$ and $x - y = 1$ is written in matrix form $A \cdot v = c$, and solved in one line:

```
A = np.array([[2, 1],
              [1, -1]])
c = np.array([5, 1])
solution = np.linalg.solve(A, c)
print(solution)          # x = 2, y = 1
print(A @ solution)      # check: gives back [5, 1]
```

OUTPUT
[2. 1.]
[5. 1.]

The same machinery fits lines and planes to thousands of data points (Chapter 16) and powers PCA (Chapter 19).

3.8 Probability basics

Probability measures how likely an event is on a scale from 0 (impossible) to 1 (certain). If all outcomes are equally likely:

$$P(\text{event}) = (\text{number of favourable outcomes}) / (\text{total number of outcomes})$$

Key rules:

Rule	Formula	Example
Complement	$P(\text{not } A) = 1 - P(A)$	$P(\text{no six on one die}) = 5/6$
Both (independent events)	$P(A \text{ and } B) = P(A) \times P(B)$	Two sixes in a row = $1/36$
Either (exclusive events)	$P(A \text{ or } B) = P(A) + P(B)$	Rolling 1 or 2 = $1/3$
Conditional	$P(A \text{ given } B) = P(A \text{ and } B) / P(B)$	See below

We can check probabilities by **simulation**: repeat the random experiment many times and count.

```
import random
random.seed(1)

rolls = 100000
sixes = sum(1 for _ in range(rolls) if random.randint(1, 6) == 6)
print("simulated P(six) =", sixes / rolls, " exact =", round(1 / 6, 4))
```

```
# two dice both showing six
both = sum(1 for _ in range(rolls)
           if random.randint(1, 6) == 6 and random.randint(1, 6) == 6)
print("simulated P(two sixes) =", both / rolls, " exact =", round(1 / 36, 4))
```

OUTPUT

```
simulated P(six) = 0.16551 exact = 0.1667
simulated P(two sixes) = 0.02745 exact = 0.0278
```

Conditional probability and Bayes' rule

Conditional probability $P(A | B)$ is the probability of A once you know that B happened. **Bayes' rule** reverses the direction:

$$P(A | B) = P(B | A) \times P(A) / P(B)$$

Suppose a disease affects 1% of people. A test detects it 95% of the time (sensitivity) but gives a false alarm for 5% of healthy people. If you test positive, how likely are you to be ill?

```
p_disease = 0.01
p_pos_given_disease = 0.95
p_pos_given_healthy = 0.05

# total chance of a positive result (law of total probability)
p_pos = p_pos_given_disease * p_disease + p_pos_given_healthy * (1 - p_disease)
p_disease_given_pos = p_pos_given_disease * p_disease / p_pos
print(f"P(positive) = {p_pos:.4f}")
print(f"P(disease | positive) = {p_disease_given_pos:.3f}")
```

OUTPUT

```
P(positive) = 0.0590
P(disease | positive) = 0.161
```

Only about 16%! Because the disease is rare, most positive results are false alarms. Bayes' rule is the heart of Bayesian statistics (Chapter 23) and shows why the *base rate* matters.

3.9 Measuring error

No measurement is perfect. Two simple ways to state how far a measured value is from the true (or accepted) one:

$$\text{absolute error} = | \text{measured} - \text{true} | \quad \text{relative error} = | \text{measured} - \text{true} | / | \text{true} |$$

```
true_g = 9.81
measured = [9.6, 9.9, 9.75, 9.85]
for m in measured:
    abs_err = abs(m - true_g)
    rel_err = abs_err / true_g
    print(f"measured {m:.2f} abs error {abs_err:.2f} relative {rel_err:.2%}")

# percent change between two values
old, new = 50, 65
print("percent change:", (new - old) / old * 100, "%")
```

OUTPUT

```
measured 9.60 abs error 0.21 relative 2.14%
measured 9.90 abs error 0.09 relative 0.92%
measured 9.75 abs error 0.06 relative 0.61%
measured 9.85 abs error 0.04 relative 0.41%
percent change: 30.0 %
```

Later chapters distinguish *random error* (noise that averages out) from *systematic error* (a bias that does not), and show how to combine errors properly (Chapter 23).

3.10 Greek letters you will meet

Symbol	Name	Typical meaning in this book
μ	mu	population mean
σ	sigma (small)	standard deviation
σ^2	sigma squared	variance
Σ	sigma (capital)	sum
ρ, r	rho, r	correlation
α	alpha	significance level (Chapter 15)
β	beta	regression coefficient
ε	epsilon	error / noise term
λ	lambda	rate (Poisson, exponential), regularisation strength
θ	theta	a generic parameter to estimate
χ^2	chi-squared	a test statistic and distribution
Δ	delta (capital)	a change in something

3.11 Chapter summary

CHAPTER SUMMARY

- Scientific notation (**1.5e-3**) and significant figures communicate size and precision.
- Σ means "add a list of terms"; it is the loop **for ... total += ...**.
- Logarithms undo exponentials; they compress ranges and straighten growth curves.
- Derivatives measure rate of change; integrals measure accumulated area. Both can be estimated with tiny steps.
- Vectors and matrices are lists and tables of numbers; the dot product and matrix product are sums of products.
- Probability lives between 0 and 1; simulation can check formulas; Bayes' rule updates beliefs.
- Absolute and relative error measure how far a value is from the truth.

3.12 Practice exercises

1. Compute Σi^2 for $i = 1$ to 10 using **sum()** and a range.
2. A solution has $[H^+] = 3.2 \times 10^{-5}$ mol/L. Compute its pH.
3. A quantity grows by 5% per year. How many years until it doubles? (Hint: solve $1.05^t = 2$ using logarithms.)
4. Estimate the derivative of $\sin(x)$ at $x = 0$ numerically. What number does it approach?
5. A test has 99% sensitivity and 2% false-positive rate for a condition with 0.5% prevalence. Use Bayes' rule to find $P(\text{condition} \mid \text{positive})$.

PART II

Core Libraries

NumPy, pandas, Matplotlib and data loading: the everyday toolkit of every scientific data analyst.

CHAPTER 4

4. NumPy Part 1: Arrays

IN THIS CHAPTER YOU WILL LEARN

- Why NumPy exists and what an array is
- Creating arrays, understanding shape, dimensions and data types
- Indexing, slicing, boolean masks and fancy indexing
- Reshaping, stacking, splitting, copies versus views

4.1 Why NumPy?

Python lists are flexible but slow for numbers, and they cannot do math on the whole list at once.

NumPy (Numerical Python) adds the **ndarray** ("n-dimensional array"): a grid of numbers, all of the same type, stored compactly in memory. Two big benefits:

1. **Speed.** Loops run in fast compiled code instead of slow Python.
2. **Convenience.** You can add, multiply or take the logarithm of a million numbers with a single expression, without writing a loop. This is called **vectorisation**.

```
import numpy as np
import time

n = 1_000_000
py_list = list(range(n))
np_array = np.arange(n)

t0 = time.perf_counter()
py_result = [v * 2 for v in py_list]
t1 = time.perf_counter()
np_result = np_array * 2
t2 = time.perf_counter()

print(f"Python list : {(t1 - t0) * 1000:.1f} ms")
print(f"NumPy array : {(t2 - t1) * 1000:.1f} ms")
```

OUTPUT

```
Python list : 33.1 ms
NumPy array : 3.6 ms
```

Your times will differ, but NumPy is typically 10 to 100 times faster here. Learning to think in whole arrays instead of loops is the biggest step towards fluent scientific Python.

4.2 Creating arrays

```
import numpy as np

a = np.array([1, 2, 3, 4])          # from a list
b = np.array([[1, 2, 3],          # from a list of lists (2 rows, 3 columns)
               [4, 5, 6]])

print(a)
print(b)
print(type(a))
```

OUTPUT

```
[1 2 3 4]
[[1 2 3]
 [4 5 6]]
```

```
<class 'numpy.ndarray'>
```

Four attributes describe every array:

Attribute	Meaning	For <code>b</code> above
<code>ndim</code>	number of dimensions (axes)	2
<code>shape</code>	size along each axis (rows, columns, ...)	(2, 3)
<code>size</code>	total number of elements	6
<code>dtype</code>	type of each element	int64

```
print(b.ndim, b.shape, b.size, b.dtype)
```

OUTPUT

```
2 (2, 3) 6 int64
```

Ready-made arrays

```
print(np.zeros(4))           # four zeros
print(np.ones((2, 3)))      # 2x3 of ones
print(np.full((2, 2), 7.5))  # filled with a chosen value
print(np.arange(0, 10, 2))   # start, stop (excluded), step
print(np.linspace(0, 1, 5))  # 5 evenly spaced numbers, stop INCLUDED
print(np.eye(3))            # identity matrix
```

OUTPUT

```
[0. 0. 0. 0.]
[[1. 1. 1.]
 [1. 1. 1.]]
[[7.5 7.5]
 [7.5 7.5]]
[0 2 4 6 8]
[0.  0.25 0.5  0.75 1.  ]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

`arange` is like `range` but produces an array and accepts decimals; `linspace` is preferred for decimals because you choose the **number of points**, avoiding rounding surprises.

Random arrays

Simulation is central to science, so random numbers are essential. Use the modern **Generator** interface and a **seed** so results are reproducible.

```
rng = np.random.default_rng(seed=42)  # create a random number generator
print(rng.random(3))                  # uniform numbers between 0 and 1
print(rng.integers(1, 7, size=5))      # dice rolls: 1 to 6
print(rng.normal(loc=10, scale=2, size=4)) # bell-curve numbers, mean 10, std 2
```

OUTPUT

```
[0.77395605 0.43887844 0.85859792]
[1 5 2 1 4]
[10.25568081  9.36751482  9.96639768  8.29391214]
```

NOTE

A **seed** fixes the starting point of the random sequence. The same seed always gives the same numbers, so your analysis can be repeated by a colleague. Without a seed, each run differs.

4.3 Data types

Every element of an array shares one **dtype**. Common ones: `int64` (whole numbers), `float64` (decimals), `bool` (True/False), `<U5` (text up to 5 characters). NumPy chooses one automatically but you can force it:

```
print(np.array([1, 2, 3]).dtype)
print(np.array([1, 2.5, 3]).dtype)      # one float upgrades everything
x = np.array([1, 2, 3], dtype=float)
print(x, x.dtype)
y = np.array([1.9, 2.7, -3.2])
print(y.astype(int))                    # converts by cutting decimals (toward zero)
```

OUTPUT
int64
float64
[1. 2. 3.] float64
[1 2 -3]

Missing and special values. `np.nan` means "not a number" (a missing value) and `np.inf` is infinity. Any calculation involving `nan` gives `nan`, so check for it with `np.isnan`.

```
z = np.array([1.0, np.nan, 3.0])
print(z.sum())                        # nan spreads
print(np.isnan(z))
print(np.nansum(z), np.nanmean(z))    # functions that ignore nan
print(1 / np.array([2.0, 0.0]))        # divide by zero gives inf (with a warning)
```

OUTPUT
nan
[False True False]
4.0 2.0
[0.5 inf]

4.4 Indexing and slicing

Indexing an array works like a list, extended to several axes. For a 2D array use `[row, column]`.

```
m = np.array([[10, 11, 12, 13],
               [20, 21, 22, 23],
               [30, 31, 32, 33]])

print(m[0, 0])      # first row, first column
print(m[1, 2])      # second row, third column
print(m[-1, -1])    # last row, last column
print(m[0])          # entire first row
print(m[:, 1])       # entire second column (: means "all")
print(m[0:2, 1:3])   # rows 0-1, columns 1-2 (a sub-block)
print(m[::2, ::2])   # every second row and column
```

OUTPUT
10
22
33
[10 11 12 13]
[11 21 31]
[[11 12]
 [21 22]]
[[10 12]
 [30 32]]

The rule for slices: `start:stop:step`, where `stop` is excluded and any part may be left blank.

Boolean masks: selecting by condition

Comparing an array with a number gives an array of **True/False**, called a **mask**. Using the mask as an index keeps only the **True** positions. This is the array way of asking "give me the values where...".

```
temps = np.array([18.5, 22.1, 30.4, 25.0, 15.2, 33.8])

mask = temps > 25
print(mask)
print(temps[mask])           # values above 25
print(temps[(temps > 20) & (temps < 31)]) # combine with & (and), | (or), ~ (not)
print(np.sum(temps > 25))    # True counts as 1: how many above 25?
print(np.mean(temps > 25))   # proportion above 25
```

OUTPUT
 [False False True False False True]
 [30.4 33.8]
 [22.1 30.4 25.]
 2
 0.3333333333333333

WATCH OUT

Combine conditions with **&**, **|**, **~** and wrap each condition in brackets. Using Python's **and/or** on arrays raises an error, because it does not know how to decide truth for a whole array.

Fancy indexing

You can pass a list of positions to pick specific elements, in any order.

```
v = np.array([100, 200, 300, 400, 500])
print(v[[0, 2, 4]])
print(v[[4, 0]])
v[[1, 3]] = 0           # assign to several positions at once
print(v)
```

OUTPUT
 [100 300 500]
 [500 100]
 [100 0 300 0 500]

4.5 Changing the shape

```
a = np.arange(12)
print(a)
print(a.reshape(3, 4))      # 3 rows, 4 columns (total must stay 12)
print(a.reshape(2, -1))    # -1 means "work it out": here 6 columns
print(a.reshape(3, 4).T)   # transpose: 4 rows, 3 columns
print(a.reshape(3, 4).flatten()) # back to 1D
```

OUTPUT
 [0 1 2 3 4 5 6 7 8 9 10 11]
 [[0 1 2 3]
 [4 5 6 7]
 [8 9 10 11]]
 [[0 1 2 3 4 5]
 [6 7 8 9 10 11]]
 [[0 4 8]
 [1 5 9]
 [2 6 10]
 [3 7 11]]
 [0 1 2 3 4 5 6 7 8 9 10 11]

Adding a dimension

Sometimes an array needs to be a column instead of a row. `np.newaxis` (or `None`) adds an axis of length 1.

```
r = np.array([1, 2, 3])
print(r.shape)
col = r[:, np.newaxis]
print(col.shape)
print(col)
```

OUTPUT

```
(3,)
(3, 1)
[[1]
 [2]
 [3]]
```

Joining and splitting

```
p = np.array([1, 2, 3])
q = np.array([4, 5, 6])
print(np.concatenate([p, q]))
print(np.vstack([p, q]))      # stack as rows (vertical)
print(np.hstack([p, q]))      # side by side (horizontal)
print(np.column_stack([p, q])) # each array becomes a column
print(np.split(np.arange(6), 3)) # three equal parts
```

OUTPUT

```
[1 2 3 4 5 6]
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
[[1 4]
 [2 5]
 [3 6]]
[array([0, 1]), array([2, 3]), array([4, 5])]
```

4.6 Copies versus views

A **slice** does not copy the data; it is a **view** of the same memory. Changing the view changes the original. This is fast and memory-efficient but can surprise you.

```
original = np.array([1, 2, 3, 4, 5])
view = original[1:4]
view[0] = 99
print(original)      # the original changed too!

safe = original[1:4].copy() # an independent copy
safe[0] = -1
print(original, safe)
```

OUTPUT

```
[ 1 99  3  4  5]
[ 1 99  3  4  5] [-1  3  4]
```

Boolean-mask and fancy indexing, in contrast, return copies. When in doubt, call `.copy()`.

4.7 Useful array methods at a glance

```
d = np.array([[3, 1, 2],
              [9, 7, 8]])
print(d.max(), d.min(), d.sum(), d.mean())
print(d.argmax(), d.argmin())      # POSITION of the largest / smallest (flattened)
print(np.sort(d, axis=1))          # sort each row
print(d.T.shape, d.ravel().shape)
```

OUTPUT

```
9 1 30 5.0
3 1
[[1 2 3]
 [7 8 9]]
(3, 2) (6,)
```

4.8 Chapter summary

CHAPTER SUMMARY

- An `ndarray` is a fast grid of same-typed numbers with `shape`, `ndim`, `size`, `dtype`.
- Create arrays with `np.array`, `zeros`, `ones`, `arange`, `linspace`, and the `default_rng` generator (use a seed).
- Index with `[row, col]`, slice with `start:stop:step`, filter with boolean masks (`&`, `|`, `~`).
- `reshape`, `T`, `concatenate`, `vstack`, `hstack` change and join arrays.
- Slices are views (shared memory); use `.copy()` for independence.
- `np.nan` marks missing values; use `np.nanmean` etc. to ignore them.

4.9 Practice exercises

1. Make an array of the even numbers from 2 to 20, then select those greater than 10.
2. Create a 4×4 array of numbers 1 to 16. Extract the middle 2×2 block.
3. Generate 10 random integers from 1 to 100 (seed 0) and count how many exceed 50.
4. Make `np.linspace(0, 2*np.pi, 5)` and print it. How is it different from `np.arange(0, 2*np.pi, 1)`?
5. Show with an experiment that `b = a[:3]` is a view but `c = a[a > 2]` is a copy.

CHAPTER 5

5. NumPy Part 2: Vectorisation, Broadcasting and Linear Algebra

IN THIS CHAPTER YOU WILL LEARN

- Vectorised arithmetic and universal functions (ufuncs)
- Broadcasting: how arrays of different shapes work together
- Aggregations along axes: sum, mean, std by row or column
- `where`, sorting, `unique`, cumulative operations
- Linear algebra and random simulation with NumPy

5.1 Vectorised arithmetic

Operations on arrays apply **element by element** with no loop.

```
import numpy as np

t = np.array([0.0, 1.0, 2.0, 3.0])      # time in seconds
v0, g = 20.0, 9.81
height = v0 * t - 0.5 * g * t ** 2      # projectile height formula, all at once
print(height)
```

OUTPUT

```
[ 0.    15.095 20.38 15.855]
```

The expression `v0 * t - 0.5 * g * t ** 2` reads exactly like the physics formula, which is one of NumPy's greatest strengths.

Universal functions (ufuncs)

A **ufunc** is a function that operates on every element: `np.sqrt`, `np.exp`, `np.log`, `np.sin`, `np.abs`, `np.round`, and so on.

```
x = np.array([1, 4, 9, 16])
print(np.sqrt(x))
print(np.log(x))
print(np.exp(np.array([0, 1, 2])))
print(np.round(np.array([1.234, 5.678]), 1))
print(np.floor(2.7), np.ceil(2.1))
```

OUTPUT

```
[1.  2.  3.  4.]
[0.    1.38629436 2.19722458 2.77258872]
[1.    2.71828183 7.3890561 ]
[1.2  5.7]
2.0 3.0
```

5.2 Broadcasting

Broadcasting is NumPy's rule for combining arrays of different shapes. The smaller array is virtually "stretched" to match the larger one, without copying data.

```
a = np.array([[1, 2, 3],
               [4, 5, 6]])
```

```
print(a + 10)           # a scalar is stretched to every element
print(a * np.array([1, 10, 100])) # a row is stretched down each row
print(a + np.array([[100], [200]])) # a column is stretched across each column
```

OUTPUT

```
[[11 12 13]
 [14 15 16]]
[[ 1 20 300]
 [ 4 50 600]]
[[101 102 103]
 [204 205 206]]
```

The rule. Compare the shapes from the **right-most** axis going left. Two sizes are compatible if they are equal, or if one of them is 1 (or missing). Size 1 gets stretched.

Array A shape	Array B shape	Result	Why
(2, 3)	(3,)	(2, 3)	B treated as (1, 3), stretched over 2 rows
(2, 3)	(2, 1)	(2, 3)	column stretched over 3 columns
(2, 3)	(2,)	error	last axes 3 and 2 do not match
(4, 1)	(1, 5)	(4, 5)	both stretched: a grid!

```
# A multiplication table using broadcasting: column times row
rows = np.arange(1, 5)[:, np.newaxis] # shape (4, 1)
cols = np.arange(1, 6)[np.newaxis, :] # shape (1, 5)
print(rows * cols)
```

OUTPUT

```
[[ 1  2  3  4  5]
 [ 2  4  6  8 10]
 [ 3  6  9 12 15]
 [ 4  8 12 16 20]]
```

Standardising columns is a very common use. Subtract each column's mean and divide by its standard deviation, letting broadcasting handle the shapes:

```
data = np.array([[170., 65.],
                 [180., 80.],
                 [160., 55.],
                 [175., 70.]]) # height (cm), weight (kg)
mean = data.mean(axis=0)      # one mean per column
std = data.std(axis=0)
z = (data - mean) / std        # (4,2) - (2,) -> works!
print(mean, std)
print(z.round(2))
```

OUTPUT

```
[171.25  67.5 ] [7.39509973 9.01387819]
[[-0.17 -0.28]
 [ 1.18  1.39]
 [-1.52 -1.39]
 [ 0.51  0.28]]
```

5.3 Aggregations and the axis argument

Functions such as `sum`, `mean`, `std`, `min`, `max` can collapse an array. The **axis** argument chooses which direction to collapse.

- `axis=0` collapses **down the rows**, producing one result per **column**.
- `axis=1` collapses **across the columns**, producing one result per **row**.
- No axis: collapse everything into a single number.

```
scores = np.array([[80, 90, 70],
```

```

        [60, 75, 95],
        [85, 65, 88],
        [90, 92, 76]])      # 4 students (rows) x 3 tests (columns)

print("overall mean :", scores.mean())
print("per test      :", scores.mean(axis=0))  # 3 numbers, one per column
print("per student   :", scores.mean(axis=1))  # 4 numbers, one per row
print("best score in each test:", scores.max(axis=0))
print("which student best in each test:", scores.argmax(axis=0))

```

OUTPUT

```

overall mean : 80.5
per test      : [78.75 80.5  82.25]
per student   : [80.        76.66666667 79.33333333 86.        ]
best score in each test: [90 92 95]
which student best in each test: [3 3 1]

```

TIP

A memory trick: the axis you name is the one that **disappears**. A (4, 3) array summed along **axis=0** becomes shape (3,); along **axis=1** becomes (4,).

Statistics functions you will use daily:

```

x = np.array([2, 4, 4, 4, 5, 5, 7, 9])
print("mean      ", np.mean(x))
print("median    ", np.median(x))
print("std       ", np.std(x))          # population std (divide by n)
print("std (n-1)", np.std(x, ddof=1))   # sample std (divide by n-1)
print("variance  ", np.var(x, ddof=1))
print("percentiles", np.percentile(x, [25, 50, 75]))
print("range     ", np.ptp(x))          # peak to peak: max - min

```

OUTPUT

```

mean      5.0
median    4.5
std       2.0
std (n-1) 2.138089935299395
variance  4.571428571428571
percentiles [4.  4.5 5.5]
range     7

```

The **ddof** (delta degrees of freedom) argument decides between dividing by n or by $n - 1$. For **samples** taken from a larger population, use **ddof=1**; Chapter 12 explains why.

5.4 Conditional selection with where

np.where(condition, a, b) creates an array that takes **a** where the condition is true and **b** elsewhere, like an **if/else** for every element.

```

temps = np.array([12, 25, 31, 18, 35])
label = np.where(temps > 30, "hot", "ok")
print(label)
adjusted = np.where(temps > 30, 30, temps)  # cap values at 30
print(adjusted)
print(np.where(temps > 30)[0])              # with only a condition: positions (indices)

```

OUTPUT

```

['ok' 'ok' 'hot' 'ok' 'hot']
[12 25 30 18 30]
[2 4]

```

For several categories use **np.select** (a list of conditions and a list of results):

```

conditions = [temps < 15, temps < 30]
choices = ["cold", "mild"]

```

```
print(np.select(conditions, choices, default="hot"))
```

OUTPUT

```
['cold' 'mild' 'hot' 'mild' 'hot']
```

5.5 Sorting, searching and unique values

```
x = np.array([30, 10, 20, 10, 40, 20, 10])
print(np.sort(x))
order = np.argsort(x)          # positions that would sort the array
print(order)
print(x[order])
values, counts = np.unique(x, return_counts=True)
print(values, counts)          # each distinct value and how often it appears
print(np.isin(x, [10, 40]))    # membership test
```

OUTPUT

```
[10 10 10 20 20 30 40]
[1 3 6 2 5 0 4]
[10 10 10 20 20 30 40]
[10 20 30 40] [3 2 1 1]
[False  True False  True  True False  True]
```

5.6 Cumulative and difference operations

```
d = np.array([1, 3, 6, 10, 15])
print(np.cumsum(d))            # running total
print(np.cumprod(np.array([1, 2, 3, 4])))
print(np.diff(d))              # differences between neighbours (n-1 values)
```

OUTPUT

```
[ 1  4 10 20 35]
[ 1  2  6 24]
[2 3 4 5]
```

`diff` is a discrete derivative (change per step) and `cumsum` a discrete integral (running total), the two ideas of Chapter 3 in one line each.

5.7 Linear algebra with NumPy

`np.linalg` contains the standard tools:

```
A = np.array([[4., 2.],
               [1., 3.]])
b = np.array([10., 7.])

print("A @ A =\n", A @ A)
print("determinant:", np.linalg.det(A))
print("inverse:\n", np.linalg.inv(A))
x = np.linalg.solve(A, b)          # solves A x = b (preferred over inverse)
print("solution x:", x)

vals, vecs = np.linalg.eig(A)      # eigenvalues and eigenvectors
print("eigenvalues:", vals)
```

OUTPUT

```
A @ A =
[[18. 14.]
 [ 7. 11.]]
determinant: 10.000000000000002
inverse:
[[ 0.3 -0.2]
 [-0.1  0.4]]
solution x: [1.6 1.8]
```

```
eigenvalues: [5. 2.]
```

THE MATH BEHIND IT

What is an eigenvector? A matrix usually rotates and stretches a vector. An **eigenvector** is a special direction that the matrix only *stretches*, never rotates. The stretch factor is the **eigenvalue** λ : $A \cdot v = \lambda \cdot v$. In PCA (Chapter 19) the eigenvectors of the data's covariance matrix point along the directions of greatest variation, and the eigenvalues say how much variation lies along each.

5.8 Random numbers and simulation

Sampling from probability distributions with the Generator:

```
rng = np.random.default_rng(2024)

print(rng.uniform(0, 5, size=3))          # any value between 0 and 5
print(rng.normal(50, 10, size=3))         # bell curve, mean 50, std 10
print(rng.binomial(n=10, p=0.5, size=5))  # number of heads in 10 coin flips
print(rng.poisson(lam=4, size=5))         # event counts with average 4
print(rng.choice(["A", "B", "C"], size=6))
data = np.arange(10)
rng.shuffle(data)
print(data)
```

OUTPUT

```
[3.37915669 1.07161601 1.54726015]
[40.26820485 36.07199904 50.67196355]
[3 4 4 3 5]
[3 5 4 3 5]
['B' 'B' 'B' 'C' 'A' 'C']
[9 1 6 5 4 0 8 2 3 7]
```

A simulation example: estimating π

Drop random points in a unit square. The fraction that lands inside the quarter-circle of radius 1 is $\pi/4$.

```
rng = np.random.default_rng(0)
n = 200_000
x = rng.random(n)
y = rng.random(n)
inside = (x ** 2 + y ** 2) <= 1          # boolean array
pi_estimate = 4 * inside.mean()
print(pi_estimate)
```

OUTPUT

```
3.14528
```

This is a **Monte Carlo** method: use random numbers to answer a mathematical question. We will use it for uncertainty in Chapter 23.

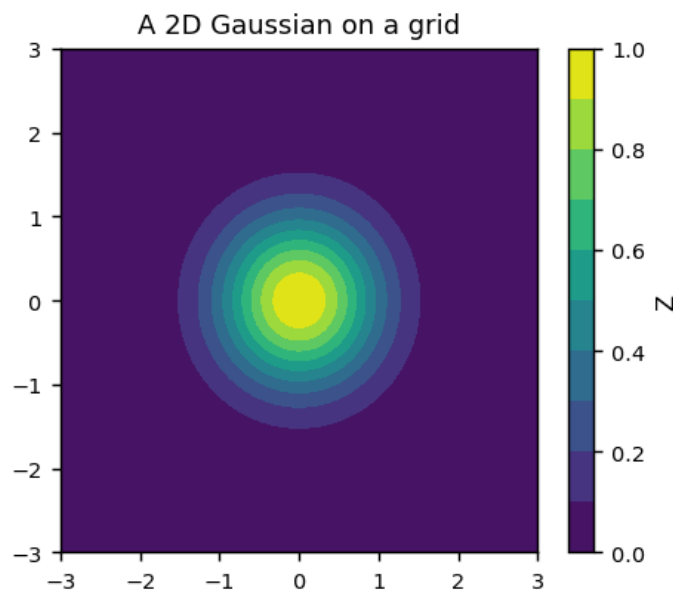
5.9 Grids of coordinates: meshgrid

To evaluate a function of two variables over a region, build a grid with `meshgrid`.

```
import matplotlib.pyplot as plt

xs = np.linspace(-3, 3, 100)
ys = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(xs, ys)          # two 100x100 arrays of x and y coordinates
Z = np.exp(-(X ** 2 + Y ** 2))      # a 2D bell-shaped surface
plt.figure(figsize=(4, 3.4))
plt.contourf(X, Y, Z, levels=10, cmap="viridis")
plt.colorbar(label="Z")
```

```
plt.title("A 2D Gaussian on a grid")
plt.show()
```



5.10 Saving and loading arrays

```
arr = np.arange(6).reshape(2, 3)
np.save("arr.npy", arr)          # binary format, exact and fast
print(np.load("arr.npy"))
np.savetxt("arr.csv", arr, delimiter=",", fmt="%d")  # plain text
print(np.loadtxt("arr.csv", delimiter=",", dtype=int))
```

OUTPUT

```
[[0 1 2]
 [3 4 5]]
[[0 1 2]
 [3 4 5]]
```

5.11 Performance tips

- Replace Python loops with vectorised expressions whenever possible.
- Avoid growing arrays one element at a time (`np.append` in a loop); create the full array first.
- Use `float32` instead of `float64` only when memory is critical; otherwise keep the default precision.
- Prefer built-ins (`np.sum`) over hand-written loops; they are implemented in optimised C.

5.12 Chapter summary

CHAPTER SUMMARY

- Arithmetic and ufuncs act on all elements at once (vectorisation).
- Broadcasting stretches size-1 axes; compare shapes from the right.
- `axis=0` collapses rows (per column result); `axis=1` collapses columns (per row result).
- `np.where/np.select` do element-wise if/else; `sort`, `argsort`, `unique`, `cumsum`, `diff` are the everyday helpers.
- `np.linalg` solves equations, inverts matrices and finds eigenvalues.

- `default_rng(seed)` generates reproducible random numbers for simulation.

5.13 Practice exercises

1. Given heights in cm `[150, 165, 172, 181]`, convert them to metres and compute the mean and sample standard deviation.
2. Build a 3×4 array of random integers (seed 1). Find the column means and the row with the largest sum.
3. Use broadcasting to subtract each row's mean from that row (Hint: `keepdims=True`).
4. Use `np.where` to replace every negative number in `[-2, 3, -1, 5]` by zero.
5. Estimate $P(\text{sum of two dice} = 7)$ by simulating 100,000 rolls with NumPy and compare with $1/6$.

CHAPTER 6

6. pandas Part 1: Series and DataFrames

IN THIS CHAPTER YOU WILL LEARN

- What pandas is and the two objects you will use all the time: Series and DataFrame
- Creating tables from dictionaries, lists and files
- Inspecting data: `head`, `info`, `describe`, `shape`, `dtypes`
- Selecting columns and rows with `[]`, `loc` and `iloc`; filtering with conditions
- Adding, changing, renaming and sorting

6.1 What is pandas?

pandas is the library for tables of data. If NumPy is a grid of numbers, pandas is a grid with **labels**: each column has a name and each row has an identifier. It feels like a spreadsheet you control with code, and it can handle millions of rows.

```
import numpy as np
import pandas as pd
```

NOTE

The pandas version in this book prints text columns with the type `str`. Older versions (including some Colab setups) print the same columns as `object`. They behave alike for everything in this book.

6.2 Series: one labelled column

A **Series** is a one-dimensional array with an **index** (labels for each value).

```
s = pd.Series([4.1, 5.3, 3.8], index=["Mon", "Tue", "Wed"], name="rain_mm")
print(s)
print(s["Tue"])          # select by label
print(s.iloc[0])         # select by position
print(s * 2)             # arithmetic works like NumPy
print(s.mean(), s.max(), s.idxmax()) # idxmax gives the LABEL of the maximum
```

OUTPUT

```
Mon    4.1
Tue    5.3
Wed    3.8
Name: rain_mm, dtype: float64
5.3
4.1
Mon     8.2
Tue    10.6
Wed     7.6
Name: rain_mm, dtype: float64
4.3999999999999995 5.3 Tue
```

6.3 DataFrame: the table

A **DataFrame** is a collection of Series sharing the same index: rows are observations (one experiment, one person, one day) and columns are variables (measurements). The simplest way to create one is from a dictionary of columns.

```
df = pd.DataFrame({
    "plant":  ["A", "B", "C", "D", "E", "F"],
    "light":  ["low", "low", "medium", "medium", "high", "high"],
    "height_cm": [12.1, 11.4, 18.3, 17.5, 25.2, 26.8],
    "leaves":  [5, 4, 8, 7, 11, 12],
})
df
```

OUTPUT

	plant	light	height_cm	leaves
0	A	low	12.1	5
1	B	low	11.4	4
2	C	medium	18.3	8
3	D	medium	17.5	7
4	E	high	25.2	11
5	F	high	26.8	12

The bold numbers on the left are the **index** and the top row holds the **column names**. Each column has a fixed **dtype**.

Looking at a DataFrame

Always start by looking. These functions are the first thing you type on any new dataset.

```
print(df.shape)          # (rows, columns)
print(df.columns.tolist())
print(df.dtypes)
df.head(3)               # first 3 rows (tail(3) gives the last 3)
```

OUTPUT

```
(6, 4)
['plant', 'light', 'height_cm', 'leaves']
plant      str
light      str
height_cm  float64
leaves     int64
dtype: object
  plant  light  height_cm  leaves
0     A    low      12.1      5
1     B    low      11.4      4
2     C  medium      18.3      8
```

```
df.info()                # columns, non-missing counts, dtypes, memory
```

OUTPUT

```
<class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   plant       6 non-null      str
1   light       6 non-null      str
2   height_cm   6 non-null      float64
3   leaves      6 non-null      int64
dtypes: float64(1), int64(1), str(2)
memory usage: 324.0 bytes
```

```
df.describe()            # summary statistics of numeric columns
```

OUTPUT

	height_cm	leaves
count	6.000000	6.000000

mean	18.550000	7.833333
std	6.421448	3.188521
min	11.400000	4.000000
25%	13.450000	5.500000
50%	17.900000	7.500000
75%	23.475000	10.250000
max	26.800000	12.000000

`describe` shows count, mean, standard deviation, minimum, the quartiles (25%, 50%, 75%) and the maximum. For text columns use `df.describe(include="object")` (or `include="all"`).

Other ways to create a DataFrame

```
# from a list of dictionaries (one per row)
rows = [{"x": 1, "y": 2.0}, {"x": 2, "y": 4.1}, {"x": 3, "y": 5.9}]
print(pd.DataFrame(rows))

# from a NumPy array, naming the columns yourself
arr = np.arange(6).reshape(3, 2)
print(pd.DataFrame(arr, columns=["a", "b"]))

# a date-indexed table
idx = pd.date_range("2024-01-01", periods=4, freq="D")
print(pd.DataFrame({"temp": [3.1, 4.0, 2.2, 5.5]}, index=idx))
```

OUTPUT

	x	y
0	1	2.0
1	2	4.1
2	3	5.9

	a	b
0	0	1
1	2	3
2	4	5

	temp
2024-01-01	3.1
2024-01-02	4.0
2024-01-03	2.2
2024-01-04	5.5

6.4 Selecting columns

```
print(df["height_cm"])          # one column -> a Series
df[["plant", "height_cm"]]     # list of columns -> a DataFrame (note the double brackets)
```

OUTPUT

0	12.1
1	11.4
2	18.3
3	17.5
4	25.2
5	26.8

Name: height_cm, dtype: float64

	plant	height_cm
0	A	12.1
1	B	11.4
2	C	18.3
3	D	17.5
4	E	25.2
5	F	26.8

A column without spaces can also be read as `df.height_cm`, but brackets are safer and required for assignment.

6.5 Selecting rows: loc and iloc

Two tools, and the difference is important:

Tool	Selects by	Slice end	Example
<code>df.loc[rows, cols]</code>	labels (index and column names)	included	<code>df.loc[0:2, "plant"]</code>
<code>df.iloc[rows, cols]</code>	integer positions	excluded	<code>df.iloc[0:2, 0]</code>

```
print(df.loc[2, "height_cm"])      # row label 2, column "height_cm"
print(df.loc[0:2, ["plant", "leaves"]]) # rows with labels 0,1,2 (end INCLUDED)
print(df.iloc[0:2, 0:2])          # first two rows and columns (end excluded)
print(df.iloc[-1])                # the last row
```

OUTPUT

```
18.3
  plant  leaves
0     A       5
1     B       4
2     C       8
  plant light
0     A   low
1     B   low
  plant      F
light      high
height_cm  26.8
leaves     12
Name: 5, dtype: object
```

6.6 Filtering rows with conditions

A condition on a column gives a True/False Series (a mask, exactly like NumPy). Put it inside `[]` to keep the matching rows.

```
tall = df[df["height_cm"] > 17]
print(tall)

# several conditions: & (and), | (or), ~ (not); brackets are required
print(df[(df["light"] == "high") & (df["leaves"] > 11)])
print(df[df["light"].isin(["low", "medium"])])
print(df.query("height_cm > 12 and leaves < 10")) # readable alternative
```

OUTPUT

```
  plant  light  height_cm  leaves
2     C  medium      18.3       8
3     D  medium      17.5       7
4     E   high      25.2      11
5     F   high      26.8      12
  plant  light  height_cm  leaves
5     F   high      26.8      12
  plant  light  height_cm  leaves
0     A   low      12.1       5
1     B   low      11.4       4
2     C  medium      18.3       8
3     D  medium      17.5       7
  plant  light  height_cm  leaves
0     A   low      12.1       5
2     C  medium      18.3       8
3     D  medium      17.5       7
```

6.7 Creating and changing columns

```
df["height_m"] = df["height_cm"] / 100          # new column from old ones
df["leaf_density"] = df["leaves"] / df["height_cm"]
df["is_tall"] = df["height_cm"] > 20
df.loc[df["plant"] == "A", "leaves"] = 6       # change one value
df["light"] = df["light"].str.upper()          # text functions via .str
df.head()
```

OUTPUT

	plant	light	height_cm	leaves	height_m	leaf_density	is_tall
0	A	LOW	12.1	6	0.121	0.413223	False
1	B	LOW	11.4	4	0.114	0.350877	False
2	C	MEDIUM	18.3	8	0.183	0.437158	False
3	D	MEDIUM	17.5	7	0.175	0.400000	False
4	E	HIGH	25.2	11	0.252	0.436508	True

WATCH OUT

Do not write `df[df["plant"] == "A"]["leaves"] = 6`. This "chained" selection may modify a temporary copy and leave `df` unchanged. Use a single `df.loc[row_condition, "column"] = value`.

Renaming, dropping and reordering

```
df = df.rename(columns={"height_cm": "height"})
df = df.drop(columns=["height_m", "is_tall"])      # remove columns
df = df[["plant", "light", "leaves", "height", "leaf_density"]] # reorder
print(df.columns.tolist())
```

OUTPUT

```
['plant', 'light', 'leaves', 'height', 'leaf_density']
```

Most pandas methods return a **new** DataFrame instead of changing the old one, so assign the result back (`df = df.drop(...)`).

Sorting

```
print(df.sort_values("height", ascending=False).head(3))
print(df.sort_values(["light", "height"]))          # sort by two columns
print(df.nlargest(2, "leaves"))                    # the two rows with most leaves
```

OUTPUT

	plant	light	leaves	height	leaf_density
5	F	HIGH	12	26.8	0.447761
4	E	HIGH	11	25.2	0.436508
2	C	MEDIUM	8	18.3	0.437158

	plant	light	leaves	height	leaf_density
4	E	HIGH	11	25.2	0.436508
5	F	HIGH	12	26.8	0.447761
1	B	LOW	4	11.4	0.350877
0	A	LOW	6	12.1	0.413223
3	D	MEDIUM	7	17.5	0.400000
2	C	MEDIUM	8	18.3	0.437158

	plant	light	leaves	height	leaf_density
5	F	HIGH	12	26.8	0.447761
4	E	HIGH	11	25.2	0.436508

6.8 The index

The index is the row label. The default is 0, 1, 2, ..., but you can choose a meaningful column.

```
d2 = df.set_index("plant")
print(d2.loc["C"])
print(d2.reset_index().head(2))      # turn the index back into a column
```

```

OUTPUT
light          MEDIUM
leaves          8
height         18.3
leaf_density    0.437158
Name: C, dtype: object
  plant light  leaves  height  leaf_density
0     A   LOW      6    12.1      0.413223
1     B   LOW      4    11.4      0.350877

```

6.9 Quick summaries of columns

```

print(df["light"].value_counts())          # counts of each category
print(df["light"].nunique(), df["light"].unique())
print(df["height"].mean(), df["height"].median(), df["height"].std())
print(df[["height", "leaves"]].corr())    # correlation matrix

```

```

OUTPUT
light
LOW      2
MEDIUM  2
HIGH     2
Name: count, dtype: int64
3 <StringArray>
['LOW', 'MEDIUM', 'HIGH']
Length: 3, dtype: str
18.55 17.9 6.421448434738069
      height  leaves
height 1.000000 0.980633
leaves 0.980633 1.000000

```

6.10 Applying your own function

`apply` runs a function on each value; `map` replaces values using a dictionary.

```

def size_class(h):
    if h < 15:
        return "small"
    elif h < 22:
        return "medium"
    return "large"

df["size"] = df["height"].apply(size_class)
df["light_code"] = df["light"].map({"LOW": 0, "MEDIUM": 1, "HIGH": 2})
df[["plant", "height", "size", "light_code"]]

```

```

OUTPUT
  plant  height  size  light_code
0     A    12.1  small           0
1     B    11.4  small           0
2     C    18.3  medium          1
3     D    17.5  medium          1
4     E    25.2  large           2
5     F    26.8  large           2

```

For pure arithmetic prefer vectorised operations (`df["a"] * 2`) over `apply`; they are much faster.

6.11 Chapter summary

CHAPTER SUMMARY

- A Series is one labelled column; a DataFrame is a table of columns sharing an index.
- Inspect with `shape`, `dtypes`, `head`, `info`, `describe`, `value_counts`.

- `df["col"]` gets a column; `loc` uses labels (end included), `iloc` uses positions (end excluded).
- Filter with masks: `df[(cond1) & (cond2)]`; change values with `df.loc[mask, "col"] = value`.
- Methods return new objects; assign the result back. `rename`, `drop`, `sort_values`, `set_index` are the essentials.

6.12 Practice exercises

1. Create a DataFrame of five planets with columns `name`, `diameter_km` and `moons` and print the planet with the most moons.
2. Add a column `diameter_ratio` giving each diameter divided by the smallest diameter.
3. Filter for planets with diameter above the average diameter.
4. Sort by `moons`, descending, and show the top three names only.
5. Explain the difference between `df.loc[0:2]` and `df.iloc[0:2]` and demonstrate it.

CHAPTER 7

7. pandas Part 2: Cleaning and Preparing Data

IN THIS CHAPTER YOU WILL LEARN

- Loading CSV files and checking their quality
- Finding and handling missing values
- Duplicates, wrong data types, inconsistent text
- Dates and times
- Detecting and treating outliers

Real data is **dirty**: blank cells, typos, numbers stored as text, duplicate rows, impossible values. Cleaning is usually most of the work, and **how** you clean can change your scientific conclusions, so document every decision.

7.1 A messy dataset to practise on

We will create a small messy file, save it, and read it back, the same way you would load your own file.

```
import numpy as np
import pandas as pd

csv_text = """id,date,site,temp_c,ph,notes
1,2024-03-01,North,18.2,7.1,ok
2,2024-03-01,north ,19.0,7.3,ok
3,2024-03-02,South,,7.0,sensor down
4,2024-03-02,South,21.5,6.9,ok
4,2024-03-02,South,21.5,6.9,ok
5,2024/03/03,East,17.8,,ok
6,2024-03-03,East,999,7.2,error code
7,2024-03-04,North,20.1,7.4,ok
8,2024-03-04,West,19.6,7.0,
"""

with open("water.csv", "w") as f:
    f.write(csv_text)

df = pd.read_csv("water.csv")
df
```

OUTPUT

	id	date	site	temp_c	ph	notes
0	1	2024-03-01	North	18.2	7.1	ok
1	2	2024-03-01	north	19.0	7.3	ok
2	3	2024-03-02	South	NaN	7.0	sensor down
3	4	2024-03-02	South	21.5	6.9	ok
4	4	2024-03-02	South	21.5	6.9	ok
5	5	2024/03/03	East	17.8	NaN	ok
6	6	2024-03-03	East	999.0	7.2	error code
7	7	2024-03-04	North	20.1	7.4	ok
8	8	2024-03-04	West	19.6	7.0	NaN

Inspect before changing anything:

```
df.info()
print(df.isna().sum())          # number of missing values per column
```

OUTPUT

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column  Non-Null Count  Dtype
---  -
0    id      9 non-null        int64
1    date     9 non-null        str
2    site     9 non-null        str
3    temp_c   8 non-null        float64
4    ph        8 non-null        float64
5    notes    8 non-null        str
dtypes: float64(2), int64(1), str(3)
memory usage: 564.0 bytes
id      0
date    0
site    0
temp_c  1
ph      1
notes   1
dtype: int64
```

Problems we can already see: a duplicate row (id 4), an impossible temperature (999), inconsistent site names ("North", "north "), dates in two formats, and missing values in `temp_c`, `ph` and `notes`.

7.2 Missing values

Missing values appear as `NaN` (not a number) or `None`. First find them:

```
print(df.isna())          # True where missing (a mask)
print(df.isna().sum())    # count per column
print(df.isna().mean().round(2)) # fraction missing per column
print(df[df["temp_c"].isna()]) # the rows that have a missing temperature
```

OUTPUT

```
id      date      site  temp_c    ph  notes
0  False  False  False   False  False  False
1  False  False  False   False  False  False
2  False  False  False    True  False  False
3  False  False  False   False  False  False
4  False  False  False   False  False  False
5  False  False  False   False   True  False
6  False  False  False   False  False  False
7  False  False  False   False  False  False
8  False  False  False   False  False   True
id      0
date    0
site    0
temp_c  1
ph      1
notes   1
dtype: int64
id      0.00
date    0.00
site    0.00
temp_c  0.11
ph      0.11
notes   0.11
dtype: float64
id      date      site  temp_c    ph  notes
2    3  2024-03-02  South    NaN  7.0  sensor down
```

Why values go missing matters

Type	Meaning	Example
MCAR: missing completely at random	no pattern at all	a file row lost by accident
MAR: missing at random	depends on other <i>measured</i> variables	older sensors fail more often
MNAR: missing not at random	depends on the missing value itself	a thermometer that fails above 40 degrees

If values are missing at random and few in number, simple fixes are fine. If they are missing *because* of the value, simple fixes introduce bias, and you should say so in your report.

Options for handling missing values

1. **Drop rows** with missing values (`dropna`). Simple, but loses data.
2. **Fill with a constant** (`fillna(0)`), only if zero truly is the right value.
3. **Fill with a typical value**: the mean or median of the column (**imputation**).
4. **Fill using neighbours**: forward-fill or interpolate, for ordered data such as time series.
5. **Model-based imputation**: predict missing values from other columns (Chapter 17).

```
clean = df.copy()
print(len(clean), "rows before")
print(len(clean.dropna()), "rows after dropna (drops any row with a NaN)")
print(len(clean.dropna(subset=["temp_c"])), "rows after dropping only missing temp")

clean["ph"] = clean["ph"].fillna(clean["ph"].median())
clean["notes"] = clean["notes"].fillna("none")
clean
```

```
OUTPUT
9 rows before
6 rows after dropna (drops any row with a NaN)
8 rows after dropping only missing temp
  id  date    site  temp_c  ph      notes
0  1  2024-03-01  North    18.2  7.10      ok
1  2  2024-03-01  north    19.0  7.30      ok
2  3  2024-03-02  South     NaN  7.00  sensor down
3  4  2024-03-02  South    21.5  6.90      ok
4  4  2024-03-02  South    21.5  6.90      ok
5  5  2024/03/03   East    17.8  7.05      ok
6  6  2024-03-03   East   999.0  7.20  error code
7  7  2024-03-04  North    20.1  7.40      ok
8  8  2024-03-04  West    19.6  7.00      none
```

TIP

Median imputation is usually safer than the mean when data has outliers. Keep a record of which values were imputed, for example by adding a boolean column `temp_imputed`, so you can test later how much your conclusions depend on the fill.

7.3 Duplicates

```
print(clean.duplicated().sum())          # how many exact duplicate rows
print(clean[clean.duplicated(keep=False)]) # show all copies
clean = clean.drop_duplicates()
print(len(clean))
```

```
OUTPUT
1
  id  date    site  temp_c  ph notes
```

```

3  4  2024-03-02  South    21.5  6.9    ok
4  4  2024-03-02  South    21.5  6.9    ok
8

```

Use `drop_duplicates(subset=["id"])` to treat rows as duplicates when only some columns match.

7.4 Fixing data types

Numbers stored as text prevent calculations. Convert with `pd.to_numeric`; `errors="coerce"` turns unparseable entries into `NaN` instead of crashing.

```

messy = pd.Series(["10", "20.5", "n/a", "30"])
print(pd.to_numeric(messy, errors="coerce"))

clean["id"] = clean["id"].astype(int)
clean["site"] = clean["site"].astype("category") # efficient type for repeated labels
print(clean.dtypes)

```

```

OUTPUT
0    10.0
1    20.5
2     NaN
3    30.0
dtype: float64
id          int64
date        str
site        category
temp_c      float64
ph          float64
notes       str
dtype: object

```

7.5 Cleaning text

The `.str` accessor applies string methods to a whole column.

```

clean["site"] = clean["site"].astype(str).str.strip().str.title()
print(clean["site"].value_counts())

s = pd.Series([" Red ", "red", "RED", "Blue "])
print(s.str.strip().str.lower().unique())
print(pd.Series(["a-1", "b-22"]).str.split("-", expand=True))
print(pd.Series(["temp=21C", "temp=19C"]).str.extract(r"(\d+)")) # regular expression
print(pd.Series(["apple", "banana"]).str.contains("an"))

```

```

OUTPUT
site
North    3
South    2
East     2
West     1
Name: count, dtype: int64
<StringArray>
['red', 'blue']
Length: 2, dtype: str
0    1
0  a    1
1  b  22
0
0  21
1  19
0  False
1   True
dtype: bool

```

7.6 Dates and times

`pd.to_datetime` converts text into real date objects, which unlock date arithmetic and the `.dt` accessor.

```
clean["date"] = pd.to_datetime(clean["date"].str.replace("/", "-"))
print(clean["date"].dtype)
clean["month"] = clean["date"].dt.month
clean["weekday"] = clean["date"].dt.day_name()
print(clean[["date", "month", "weekday"]].head(3))

d1 = pd.Timestamp("2024-03-10")
d2 = pd.Timestamp("2024-03-01")
print(d1 - d2)           # a Timedelta: 9 days
print((d1 - d2).days)
print(pd.date_range("2024-01-01", "2024-01-10", freq="3D"))
```

OUTPUT

```
datetime64[us]
      date  month  weekday
0 2024-03-01      3   Friday
1 2024-03-01      3   Friday
2 2024-03-02      3  Saturday
9 days 00:00:00
9
DatetimeIndex(['2024-01-01', '2024-01-04', '2024-01-07', '2024-01-10'], dtype='datetime64[us]', fr
eq='3D')
```

Format code	Meaning	Example
<code>%Y</code>	4-digit year	2024
<code>%m / %d</code>	month / day number	03 / 07
<code>%H:%M:%S</code>	hour, minute, second	14:05:30
<code>%b / %B</code>	month name short / long	Mar / March

Use `pd.to_datetime(col, format="%d/%m/%Y")` when dates are not in a standard order, to avoid day-month confusion.

7.7 Outliers: unusual values

An **outlier** is a value far from the rest. It might be an *error* (a sensor code 999 meaning "failed") or a *real extreme* (a true heatwave). Deleting real extremes destroys scientific information, so investigate before you remove.

Here the 999 is obviously a sentinel code: no water is 999 degrees. We convert it to missing.

```
clean["temp_c"] = clean["temp_c"].replace(999, np.nan)
clean["temp_c"] = clean["temp_c"].fillna(clean["temp_c"].median())
print(clean[["id", "temp_c"]])
```

OUTPUT

```
id  temp_c
0   1   18.2
1   2   19.0
2   3   19.3
3   4   21.5
5   5   17.8
6   6   19.3
7   7   20.1
8   8   19.6
```

Detecting outliers statistically

Two standard rules:

1. The IQR rule (robust). Let Q1 and Q3 be the 25th and 75th percentiles and $IQR = Q3 - Q1$ (the *interquartile range*). Flag values outside

$$[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$$

2. The z-score rule. The z-score counts how many standard deviations a value is from the mean: $z = (x - \text{mean}) / \text{std}$. A common rule flags $|z| > 3$.

```
rng = np.random.default_rng(3)
values = pd.Series(np.append(rng.normal(50, 5, 200), [95, 2]))

q1, q3 = values.quantile([0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(f"IQR fences: {low:.1f} to {high:.1f}")
print("IQR outliers:", values[(values < low) | (values > high)].round(1).tolist())

z = (values - values.mean()) / values.std()
print("z-score outliers:", values[z.abs() > 3].round(1).tolist())
```

OUTPUT

```
IQR fences: 36.8 to 63.9
IQR outliers: [66.6, 35.9, 95.0, 2.0]
z-score outliers: [95.0, 2.0]
```

The IQR rule is more reliable because one huge outlier inflates the mean and standard deviation, which can hide it from the z-score test (masking).

Treating outliers

- **Keep** them if real and relevant (and use robust statistics such as the median).
- **Remove** them if proven errors.
- **Cap (winsorise)** them at the fences: `values.clip(low, high)`.
- **Transform** the data (for example a logarithm) to reduce their influence (Chapter 17).

```
capped = values.clip(lower=low, upper=high)
print(values.max(), "->", capped.max().round(1))
```

OUTPUT

```
95.0 -> 63.9
```

7.8 Validating with simple rules

A good habit is to **assert** what must be true. If a rule fails the notebook stops, so problems never slip by silently.

```
assert clean["ph"].between(0, 14).all(), "pH outside 0-14!"
assert clean["id"].is_unique, "duplicate ids!"
print("all checks passed")
clean
```

OUTPUT

```
all checks passed
```

	id	date	site	temp_c	ph	notes	month	weekday
0	1	2024-03-01	North	18.2	7.10	ok	3	Friday
1	2	2024-03-01	North	19.0	7.30	ok	3	Friday
2	3	2024-03-02	South	19.3	7.00	sensor down	3	Saturday
3	4	2024-03-02	South	21.5	6.90	ok	3	Saturday

5	5	2024-03-03	East	17.8	7.05		ok	3	Sunday
6	6	2024-03-03	East	19.3	7.20	error	code	3	Sunday
7	7	2024-03-04	North	20.1	7.40		ok	3	Monday
8	8	2024-03-04	West	19.6	7.00		none	3	Monday

7.9 A reusable cleaning function

Once you have cleaned a file by hand, wrap the steps in a function so the same clean-up can be applied to new data and is documented in code.

```
def clean_water(path):
    d = pd.read_csv(path)
    d = d.drop_duplicates()
    d["site"] = d["site"].str.strip().str.title()
    d["date"] = pd.to_datetime(d["date"].str.replace("/", "-"))
    d["temp_c"] = d["temp_c"].replace(999, np.nan)
    d["temp_c"] = d["temp_c"].fillna(d["temp_c"].median())
    d["ph"] = d["ph"].fillna(d["ph"].median())
    d["notes"] = d["notes"].fillna("none")
    return d

final = clean_water("water.csv")
final
```

OUTPUT

	id	date	site	temp_c	ph	notes
0	1	2024-03-01	North	18.2	7.1	ok
1	2	2024-03-01	North	19.0	7.3	ok
2	3	2024-03-02	South	19.3	7.0	sensor down
3	4	2024-03-02	South	21.5	6.9	ok
5	5	2024-03-03	East	17.8	7.1	ok
6	6	2024-03-03	East	19.3	7.2	error code
7	7	2024-03-04	North	20.1	7.4	ok
8	8	2024-03-04	West	19.6	7.0	none

7.10 Chapter summary

CHAPTER SUMMARY

- Always inspect first: `info`, `isna().sum()`, `describe`, `value_counts`.
- Missing values: understand *why*; then drop, fill (median is robust) or model them.
- Remove duplicates with `drop_duplicates`; fix types with `astype`, `to_numeric`, `to_datetime`.
- Clean text with `.str` (strip, lower, replace, extract); dates with `.dt`.
- Outliers: IQR rule (robust) or z-score; investigate before deleting; consider capping or transforming.
- Put cleaning steps in a function and add `assert` checks.

7.11 Practice exercises

1. In the water data, find the average temperature per site after cleaning.
2. Create a column `temp_f` (Fahrenheit) and verify with `assert` that no value is below the freezing point of water (32).
3. Generate 100 numbers from a normal distribution, add one extreme value, and compare the mean and median before and after.
4. Convert the strings `["2024-01-05", "05/02/2024", "March 3, 2024"]` to dates. What problem arises with mixed formats?

5. Write a function that reports, for any DataFrame, the percentage of missing values per column, sorted from highest to lowest.

CHAPTER 8

8. pandas Part 3: Grouping, Combining and Reshaping

IN THIS CHAPTER YOU WILL LEARN

- Split-apply-combine with **groupby** and aggregation
- Pivot tables and crosstabs
- Combining tables with **concat** and **merge** (joins)
- Wide versus long format: **melt** and **pivot**
- Time-series basics: resampling and rolling windows
- Method chaining and performance tips

8.1 A dataset for this chapter

We will simulate an experiment: three fertiliser treatments tested on plants in two greenhouses over several weeks. Simulated data is a great way to learn, because you know the truth you are trying to recover.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(10)
n = 60
df = pd.DataFrame({
    "plant_id": np.arange(n),
    "treatment": rng.choice(["control", "fert_A", "fert_B"], size=n),
    "greenhouse": rng.choice(["G1", "G2"], size=n),
    "week": rng.integers(1, 5, size=n),
})
base = {"control": 10, "fert_A": 13, "fert_B": 15}
df["height"] = df["treatment"].map(base) + 1.5 * df["week"] + rng.normal(0, 1.5, n)
df["height"] = df["height"].round(1)
df.head()
```

OUTPUT

	plant_id	treatment	greenhouse	week	height
0	0	fert_B	G1	1	14.9
1	1	fert_B	G1	2	18.9
2	2	control	G1	3	13.6
3	3	control	G1	4	16.0
4	4	fert_B	G2	4	22.2

8.2 groupby: split, apply, combine

groupby performs three steps: **split** the table into groups by the values of one or more columns, **apply** a calculation to each group, and **combine** the results into a new table. It answers "what is the average X for each Y?".

```
print(df.groupby("treatment")["height"].mean())
```

OUTPUT

treatment	height
control	13.252632
fert_A	16.960000

```
fert_B      18.192308
Name: height, dtype: float64
```

You can group by several columns and compute several statistics at once with **agg**:

```
summary = df.groupby("treatment")["height"].agg(["count", "mean", "std", "min", "max"])
print(summary.round(2))
```

```
by_two = df.groupby(["treatment", "greenhouse"])["height"].mean().round(2)
print(by_two)
```

OUTPUT

	count	mean	std	min	max
treatment					
control	19	13.25	1.77	9.7	16.0
fert_A	15	16.96	1.89	14.2	19.6
fert_B	26	18.19	2.56	13.9	22.8
treatment greenhouse					
control G1			12.54		
G2			13.89		
fert_A G1			16.94		
G2			16.98		
fert_B G1			18.59		
G2			17.55		

Name: height, dtype: float64

Named aggregation

Named aggregation gives each result a clear column name and lets you use different columns.

```
result = df.groupby("treatment").agg(
    n_plants=("plant_id", "count"),
    mean_height=("height", "mean"),
    sd_height=("height", "std"),
    max_week=("week", "max"),
).round(2)
result
```

OUTPUT

	n_plants	mean_height	sd_height	max_week
treatment				
control	19	13.25	1.77	4
fert_A	15	16.96	1.89	4
fert_B	26	18.19	2.56	4

transform and filter

- **transform** returns a result with the **same length** as the original, handy for adding group statistics to each row.
- **filter** keeps or drops whole groups by a condition.

```
df["group_mean"] = df.groupby("treatment")["height"].transform("mean")
df["diff_from_group"] = (df["height"] - df["group_mean"]).round(2)
print(df[["treatment", "height", "group_mean", "diff_from_group"]].head(4))
```

```
big_groups = df.groupby("greenhouse").filter(lambda g: len(g) > 25)
print(big_groups["greenhouse"].value_counts())
```

OUTPUT

	treatment	height	group_mean	diff_from_group
0	fert_B	14.9	18.192308	-3.29
1	fert_B	18.9	18.192308	0.71
2	control	13.6	13.252632	0.35
3	control	16.0	13.252632	2.75
greenhouse				
	G1	34		
	G2	26		

Name: count, dtype: int64

NOTE

`groupby` turns the group keys into the **index** of the result. Add `.reset_index()` (or `as_index=False`) to get them back as normal columns.

8.3 Pivot tables and crosstabs

A **pivot table** spreads one variable across columns, like an Excel pivot.

```
pt = df.pivot_table(index="treatment", columns="greenhouse",
                    values="height", aggfunc="mean").round(1)
print(pt)

# counts of combinations
print(pd.crosstab(df["treatment"], df["greenhouse"]))
print(pd.crosstab(df["treatment"], df["greenhouse"], normalize="index").round(2))
```

OUTPUT

greenhouse	G1	G2
treatment		
control	12.5	13.9
fert_A	16.9	17.0
fert_B	18.6	17.6

greenhouse	G1	G2
treatment		
control	9	10
fert_A	9	6
fert_B	16	10

greenhouse	G1	G2
treatment		
control	0.47	0.53
fert_A	0.60	0.40
fert_B	0.62	0.38

8.4 Combining tables

Stacking: concat

`pd.concat` glues tables together, either below each other (`axis=0`) or side by side (`axis=1`).

```
jan = pd.DataFrame({"id": [1, 2], "value": [10, 12]})
feb = pd.DataFrame({"id": [3, 4], "value": [9, 14]})
print(pd.concat([jan, feb], ignore_index=True))
```

OUTPUT

	id	value
0	1	10
1	2	12
2	3	9
3	4	14

Joining on a key: merge

`merge` matches rows of two tables by a shared column, exactly like a database **join**. Suppose we have a second table describing each treatment.

```
info = pd.DataFrame({
    "treatment": ["control", "fert_A", "fert_B", "fert_C"],
    "cost_per_plant": [0.0, 0.40, 0.65, 0.90],
})
left = df[["plant_id", "treatment", "height"]].head(6)
```

```
print(left.merge(info, on="treatment", how="left"))
```

OUTPUT

```
  plant_id treatment  height  cost_per_plant
0         0   fert_B    14.9             0.65
1         1   fert_B    18.9             0.65
2         2  control    13.6             0.00
3         3  control    16.0             0.00
4         4   fert_B    22.2             0.65
5         5   fert_B    16.7             0.65
```

The **how** argument chooses what to keep:

how	Keeps	Use when
inner	only keys found in both tables	you need complete matches
left	all rows of the left table (missing filled with NaN)	the left table is your main dataset
right	all rows of the right table	rarely; swap the tables instead
outer	all rows of both	you want to see everything that does or does not match

```
left_small = pd.DataFrame({"key": ["a", "b", "c"], "x": [1, 2, 3]})
right_small = pd.DataFrame({"key": ["b", "c", "d"], "y": [20, 30, 40]})
for how in ["inner", "left", "outer"]:
    print(how)
    print(left_small.merge(right_small, on="key", how=how))
```

OUTPUT

```
inner
  key  x  y
0   b  2  20
1   c  3  30
left
  key  x    y
0   a  1  NaN
1   b  2  20.0
2   c  3  30.0
outer
  key  x    y
0   a  1.0  NaN
1   b  2.0  20.0
2   c  3.0  30.0
3   d  NaN  40.0
```

WATCH OUT

Before merging, check that the key is **unique** in the table you expect it to be unique in. Duplicate keys multiply rows. Use `validate="many_to_one"` in `merge` to get an error instead of a silent surprise.

8.5 Wide versus long format

- **Wide** format: one row per subject, one column per measurement (good for humans and spreadsheets).
- **Long** ("tidy") format: one row per **measurement**, with a column for the variable name and a column for the value. Most plotting and statistics tools prefer long format.

`melt` goes wide to long; `pivot` goes long to wide.

```
wide = pd.DataFrame({
    "subject": ["s1", "s2"],
    "week1": [5.1, 4.8],
    "week2": [6.0, 5.5],
    "week3": [7.2, 6.1],
```

```

})
long = wide.melt(id_vars="subject", var_name="week", value_name="height")
print(long)
print(long.pivot(index="subject", columns="week", values="height"))

```

OUTPUT

```

  subject  week  height
0      s1 week1    5.1
1      s2 week1    4.8
2      s1 week2    6.0
3      s2 week2    5.5
4      s1 week3    7.2
5      s2 week3    6.1
week      week1  week2  week3
subject
s1         5.1    6.0    7.2
s2         4.8    5.5    6.1

```

Tidy data has three rules: every variable is a column, every observation is a row, every value is a cell. Reshape your data to that form first and the rest of the analysis becomes easy.

8.6 Working with time series

With a date index, pandas can **resample** (change the time step) and compute **rolling windows**.

```

days = pd.date_range("2024-01-01", periods=60, freq="D")
rng = np.random.default_rng(5)
temp = pd.Series(10 + 5 * np.sin(np.arange(60) / 9) + rng.normal(0, 1, 60), index=days)

print(temp.resample("W").mean().round(2).head())          # weekly averages
print(temp.resample("MS").agg(["mean", "min", "max"]).round(2))

```

OUTPUT

```

2024-01-07    11.42
2024-01-14    14.55
2024-01-21    13.97
2024-01-28    12.12
2024-02-04     8.00
Freq: W-SUN, dtype: float64
      mean  min  max
2024-01-01  12.61  7.76  16.56
2024-02-01   6.85  3.01  13.23

```

A **rolling window** moves along the data computing a statistic on the last k values, which smooths noise.

```

smooth = temp.rolling(window=7).mean()          # 7-day moving average
print(smooth.iloc[5:9].round(2))                # first 6 values are NaN: not enough history
print(temp.diff().head(3).round(2))             # change from the previous day
print(temp.shift(1).head(3).round(2))           # the previous day's value
print(temp.pct_change().head(3).round(3))       # percent change
print(temp.expanding().mean().tail(2).round(2)) # cumulative mean so far

```

OUTPUT

```

2024-01-06    NaN
2024-01-07    11.42
2024-01-08    11.92
2024-01-09    12.69
Freq: D, dtype: float64
2024-01-01    NaN
2024-01-02    0.03
2024-01-03    1.62
Freq: D, dtype: float64
2024-01-01    NaN
2024-01-02    9.20
2024-01-03    9.23
Freq: D, dtype: float64
2024-01-01    NaN
2024-01-02    0.003

```

```
2024-01-03    0.176
Freq: D, dtype: float64
2024-02-28    9.79
2024-02-29    9.82
Freq: D, dtype: float64
```

Function	Meaning
<code>shift(k)</code>	move values down by k (lag)
<code>diff(k)</code>	value minus value k steps earlier
<code>rolling(w).mean()</code>	moving average over w points
<code>expanding().mean()</code>	average of everything so far
<code>resample("W")</code>	group by week (also D , MS , QS , YS)

8.7 Categorical data

For columns with few repeated labels, the `category` type saves memory and allows a custom order.

```
levels = pd.Categorical(["low", "high", "medium", "low"],
                        categories=["low", "medium", "high"], ordered=True)
s = pd.Series(levels)
print(s.sort_values().tolist())
print(s > "low")
```

```
OUTPUT
['low', 'low', 'medium', 'high']
0    False
1     True
2     True
3    False
dtype: bool
```

Binning numbers into groups

```
ages = pd.Series([3, 17, 25, 41, 67, 88])
print(pd.cut(ages, bins=[0, 18, 65, 120], labels=["child", "adult", "senior"]))
q = pd.qcut(pd.Series(range(1, 13)), q=4, labels=["Q1", "Q2", "Q3", "Q4"])
print(q.value_counts().sort_index())
```

```
OUTPUT
0    child
1    child
2    adult
3    adult
4    senior
5    senior
dtype: category
Categories (3, str): ['child' < 'adult' < 'senior']
Q1    3
Q2    3
Q3    3
Q4    3
Name: count, dtype: int64
```

`cut` makes bins of chosen edges; `qcut` makes bins with (roughly) equal numbers of observations.

8.8 Method chaining

Instead of creating many temporary variables, you can chain operations. Each step returns a new DataFrame to the next. Wrap the chain in brackets and put one step per line.

```
report = (
```

```

df
    .query("week >= 2")
    .assign(height_m=lambda d: d["height"] / 100)
    .groupby(["treatment", "greenhouse"], as_index=False)
    .agg(mean_height_m=("height_m", "mean"), n=("plant_id", "count"))
    .round(3)
    .sort_values("mean_height_m", ascending=False)
)
report

```

OUTPUT

	treatment	greenhouse	mean_height_m	n
5	fert_B	G2	0.195	5
4	fert_B	G1	0.191	13
2	fert_A	G1	0.182	6
3	fert_A	G2	0.170	6
1	control	G2	0.142	9
0	control	G1	0.132	5

Chains read like a recipe and are easy to audit.

8.9 Performance tips

- Use vectorised operations and built-in methods, not Python loops or `apply`, for arithmetic.
- Use `category` for repeated text, and downcast numbers when memory is tight.
- Read only the columns you need: `pd.read_csv(path, usecols=[...])`.
- For files too big for memory, read in pieces: `pd.read_csv(path, chunksize=100000)`.
- Check memory with `df.memory_usage(deep=True).sum()`.

```

big = pd.DataFrame({"g": np.random.default_rng(1).choice(["a", "b", "c"], 100000)})
before = big.memory_usage(deep=True).sum()
big["g"] = big["g"].astype("category")
after = big.memory_usage(deep=True).sum()
print(f"memory: {before/1e6:.2f} MB -> {after/1e6:.2f} MB")

```

OUTPUT

memory: 5.00 MB -> 0.10 MB

8.10 Chapter summary

CHAPTER SUMMARY

- `groupby(...).agg(...)` answers "statistic X for each group Y"; `transform` adds group results back to every row.
- `pivot_table` and `crosstab` summarise two variables against each other.
- `concat` stacks tables; `merge` joins on keys (`inner`, `left`, `outer`); check keys are unique.
- Tidy (long) data is easiest to analyse; `melt` and `pivot` convert.
- Dates: `resample`, `rolling`, `shift`, `diff`. Bins: `cut` and `qcut`.
- Method chains make pipelines readable.

8.11 Practice exercises

1. From the plant dataset, compute the median height per week.
2. Build a pivot table with `week` as rows and `treatment` as columns of mean height.
3. Merge `df` with the `info` table and compute the total treatment cost per greenhouse.

4. Convert **wide** to long, then compute each subject's average height with **groupby**.
5. On the temperature series, compute a 5-day rolling standard deviation and find the day with the largest value.

CHAPTER 9

9. Data Visualisation: Matplotlib and Seaborn

IN THIS CHAPTER YOU WILL LEARN

- The anatomy of a Matplotlib figure: Figure, Axes, artists
- Line, scatter, bar, histogram, box, violin, heatmap and error-bar plots
- Subplots, labels, legends, colours, styles, saving figures
- Seaborn for statistical plots; pandas plotting shortcuts
- Choosing the right chart and designing honest figures

A plot is often the fastest way to see a pattern, catch an error or convince a reader. Anscombe's famous four datasets have identical means, variances and correlations, but look completely different when plotted. **Always plot your data.**

9.1 Matplotlib: the anatomy of a figure

Matplotlib builds a plot from objects:

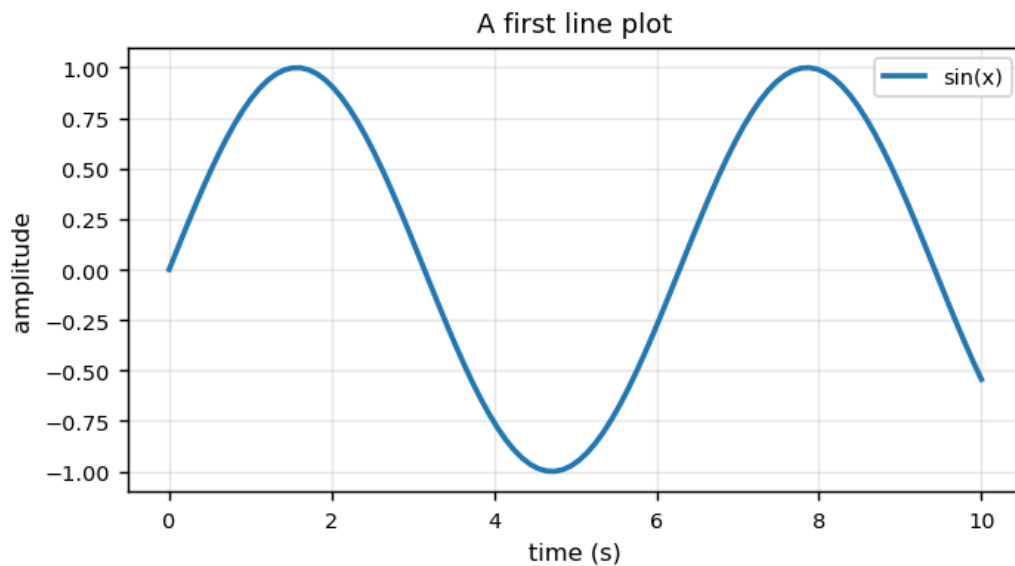
- **Figure:** the whole canvas (the window or image).
- **Axes:** one plot area with its x and y axis (a figure may hold many).
- **Axis, ticks, labels, title, legend, lines:** the parts drawn on the Axes.

The recommended style is the **object-oriented** one: create a figure and axes with `plt.subplots()`, then call methods on the axes.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
y = np.sin(x)

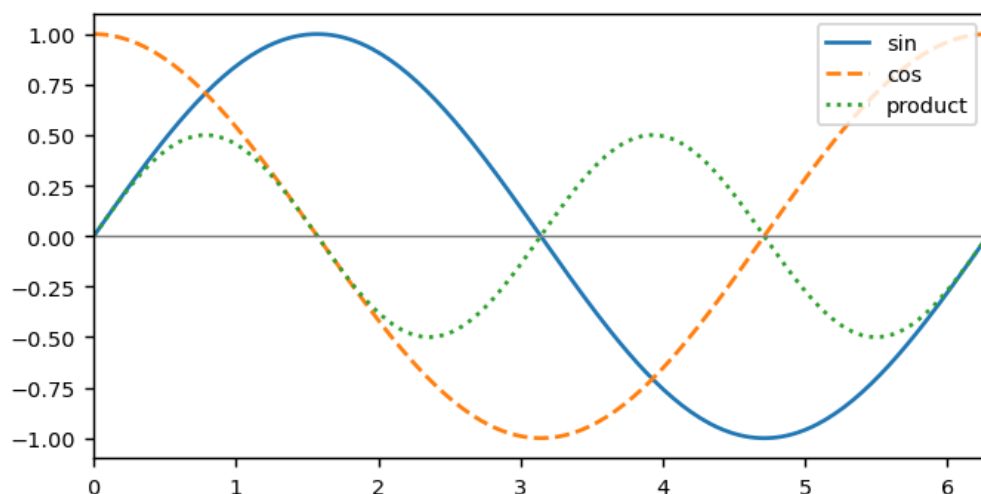
fig, ax = plt.subplots(figsize=(6, 3))    # figsize in inches: (width, height)
ax.plot(x, y, label="sin(x)", color="tab:blue", linewidth=2)
ax.set_xlabel("time (s)")
ax.set_ylabel("amplitude")
ax.set_title("A first line plot")
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```

**TIP**

In Colab, figures display automatically after the cell runs. Always label axes **with units**, add a title or caption, and use a legend only when there is more than one thing to tell apart.

9.2 Line plots with several series

```
t = np.linspace(0, 2 * np.pi, 200)
fig, ax = plt.subplots(figsize=(6, 3))
ax.plot(t, np.sin(t), "-", label="sin")           # solid line
ax.plot(t, np.cos(t), "--", label="cos")          # dashed
ax.plot(t, np.sin(t) * np.cos(t), ":", label="product") # dotted
ax.axhline(0, color="grey", linewidth=0.8)       # horizontal reference line
ax.set_xlim(0, 2 * np.pi)
ax.legend(loc="upper right")
plt.show()
```

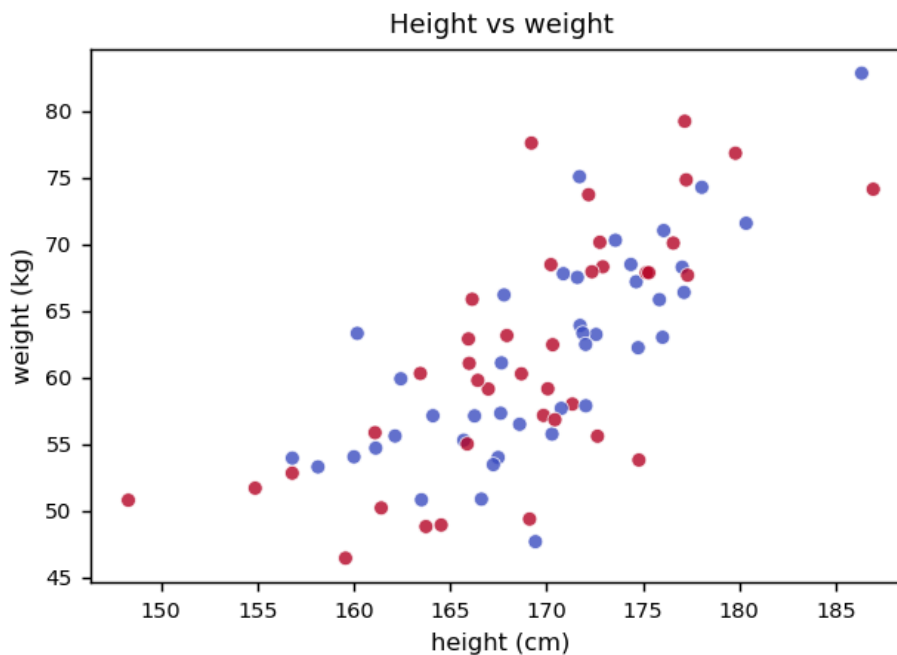


Line style codes: "-" solid, "--" dashed, ":" dotted, "-." dash-dot. Marker codes: "o" circle, "s" square, "^" triangle, "x" cross.

9.3 Scatter plots: relationships between two variables

```
rng = np.random.default_rng(1)
height = rng.normal(170, 8, 80)
weight = 0.9 * (height - 100) + rng.normal(0, 6, 80)
group = rng.integers(0, 2, 80)

fig, ax = plt.subplots(figsize=(5.5, 3.6))
sc = ax.scatter(height, weight, c=group, cmap="coolwarm", s=30, alpha=0.8,
                edgecolor="white", linewidth=0.5)
ax.set_xlabel("height (cm)")
ax.set_ylabel("weight (kg)")
ax.set_title("Height vs weight")
plt.show()
```

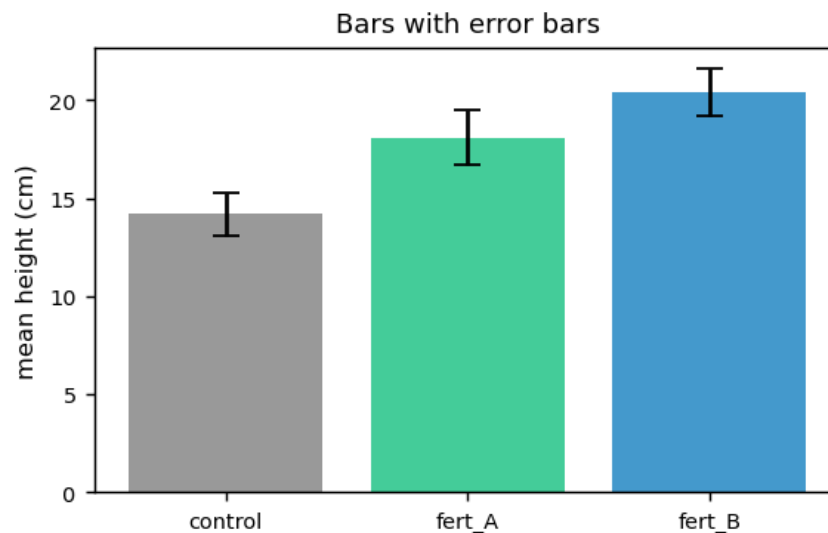


Parameters: **c** colours points by a variable, **s** sets the size, **alpha** makes points semi-transparent (vital when points overlap), **cmap** is the colour map.

9.4 Bar charts: comparing categories

```
labels = ["control", "fert_A", "fert_B"]
means = [14.2, 18.1, 20.4]
errors = [1.1, 1.4, 1.2]

fig, ax = plt.subplots(figsize=(5, 3))
ax.bar(labels, means, yerr=errors, capsize=5, color=["#999", "#4c9", "#49c"])
ax.set_ylabel("mean height (cm)")
ax.set_title("Bars with error bars")
plt.show()
```



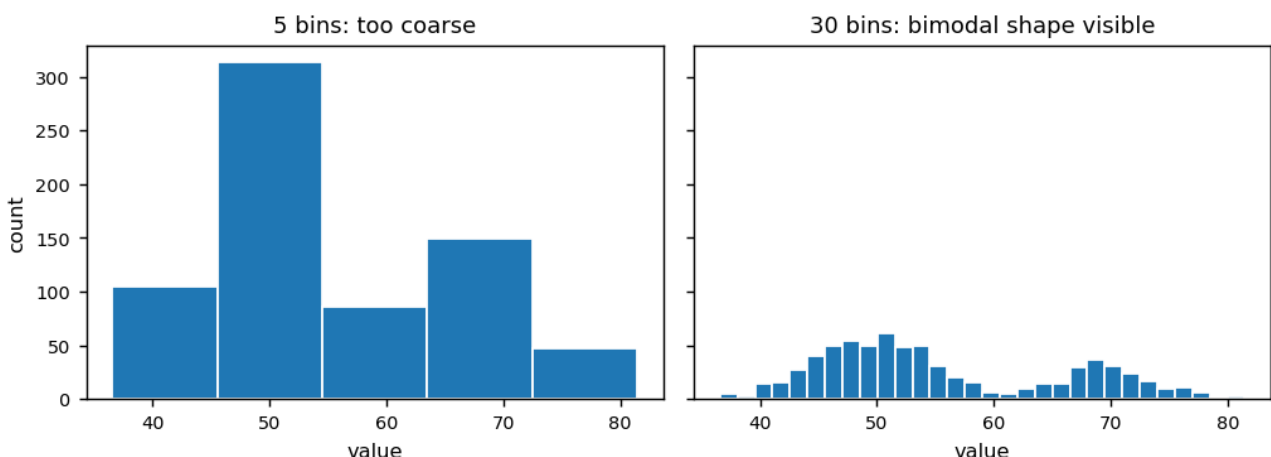
The `yerr` argument draws **error bars** (uncertainty). Whatever error measure you use (standard deviation, standard error, confidence interval) **state it in the caption**.

9.5 Histograms: the shape of one variable

A histogram cuts the range into **bins**, counts how many observations fall in each and draws bars. It shows centre, spread, skewness and outliers.

```
rng = np.random.default_rng(2)
data = np.concatenate([rng.normal(50, 5, 500), rng.normal(70, 4, 200)]) # two groups mixed

fig, axes = plt.subplots(1, 2, figsize=(8, 3), sharey=True)
axes[0].hist(data, bins=5, color="tab:blue", edgecolor="white")
axes[0].set_title("5 bins: too coarse")
axes[1].hist(data, bins=30, color="tab:blue", edgecolor="white")
axes[1].set_title("30 bins: bimodal shape visible")
for ax in axes:
    ax.set_xlabel("value")
axes[0].set_ylabel("count")
plt.tight_layout()
plt.show()
```



NOTE

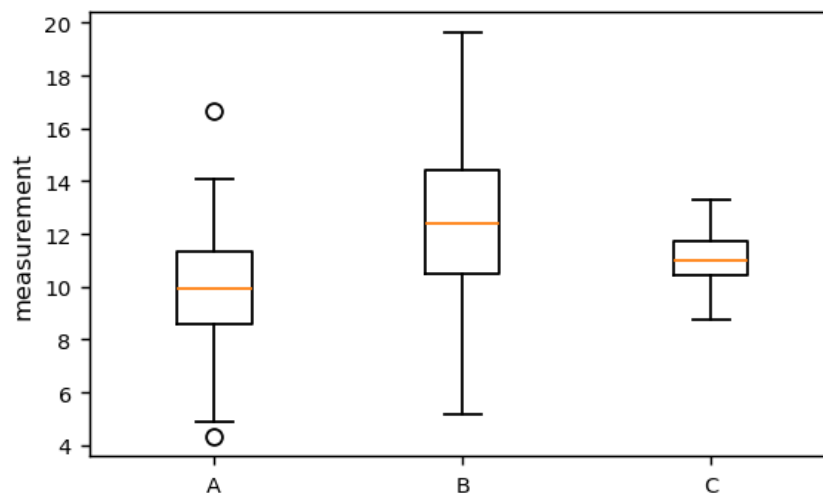
The number of bins changes the story. Too few hides structure; too many shows noise. Try several, or use `bins="auto"`. Use `density=True` to draw a density (area = 1) so histograms of different sample sizes can be compared.

9.6 Box plots and violin plots: comparing distributions

A **box plot** shows the median (line), the quartiles Q1 to Q3 (box), whiskers reaching to the most extreme non-outlier points (within 1.5 IQR) and dots for outliers.

```
rng = np.random.default_rng(3)
groups = [rng.normal(10, 2, 100), rng.normal(12, 3, 100), rng.normal(11, 1, 100)]

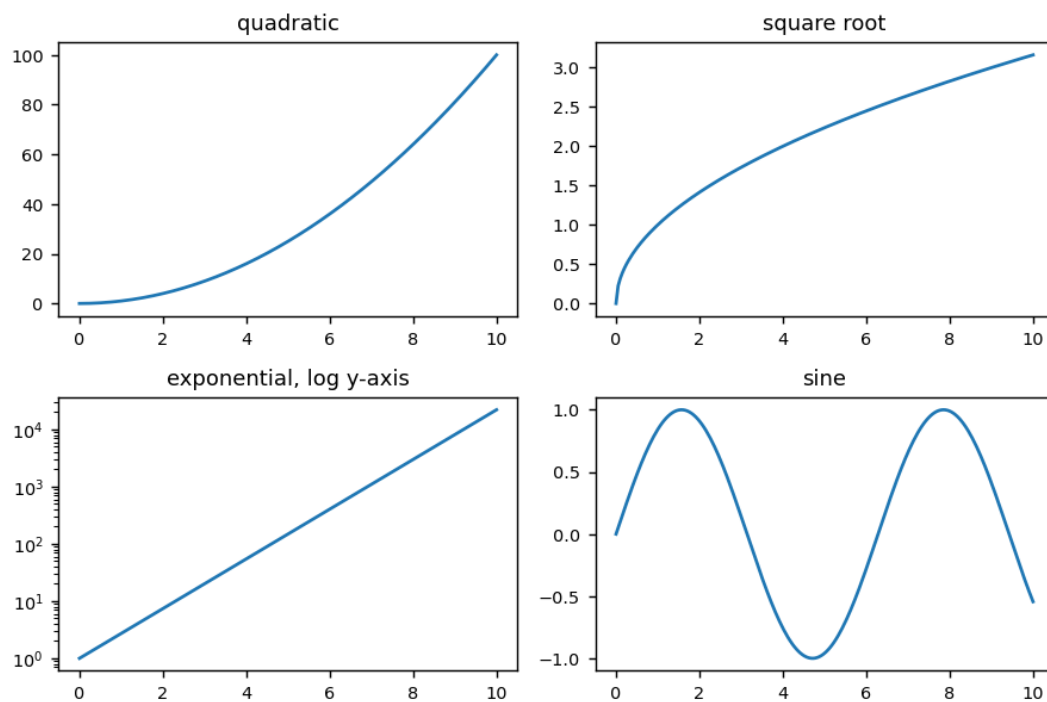
fig, ax = plt.subplots(figsize=(5, 3))
ax.boxplot(groups, tick_labels=["A", "B", "C"])
ax.set_ylabel("measurement")
plt.show()
```



9.7 Subplots: several panels

```
x = np.linspace(0, 10, 200)
fig, axes = plt.subplots(2, 2, figsize=(7, 5))
axes[0, 0].plot(x, x ** 2); axes[0, 0].set_title("quadratic")
axes[0, 1].plot(x, np.sqrt(x)); axes[0, 1].set_title("square root")
axes[1, 0].semilogy(x, np.exp(x)); axes[1, 0].set_title("exponential, log y-axis")
axes[1, 1].plot(x, np.sin(x)); axes[1, 1].set_title("sine")
fig.suptitle("Four views", y=1.02)
fig.tight_layout()
plt.show()
```

Four views

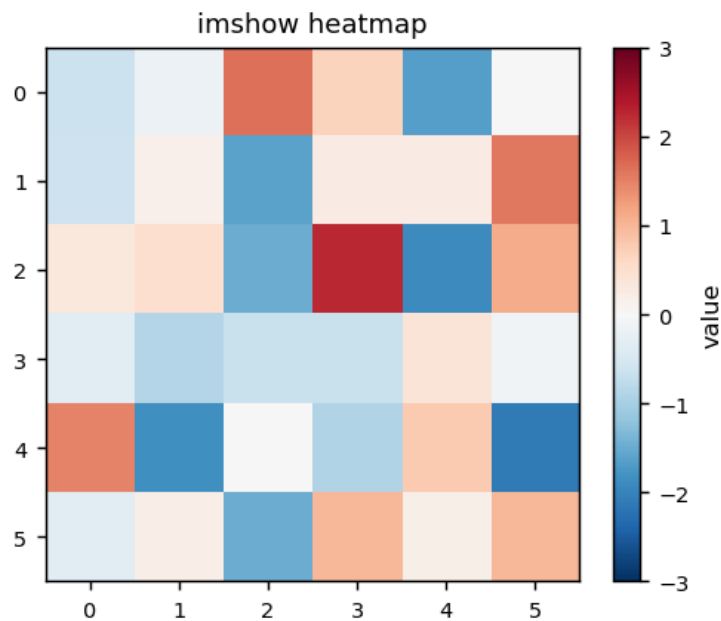


`sharex=True/sharey=True` share axis ranges across panels. `semilogy`, `semilogx` and `loglog` use logarithmic axes: essential when data spans several orders of magnitude or follows a power law or exponential.

9.8 Heatmaps and images

A **heatmap** shows a matrix of values as colours. It is ideal for correlation matrices and 2D measurements.

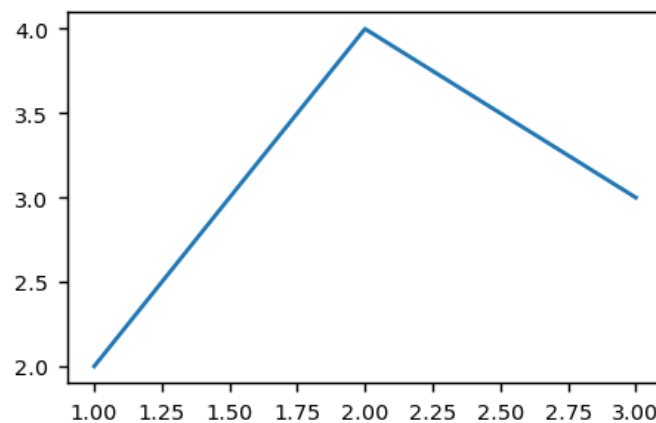
```
rng = np.random.default_rng(4)
matrix = rng.normal(size=(6, 6))
fig, ax = plt.subplots(figsize=(4.5, 3.6))
im = ax.imshow(matrix, cmap="RdBu_r", vmin=-3, vmax=3)
fig.colorbar(im, ax=ax, label="value")
ax.set_title("imshow heatmap")
plt.show()
```



Colour map advice: use a **sequential** map (`viridis`, `plasma`) for ordered values from low to high, a **diverging** map (`RdBu_r`, `coolwarm`) when values have a meaningful centre (zero), and a **qualitative** map (`tab10`) for categories. Avoid `jet` (rainbow); it creates fake edges and is unreadable in greyscale.

9.9 Saving figures

```
fig, ax = plt.subplots(figsize=(4, 2.5))
ax.plot([1, 2, 3], [2, 4, 3])
fig.savefig("my_plot.png", dpi=300, bbox_inches="tight") # high resolution for papers
fig.savefig("my_plot.pdf") # vector graphic, sharp at any zoom
plt.show()
```



In Colab, find saved files in the left folder panel, or download with `from google.colab import files; files.download("my_plot.png")`.

9.10 Seaborn: statistical plots in one line

Seaborn sits on top of Matplotlib and works directly with DataFrames in long format. It does grouping, colouring and statistics for you.

```
import seaborn as sns
import pandas as pd

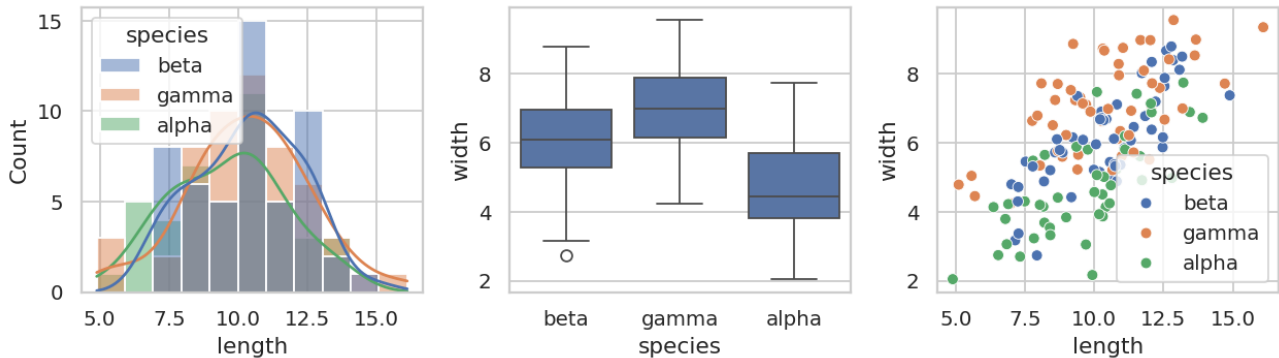
rng = np.random.default_rng(6)
```

```

n = 150
df = pd.DataFrame({
    "species": rng.choice(["alpha", "beta", "gamma"], n),
    "length": rng.normal(10, 2, n),
})
shift = df["species"].map({"alpha": 0, "beta": 1, "gamma": 2})
df["width"] = 0.5 * df["length"] + rng.normal(0, 1, n) + shift

sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(1, 3, figsize=(10, 3))
sns.histplot(data=df, x="length", hue="species", kde=True, ax=axes[0])
sns.boxplot(data=df, x="species", y="width", ax=axes[1])
sns.scatterplot(data=df, x="length", y="width", hue="species", ax=axes[2])
plt.tight_layout()
plt.show()

```

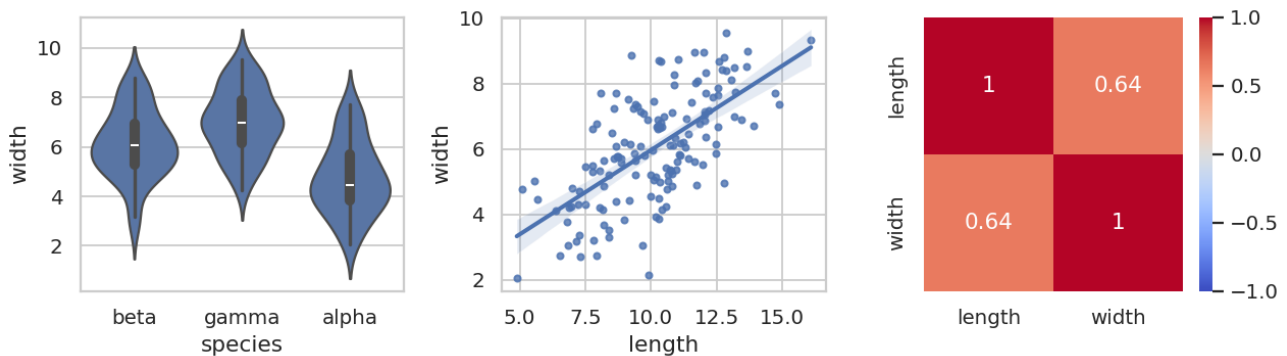


Other Seaborn favourites:

```

fig, axes = plt.subplots(1, 3, figsize=(10, 3))
sns.violinplot(data=df, x="species", y="width", ax=axes[0]) # distribution shape
sns.regplot(data=df, x="length", y="width", ax=axes[1], # scatter + fit line
            scatter_kws={"s": 12})
sns.heatmap(df[["length", "width"]].corr(), annot=True, cmap="coolwarm",
            vmin=-1, vmax=1, ax=axes[2])
plt.tight_layout()
plt.show()

```

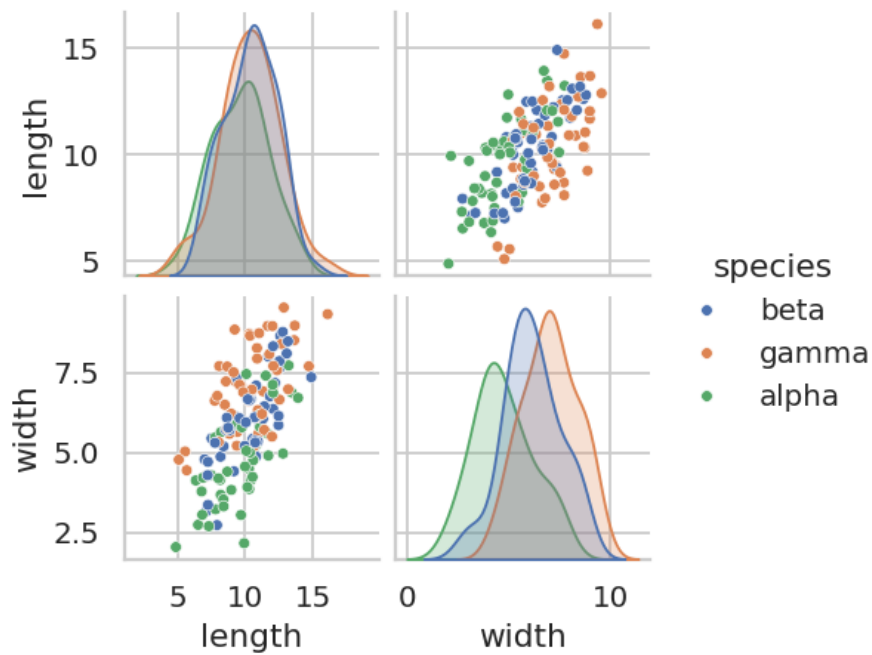


`sns.pairplot(df, hue="species")` draws every variable against every other: an excellent first look at a dataset with several numeric columns.

```

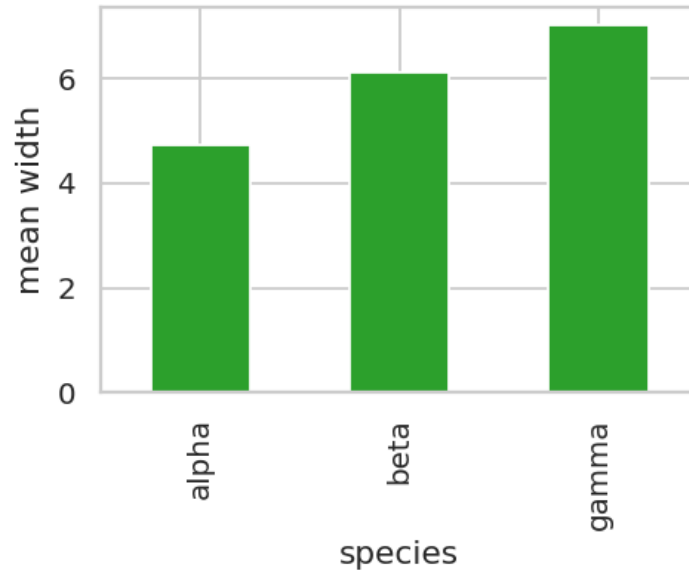
g = sns.pairplot(df, hue="species", height=1.8, plot_kws={"s": 18})
plt.show()

```



9.11 Plotting directly from pandas

```
summary = df.groupby("species")["width"].mean()
ax = summary.plot(kind="bar", figsize=(4, 2.6), color="tab:green")
ax.set_ylabel("mean width")
plt.show()
```

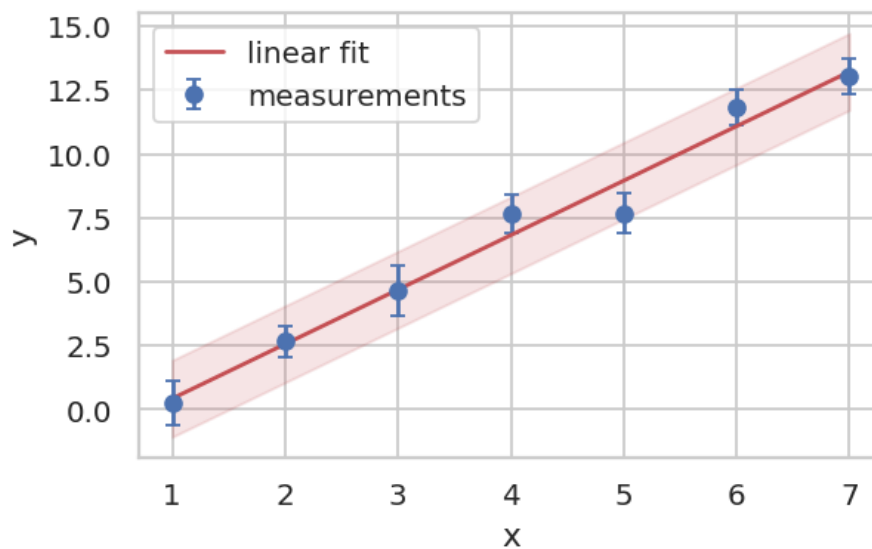


Any Series or DataFrame has `.plot()` with `kind` set to `line`, `bar`, `barh`, `hist`, `box`, `scatter`, `area` or `kde`.

9.12 Error bars and confidence bands

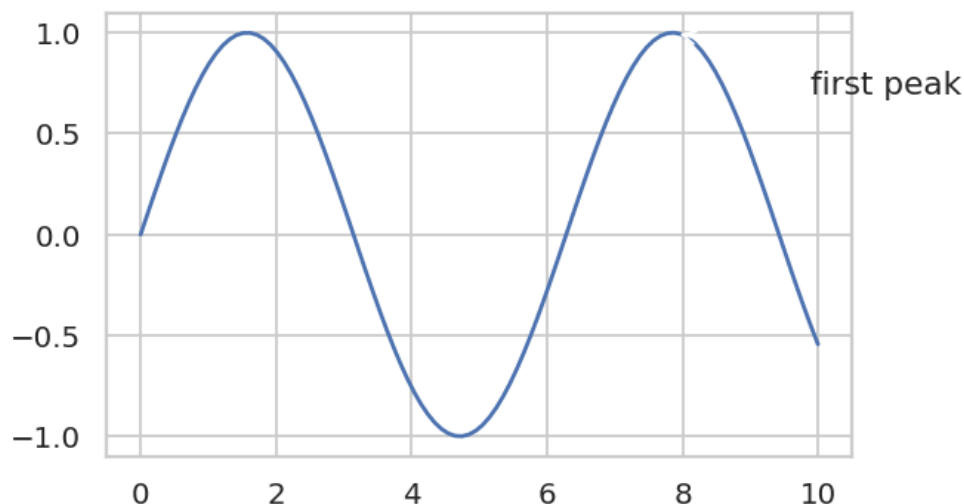
```
x = np.arange(1, 8)
rng = np.random.default_rng(8)
y = 2 * x + rng.normal(0, 1, 7)
err = rng.uniform(0.5, 1.5, 7)

fig, ax = plt.subplots(figsize=(5, 3))
ax.errorbar(x, y, yerr=err, fmt="o", capsize=3, label="measurements")
fit = np.polyfit(x, y, 1)
ax.plot(x, np.polyval(fit, x), "r-", label="linear fit")
ax.fill_between(x, np.polyval(fit, x) - 1.5, np.polyval(fit, x) + 1.5, color="r", alpha=0.15)
ax.legend()
ax.set_xlabel("x"); ax.set_ylabel("y")
plt.show()
```



9.13 Annotations and text

```
x = np.linspace(0, 10, 100)
fig, ax = plt.subplots(figsize=(5, 3))
ax.plot(x, np.sin(x))
peak = x[np.argmax(np.sin(x))]
ax.annotate("first peak", xy=(peak, 1), xytext=(peak + 2, 0.7),
           arrowprops=dict(arrowstyle="->"))
plt.show()
```



9.14 Choosing the right chart

Question	Good chart	Avoid
How is one variable distributed?	histogram, density, box plot	pie chart
How do two numeric variables relate?	scatter plot (+ fit line)	bar chart
How does something change over time?	line plot	3D bars
Compare a number across categories	bar chart (start at zero)	pie with many slices
Compare distributions across groups	box / violin / strip plot	bars of means alone
Many variables at once	heatmap, pair plot	overloaded 3D plots
Composition (parts of a whole)	stacked bar, few-slice pie	more than 5 slices

9.15 Honest and readable figures

1. **Label axes with units.** Nobody can interpret "y".
2. **Start bar charts at zero.** Bar length encodes value; a cut axis exaggerates differences.
3. **Show the data,** not only the summary: overlay points on bars or boxes.
4. **Report uncertainty:** error bars, bands, sample size.
5. **Use colourblind-friendly colours** (about 8% of men are colour-blind). Seaborn's `"colorblind"` palette and Matplotlib's `viridis` are good.
6. **Keep text big enough** to read when the figure is shrunk to a paper column.
7. **One message per figure.** Remove clutter: extra gridlines, borders, heavy 3D effects.

```
plt.style.use("default")
print(plt.style.available[:6])      # other ready-made styles, e.g. "seaborn-v0_8", "ggplot"
```

OUTPUT

```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic']
```

TIP

Interactive charts (zoom, hover) are available with the Plotly library: `import plotly.express as px; px.scatter(df, x="length", y="width", color="species")`. They work in Colab too and suit exploration; for papers use static Matplotlib or Seaborn figures.

9.16 Chapter summary

CHAPTER SUMMARY

- Use the object-oriented pattern: `fig, ax = plt.subplots()`, then `ax.plot/scatter/bar/hist/boxplot`.
- Always set labels (with units), titles, legends; save with `fig.savefig(..., dpi=300)`.
- Histogram for one variable, scatter for two, line for time, bar for categories, box/violin for group comparison, heatmap for matrices.
- Seaborn plots DataFrames with grouping in one line: `histplot`, `boxplot`, `scatterplot`, `violinplot`, `regplot`, `heatmap`, `pairplot`.
- Honest plots: zero-based bars, visible data, stated uncertainty, colourblind-safe palettes.

9.17 Practice exercises

1. Plot $y = x^2$, $y = x^3$ and $y = x^4$ on one axes for x from 0 to 2 with a legend.
2. Generate 1000 samples from an exponential distribution and draw histograms with 10, 30 and 100 bins side by side. What do you notice?
3. Make a scatter plot of two correlated variables coloured by a third categorical one, and add the fitted line.
4. Use `sns.boxplot` with `sns.stripplot` overlaid on the same Axes to show both summary and raw points.
5. Create a 2×2 grid showing the same data as line, scatter, bar and histogram. Which tells the truth best?

CHAPTER 10

10. Getting Data In and Out

IN THIS CHAPTER YOU WILL LEARN

- Reading and writing CSV, Excel, JSON and text files
- Using Google Drive and file uploads in Colab
- Reading data from URLs and web APIs
- Working with SQL databases using Python
- Good data-management habits

10.1 Files in Colab: where things live

Colab's temporary machine has a folder `/content/`. Code like `pd.read_csv("data.csv")` looks in the **current working directory**, which is `/content/` by default. Check it and list the files:

```
import os
print(os.getcwd())
print(os.listdir("."))
```

OUTPUT
 /tmp/bookrun/ch10
 []

Uploading files from your computer

```
from google.colab import files
uploaded = files.upload()           # opens a file picker
name = list(uploaded.keys())[0]
df = pd.read_csv(name)
```

Mounting Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
df = pd.read_csv('/content/drive/MyDrive/data/experiment.csv')
df.to_csv('/content/drive/MyDrive/data/cleaned.csv', index=False)
```

Files saved inside `/content/drive/MyDrive/` are permanent. Files saved elsewhere vanish when the runtime is recycled.

10.2 CSV files

CSV (comma-separated values) is the universal exchange format: plain text, one row per line.

```
import numpy as np
import pandas as pd

df = pd.DataFrame({"t": [0, 1, 2, 3], "signal": [0.1, 0.9, 2.1, 2.8]})
df.to_csv("signal.csv", index=False)      # index=False: do not write the row numbers
print(open("signal.csv").read())
print(pd.read_csv("signal.csv"))
```

OUTPUT
 t,signal
 0,0.1

```

1,0.9
2,2.1
3,2.8

   t  signal
0  0    0.1
1  1    0.9
2  2    2.1
3  3    2.8

```

`read_csv` has many helpful options:

Option	Purpose	Example
<code>sep</code>	separator other than comma	<code>sep=";", sep="\t"</code>
<code>header</code>	which row holds names (<code>None</code> if none)	<code>header=None</code>
<code>names</code>	provide your own column names	<code>names=["a", "b"]</code>
<code>usecols</code>	read only some columns	<code>usecols=["t"]</code>
<code>nrows</code>	read only the first n rows	<code>nrows=1000</code>
<code>skiprows</code>	skip junk lines at the top	<code>skiprows=3</code>
<code>na_values</code>	extra strings meaning missing	<code>na_values=["NA", "-999"]</code>
<code>parse_dates</code>	convert columns to dates	<code>parse_dates=["date"]</code>
<code>dtype</code>	force column types	<code>dtype={"id": str}</code>
<code>index_col</code>	use a column as the index	<code>index_col="date"</code>
<code>encoding</code>	text encoding	<code>encoding="latin-1"</code>
<code>chunksize</code>	read in pieces (huge files)	<code>chunksize=100000</code>

```

text = """# instrument log
# calibrated 2024-01-01
time;value;flag
2024-01-01 00:00;1.5;ok
2024-01-01 01:00;1.7;ok
2024-01-01 02:00;-999;bad
"""
open("log.txt", "w").write(text)

log = pd.read_csv("log.txt", sep=";", skiprows=2, decimal=".",
                  na_values=["-999"], parse_dates=["time"], index_col="time")
print(log)
print(log.dtypes)

```

OUTPUT

```

              value flag
time
2024-01-01 00:00:00    1.5  ok
2024-01-01 01:00:00    1.7  ok
2024-01-01 02:00:00   NaN bad
value      float64
flag        str
dtype: object

```

10.3 Excel files

```

df.to_excel("signal.xlsx", index=False, sheet_name="run1")
print(pd.read_excel("signal.xlsx", sheet_name="run1"))

with pd.ExcelWriter("multi.xlsx") as writer:      # several sheets in one file
    df.to_excel(writer, sheet_name="raw", index=False)
    df.describe().to_excel(writer, sheet_name="summary")
print(pd.ExcelFile("multi.xlsx").sheet_names)

```

OUTPUT

```

t  signal
0  0      0.1
1  1      0.9
2  2      2.1
3  3      2.8
['raw', 'summary']

```

Excel reading needs the `openpyxl` package, which Colab already has.

10.4 JSON files

JSON is the standard format for web data: nested dictionaries and lists.

```

import json

record = {"experiment": "E7", "params": {"temp": 37, "ph": 7.2}, "values": [1.2, 1.5, 1.4]}
with open("exp.json", "w") as f:
    json.dump(record, f, indent=2)

with open("exp.json") as f:
    loaded = json.load(f)
print(loaded["params"]["temp"], loaded["values"])

records = [{"id": 1, "x": 0.5}, {"id": 2, "x": 0.7}]
print(pd.DataFrame(records))
nested = pd.json_normalize([{"id": 1, "loc": {"lat": 10, "lon": 20}}]) # flatten
print(nested)

```

OUTPUT

```

37 [1.2, 1.5, 1.4]
   id  x
0   1  0.5
1   2  0.7
   id  loc.lat  loc.lon
0   1         10        20

```

10.5 Other formats worth knowing

Format	Use it for	pandas call
Parquet	large tables, fast and compact, keeps dtypes	<code>to_parquet</code> , <code>read_parquet</code>
Pickle	saving any Python object quickly (only your own files!)	<code>to_pickle</code> , <code>read_pickle</code>
HDF5 / NetCDF	large multi-dimensional scientific arrays (climate, imaging)	<code>h5py</code> , <code>xarray</code> libraries
FITS	astronomy images and tables	<code>astropy</code> library
TXT / DAT	instrument output	<code>np.loadtxt</code> , <code>pd.read_csv(sep=...)</code>
Fixed width	old formats	<code>pd.read_fwf</code>

WATCH OUT

Never unpickle a file from an untrusted source. Pickle can execute code when loading. Use CSV or Parquet when sharing.

10.6 Reading directly from the web

`read_csv` accepts a URL:

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
```

```
iris = pd.read_csv(url)
iris.head()
```

For a web API that returns JSON, use the `requests` library:

```
import requests
r = requests.get("https://api.open-meteo.com/v1/forecast",
                 params={"latitude": 18.5, "longitude": 73.8,
                        "hourly": "temperature_2m", "forecast_days": 1})
data = r.json()
weather = pd.DataFrame({"time": data["hourly"]["time"],
                       "temp": data["hourly"]["temperature_2m"]})
```

Check that the server says `r.status_code == 200` (success) and respect usage limits.

Built-in practice datasets

The `scikit-learn` library ships small classic datasets, always available offline:

```
from sklearn.datasets import load_iris, load_wine, load_diabetes, load_breast_cancer

iris = load_iris(as_frame=True)
print(iris.frame.head(3))
print(iris.target_names)
print(load_wine(as_frame=True).frame.shape, load_diabetes(as_frame=True).frame.shape)
```

OUTPUT

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0

['setosa' 'versicolor' 'virginica']
(178, 14) (442, 11)

Seaborn also provides `sns.load_dataset("penguins")`, `"tips"`, `"titanic"` and more (internet required in Colab).

10.7 Databases and SQL

Large research projects often keep data in a **relational database**. **SQL** (Structured Query Language) asks questions of it. Python includes `sqlite3`, a tiny database stored in one file, which is perfect for practising.

```
import sqlite3

conn = sqlite3.connect("lab.db")
df = pd.DataFrame({
    "sample": ["s1", "s2", "s3", "s4"],
    "site": ["N", "N", "S", "S"],
    "conc": [1.2, 1.5, 2.8, 3.1],
})
df.to_sql("measurements", conn, if_exists="replace", index=False)

query = """
SELECT site, AVG(conc) AS mean_conc, COUNT(*) AS n
FROM measurements
GROUP BY site
ORDER BY mean_conc DESC
"""
print(pd.read_sql(query, conn))
conn.close()
```

OUTPUT

	site	mean_conc	n
0	S	2.95	2
1	N	1.35	2

The four SQL clauses above map directly to pandas: **SELECT** = choose columns, **WHERE** = filter rows, **GROUP BY** = `groupby`, **ORDER BY** = `sort_values`, and **JOIN** = `merge`.

10.8 Saving your analysis results

- Save **cleaned data** separately from **raw data**. Never overwrite the raw file.
- Use descriptive names with dates: `water_clean_2024-03-05.csv`.
- Record the code that created each file (your notebook is the record).
- Use plain, open formats (CSV, Parquet) so others can read them years later.

```
folder = "project"
os.makedirs(folder + "/raw", exist_ok=True)
os.makedirs(folder + "/clean", exist_ok=True)
os.makedirs(folder + "/figures", exist_ok=True)
print(sorted(os.listdir(folder)))
```

OUTPUT

```
['clean', 'figures', 'raw']
```

10.9 Chapter summary

CHAPTER SUMMARY

- Colab files live in `/content/` and are temporary; mount Drive for permanence.
- `read_csv` options (`sep`, `skiprows`, `na_values`, `parse_dates`, `usecols`, `chunksize`) handle messy real files.
- `to_csv(index=False)`, `to_excel`, `json` and Parquet cover output formats.
- `requests` fetches web APIs; `sqlite3` with `read_sql` and `to_sql` connects to databases.
- Keep raw data untouched, cleaned data separate, and code as the record.

10.10 Practice exercises

1. Write a CSV with a semicolon separator and a header row of comments; read it back correctly.
2. Save a DataFrame to Excel with two sheets and read back only the second sheet.
3. Store the wine dataset from scikit-learn in an SQLite database and use SQL to compute the average alcohol content per class.
4. Read only columns `a` and `c` of a CSV with `usecols`, and only the first 100 rows.
5. Why might you choose Parquet over CSV for a 5 GB table?

PART III

Exploration and Statistics

Learn to explore data, describe it with numbers, understand randomness, and draw honest conclusions with confidence intervals and tests.

CHAPTER 11

11. Exploratory Data Analysis (EDA)

IN THIS CHAPTER YOU WILL LEARN

- The EDA mindset and a repeatable checklist
- Univariate, bivariate and multivariate exploration
- Spotting data problems, patterns, groups and relationships
- A complete worked example on a real dataset

Exploratory Data Analysis (EDA), a phrase popularised by the statistician John Tukey, means looking at your data from many angles **before** formal modelling. The goal is to understand its structure, find errors and form questions worth testing. EDA does not prove anything; it suggests.

11.1 The EDA checklist

Stage	Questions	Tools
1. Structure	How many rows and columns? What does each column mean, and what are its units and types?	<code>shape</code> , <code>info</code> , <code>dtypes</code>
2. Quality	Missing values? Duplicates? Impossible values?	<code>isna</code> , <code>duplicated</code> , <code>describe</code>
3. One variable	What is typical? How spread out? Any skew or outliers?	histogram, box plot, <code>describe</code>
4. Two variables	Are two variables related? How?	scatter, correlation, group comparisons
5. Many variables	Are there clusters, redundant variables, hidden groups?	pair plot, heatmap, PCA (Chapter 19)
6. Questions	What hypotheses or models should I test next?	notes in the notebook

11.2 A worked example: the wine dataset

We analyse 178 wines from three Italian cultivars, each described by 13 chemical measurements. The dataset is bundled with scikit-learn.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_wine

wine = load_wine(as_frame=True)
df = wine.frame.copy()
df["cultivar"] = df["target"].map(dict(enumerate(wine.target_names)))
df = df.drop(columns="target")
df.head()
```

OUTPUT

```
   alcohol  malic_acid  ash  alcalinity_of_ash  magnesium  total_phenols  ...  \
0    14.23         1.71  2.43             15.6         127.0           2.80  ...
1    13.20         1.78  2.14             11.2         100.0           2.65  ...
```

```

2    13.16    2.36  2.67    18.6    101.0    2.80 ...
3    14.37    1.95  2.50    16.8    113.0    3.85 ...
4    13.24    2.59  2.87    21.0    118.0    2.80 ...

   proanthocyanins  color_intensity  hue  od280/od315_of_diluted_wines  proline  \
0                2.29             5.64  1.04                      3.92    1065.0
1                1.28             4.38  1.05                      3.40    1050.0
2                2.81             5.68  1.03                      3.17    1185.0
3                2.18             7.80  0.86                      3.45    1480.0
4                1.82             4.32  1.04                      2.93     735.0

   cultivar
0  class_0
1  class_0
2  class_0
3  class_0
4  class_0

[5 rows x 14 columns]

```

Stage 1 and 2: structure and quality

```

print("shape:", df.shape)
print("missing values:", int(df.isna().sum().sum()))
print("duplicate rows:", int(df.duplicated().sum()))
print(df["cultivar"].value_counts())
df.describe().T.round(2).head(6)

```

OUTPUT

```

shape: (178, 14)
missing values: 0
duplicate rows: 0
cultivar
class_1    71
class_0    59
class_2    48
Name: count, dtype: int64

   count  mean  std  min  25%  50%  75%  max
alcohol  178.0  13.00  0.81  11.03  12.36  13.05  13.68  14.83
malic_acid  178.0  2.34  1.12  0.74  1.60  1.87  3.08  5.80
ash  178.0  2.37  0.27  1.36  2.21  2.36  2.56  3.23
alcalinity_of_ash  178.0  19.49  3.34  10.60  17.20  19.50  21.50  30.00
magnesium  178.0  99.74  14.28  70.00  88.00  98.00  107.00  162.00
total_phenols  178.0  2.30  0.63  0.98  1.74  2.36  2.80  3.88

```

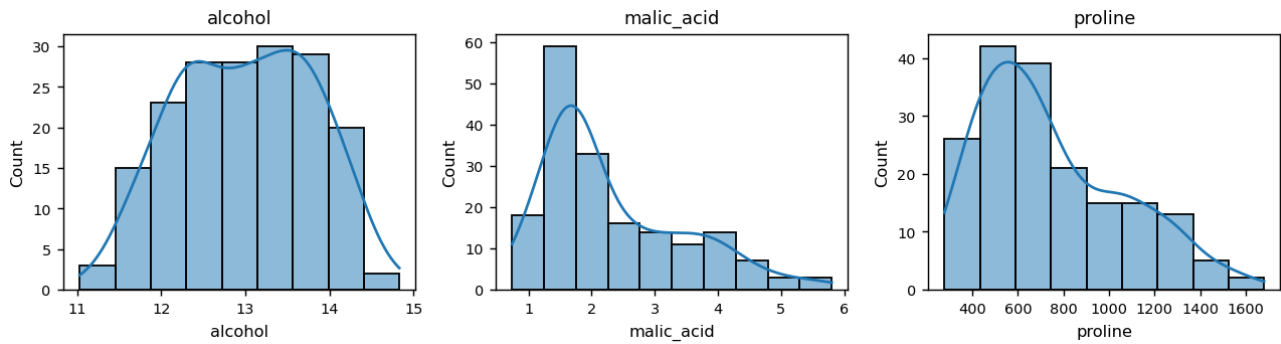
The `.T` transposes the table so each variable is a row: easier to read with many columns. Check that the `min` and `max` are plausible and that `mean` and `50%` (the median) are close; a large gap hints at skew.

Stage 3: one variable at a time

```

fig, axes = plt.subplots(1, 3, figsize=(10, 2.8))
for ax, col in zip(axes, ["alcohol", "malic_acid", "proline"]):
    sns.histplot(df[col], kde=True, ax=ax, color="tab:blue")
    ax.set_title(col)
plt.tight_layout()
plt.show()

```



Observe: alcohol is roughly symmetric, malic acid is skewed right (a long tail of high values), proline is strongly right-skewed. Skewed variables may benefit from a log transform before some analyses (Chapter 17).

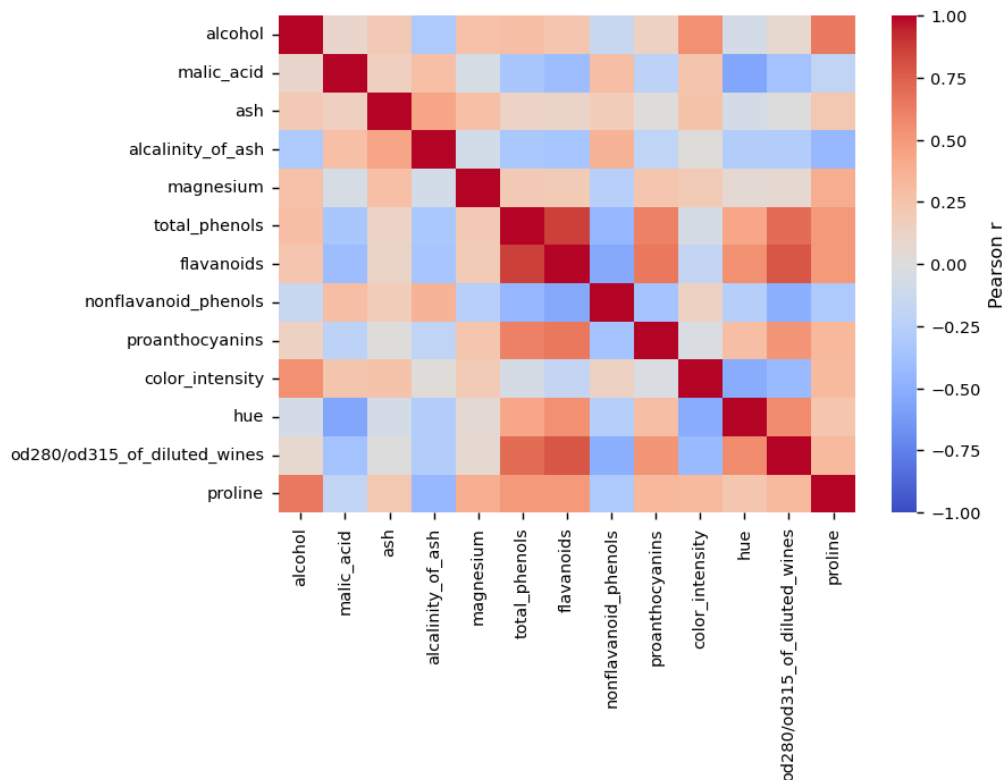
```
skew = df.drop(columns="cultivar").skew().sort_values()
print(skew.round(2).tail(4))
```

```
OUTPUT
proline          0.77
color_intensity  0.87
malic_acid       1.04
magnesium        1.10
dtype: float64
```

Stage 4: two variables

Numeric versus numeric. Correlation measures linear association from -1 to $+1$ (Chapter 12).

```
num = df.drop(columns="cultivar")
corr = num.corr()
fig, ax = plt.subplots(figsize=(7, 5.5))
sns.heatmap(corr, cmap="coolwarm", vmin=-1, vmax=1, center=0, ax=ax,
            cbar_kws={"label": "Pearson r"})
plt.tight_layout()
plt.show()
```



```
# strongest correlated pairs
pairs = corr.where(np.triu(np.ones(corr.shape, dtype=bool), k=1)).stack()
print(pairs.abs().sort_values(ascending=False).head(5).round(2))
```

OUTPUT

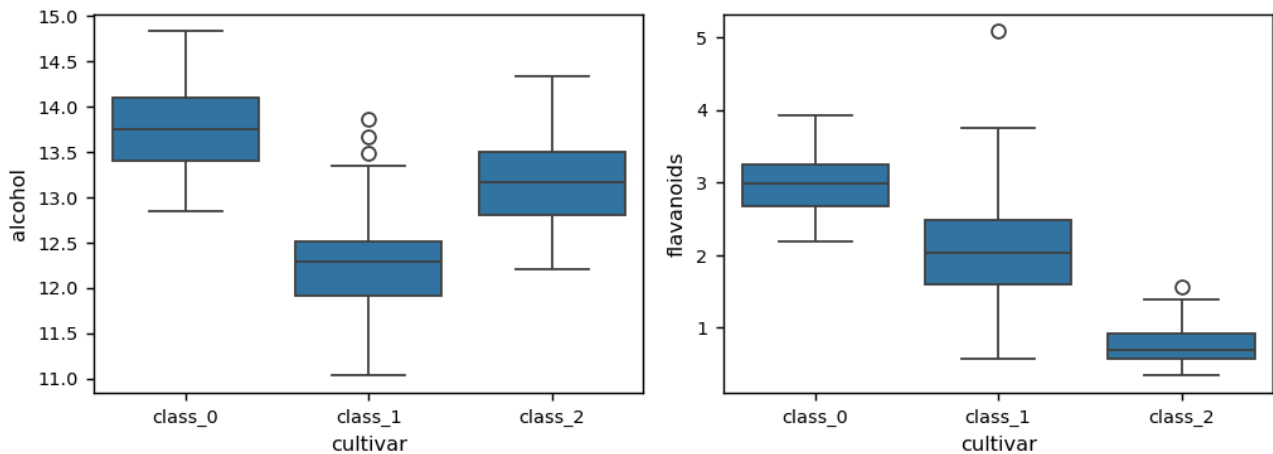
```
total_phenols  flavanoids                0.86
flavanoids    od280/od315_of_diluted_wines  0.79
total_phenols od280/od315_of_diluted_wines  0.70
flavanoids    proanthocyanins             0.65
alcohol       proline                     0.64
dtype: float64
```

Numeric versus category. Compare a variable across groups:

```
fig, axes = plt.subplots(1, 2, figsize=(8, 3))
sns.boxplot(data=df, x="cultivar", y="alcohol", ax=axes[0])
sns.boxplot(data=df, x="cultivar", y="flavanoids", ax=axes[1])
plt.tight_layout()
plt.show()
print(df.groupby("cultivar")[["alcohol", "flavanoids", "proline"]].mean().round(2))
```

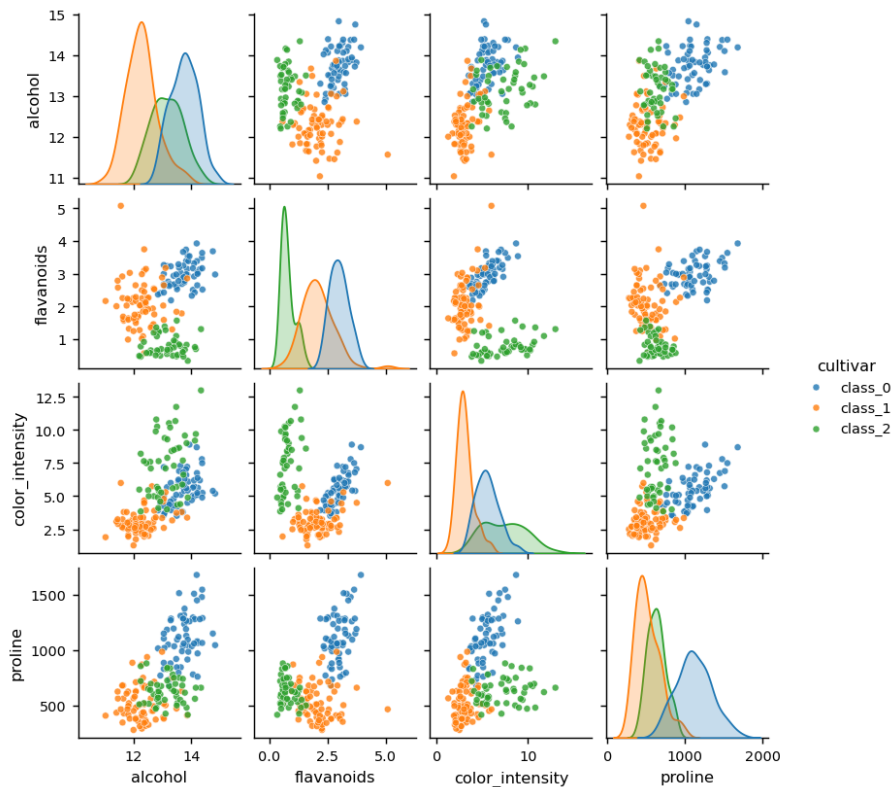
OUTPUT

```
          alcohol  flavanoids  proline
cultivar
class_0      13.74         2.98  1115.71
class_1      12.28         2.08   519.51
class_2      13.15         0.78   629.90
```



Stage 5: many variables at once

```
cols = ["alcohol", "flavanoids", "color_intensity", "proline"]
g = sns.pairplot(df, vars=cols, hue="cultivar", height=1.6, plot_kws={"s": 15, "alpha": 0.8})
plt.show()
```



The pair plot shows that cultivars separate clearly in **flavanoids** and **color_intensity**: a hint that these variables could classify wines (and indeed we will do that in Chapter 18).

11.3 Questions to ask of every dataset

1. **Where did the data come from?** Who collected it, how and why? Sampling method determines what conclusions are valid.
2. **What is one row?** A person, a day, a measurement? Are rows independent or repeated measures of the same subject?
3. **What are the units and scales?** Mixed units are a classic error source.

4. **Is anything impossible or suspicious?** Negative ages, future dates, constant columns.
5. **Are the groups balanced?** If one group is 90% of the data, results mostly describe it.
6. **Is there a time order?** If yes, treat the data as a time series (Chapter 20).

11.4 Simple automated overview

A small helper function saves time on every new dataset:

```
def overview(d):
    out = pd.DataFrame({
        "dtype": d.dtypes.astype(str),
        "n_missing": d.isna().sum(),
        "n_unique": d.nunique(),
    })
    numeric = d.select_dtypes("number")
    out["mean"] = numeric.mean()
    out["std"] = numeric.std()
    out["min"] = numeric.min()
    out["max"] = numeric.max()
    return out.round(2)
```

```
overview(df).head(5)
```

OUTPUT

	dtype	n_missing	n_unique	mean	std	min	max
alcohol	float64	0	126	13.00	0.81	11.03	14.83
malic_acid	float64	0	133	2.34	1.12	0.74	5.80
ash	float64	0	79	2.37	0.27	1.36	3.23
alcalinity_of_ash	float64	0	63	19.49	3.34	10.60	30.00
magnesium	float64	0	53	99.74	14.28	70.00	162.00

11.5 Common EDA pitfalls

WATCH OUT

- **Simpson's paradox:** a trend in every subgroup can reverse when groups are combined. Always check by subgroup.
- **Correlation is not causation.** Ice cream sales and drownings correlate; both rise in summer.
- **Fishing for patterns:** if you plot 100 relationships, about 5 will look "significant" by chance. Note what you explored and test findings on fresh data.
- **Survivorship bias:** data that only includes the cases that "survived" a filter (for example published studies) misleads.

```
# Simpson's paradox in a nutshell
rng = np.random.default_rng(0)
frames = []
for shift, label in [(0, "group 1"), (6, "group 2"), (12, "group 3")]:
    x = rng.normal(shift, 1.0, 60)
    y = -0.8 * (x - shift) + 2.0 * shift + rng.normal(0, 0.6, 60)
    frames.append(pd.DataFrame({"x": x, "y": y, "group": label}))
sim = pd.concat(frames)
print("overall correlation:", round(sim["x"].corr(sim["y"]), 2))
for name, g in sim.groupby("group"):
    print(name, "correlation:", round(g["x"].corr(g["y"]), 2))
```

OUTPUT

```
overall correlation: 0.96
group 1 correlation: -0.8
group 2 correlation: -0.75
group 3 correlation: -0.82
```

Overall the variables look **positively** related but inside every group they are **negatively** related. The hidden grouping variable flips the story.

11.6 Chapter summary

CHAPTER SUMMARY

- EDA = structure, quality, one variable, two variables, many variables, then questions.
- Use histograms and box plots for single variables, scatter plots and correlation for pairs, pair plots for many.
- Compare groups with `groupby` and box plots; check balance.
- Beware Simpson's paradox, confusing correlation with causation, and over-interpreting chance patterns.
- Wrap repeated steps in helper functions and write your observations in text cells.

11.7 Practice exercises

1. Load `load_diabetes(as_frame=True)` and produce a full EDA following the checklist. Write three observations.
2. Which two wine variables have the strongest negative correlation? Plot them with colours for cultivar.
3. Find variables in the wine data with `|skewness|` above 0.8. Plot them before and after a log transform.
4. Write an `overview` function that also reports the fraction of missing values and flags constant columns.
5. Create your own Simpson's paradox data set with different parameters and show it.

CHAPTER 12

12. Descriptive Statistics

IN THIS CHAPTER YOU WILL LEARN

- Measures of centre: mean, median, mode (and when each is best)
- Measures of spread: range, variance, standard deviation, IQR, MAD
- Shape: skewness, kurtosis; percentiles and quantiles
- Covariance and correlation (Pearson, Spearman); z-scores
- Why we divide by $n - 1$

Descriptive statistics summarise a dataset with a few numbers. Good summaries answer two basic questions: *where is the data centred?* and *how spread out is it?*

We will use the same tiny sample throughout: the resting heart rates (beats per minute) of 9 people.

```
import numpy as np
import pandas as pd
from scipy import stats

hr = np.array([62, 65, 68, 70, 71, 73, 75, 80, 110])
print(hr)
```

OUTPUT

```
[ 62  65  68  70  71  73  75  80 110]
```

12.1 Measures of centre

Mean

The **mean** (average) adds all values and divides by how many there are.

$$\bar{x} = (x_1 + x_2 + \dots + x_n) / n = (1/n) \sum x_i$$

Here $\bar{x}$ ("x-bar") is the sample mean, x_i is the i -th value and n is the number of values. The mean is the **balance point** of the data: if each value were a weight on a ruler, the ruler balances at the mean.

Median

The **median** is the middle value after sorting (for an even count, the average of the middle two). Half the data lies below it and half above.

Mode

The **mode** is the most frequent value; it is the only centre measure that also works for categories.

```
print("mean :", hr.mean().round(2))
print("median:", np.median(hr))
print("mode :", stats.mode(hr, keepdims=False).mode) # all values tie: smallest shown

colors = pd.Series(["red", "blue", "red", "green", "red", "blue"])
print(colors.mode().tolist())
```

OUTPUT

```
mean : 74.89
median: 71.0
```

```
mode : 62
['red']
```

Which to use? The value 110 is unusual (perhaps a person just after climbing stairs). It pulls the mean to 74.9 while the median stays at 71. The mean is sensitive to outliers; the median is **robust**.

Situation	Better centre
Symmetric, no outliers	mean (uses all the information)
Skewed or has outliers (incomes, reaction times)	median
Categories	mode
Growth rates, ratios	geometric mean

```
growth = np.array([1.10, 1.50, 0.80])      # +10 %, +50 %, -20 %
print("arithmetic mean:", growth.mean().round(4))
print("geometric mean :", stats.gmean(growth).round(4)) # right average for growth
print("harmonic mean :", stats.hmean([40, 60]).round(2)) # for speeds
```

OUTPUT

```
arithmetic mean: 1.1333
geometric mean : 1.097
harmonic mean  : 48.0
```

Other variants: the **trimmed mean** drops a fraction of the smallest and largest values, and the **weighted mean** gives some values more importance.

```
print("10% trimmed mean:", stats.trim_mean(hr, 0.1).round(2))
values, weights = [80, 90, 70], [1, 2, 1]
print("weighted mean:", np.average(values, weights=weights))
```

OUTPUT

```
10% trimmed mean: 74.89
weighted mean: 82.5
```

12.2 Measures of spread

Knowing the centre is not enough: heights of 170 ± 2 cm and 170 ± 20 cm are very different populations.

Range and IQR

The **range** is $\max - \min$. It is simple but dominated by extremes. The **interquartile range** $IQR = Q3 - Q1$ measures the spread of the middle half of the data and is robust.

Variance and standard deviation

Start from **deviations**, the distance of each value from the mean: $x_i - \bar{x}$. They sum to zero, so we **square** them before averaging. The result is the **variance**:

$$s^2 = \sum (x_i - \bar{x})^2 / (n - 1)$$

The **standard deviation** is its square root, $s = \sqrt{s^2}$, which returns to the original units. Roughly, it is "the typical distance of a point from the mean".

```
dev = hr - hr.mean()
print("deviations:", dev.round(1))
print("sum of deviations:", dev.sum().round(10))
print("variance:", (dev ** 2).sum() / (len(hr) - 1))
print("std dev :", np.sqrt((dev ** 2).sum() / (len(hr) - 1)).round(3))
print("numpy   :", hr.var(ddof=1).round(3), hr.std(ddof=1).round(3))
```

OUTPUT

```

deviations: [-12.9 -9.9 -6.9 -4.9 -3.9 -1.9  0.1  5.1 35.1]
sum of deviations: 0.0
variance: 201.61111111111111
std dev : 14.199
numpy   : 201.611 14.199

```

THE MATH BEHIND IT

Why $n - 1$? If we knew the *true* population mean μ , we could average $(x - \mu)^2$ over n values. But we only have the sample mean $\bar{x}$, which is, by construction, the value closest to the sample points. Distances to $\bar{x}$ are therefore slightly *smaller* than distances to the true μ , so dividing by n underestimates the true variance. Dividing by $n - 1$ (Bessel's correction) exactly removes this bias on average. Another view: once $\bar{x}$ is fixed, only $n - 1$ of the deviations are free (the last one is forced so they sum to zero). Those $n - 1$ free pieces are the **degrees of freedom**.

We can **verify** the claim by simulation: draw many small samples from a population with known variance 25 and compare both formulas.

```

rng = np.random.default_rng(1)
true_var = 25.0
samples = rng.normal(0, 5, size=(100000, 5)) # 100,000 samples of size 5
print("divide by n      :", samples.var(axis=1, ddof=0).mean().round(2))
print("divide by n - 1:", samples.var(axis=1, ddof=1).mean().round(2))
print("true variance   :", true_var)

```

OUTPUT

```

divide by n      : 19.95
divide by n - 1: 24.94
true variance    : 25.0

```

Dividing by n is systematically too small (about 20 instead of 25); dividing by $n - 1$ recovers 25.

Other robust and relative measures

```

mad = np.median(np.abs(hr - np.median(hr))) # median absolute deviation
q1, q3 = np.percentile(hr, [25, 75])
print("range:", np.ptp(hr), " IQR:", q3 - q1, " MAD:", mad)
print("std   :", hr.std(ddof=1).round(2))

cv = hr.std(ddof=1) / hr.mean() # coefficient of variation
print("coefficient of variation:", f"{cv:.1%}")
print("standard error of the mean:", (hr.std(ddof=1) / np.sqrt(len(hr))).round(2))

```

OUTPUT

```

range: 48 IQR: 7.0 MAD: 4.0
std   : 14.2
coefficient of variation: 19.0%
standard error of the mean: 4.73

```

- The **coefficient of variation** (std / mean) compares spread of variables with different units or scales.
- The **standard error of the mean** ($SEM = s/\sqrt{n}$) measures how precisely the *mean* is known. It shrinks as the sample grows, while the standard deviation does not. Do not confuse them: SD describes the data; SEM describes the uncertainty of the mean.

12.3 Percentiles and quantiles

The **p-th percentile** is the value below which p percent of the data lies. The median is the 50th percentile; quartiles Q1, Q2, Q3 are the 25th, 50th and 75th.

```
print(np.percentile(hr, [10, 25, 50, 75, 90]))
```

```
print(pd.Series(hr).quantile([0.25, 0.5, 0.75]).tolist())
print(pd.Series(hr).describe().round(2))
```

OUTPUT

```
[64.4 68.  71.  75.  86. ]
[68.0, 71.0, 75.0]
count      9.00
mean       74.89
std        14.20
min        62.00
25%        68.00
50%        71.00
75%        75.00
max        110.00
dtype: float64
```

The **five-number summary** (min, Q1, median, Q3, max) is exactly what a box plot draws.

12.4 Shape: skewness and kurtosis

Skewness measures asymmetry. Positive skew: a long right tail (mean > median). Negative skew: a long left tail.

Kurtosis measures tail heaviness. A normal (bell-curve) distribution has *excess kurtosis* of 0; larger means heavier tails and more extreme outliers.

$$\text{skewness} = (1/n) \sum (x_i - \bar{x}) / s)^3 \quad \text{kurtosis} = (1/n) \sum (x_i - \bar{x}) / s)^4 - 3$$

```
rng = np.random.default_rng(2)
symmetric = rng.normal(0, 1, 5000)
right_skew = rng.exponential(1, 5000)
heavy = rng.standard_t(3, 5000)
for name, d in [("normal", symmetric), ("exponential", right_skew), ("t (3 df)", heavy)]:
    print(f"{name:12s} skew {stats.skew(d):6.2f} excess kurtosis {stats.kurtosis(d):6.2f}")
```

OUTPUT

```
normal      skew -0.00 excess kurtosis -0.10
exponential skew  1.85 excess kurtosis  4.83
t (3 df)    skew -1.01 excess kurtosis 18.26
```

12.5 Covariance and correlation

Two variables **covary** if they move together. The **covariance** averages the product of the two deviations:

$$\text{cov}(x, y) = \sum (x_i - \bar{x})(y_i - \bar{y}) / (n - 1)$$

If high x values go with high y values, both deviations are positive (or both negative), their product is positive, and the covariance is positive. Covariance depends on units, so we rescale by both standard deviations to get the **Pearson correlation coefficient** r , always between -1 and $+1$:

$$r = \text{cov}(x, y) / (s_x s_y)$$

r	Interpretation
+1	perfect increasing straight-line relationship
0	no linear relationship

r	Interpretation
-1	perfect decreasing straight-line relationship
about 0.1 / 0.3 / 0.5+	weak / moderate / strong (rough guide; depends on the field)

```
hours = np.array([1, 2, 3, 4, 5, 6, 7, 8])
score = np.array([52, 55, 61, 64, 70, 72, 78, 85])
```

```
cov = np.cov(hours, score)[0, 1]
r = np.corrcoef(hours, score)[0, 1]
print("covariance:", round(cov, 2))
print("correlation r:", round(r, 4))
print("scipy:", stats.pearsonr(hours, score))
```

OUTPUT

```
covariance: 27.5
correlation r: 0.9941
scipy: PearsonRResult(statistic=np.float64(0.9940557850184222), pvalue=np.float64(5.22739551711967
4e-07))
```

WATCH OUT

Pearson's r detects only **straight-line** relationships. A perfect parabola can give $r \approx 0$. Always look at the scatter plot. And r is sensitive to outliers.

```
x = np.linspace(-3, 3, 50)
y = x ** 2
print("r for a perfect parabola:", round(np.corrcoef(x, y)[0, 1], 3))
```

OUTPUT

```
r for a perfect parabola: -0.0
```

Rank correlation (Spearman and Kendall)

Spearman's ρ is Pearson's r applied to the **ranks** (1st, 2nd, 3rd...) of the values. It detects any *monotonic* (always increasing or always decreasing) relationship and resists outliers. **Kendall's τ** counts concordant versus discordant pairs.

```
x = np.arange(1, 11)
y = x ** 3 # increasing but not linear
print("Pearson :", round(stats.pearsonr(x, y)[0], 3))
print("Spearman:", round(stats.spearmanr(x, y)[0], 3))
print("Kendall :", round(stats.kendalltau(x, y)[0], 3))
```

```
df = pd.DataFrame({"a": x, "b": y, "c": np.sqrt(x)})
print(df.corr(method="spearman").round(2))
```

OUTPUT

```
Pearson : 0.928
Spearman: 1.0
Kendall : 1.0
   a  b  c
a  1.0 1.0 1.0
b  1.0 1.0 1.0
c  1.0 1.0 1.0
```

12.6 Standardising: z-scores

A **z-score** converts a value to "number of standard deviations from the mean", making different variables comparable.

$$z = (x - \bar{x}) / s$$

```
z = stats.zscore(hr, ddof=1)
print(pd.DataFrame({"hr": hr, "z": z.round(2)}).T)
```

OUTPUT

	0	1	2	3	4	5	6	7	8
hr	62.00	65.0	68.00	70.00	71.00	73.00	75.00	80.00	110.00
z	-0.91	-0.7	-0.49	-0.34	-0.27	-0.13	0.01	0.36	2.47

A z-score of 0 is the mean; +2 is two standard deviations above. For bell-shaped data about 68% of values have $|z| < 1$, 95% have $|z| < 2$ and 99.7% have $|z| < 3$.

12.7 Frequencies and proportions for categories

```
species = pd.Series(["cat", "dog", "cat", "bird", "dog", "cat", "cat", "dog"])
print(species.value_counts())
print(species.value_counts(normalize=True).round(3))
```

OUTPUT

```
cat    4
dog    3
bird   1
Name: count, dtype: int64
cat    0.500
dog    0.375
bird   0.125
Name: proportion, dtype: float64
```

12.8 Weighted and grouped statistics in one table

```
df = pd.DataFrame({"group": list("AABBBBC"), "v": [3, 5, 2, 4, 9, 7]})
print(df.groupby("group")["v"].agg(["count", "mean", "median", "std", "min", "max"]))
```

OUTPUT

	count	mean	median	std	min	max
group						
A	2	4.0	4.0	1.414214	3	5
B	3	5.0	4.0	3.605551	2	9
C	1	7.0	7.0	NaN	7	7

12.9 Chapter summary

CHAPTER SUMMARY

- Centre: mean (balance point, sensitive to outliers), median (robust), mode (categories).
- Spread: range, IQR (robust), variance and standard deviation (with $n - 1$ for samples), MAD, coefficient of variation.
- $SEM = s/\sqrt{n}$ is the precision of the mean, not the spread of the data.
- Percentiles and the five-number summary describe distributions; skewness and kurtosis describe shape.
- Covariance and Pearson correlation measure straight-line association; Spearman/Kendall measure monotonic association; always plot.
- z-scores put values on a common scale.

12.10 Practice exercises

1. For `[4, 8, 15, 16, 23, 42]` compute the mean, median, sample variance and standard deviation by hand, then confirm with NumPy.
2. Show with a simulation that the median is more stable than the mean when an outlier is added.
3. Generate two variables that have $r \approx 0$ but are clearly related. Plot them.
4. Compute the z-score of a value of 85 in a sample with mean 70 and standard deviation 10. What percentile does it roughly correspond to?
5. A dataset has a mean of 50 and median of 35. Is it likely skewed left or right? Why?

CHAPTER 13

13. Probability Distributions

IN THIS CHAPTER YOU WILL LEARN

- Random variables, PMF, PDF and CDF
- The key discrete distributions: Bernoulli, binomial, Poisson
- The key continuous distributions: uniform, normal, exponential, t, chi-square, F
- The Law of Large Numbers and the Central Limit Theorem
- Using `scipy.stats` fluently

A **random variable** is a number produced by a random process: the count of heads in 10 coin flips, the time until the next radioactive decay, the measurement error of a balance. A **probability distribution** describes which values are likely.

13.1 Discrete versus continuous

- **Discrete** variables take separate values (0, 1, 2, ...). Their distribution is a **probability mass function** (PMF): $P(X = k)$ for each k .
- **Continuous** variables take any value in a range. The **probability density function** (PDF) $f(x)$ gives the *density*; probability comes from the **area** under it: $P(a < X < b) = \int_a^b f(x) dx$. A single exact value has probability zero.
- The **cumulative distribution function** (CDF) $F(x) = P(X \leq x)$ works for both. It rises from 0 to 1.

The **expected value** $E[X]$ is the long-run average, and the **variance** $\text{Var}(X) = E[(X - \mu)^2]$ measures the spread. They are the theoretical twins of the sample mean and sample variance.

13.2 scipy.stats: one interface for everything

Each distribution in `scipy.stats` has the same methods:

Method	What it does
<code>.pmf(k) / .pdf(x)</code>	probability (mass) or density at a value
<code>.cdf(x)</code>	$P(X \leq x)$
<code>.sf(x)</code>	survival function: $P(X > x) = 1 - \text{cdf}$
<code>.ppf(q)</code>	inverse CDF (percent-point function): the value with $P(X \leq \text{value}) = q$
<code>.rvs(size=n, random_state=s)</code>	draw random samples
<code>.mean(), .var(), .std()</code>	theoretical moments

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

13.3 Discrete distributions

Bernoulli and binomial

A **Bernoulli** trial has two outcomes (success with probability p , failure with $1 - p$): a coin flip, a patient cured or not. Repeating n independent trials and counting the successes gives the **binomial** distribution:

$$P(X = k) = C(n, k) \cdot p^k \cdot (1 - p)^{n - k}$$

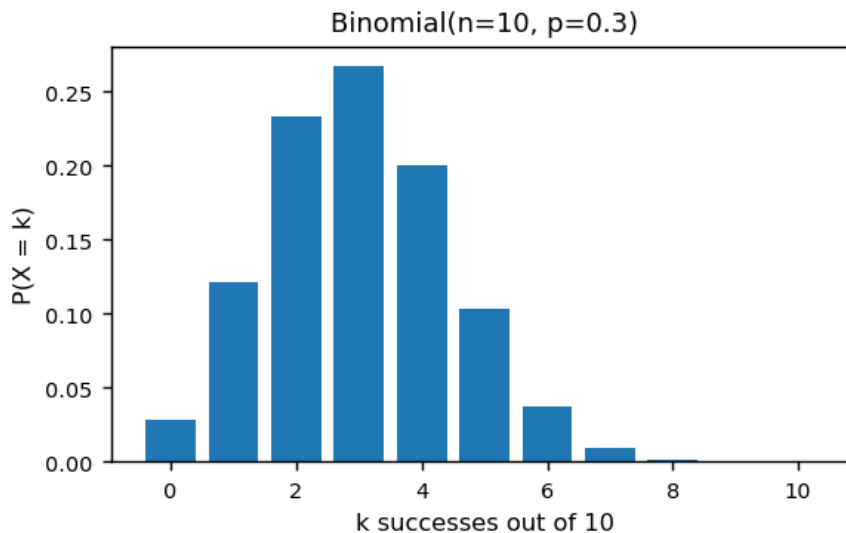
where $C(n, k) = n! / (k! (n - k)!)$ is the number of ways to choose which k of the n trials succeed. Mean: np . Variance: $np(1 - p)$.

```
n, p = 10, 0.3
X = stats.binom(n, p)
print("P(X = 3):", X.pmf(3).round(4))
print("P(X <= 3):", X.cdf(3).round(4))
print("P(X >= 5):", X.sf(4).round(4))
print("mean, var:", X.mean(), X.var().round(2))

k = np.arange(0, n + 1)
fig, ax = plt.subplots(figsize=(5, 2.8))
ax.bar(k, X.pmf(k), color="tab:blue")
ax.set_xlabel("k successes out of 10"); ax.set_ylabel("P(X = k)")
ax.set_title("Binomial(n=10, p=0.3)")
plt.show()
```

OUTPUT

```
P(X = 3): 0.2668
P(X <= 3): 0.6496
P(X >= 5): 0.1503
mean, var: 3.0 2.1
```



Example: if a drug works for 30% of patients, the chance that 5 or more out of 10 respond is about 15%.

Poisson

The **Poisson** distribution counts **events in a fixed interval** when events occur independently at a constant average rate λ : photons hitting a detector per second, mutations per gene, calls per hour.

$$P(X = k) = \lambda^k e^{-\lambda} / k!$$

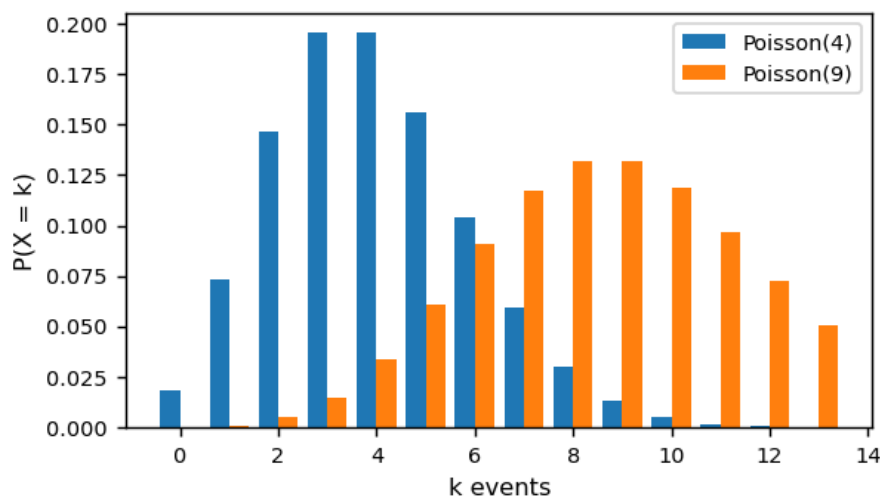
Mean = variance = λ . That equality is a useful diagnostic: if the variance is much larger than the mean the data is "overdispersed" and Poisson does not fit.

```
lam = 4
P = stats.poisson(lam)
print("P(X = 0):", P.pmf(0).round(4))
print("P(X >= 8):", P.sf(7).round(4))

k = np.arange(0, 14)
fig, ax = plt.subplots(figsize=(5, 2.8))
ax.bar(k - 0.2, P.pmf(k), width=0.4, label="Poisson(4)")
ax.bar(k + 0.2, stats.poisson(9).pmf(k), width=0.4, label="Poisson(9)")
ax.legend(); ax.set_xlabel("k events"); ax.set_ylabel("P(X = k)")
plt.show()
```

OUTPUT

```
P(X = 0): 0.0183
P(X >= 8): 0.0511
```



13.4 Continuous distributions

Uniform

Every value in $[a, b]$ is equally likely. Random number generators produce $\text{Uniform}(0, 1)$.

Normal (Gaussian)

The **normal distribution** is the famous bell curve, described by its mean μ (centre) and standard deviation σ (width):

$$f(x) = (1 / (\sigma \sqrt{2\pi})) \cdot \exp(- (x - \mu)^2 / (2\sigma^2))$$

It appears everywhere because sums and averages of many small independent effects tend towards it (see the Central Limit Theorem below). The **68-95-99.7 rule**: about 68% of values lie within 1σ of the mean, 95% within 2σ and 99.7% within 3σ . The **standard normal** has $\mu = 0$ and $\sigma = 1$.

```
Z = stats.norm(0, 1)
print("P(-1 < Z < 1):", (Z.cdf(1) - Z.cdf(-1)).round(4))
print("P(-2 < Z < 2):", (Z.cdf(2) - Z.cdf(-2)).round(4))
print("P(-3 < Z < 3):", (Z.cdf(3) - Z.cdf(-3)).round(4))
print("97.5th percentile:", Z.ppf(0.975).round(3)) # the famous 1.96

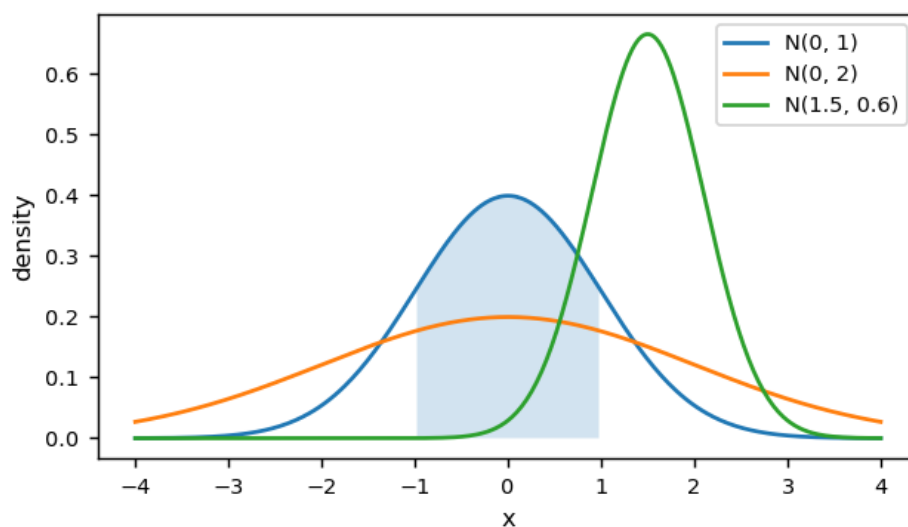
# IQ scores: mean 100, std 15. What fraction is above 130?
iq = stats.norm(100, 15)
```

```
print("P(IQ > 130):", iq.sf(130).round(4))
print("score needed for top 5%:", iq.ppf(0.95).round(1))

x = np.linspace(-4, 4, 300)
fig, ax = plt.subplots(figsize=(5.5, 3))
ax.plot(x, Z.pdf(x), label="N(0, 1)")
ax.plot(x, stats.norm(0, 2).pdf(x), label="N(0, 2)")
ax.plot(x, stats.norm(1.5, 0.6).pdf(x), label="N(1.5, 0.6)")
ax.fill_between(x, Z.pdf(x), where=(x > -1) & (x < 1), alpha=0.2)
ax.set_xlabel("x"); ax.set_ylabel("density")
ax.legend()
plt.show()
```

OUTPUT

```
P(-1 < Z < 1): 0.6827
P(-2 < Z < 2): 0.9545
P(-3 < Z < 3): 0.9973
97.5th percentile: 1.96
P(IQ > 130): 0.0228
score needed for top 5%: 124.7
```



Exponential

The **exponential** distribution models the **waiting time** between Poisson events: time until the next radioactive decay, the lifetime of a component. With rate λ : $f(x) = \lambda e^{-\lambda x}$, mean = $1/\lambda$. It is **memoryless**: having waited already does not change the remaining waiting time.

```
decay = stats.expon(scale=1 / 0.5)          # rate 0.5 per second -> mean wait 2 s
print("mean wait:", decay.mean())
print("P(wait > 3 s):", decay.sf(3).round(4))
print("median wait:", decay.median().round(3))
```

OUTPUT

```
mean wait: 2.0
P(wait > 3 s): 0.2231
median wait: 1.386
```

Distributions used for inference: t, chi-square and F

These arise when we *estimate* quantities from samples. You will meet them in every test in Chapter 15.

Distribution	Where it comes from	Used for
Student's t (df)	$(\bar{x} - \mu) / (s/\sqrt{n})$ when σ is estimated	t-tests, confidence intervals for means

Distribution	Where it comes from	Used for
Chi-square χ^2 (df)	sum of squared standard normals	variance tests, goodness of fit, independence tests
F (df1, df2)	ratio of two scaled chi-squares	ANOVA, comparing variances, regression tests

The t distribution looks like the normal but with **heavier tails** (more uncertainty when the sample is small); as the degrees of freedom grow it becomes the normal.

```
x = np.linspace(-5, 5, 300)
fig, axes = plt.subplots(1, 3, figsize=(10, 2.8))
axes[0].plot(x, stats.norm.pdf(x), "k--", label="normal")
for df_ in [2, 5, 30]:
    axes[0].plot(x, stats.t.pdf(x, df_), label=f"t, df={df_}")
axes[0].legend(); axes[0].set_title("Student t")

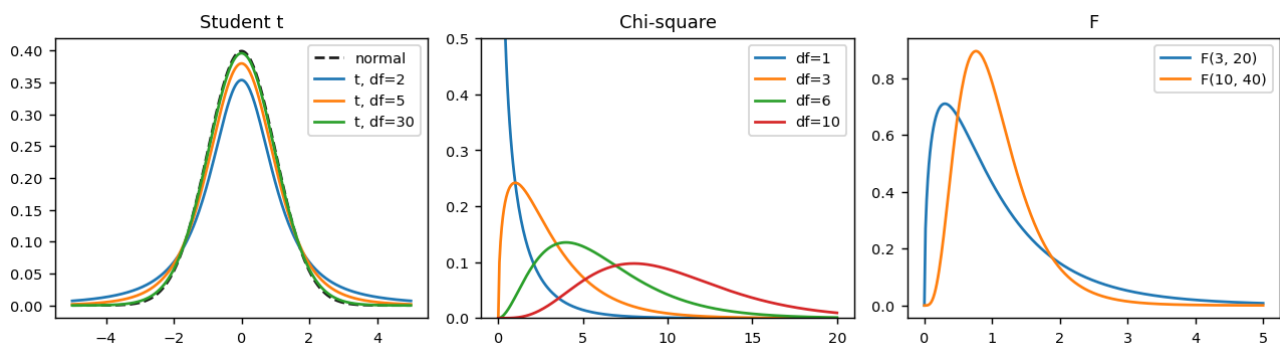
x2 = np.linspace(0, 20, 300)
for df_ in [1, 3, 6, 10]:
    axes[1].plot(x2, stats.chi2.pdf(x2, df_), label=f"df={df_}")
axes[1].set_ylim(0, 0.5); axes[1].legend(); axes[1].set_title("Chi-square")

x3 = np.linspace(0, 5, 300)
axes[2].plot(x3, stats.f.pdf(x3, 3, 20), label="F(3, 20)")
axes[2].plot(x3, stats.f.pdf(x3, 10, 40), label="F(10, 40)")
axes[2].legend(); axes[2].set_title("F")
plt.tight_layout()
plt.show()

print("t critical values for 95% two-sided:")
for df_ in [2, 5, 10, 30, 1000]:
    print(f" df = {df_:>4}: {stats.t.ppf(0.975, df_):.3f}")
```

OUTPUT

```
t critical values for 95% two-sided:
df =    2: 4.303
df =    5: 2.571
df =   10: 2.228
df =   30: 2.042
df = 1000: 1.962
```



As df grows, the critical value approaches the normal's 1.96.

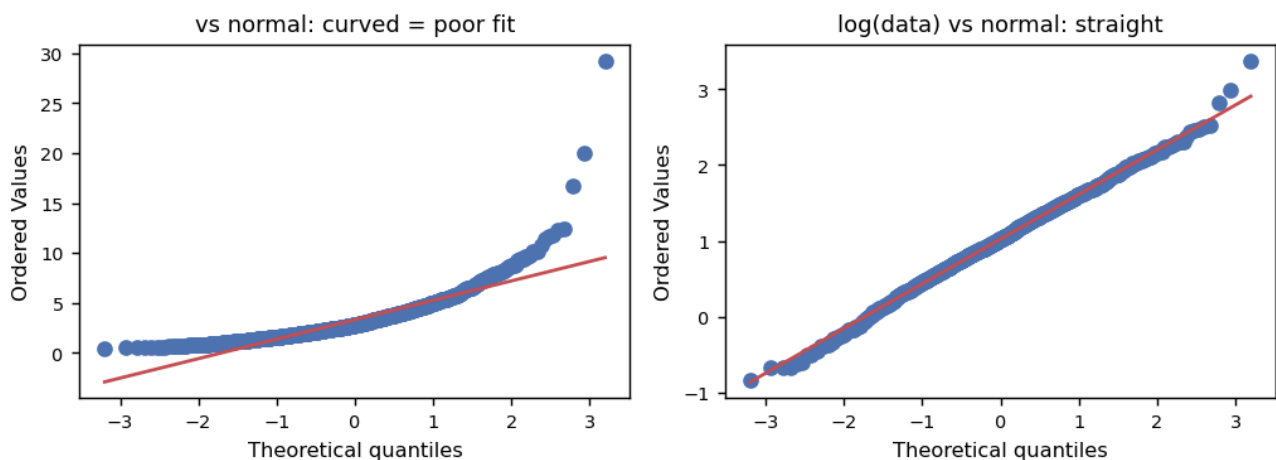
13.5 Which distribution fits my data?

Data looks like...	Consider
Counts of successes out of a fixed number	binomial
Counts of rare events per time or space	Poisson
Symmetric, bell-shaped measurements	normal
Positive, right-skewed (sizes, concentrations)	log-normal, gamma
Waiting times between random events	exponential, Weibull
Proportions between 0 and 1	beta
Heavy tails and outliers	t, Cauchy

```
data = stats.lognorm(s=0.6, scale=np.exp(1)).rvs(1000, random_state=1)
shape, loc, scale = stats.lognorm.fit(data, floc=0) # maximum likelihood fit
print("fitted shape, scale:", round(shape, 3), round(scale, 3))

# Q-Q plot: points follow the line if the distribution fits
fig, axes = plt.subplots(1, 2, figsize=(8, 3))
stats.probplot(data, dist="norm", plot=axes[0])
axes[0].set_title("vs normal: curved = poor fit")
stats.probplot(np.log(data), dist="norm", plot=axes[1])
axes[1].set_title("log(data) vs normal: straight")
plt.tight_layout()
plt.show()
```

OUTPUT
fitted shape, scale: 0.589 2.782



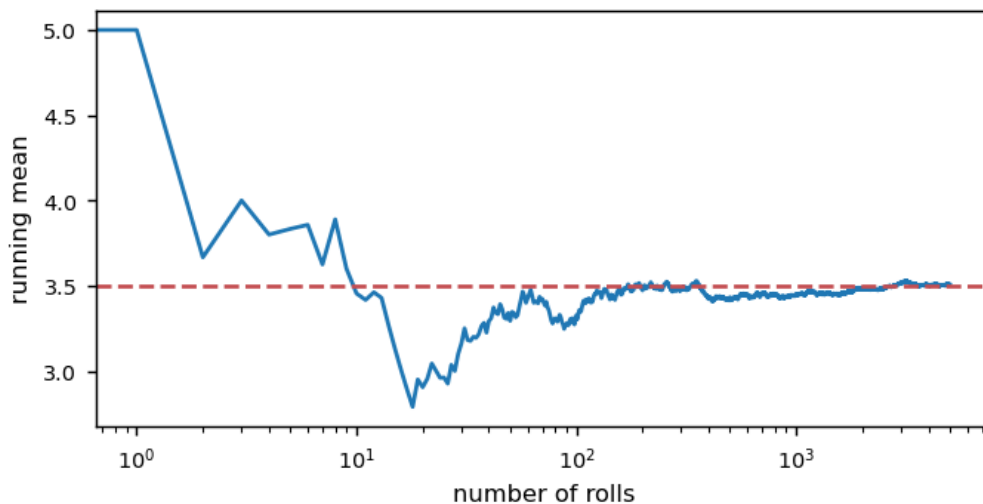
A **Q-Q plot** compares your data's quantiles with the theoretical quantiles: points along the straight line indicate a good match.

13.6 The Law of Large Numbers

The **Law of Large Numbers** says the sample mean gets closer to the true mean as the sample grows. Simulation shows it beautifully:

```
rng = np.random.default_rng(5)
rolls = rng.integers(1, 7, 5000) # fair die, true mean 3.5
running_mean = np.cumsum(rolls) / np.arange(1, 5001)

fig, ax = plt.subplots(figsize=(6, 2.8))
ax.plot(running_mean)
ax.axhline(3.5, color="r", linestyle="--")
ax.set_xscale("log"); ax.set_xlabel("number of rolls"); ax.set_ylabel("running mean")
plt.show()
```



13.7 The Central Limit Theorem (CLT)

The **Central Limit Theorem** is arguably the most important result in statistics:

THE MATH BEHIND IT

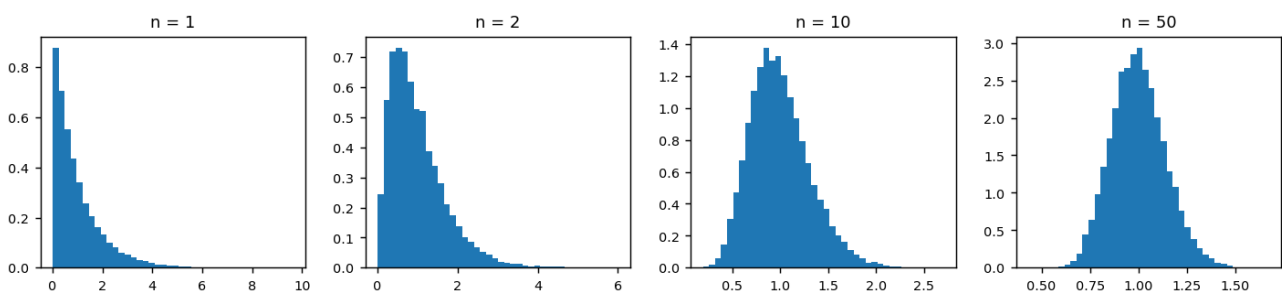
Take many independent samples of size n from **any** population with mean μ and standard deviation σ (finite). The distribution of the **sample means** approaches a normal distribution with mean μ and standard deviation $\sigma / \sqrt{n}$ as n grows, regardless of the population's shape.

This is why the normal distribution is everywhere, and why we can build confidence intervals and tests for means even when the raw data is not normal. Let us watch it happen with a very non-normal population (exponential, strongly skewed):

```
rng = np.random.default_rng(7)
fig, axes = plt.subplots(1, 4, figsize=(11, 2.6), sharey=False)
for ax, n in zip(axes, [1, 2, 10, 50]):
    means = rng.exponential(1.0, size=(20000, n)).mean(axis=1)
    ax.hist(means, bins=40, color="tab:blue", density=True)
    ax.set_title(f"n = {n}")
    print(f"n = {n}: std of means = {means.std():.3f}    theory 1/sqrt(n) = {1/np.sqrt(n):.3f}")
plt.tight_layout()
plt.show()
```

OUTPUT

```
n = 1: std of means = 1.008    theory 1/sqrt(n) = 1.000
n = 2: std of means = 0.698    theory 1/sqrt(n) = 0.707
n = 10: std of means = 0.313    theory 1/sqrt(n) = 0.316
n = 50: std of means = 0.140    theory 1/sqrt(n) = 0.141
```



Notice the histogram becoming a symmetric bell and the spread shrinking as $1/\sqrt{n}$. Four times the data only halves the uncertainty, a fact that sets the cost of precision in experiments.

13.8 Fitting and sampling recipe

```
rng = np.random.default_rng(11)
sample = rng.normal(loc=20, scale=3, size=200)
mu_hat, sigma_hat = stats.norm.fit(sample)          # estimates of the parameters
print(round(mu_hat, 2), round(sigma_hat, 2))
print("P(sample value > 25):", stats.norm(mu_hat, sigma_hat).sf(25).round(4))
print("empirical fraction :", (sample > 25).mean())
```

OUTPUT

```
20.05 2.77
P(sample value > 25): 0.0368
empirical fraction : 0.04
```

13.9 Chapter summary

CHAPTER SUMMARY

- A distribution tells which values a random variable takes and how often: PMF (discrete), PDF (continuous), CDF (cumulative).
- Binomial: successes in n trials; Poisson: event counts (mean = variance = λ); normal: bell curve; exponential: waiting times; t , χ^2 and F : inference.
- `scipy.stats` methods `pmf/pdf`, `cdf`, `sf`, `ppf`, `rvs`, `fit` work for all distributions.
- Check fit with histograms and Q-Q plots; log-transform for right skew.
- Law of Large Numbers: sample means converge to the truth. CLT: means are approximately normal with spread $\sigma/\sqrt{n}$.

13.10 Practice exercises

1. A fair coin is flipped 20 times. Find $P(\text{exactly 10 heads})$ and $P(\text{at least 15 heads})$.
2. A detector counts on average 6 particles per minute. What is the probability of zero counts in a minute? Of more than 10?
3. Heights are normal with mean 170 cm and standard deviation 8 cm. What height marks the tallest 10 percent?
4. Verify the CLT with samples from a uniform distribution on $[0, 1]$: what is the expected standard deviation of the mean of $n = 25$?
5. Draw 500 values from a gamma distribution and produce a Q-Q plot against the normal. Is it straight?

CHAPTER 14

14. Sampling, Estimation and Confidence Intervals

IN THIS CHAPTER YOU WILL LEARN

- Population versus sample, and why sampling method matters
- Point estimates, bias and the standard error
- Confidence intervals for a mean and a proportion, and what they really mean
- The bootstrap: confidence intervals for anything
- Choosing a sample size; the idea of maximum likelihood

14.1 Population and sample

The **population** is the whole group we care about (every fish in a lake, every measurement a detector could make). The **sample** is the part we actually measure. A number computed from the population is a **parameter** (μ , σ , p); one computed from the sample is a **statistic** ($\bar{x}$, s , $\hat{p}$). Statistics *estimate* parameters, and because samples vary, every estimate carries uncertainty. Quantifying that uncertainty is the heart of inference.

Quantity	Population (true, usually unknown)	Sample (computed)
Mean	μ	$\bar{x}$
Standard deviation	σ	s
Proportion	p	$\hat{p}$
Size	N	n

Sampling methods

Method	Idea	Risk
Simple random	every unit equally likely to be chosen	needs a list of the population
Stratified	split into groups (strata), sample within each	need group labels; gives precise estimates
Cluster	randomly choose groups, measure all inside	less precise, cheaper
Systematic	every k-th unit	hidden periodic patterns
Convenience	whoever is easy to reach	biased ; unrepresentative

WATCH OUT

No amount of clever statistics repairs a biased sample. A survey of people walking past a gym says little about the general population's fitness. The key is **random selection** and **independence**.

14.2 The sampling distribution and the standard error

If we repeatedly drew samples and computed the mean each time, the means would form the **sampling distribution of the mean**. By the Central Limit Theorem it is approximately normal, centred on μ , with spread

$$SE = \sigma / \sqrt{n} \quad \text{estimated by} \quad SE = s / \sqrt{n}$$

called the **standard error**. Let us see this with a simulated population where we know the truth.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

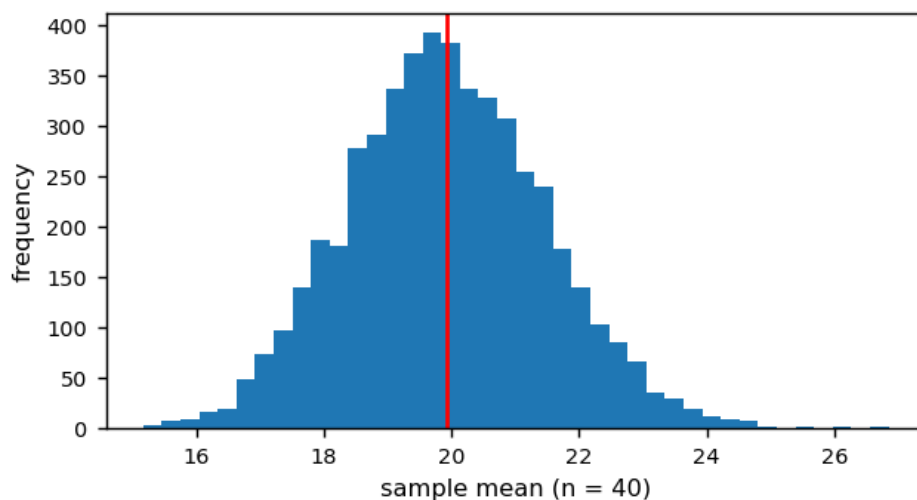
rng = np.random.default_rng(1)
population = rng.gamma(shape=4, scale=5, size=100000)      # skewed population
mu, sigma = population.mean(), population.std()
print(f"population mean {mu:.2f}, sd {sigma:.2f}")

n = 40
means = np.array([rng.choice(population, n).mean() for _ in range(5000)])
print("mean of sample means  :", means.mean().round(2))
print("sd of sample means    :", means.std().round(3))
print("theory sigma/sqrt(n)  :", round(sigma / np.sqrt(n), 3))

fig, ax = plt.subplots(figsize=(5.5, 2.8))
ax.hist(means, bins=40, color="tab:blue")
ax.axvline(mu, color="red")
ax.set_xlabel("sample mean (n = 40)"); ax.set_ylabel("frequency")
plt.show()
```

OUTPUT

```
population mean 19.93, sd 9.95
mean of sample means : 19.92
sd of sample means   : 1.544
theory sigma/sqrt(n) : 1.574
```



14.3 Point estimates and bias

A **point estimate** is a single best guess ($\bar{x}$ for μ). An estimator is **unbiased** if its average over many samples equals the true parameter: $\bar{x}$ is unbiased for μ ; s^2 with $n - 1$ is unbiased for σ^2 ; s^2 with n is biased (Chapter 12).

14.4 Confidence intervals

A point estimate alone says nothing about precision. A **confidence interval** (CI) gives a range that is likely to contain the true parameter.

For a mean, when σ is unknown (the usual case) use the **t distribution**:

$$\bar{x} \pm t_c \times s / \sqrt{n}$$

Here t_c is the critical value from the t distribution with $n - 1$ degrees of freedom that leaves 2.5% in each tail for a 95% interval. The piece $t_c \times SE$ is the **margin of error**.

```
sample = np.array([9.8, 10.4, 10.1, 9.7, 10.6, 10.2, 9.9, 10.3, 10.0, 10.5])
n = len(sample)
mean = sample.mean()
se = sample.std(ddof=1) / np.sqrt(n)
t_crit = stats.t.ppf(0.975, df=n - 1)
low, high = mean - t_crit * se, mean + t_crit * se
print(f"mean = {mean:.3f}, SE = {se:.3f}, t* = {t_crit:.3f}")
print(f"95% CI = ({low:.3f}, {high:.3f})")

# the same in one call
print(stats.t.interval(0.95, df=n - 1, loc=mean, scale=se))
```

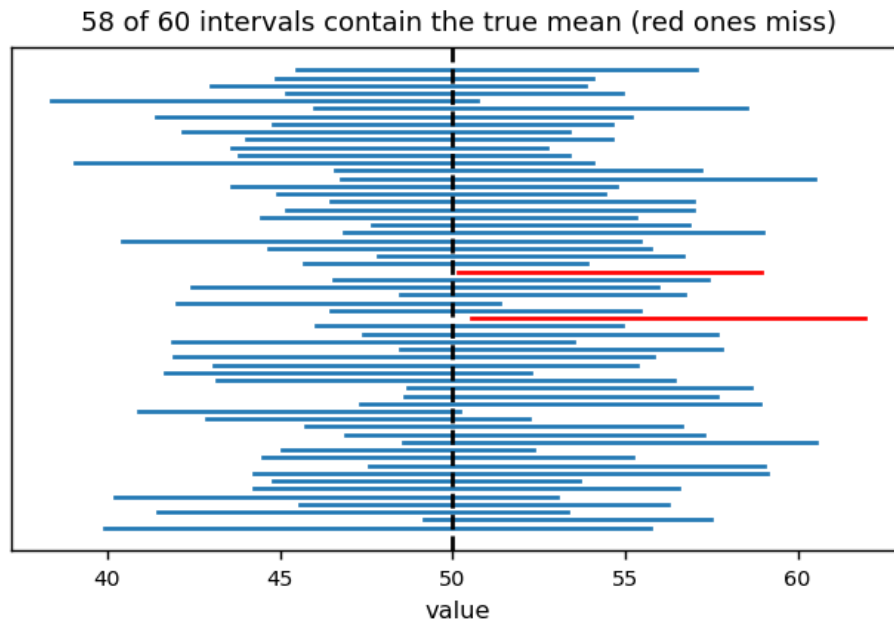
OUTPUT

```
mean = 10.150, SE = 0.096, t* = 2.262
95% CI = (9.933, 10.367)
(np.float64(9.933414941033183), np.float64(10.366585058966818))
```

What does "95% confident" mean?

It does **not** mean "there is a 95% probability that μ is in this particular interval". μ is a fixed number; the interval is what varies. The correct meaning is about the **procedure**: if we repeated the whole experiment many times and built an interval each time, about 95% of those intervals would contain the true mean. Simulation makes this concrete:

```
rng = np.random.default_rng(3)
true_mu, true_sd, n = 50, 10, 15
hits = 0
fig, ax = plt.subplots(figsize=(6, 3.4))
for i in range(60):
    s = rng.normal(true_mu, true_sd, n)
    se = s.std(ddof=1) / np.sqrt(n)
    lo, hi = stats.t.interval(0.95, n - 1, loc=s.mean(), scale=se)
    covered = lo <= true_mu <= hi
    hits += covered
    ax.plot([lo, hi], [i, i], color="tab:blue" if covered else "red")
ax.axvline(true_mu, color="black", linestyle="--")
ax.set_yticks([]); ax.set_xlabel("value")
ax.set_title(f"{hits} of 60 intervals contain the true mean (red ones miss)")
plt.show()
```



What controls the width?

$$\text{width} \propto t_c \times s / \sqrt{n}$$

- **Higher confidence** (99% instead of 95%) gives a **wider** interval.
- **More data** (larger n) gives a **narrower** interval, shrinking as $1/\sqrt{n}$.
- **More variable data** (larger s) gives a **wider** interval.

```
for n in [5, 20, 80, 320]:
    half = stats.t.ppf(0.975, n - 1) * 10 / np.sqrt(n)
    print(f"n = {n:>3}: margin of error = +/- {half:.2f}")
```

OUTPUT

```
n = 5: margin of error = +/- 12.42
n = 20: margin of error = +/- 4.68
n = 80: margin of error = +/- 2.23
n = 320: margin of error = +/- 1.10
```

Quadrupling the sample size halves the margin of error.

14.5 Confidence interval for a proportion

If x out of n subjects show a trait, $\hat{p} = x/n$. The simple (Wald) interval is $\hat{p} \pm z_c \sqrt{\hat{p}(1 - \hat{p})/n}$. It works badly for small n or $\hat{p}$ near 0 or 1; the **Wilson** interval is better. `scipy` provides it through the binomial test result.

```
x, n = 18, 60
p_hat = x / n
z = stats.norm.ppf(0.975)
se = np.sqrt(p_hat * (1 - p_hat) / n)
print(f"p-hat = {p_hat:.3f}, Wald 95% CI = ({p_hat - z*se:.3f}, {p_hat + z*se:.3f})")

res = stats.binomtest(x, n)
print("Wilson CI :", res.proportion_ci(confidence_level=0.95, method="wilson"))
print("Exact CI :", res.proportion_ci(confidence_level=0.95, method="exact"))
```

OUTPUT

```
p-hat = 0.300, Wald 95% CI = (0.184, 0.416)
Wilson CI : ConfidenceInterval(low=np.float64(0.19898166138769108), high=np.float64(0.425087079469
```

```
14487))
Exact CI : ConfidenceInterval(low=0.18845144541969047, high=0.4320831338760616)
```

14.6 The bootstrap: intervals without formulas

Formulas exist for means and proportions, but what about the median, a correlation or a ratio of two means? The **bootstrap** (Bradley Efron, 1979) handles all of them with one idea: treat your sample as if it were the population, and **resample from it with replacement** many times.

Recipe

1. Draw n values from your sample **with replacement** (some values repeat, some are left out).
2. Compute your statistic on this new "bootstrap sample".
3. Repeat thousands of times.
4. The spread of the bootstrap statistics approximates the spread you would see from real repeated sampling. The 2.5th and 97.5th percentiles give a 95% **percentile interval**.

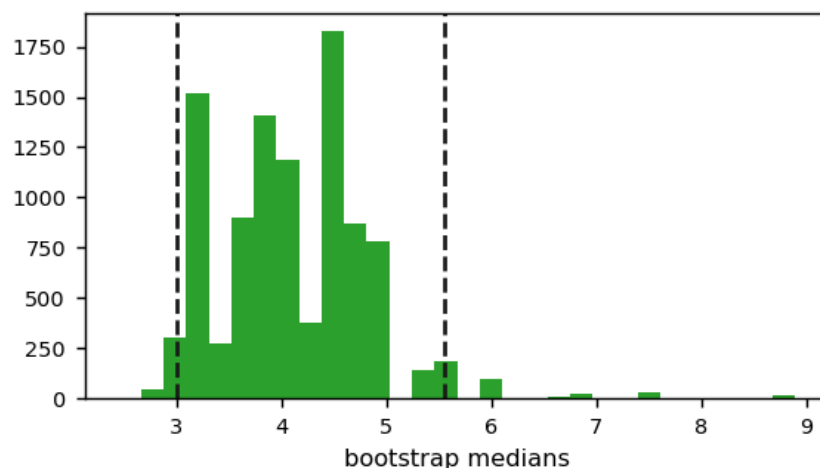
```
rng = np.random.default_rng(42)
data = np.array([3.1, 4.7, 2.2, 8.9, 3.8, 5.0, 2.9, 12.4, 4.4, 3.3, 6.1, 2.7])
print("sample median:", np.median(data))

boot = np.array([np.median(rng.choice(data, size=len(data), replace=True))
                 for _ in range(10000)])
lo, hi = np.percentile(boot, [2.5, 97.5])
print(f"bootstrap 95% CI for the median: ({lo:.2f}, {hi:.2f})")
print("bootstrap standard error:", boot.std().round(3))

fig, ax = plt.subplots(figsize=(5, 2.6))
ax.hist(boot, bins=30, color="tab:green")
ax.axvline(lo, color="k", linestyle="--"); ax.axvline(hi, color="k", linestyle="--")
ax.set_xlabel("bootstrap medians")
plt.show()
```

OUTPUT

```
sample median: 4.1
bootstrap 95% CI for the median: (3.00, 5.55)
bootstrap standard error: 0.723
```



SciPy has a ready-made function:

```
res = stats.bootstrap((data,), np.median, confidence_level=0.95,
                     n_resamples=5000, random_state=1, method="BCa")
print(res.confidence_interval)
```

OUTPUT

```
ConfidenceInterval(low=np.float64(3.0), high=np.float64(5.55))
```

BCa (bias-corrected and accelerated) is an improved version of the percentile method. The bootstrap works best for moderately large samples; it cannot create information that your sample does not contain, so it fails for very tiny samples and for extreme statistics such as the maximum.

Bootstrapping a correlation

```
rng = np.random.default_rng(0)
x = rng.normal(size=30)
y = 0.6 * x + rng.normal(scale=0.8, size=30)
r_obs = np.corrcoef(x, y)[0, 1]
idx = rng.integers(0, 30, size=(5000, 30)) # resample row positions together!
r_boot = np.array([np.corrcoef(x[i], y[i])[0, 1] for i in idx])
print(f"r = {r_obs:.2f}, 95% CI = {np.percentile(r_boot, [2.5, 97.5]).round(2)}")
```

OUTPUT

```
r = 0.32, 95% CI = [0.06 0.55]
```

Notice that we resample **pairs** (rows) so that x and y stay matched.

14.7 How large a sample do I need?

For a mean with desired margin of error E and (estimated) standard deviation σ :

$$n \approx (z_c \times \sigma / E)^2$$

For a proportion, use $n \approx z_c^2 p(1 - p) / E^2$, with $p = 0.5$ as the cautious worst case.

```
z = stats.norm.ppf(0.975)
sigma, E = 12.0, 2.0
print("n for a mean (sigma = 12, E = 2):", int(np.ceil((z * sigma / E) ** 2)))
E = 0.03
print("n for a proportion (E = 3%)      :", int(np.ceil(z ** 2 * 0.25 / E ** 2)))
```

OUTPUT

```
n for a mean (sigma = 12, E = 2): 139
n for a proportion (E = 3%)      : 1068
```

This is why opinion polls of about 1000 people give about $\pm 3\%$.

14.8 Maximum likelihood: how estimates are "derived"

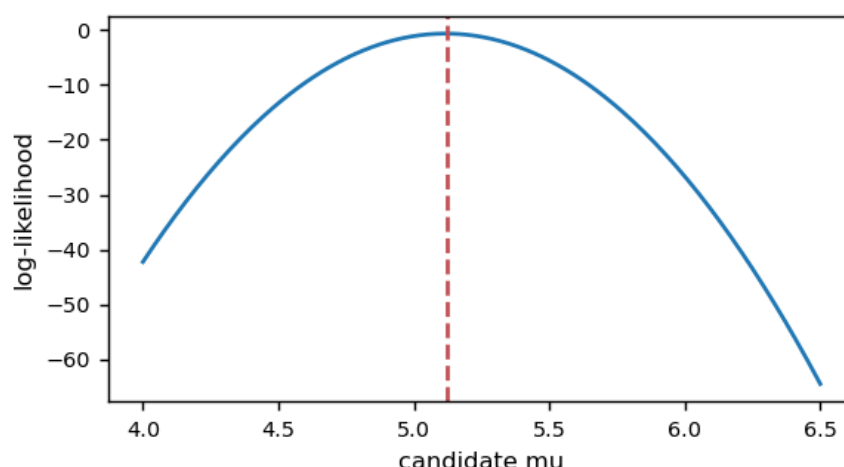
How do we know $\bar{x}$ is the right estimate of a normal mean? **Maximum likelihood estimation (MLE)** picks the parameter value that makes the observed data **most probable**. The **likelihood** $L(\theta)$ is the probability (density) of the data as a function of the parameter θ . Computers usually maximise the log-likelihood, because products of many small numbers become sums.

```
data = np.array([4.8, 5.1, 5.6, 4.9, 5.3, 5.0])
mus = np.linspace(4, 6.5, 251)
loglik = np.array([stats.norm(m, 0.3).logpdf(data).sum() for m in mus])
best = mus[np.argmax(loglik)]
print("MLE of mu:", round(best, 3), " sample mean:", round(data.mean(), 3))
```

```
fig, ax = plt.subplots(figsize=(5, 2.6))
ax.plot(mus, loglik)
ax.axvline(best, color="r", linestyle="--")
ax.set_xlabel("candidate mu"); ax.set_ylabel("log-likelihood")
plt.show()
```

OUTPUT

```
MLE of mu: 5.12 sample mean: 5.117
```



The peak sits exactly at the sample mean. MLE is the engine behind regression, logistic regression and many scientific model fits (Chapters 16 and 21).

14.9 Chapter summary

CHAPTER SUMMARY

- Samples estimate populations; random sampling avoids bias; estimates have a standard error $SE = s/\sqrt{n}$.
- A 95% CI is $\bar{x} \pm t^* \times SE$. The confidence level describes the *procedure*: 95% of such intervals catch the true value.
- More data narrows the interval by $1/\sqrt{n}$; higher confidence or noisier data widen it.
- The bootstrap resamples the data with replacement to get intervals for any statistic.
- Sample size for a margin E : $n \approx (z \sigma / E)^2$. MLE finds the parameters that make the data most probable.

14.10 Practice exercises

1. Measurements [12.1, 11.8, 12.5, 12.0, 12.3, 11.9]: compute the 90%, 95% and 99% confidence intervals for the mean and comment on the widths.
2. In a clinical audit 45 of 200 patients reported side effects. Give a 95% Wilson interval.
3. Bootstrap the standard deviation of a dataset of your choice and compare the interval with the normal-theory formula.
4. How large a sample is needed to estimate a mean to ± 0.5 when σ is about 4?
5. Show by simulation that the 95% t-interval covers the truth about 95% of the time even for skewed data when $n = 50$, but less well when $n = 5$.

CHAPTER 15

15. Hypothesis Testing

IN THIS CHAPTER YOU WILL LEARN

- The logic of a statistical test: null hypothesis, p-value, significance level
- Type I and type II errors, power, effect size
- One-sample, two-sample and paired t-tests
- ANOVA and post-hoc comparisons; chi-square tests; non-parametric tests
- Checking assumptions; permutation tests; multiple comparisons

15.1 The logic of hypothesis testing

You have data showing the average of a treated group is higher than the average of a control group. Is the treatment effective, or could the difference be **random luck** from sampling? Hypothesis testing answers this with a proof-by-contradiction style of reasoning.

1. State the **null hypothesis H_0** : "there is no effect" (for example, the two means are equal). State the **alternative H_1** : "there is an effect".
2. Choose a **significance level α** , commonly 0.05: the risk of a false alarm you will tolerate.
3. Compute a **test statistic** that measures how far the data is from what H_0 predicts, in standard-error units.
4. Compute the **p-value**: the probability of seeing a result at least this extreme **if H_0 were true**.
5. If $p \leq \alpha$, **reject H_0** (the result is "statistically significant"). Otherwise, **fail to reject** it, which is not the same as proving it true.

WATCH OUT

What a p-value is NOT. It is not the probability that H_0 is true, nor the probability the result happened by chance in a single-dataset sense, and not the size or importance of the effect. It is $P(\text{data this extreme} \mid H_0)$. A tiny effect can have a tiny p-value with a huge sample; a large effect can have a large p-value with a tiny sample. Always report the **effect size and a confidence interval** with the p-value.

The two kinds of mistake

	H_0 true (no effect)	H_0 false (real effect)
Reject H_0	Type I error (false positive), probability α	correct decision, probability = power ($1 - \beta$)
Do not reject	correct	Type II error (false negative), probability β

Power is the chance of detecting a real effect. It grows with larger effects, larger samples, lower noise and higher α . Aim for at least 80%.

15.2 A simulation to build intuition

Imagine flipping a coin 100 times and getting 62 heads. Is the coin fair? Simulate what a fair coin does.

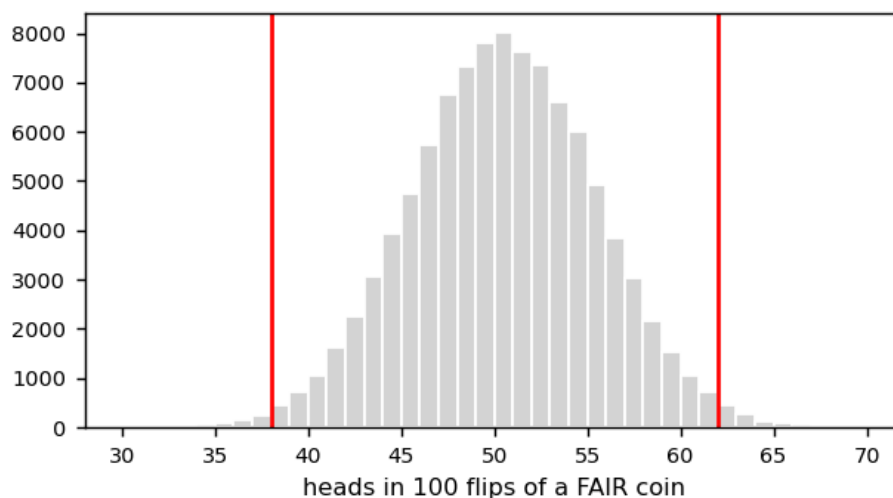
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(0)
fair = rng.binomial(n=100, p=0.5, size=100000) # heads in 100 flips, 100,000 times
p_sim = np.mean(np.abs(fair - 50) >= 12) # as extreme as 62 in either direction
print("simulated two-sided p-value:", p_sim)
print("exact binomial test p-value:", round(stats.binomtest(62, 100, 0.5).pvalue, 4))

fig, ax = plt.subplots(figsize=(5.5, 2.8))
ax.hist(fair, bins=range(30, 71), color="lightgrey", edgecolor="white")
ax.axvline(62, color="red"); ax.axvline(38, color="red")
ax.set_xlabel("heads in 100 flips of a FAIR coin")
plt.show()
```

OUTPUT

```
simulated two-sided p-value: 0.02109
exact binomial test p-value: 0.021
```



The p-value (about 0.02) is the fraction of fair-coin worlds that look at least as odd as ours. It is small, so we doubt the coin is fair.

15.3 The t-tests

One-sample t-test

Does a sample come from a population with a known mean μ_0 ?

$$t = (\bar{x} - \mu_0) / (s / \sqrt{n})$$

The statistic t is the difference measured in standard errors. Under H_0 it follows the t distribution with $n - 1$ degrees of freedom.

```
# A supplier claims bolts average 50.0 mm. We measure 12 of them.
bolts = np.array([50.3, 50.8, 49.9, 50.6, 50.4, 50.9, 50.2, 50.7, 50.1, 50.5, 50.6, 50.4])
res = stats.ttest_lsamp(bolts, popmean=50.0)
print("t =", round(res.statistic, 3), " p =", round(res.pvalue, 5), " df =", res.df)
print("95% CI of the mean:", res.confidence_interval(0.95))
```

OUTPUT

```
t = 5.304  p = 0.00025  df = 11
95% CI of the mean: ConfidenceInterval(low=np.float64(50.263279521598406), high=np.float64(50.636720478401585))
```

Two-sample (independent) t-test

Compare the means of two separate groups.

$$t = (\bar{x}_1 - \bar{x}_2) / \sqrt{(s_1^2/n_1 + s_2^2/n_2)}$$

The default version in SciPy is the standard pooled test; **Welch's t-test** (`equal_var=False`) does not assume equal variances and is the safer default.

```
rng = np.random.default_rng(1)
control = rng.normal(100, 15, 30)
treated = rng.normal(110, 15, 30)

res = stats.ttest_ind(treated, control, equal_var=False) # Welch
print(f"means: {treated.mean():.1f} vs {control.mean():.1f}")
print(f"Welch t = {res.statistic:.3f}, p = {res.pvalue:.4f}")
print("95% CI of difference:", np.round(res.confidence_interval(0.95), 2))
```

OUTPUT

```
means: 109.5 vs 99.0
Welch t = 3.093, p = 0.0031
95% CI of difference: [ 3.71 17.32]
```

Effect size: how big is the difference?

Cohen's d expresses a difference in units of pooled standard deviation:

$$d = (\bar{x}_1 - \bar{x}_2) / s_{pooled}$$

Rough guide: 0.2 small, 0.5 medium, 0.8 large (context matters!).

```
def cohens_d(a, b):
    na, nb = len(a), len(b)
    pooled = np.sqrt(((na - 1) * a.var(ddof=1) + (nb - 1) * b.var(ddof=1)) / (na + nb - 2))
    return (a.mean() - b.mean()) / pooled

print("Cohen's d:", round(cohens_d(treated, control), 2))
```

OUTPUT

```
Cohen's d: 0.8
```

Paired t-test

When the two measurements come from the **same subjects** (before and after), analyse the **differences**. This removes person-to-person variation and is far more powerful.

```
before = np.array([120, 135, 142, 128, 150, 138, 145, 130])
after = np.array([115, 130, 140, 120, 141, 133, 139, 128])

res = stats.ttest_rel(after, before)
print("mean difference:", (after - before).mean())
print(f"paired t = {res.statistic:.3f}, p = {res.pvalue:.4f}")

# Equivalent: a one-sample test on the differences
print(stats.ttest_1samp(after - before, 0).pvalue)
```

OUTPUT

```
mean difference: -5.25
paired t = -5.957, p = 0.0006
0.0005662259802804029
```

One-sided versus two-sided

A **two-sided** test asks "is there a difference in either direction?" A **one-sided** test asks about one direction only (`alternative="greater"` or `"less"`). Decide **before** looking at the data; choosing a direction afterwards is cheating.

```
print(stats.ttest_ind(treated, control, alternative="greater", equal_var=False).pvalue)
```

OUTPUT
0.0015255453272094526

15.4 Checking assumptions

The t-tests assume (1) independent observations, (2) roughly normal data or a reasonably large n (thanks to the CLT), and, for the pooled version, (3) equal variances.

```
print("Shapiro-Wilk (H0: data are normal):", stats.shapiro(control))
print("Levene (H0: equal variances)      :", stats.levene(control, treated))

skewed = rng.exponential(2, 60)
print("Shapiro on skewed data p =", round(stats.shapiro(skewed).pvalue, 6))
```

OUTPUT
Shapiro-Wilk (H0: data are normal): ShapiroResult(statistic=np.float64(0.915672557773347), pvalue=np.float64(0.020742617537824873))
Levene (H0: equal variances) : LeveneResult(statistic=np.float64(0.5630420362506733), pvalue=np.float64(0.456070477430025))
Shapiro on skewed data p = 0.0

TIP

A formal normality test on a small sample has little power, and on a huge sample rejects trivial departures. Always **look** at histograms and Q-Q plots too. The t-test is fairly robust; heavy skew with small n , or outliers, are the real dangers.

15.5 Non-parametric tests

When assumptions fail, use **rank-based** tests. They replace values with their ranks and so ignore outliers and do not need normality.

Situation	Parametric test	Non-parametric alternative
One sample vs a value	one-sample t	Wilcoxon signed-rank
Two independent groups	two-sample t	Mann-Whitney U
Paired measurements	paired t	Wilcoxon signed-rank
3+ independent groups	one-way ANOVA	Kruskal-Wallis
Correlation	Pearson	Spearman / Kendall

```
a = np.array([1.2, 1.9, 2.3, 2.8, 3.1, 14.0]) # one wild value
b = np.array([2.9, 3.4, 3.8, 4.4, 5.1, 5.6])
print("t-test      p =", round(stats.ttest_ind(a, b).pvalue, 4))
print("Mann-Whitney p =", round(stats.mannwhitneyu(a, b).pvalue, 4))
print("Wilcoxon paired p =", round(stats.wilcoxon(after, before).pvalue, 4))
```

OUTPUT
t-test p = 0.9936
Mann-Whitney p = 0.0931
Wilcoxon paired p = 0.0078

15.6 Comparing more than two groups: ANOVA

Running many t-tests inflates the false-positive rate. **One-way ANOVA** (Analysis of Variance) tests all groups at once: H_0 says all group means are equal.

THE MATH BEHIND IT

The idea of ANOVA. Split the total variation into two parts. **Between-group** variation measures how far group means are from the overall mean; **within-group** variation measures how spread out the values are inside each group. If the groups truly differ, the between-group part is large *relative to* the within-group part. The **F statistic** is their ratio:

$F = (\text{between-group variance}) / (\text{within-group variance})$, with $(k - 1, N - k)$ degrees of freedom where k is the number of groups and N the total number of observations. A large F (small p) means the means are not all equal.

```
rng = np.random.default_rng(4)
g1 = rng.normal(20, 4, 25)
g2 = rng.normal(23, 4, 25)
g3 = rng.normal(27, 4, 25)

F, p = stats.f_oneway(g1, g2, g3)
print(f"F = {F:.2f}, p = {p:.5f}")

# compute F by hand to see what it is
groups = [g1, g2, g3]
allv = np.concatenate(groups)
grand = allv.mean()
ss_between = sum(len(g) * (g.mean() - grand) ** 2 for g in groups)
ss_within = sum(((g - g.mean()) ** 2).sum() for g in groups)
k, N = len(groups), len(allv)
F_hand = (ss_between / (k - 1)) / (ss_within / (N - k))
print("F by hand:", round(F_hand, 2))
print("Kruskal-Wallis p:", round(stats.kruskal(g1, g2, g3).pvalue, 5))
```

OUTPUT

```
F = 22.70, p = 0.00000
F by hand: 22.7
Kruskal-Wallis p: 0.0
```

ANOVA says *some* difference exists, not *which*. **Post-hoc tests** compare pairs while controlling the overall error rate. Tukey's HSD is the classic:

```
res = stats.tukey_hsd(g1, g2, g3)
print(res)
```

OUTPUT

```
Pairwise Group Comparisons (95.0% Confidence Interval)
Comparison  Statistic  p-value  Lower CI  Upper CI
(0 - 1)      -2.318    0.112    -5.050    0.413
(0 - 2)      -7.510    0.000   -10.241   -4.778
(1 - 0)       2.318    0.112    -0.413    5.050
(1 - 2)      -5.191    0.000    -7.923   -2.460
(2 - 0)       7.510    0.000     4.778   10.241
(2 - 1)       5.191    0.000     2.460    7.923
```

15.7 Chi-square tests for counts

Test of independence

Are two categorical variables related? The **chi-square test** compares observed counts O in a contingency table with counts E expected if the variables were independent:

$$\chi^2 = \sum (O - E)^2 / E$$

with expected count $E = (\text{row total} \times \text{column total}) / \text{grand total}$.

```
#               recovered  not recovered
table = np.array([[45, 15],      # new drug
                  [30, 30]])     # placebo
chi2, p, dof, expected = stats.chi2_contingency(table)
print(f"chi2 = {chi2:.3f}, dof = {dof}, p = {p:.4f}")
print("expected counts:\n", expected.round(1))
print("Fisher exact test (good for small counts):", stats.fisher_exact(table))
```

OUTPUT

```
chi2 = 6.969, dof = 1, p = 0.0083
expected counts:
[[37.5 22.5]
 [37.5 22.5]]
Fisher exact test (good for small counts): SignificanceResult(statistic=np.float64(3.0), pvalue=np.float64(0.007953318732958611))
```

Rule of thumb: the chi-square approximation is fine when all expected counts are at least 5; otherwise use Fisher's exact test.

Goodness of fit

Does observed data match a theoretical distribution? A die rolled 120 times:

```
observed = np.array([25, 17, 15, 23, 24, 16])
print(stats.chisquare(observed))           # H0: all faces equally likely (20 each)
```

OUTPUT

```
Power_divergenceResult(statistic=np.float64(5.0), pvalue=np.float64(0.4158801869955079))
```

15.8 Permutation tests: a test without formulas

A **permutation test** builds the null distribution by computer. If the group labels do not matter (H_0), shuffling them should not change the result much. Shuffle thousands of times, recompute the statistic each time and see where the real value falls.

```
rng = np.random.default_rng(9)
x = np.array([14.2, 15.1, 13.8, 16.0, 15.5, 14.9])
y = np.array([12.9, 13.5, 14.1, 12.2, 13.0, 13.8])
observed = x.mean() - y.mean()

pooled = np.concatenate([x, y])
diffs = []
for _ in range(20000):
    rng.shuffle(pooled)
    diffs.append(pooled[:6].mean() - pooled[6:].mean())
diffs = np.array(diffs)
p_perm = np.mean(np.abs(diffs) >= abs(observed))
print(f"observed difference {observed:.2f}, permutation p = {p_perm:.4f}")

res = stats.permutation_test((x, y), lambda a, b: a.mean() - b.mean(),
                             n_resamples=9999, random_state=1)
print("scipy permutation p =", round(res.pvalue, 4))
```

OUTPUT

```
observed difference 1.67, permutation p = 0.0053
scipy permutation p = 0.0065
```

Permutation tests make very few assumptions and work for any statistic.

15.9 The multiple-comparisons problem

If you test 20 true null hypotheses at $\alpha = 0.05$, you expect **one** false positive. Testing many outcomes (genes, sensors, subgroups) guarantees "discoveries" by chance.

```
rng = np.random.default_rng(12)
pvals = np.array([stats.ttest_ind(rng.normal(0, 1, 20), rng.normal(0, 1, 20)).pvalue
                    for _ in range(100)]) # 100 tests where NOTHING is going on
print("false positives at alpha = 0.05:", (pvals < 0.05).sum())
```

OUTPUT

```
false positives at alpha = 0.05: 5
```

Corrections:

- **Bonferroni**: use α/m (simple, conservative).
- **Holm**: step-down improvement of Bonferroni.
- **Benjamini-Hochberg (FDR)**: controls the expected *fraction* of false discoveries; standard in genomics.

```
def bh_reject(p, q=0.05):
    m = len(p)
    order = np.argsort(p)
    thresh = q * (np.arange(1, m + 1) / m)
    ok = p[order] <= thresh
    k = np.max(np.where(ok)[0]) + 1 if ok.any() else 0
    reject = np.zeros(m, bool)
    reject[order[:k]] = True
    return reject

print("Bonferroni rejections:", (pvals < 0.05 / 100).sum())
print("Benjamini-Hochberg rejections:", bh_reject(pvals).sum())
```

OUTPUT

```
Bonferroni rejections: 0
Benjamini-Hochberg rejections: 0
```

15.10 Estimating power by simulation

How many samples do I need to detect a difference of 5 units when $\sigma = 10$? Simulate the experiment many times and count how often it detects the effect.

```
def power(n, effect=5, sd=10, sims=2000, alpha=0.05, seed=0):
    rng = np.random.default_rng(seed)
    hits = 0
    for _ in range(sims):
        a = rng.normal(0, sd, n)
        b = rng.normal(effect, sd, n)
        hits += stats.ttest_ind(a, b).pvalue < alpha
    return hits / sims

for n in [10, 30, 64, 100]:
    print(f"n per group = {n}>3: power = {power(n):.2f}")
```

OUTPUT

```
n per group = 10: power = 0.16
n per group = 30: power = 0.47
n per group = 64: power = 0.81
n per group = 100: power = 0.93
```

Power reaches about 80% near 64 per group for a medium effect ($d = 0.5$). Planning this **before** the experiment is essential.

15.11 Choosing a test

Question	Data	Test
Mean equals a value?	numeric	one-sample t (or Wilcoxon)
Two groups differ?	numeric, independent	Welch t (or Mann-Whitney)
Before vs after?	numeric, paired	paired t (or Wilcoxon signed-rank)
3+ groups differ?	numeric	one-way ANOVA (or Kruskal-Wallis) + post-hoc
Two categories related?	counts	chi-square / Fisher
Data follows a distribution?	numeric	Shapiro-Wilk, KS test, Q-Q plot
Two variables correlated?	numeric	Pearson / Spearman test
Anything else	any	permutation test / bootstrap

```
x = np.arange(20)
y = 0.5 * x + np.random.default_rng(0).normal(0, 3, 20)
print(stats.pearsonr(x, y))
print(stats.ks_2samp(control, treated))      # do two samples share a distribution?
```

OUTPUT

```
PearsonRRResult(statistic=np.float64(0.6919881927377187), pvalue=np.float64(0.0007237688945779195))
KstestResult(statistic=np.float64(0.36666666666666664), pvalue=np.float64(0.034580084963334506), s
  tatic_location=np.float64(108.2006947991867), statistic_sign=np.int8(1))
```

15.12 Reporting results well

A good sentence contains the **estimate**, its **uncertainty**, the **test** and the **p-value**:

> "Treated plants were on average 9.8 cm taller than controls (95% CI 2.1 to 17.5; Welch t-test, $t = 2.6$, $p = 0.012$; Cohen's $d = 0.7$)."

Avoid "proved" and "no difference"; say "we found no evidence of a difference, with a confidence interval of ..." instead.

15.13 Chapter summary

CHAPTER SUMMARY

- Test logic: assume H_0 , compute how surprising the data is (p-value), compare with α . A small p is evidence against H_0 , not proof of H_1 , and not the probability H_0 is true.
- Type I error (false positive, α), type II error (false negative, β), power = $1 - \beta$.
- t-tests: one-sample, Welch two-sample, paired. Report effect size (Cohen's d) and a CI.
- ANOVA for 3+ groups ($F = \text{between} / \text{within variance}$), then post-hoc (Tukey); Kruskal-Wallis as non-parametric backup.
- Chi-square tests handle counts; Fisher's for small counts. Mann-Whitney/Wilcoxon avoid normality.
- Permutation tests and the bootstrap make few assumptions. Correct for multiple testing (Bonferroni, Holm, BH).

15.14 Practice exercises

1. A manufacturer claims a battery lasts 500 hours. You test 15 batteries (simulate them with mean 490 and sd 20). Carry out a one-sample t-test and state your conclusion.

2. Simulate two groups with a true difference of 3 and sd of 5, $n = 12$ each. How often does Welch's test reject at $\alpha = 0.05$ in 1000 runs?
3. Perform ANOVA and Tukey on three groups you create; then repeat with Kruskal-Wallis.
4. Test whether a 2×3 table $\begin{bmatrix} 20 & 15 & 30 \\ 25 & 25 & 15 \end{bmatrix}$ shows association.
5. Explain why running 15 t-tests between 6 groups is problematic, and show the inflation of false positives by simulation.

CHAPTER 16

16. Regression: Modelling Relationships

IN THIS CHAPTER YOU WILL LEARN

- Simple linear regression: the model, least squares, interpretation
- Measuring fit: R^2 , residuals, RMSE; inference on coefficients
- Checking assumptions with residual plots
- Multiple regression, categorical predictors, interactions, multicollinearity
- Polynomial regression, regularisation (Ridge, Lasso)
- Logistic regression for yes/no outcomes

Regression models how an outcome y (the *response* or *dependent* variable) depends on one or more inputs x (the *predictors*, *features* or *independent* variables). It serves two goals: **explanation** (how much does y change when x changes?) and **prediction** (what y should I expect for new x ?).

16.1 Simple linear regression

The model assumes a straight-line relationship plus random noise:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

where

- y_i : the measured outcome for observation i
- x_i : the predictor for observation i
- β_0 (intercept): the expected y when $x = 0$
- β_1 (slope): the expected *change in* y for a one-unit increase in x
- ε_i (epsilon): the error, the part of y that the line does not explain (assumed random with mean 0)

Fitting the line: least squares

Which line is "best"? For any candidate line, each point has a **residual** $e_i = y_i - \hat{y}_i$, the vertical gap between the actual value and the line's prediction $\hat{y}_i$ (read "y-hat"). **Ordinary Least Squares (OLS)** chooses β_0 and β_1 to minimise the **sum of squared residuals**:

$$SSR = \sum (y_i - \beta_0 - \beta_1 x_i)^2$$

Why squares? They treat positive and negative errors alike, punish big misses strongly and lead to clean formulas. Setting the slopes of SSR to zero (calculus) gives closed-form answers:

$$\beta_1 = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sum (x_i - \bar{x})^2 = r \times (s_y / s_x) \quad \beta_0 = \bar{y} - \beta_1 \bar{x}$$

The slope is the correlation r rescaled by the ratio of the standard deviations, and the line always passes through the point of means $(\bar{x}, \bar{y})$.

We will use an experiment: a spring stretched by hanging masses. Hooke's law says extension grows linearly with mass.

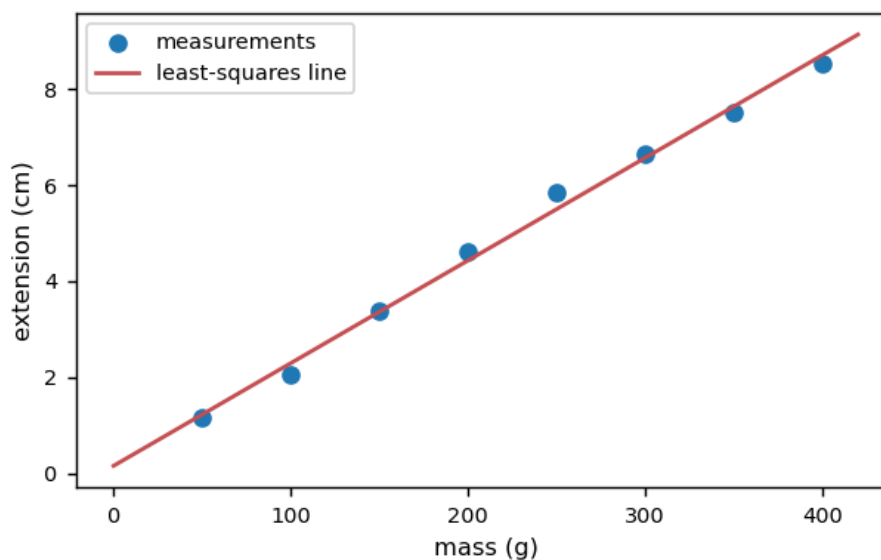
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(5)
mass = np.array([50, 100, 150, 200, 250, 300, 350, 400.0]) # grams
extension = 0.021 * mass + 0.3 + rng.normal(0, 0.25, len(mass)) # cm (true slope 0.021)

# least squares by the formulas
xbar, ybar = mass.mean(), extension.mean()
b1 = ((mass - xbar) * (extension - ybar)).sum() / ((mass - xbar) ** 2).sum()
b0 = ybar - b1 * xbar
print(f"slope b1 = {b1:.5f} cm/g, intercept b0 = {b0:.3f} cm")

fig, ax = plt.subplots(figsize=(5.5, 3.2))
ax.scatter(mass, extension, label="measurements")
xs = np.linspace(0, 420, 50)
ax.plot(xs, b0 + b1 * xs, "r-", label="least-squares line")
ax.set_xlabel("mass (g)"); ax.set_ylabel("extension (cm)"); ax.legend()
plt.show()
```

OUTPUT
slope b1 = 0.02134 cm/g, intercept b0 = 0.159 cm



The same fit with the library tools:

```
print(np.polyfit(mass, extension, deg=1)) # [slope, intercept]
res = stats.linregress(mass, extension)
print(res)
```

OUTPUT
[0.02134336 0.15881056]
LinregressResult(slope=np.float64(0.02134335840616066), intercept=np.float64(0.15881056020821482),
rvalue=np.float64(0.99730732285102), pvalue=np.float64(4.870969255058932e-08), stderr=np.float
64(0.0006407265401853059), intercept_stderr=np.float64(0.16177552141852863))

`linregress` returns the slope, intercept, correlation `rvalue`, the p-value for the slope (H_0 : slope = 0) and its standard error.

Interpreting the result

The slope 0.021 means: each extra gram of mass stretches the spring by about 0.021 cm, so the spring constant is roughly $1/0.021 \approx 48$ g per cm. The intercept is the (fitted) extension with no mass. The units of the slope are always units of y per unit of x.

WATCH OUT

Do not **extrapolate** far beyond the range of your x data. The line describes the region where you measured; springs stop being linear when overstretched.

16.2 How good is the fit?

Residuals, RMSE and R^2

Three quantities describe the variation:

- **SST** (total): $\sum (y - \bar{y})^2$, the variation of y around its mean: what an "average-only" model leaves.
- **SSR** or **SSE** (residual): $\sum (y - \hat{y})^2$, the variation left unexplained by the line.
- **SSM** (model): $SST - SSR$, the variation the line explains.

$$R^2 = 1 - SSR / SST$$

R^2 (the coefficient of determination) is the fraction of variance in y explained by the model: 0 means no better than the mean, 1 is a perfect fit. For simple regression $R^2 = r^2$. The **RMSE** (root mean squared error) = $\sqrt{SSR/n}$ is a typical prediction error in the units of y.

```
pred = b0 + b1 * mass
resid = extension - pred
sst = ((extension - ybar) ** 2).sum()
ssr = (resid ** 2).sum()
print("R^2 :", round(1 - ssr / sst, 4), " r^2:", round(res.rvalue ** 2, 4))
print("RMSE :", round(np.sqrt(ssr / len(mass)), 3), "cm")
print("residuals:", resid.round(2))
print("sum of residuals:", round(resid.sum(), 10))
```

OUTPUT

```
R^2 : 0.9946 r^2: 0.9946
RMSE : 0.18 cm
residuals: [-0.08 -0.22  0.03  0.18  0.34  0.07 -0.12 -0.19]
sum of residuals: -0.0
```

NOTE

A high R^2 does not guarantee a good model, and a low R^2 is not always bad (noisy biological data may have R^2 of 0.2 and still reveal a real effect). Always look at residual plots.

Inference on the slope

The slope b_1 is an estimate with uncertainty. Its standard error is

$$SE(b_1) = s / \sqrt{\sum (x_i - \bar{x})^2} \quad s = \sqrt{SSR / (n - 2)}$$

where s is the residual standard deviation (we lose 2 degrees of freedom for estimating two parameters). Then $t = b_1 / SE(b_1)$ follows a t distribution with $n - 2$ degrees of freedom under $H_0: \beta_1 = 0$, and a 95% CI is $b_1 \pm t_c SE(b_1)$.

```

n = len(mass)
s = np.sqrt(ssr / (n - 2))
se_b1 = s / np.sqrt(((mass - xbar) ** 2).sum())
t_stat = b1 / se_b1
p_val = 2 * stats.t.sf(abs(t_stat), df=n - 2)
tcrit = stats.t.ppf(0.975, n - 2)
print(f"SE(b1) = {se_b1:.5f}, t = {t_stat:.1f}, p = {p_val:.2e}")
print(f"95% CI for slope: ({b1 - tcrit*se_b1:.5f}, {b1 + tcrit*se_b1:.5f})")

```

OUTPUT

```

SE(b1) = 0.00064, t = 33.3, p = 4.87e-08
95% CI for slope: (0.01978, 0.02291)

```

16.3 Confidence versus prediction bands

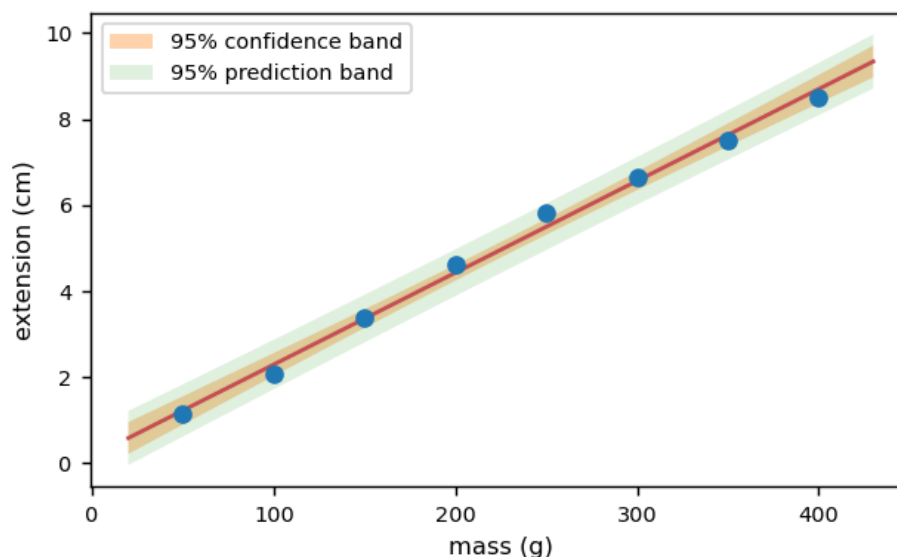
- A **confidence band** shows uncertainty about the **average** y at each x (where the true line might be).
- A **prediction band** is wider: it includes individual-point noise and answers "where will a *new measurement* fall?".

```

xs = np.linspace(20, 430, 100)
se_fit = s * np.sqrt(1 / n + (xs - xbar) ** 2 / ((mass - xbar) ** 2).sum())
se_pred = s * np.sqrt(1 + 1 / n + (xs - xbar) ** 2 / ((mass - xbar) ** 2).sum())
yhat = b0 + b1 * xs

fig, ax = plt.subplots(figsize=(5.5, 3.2))
ax.scatter(mass, extension, zorder=3)
ax.plot(xs, yhat, "r-")
ax.fill_between(xs, yhat - tcrit * se_fit, yhat + tcrit * se_fit,
               alpha=0.35, label="95% confidence band")
ax.fill_between(xs, yhat - tcrit * se_pred, yhat + tcrit * se_pred,
               alpha=0.15, label="95% prediction band")
ax.set_xlabel("mass (g)"); ax.set_ylabel("extension (cm)"); ax.legend()
plt.show()

```



Both bands are narrowest at $\bar{x}$ and flare out at the edges: predictions near the middle of your data are the most reliable.

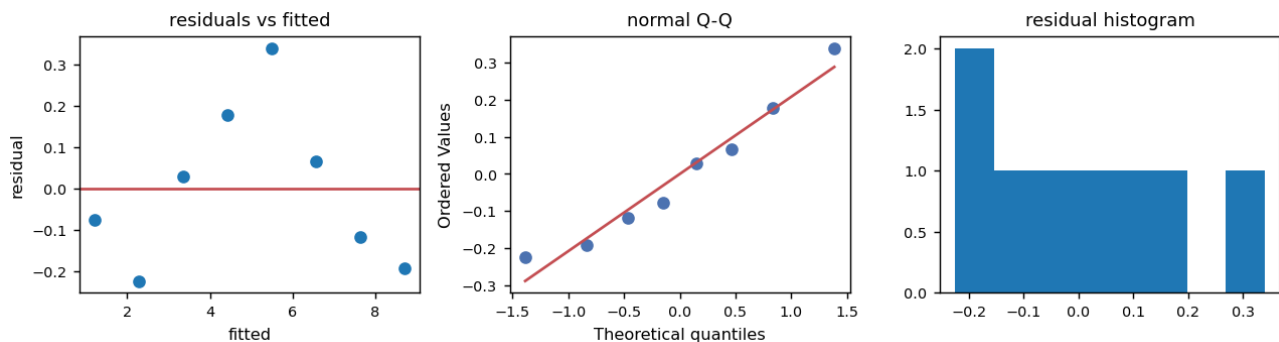
16.4 Assumptions and residual diagnostics

OLS inference works best when (remember **LINE**):

	Assumption	How to check
Linearity	the true relationship is a line	residuals vs fitted: no curve
Independence	errors are unrelated to each other	time order, autocorrelation plot
Normality	errors are roughly normal	Q-Q plot of residuals, histogram
Equal variance	errors have constant spread (homoscedasticity)	residuals vs fitted: no funnel shape

```
def diagnostics(x, y):
    slope, intercept = np.polyfit(x, y, 1)
    fitted = intercept + slope * x
    resid = y - fitted
    fig, axes = plt.subplots(1, 3, figsize=(10, 2.8))
    axes[0].scatter(fitted, resid); axes[0].axhline(0, color="r")
    axes[0].set_xlabel("fitted"); axes[0].set_ylabel("residual")
    axes[0].set_title("residuals vs fitted")
    stats.probplot(resid, plot=axes[1]); axes[1].set_title("normal Q-Q")
    axes[2].hist(resid, bins=8); axes[2].set_title("residual histogram")
    plt.tight_layout()
    plt.show()
```

```
# good case
diagnostics(mass, extension)
```

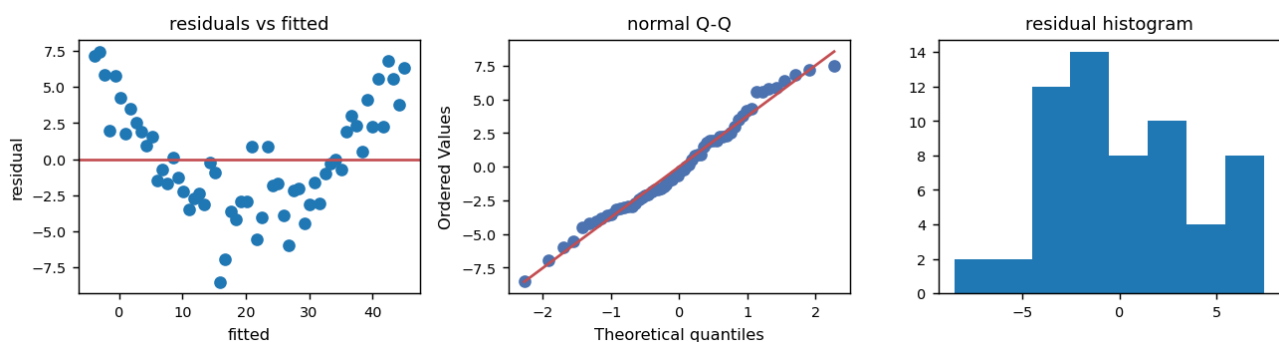


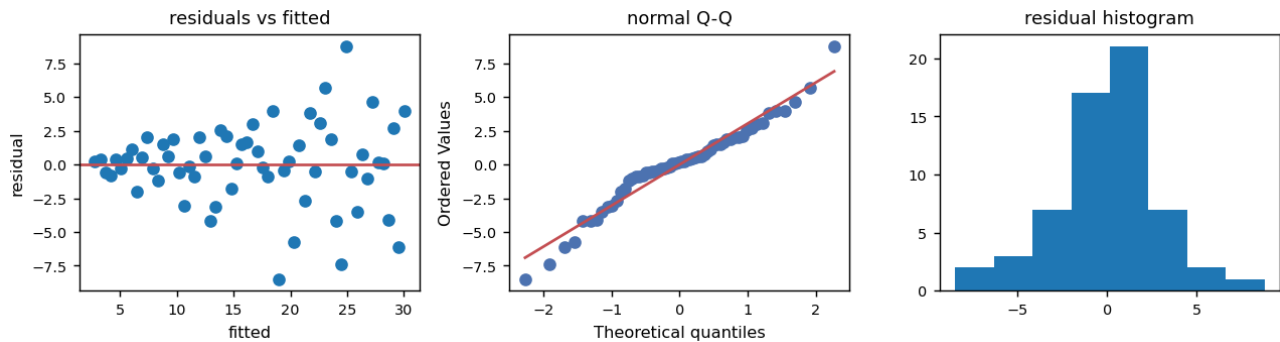
Now two bad cases: a **curved** truth and **growing noise** (a funnel). Notice how the residual plot reveals the problem even when R^2 looks high.

```
rng = np.random.default_rng(1)
x = np.linspace(1, 10, 60)
y_curve = 2 + 0.5 * x ** 2 + rng.normal(0, 2, 60) # quadratic truth
y_funnel = 3 * x + rng.normal(0, 0.6 * x) # noise grows with x

for name, y in [("curved", y_curve), ("funnel", y_funnel)]:
    print(name, "R^2 =", round(np.corrcoef(x, y)[0, 1] ** 2, 3))
    diagnostics(x, y)
```

OUTPUT
 curved $R^2 = 0.939$
 funnel $R^2 = 0.875$





Remedies: add a squared term or transform (curvature); transform y (for example $\log y$) or use weighted least squares (funnel); use robust regression for outliers; use time-series models for autocorrelated errors.

Outliers and influential points

A point with an extreme x has high **leverage**; if it also lies off the trend it is **influential** and drags the line towards itself.

```
rng = np.random.default_rng(2)
x = np.arange(10.0)
y = 2 * x + 1 + rng.normal(0, 1, 10)
x2 = np.append(x, 20.0); y2 = np.append(y, 10.0)  # an extreme, off-trend point

print("slope without:", np.polyfit(x, y, 1)[0].round(2))
print("slope with  :", np.polyfit(x2, y2, 1)[0].round(2))
```

OUTPUT
slope without: 2.06
slope with : 0.57

16.5 Multiple linear regression

With several predictors the model becomes

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$$

Each coefficient β_j is the expected change in y for a one-unit rise in x_j **holding all the other predictors fixed**. In matrix form, $y = X\beta + \varepsilon$, where $\mathbf{X}$ is the table of predictors with a column of ones (for the intercept). The least-squares solution is given by the **normal equations**:

$$\beta = (X^T X)^{-1} X^T y$$

```
rng = np.random.default_rng(8)
n = 120
temp = rng.normal(20, 5, n)
humid = rng.normal(60, 10, n)
yield_ = 5 + 0.8 * temp - 0.1 * humid + rng.normal(0, 2, n)  # crop yield

X = np.column_stack([np.ones(n), temp, humid])  # first column = intercept
beta = np.linalg.solve(X.T @ X, X.T @ yield_)
print("intercept, temp, humid:", beta.round(3))
```

OUTPUT
intercept, temp, humid: [2.75 0.811 -0.069]

Real software uses safer numerical methods, but the answer is the same. Here is a function that reproduces what a full regression report contains:

```
def ols(X, y, names):
    n, k = X.shape
    beta = np.linalg.solve(X.T @ X, X.T @ y)
    resid = y - X @ beta
    s2 = resid @ resid / (n - k)          # residual variance
    cov = s2 * np.linalg.inv(X.T @ X)    # covariance matrix of the estimates
    se = np.sqrt(np.diag(cov))
    t = beta / se
    p = 2 * stats.t.sf(np.abs(t), n - k)
    tc = stats.t.ppf(0.975, n - k)
    r2 = 1 - resid @ resid / ((y - y.mean()) @ (y - y.mean()))
    adj = 1 - (1 - r2) * (n - 1) / (n - k)
    table = pd.DataFrame({"coef": beta, "std err": se, "t": t, "P>|t|": p,
                          "[0.025": beta - tc * se, "0.975]": beta + tc * se}, index=names)
    return table.round(3), r2, adj

table, r2, adj = ols(X, yield_, ["intercept", "temp", "humid"])
print(table)
print(f"R^2 = {r2:.3f}, adjusted R^2 = {adj:.3f}")
```

OUTPUT

	coef	std err	t	P> t	[0.025	0.975]
intercept	2.750	1.254	2.192	0.03	0.266	5.234
temp	0.811	0.034	24.141	0.00	0.744	0.878
humid	-0.069	0.017	-4.075	0.00	-0.103	-0.035

R² = 0.839, adjusted R² = 0.836

Reading the table

- **coef**: the estimated effect. Each extra degree adds about 0.8 to the yield.
- **std err**: the uncertainty of the estimate.
- **t** and **P>|t|**: test of H_0 : this coefficient is 0 (given the others).
- **[0.025, 0.975]**: 95% confidence interval.
- **Adjusted R²** penalises useless predictors; plain R² can never fall when you add a variable, so compare models with adjusted R² (or cross-validation, Chapter 18).

The same report with the `statsmodels` library (installed in Colab):

```
import statsmodels.api as sm
Xc = sm.add_constant(pd.DataFrame({"temp": temp, "humid": humid}))
model = sm.OLS(yield_, Xc).fit()
print(model.summary())
```

```
# the formula interface works directly with DataFrames
import statsmodels.formula.api as smf
df = pd.DataFrame({"yield_": yield_, "temp": temp, "humid": humid})
model = smf.ols("yield_ ~ temp + humid", data=df).fit()
print(model.params, model.pvalues, model.rsquared)
```

The `summary()` table contains exactly the numbers we just computed, plus a global **F-test** (H_0 : all slopes are zero), AIC/BIC (criteria for comparing models) and the Durbin-Watson statistic (autocorrelation of residuals; near 2 is good).

With scikit-learn

scikit-learn's `LinearRegression` is designed for prediction workflows (it does not print p-values).

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

feat = np.column_stack([temp, humid])
```

```
lr = LinearRegression().fit(feats, yield_)
print("intercept:", lr.intercept_.round(3), "coefficients:", lr.coef_.round(3))
print("R^2:", round(lr.score(feats, yield_), 3))
print("prediction for temp 25, humid 55:", lr.predict([[25, 55]]).round(2))
```

OUTPUT

```
intercept: 2.75 coefficients: [ 0.811 -0.069]
R^2: 0.839
prediction for temp 25, humid 55: [19.23]
```

16.6 Categorical predictors: dummy variables

A category must be turned into numbers. **Dummy (one-hot) coding** makes a 0/1 column for each category except one (the reference level, absorbed into the intercept).

```
rng = np.random.default_rng(3)
d = pd.DataFrame({"fert": rng.choice(["none", "A", "B"], 90)})
d["growth"] = d["fert"].map({"none": 10, "A": 13, "B": 16}) + rng.normal(0, 2, 90)

dummies = pd.get_dummies(d["fert"], drop_first=True, dtype=float) # reference = "A"
print(dummies.head(3))
Xd = np.column_stack([np.ones(len(d)), dummies.values])
table, r2, adj = ols(Xd, d["growth"].values, ["intercept (ref = A)"] + list(dummies.columns))
print(table[["coef", "std err", "P>|t|"]])
```

OUTPUT

```
      B  none
0  1.0   0.0
1  0.0   1.0
2  0.0   1.0
```

	coef	std err	P> t
intercept (ref = A)	12.670	0.394	0.0
B	3.405	0.568	0.0
none	-2.344	0.537	0.0

Each coefficient is the **difference in mean** between that category and the reference category. Regression with dummy variables is ANOVA in disguise (Chapter 15).

16.7 Interactions

An **interaction** term $x_1 \cdot x_2$ lets the effect of one predictor depend on another: fertiliser may help much more in warm weather.

```
rng = np.random.default_rng(6)
n = 200
warm = rng.uniform(0, 1, n)
fert = rng.integers(0, 2, n).astype(float)
y = 5 + 1 * fert + 2 * warm + 4 * fert * warm + rng.normal(0, 0.7, n)

X = np.column_stack([np.ones(n), fert, warm, fert * warm])
table, r2, adj = ols(X, y, ["intercept", "fert", "warm", "fert x warm"])
print(table[["coef", "P>|t|"]])
```

OUTPUT

	coef	P> t
intercept	5.201	0.000
fert	0.529	0.012
warm	1.674	0.000
fert x warm	4.729	0.000

16.8 Multicollinearity

If two predictors are highly correlated they carry nearly the same information, and OLS cannot separate their effects: coefficients become unstable with huge standard errors, and may flip sign. Detect it with the **variance inflation factor**:

$$VIF_j = 1 / (1 - R_j^2)$$

where R_j^2 comes from regressing predictor j on all the other predictors. A VIF above about 5 to 10 signals trouble.

```
rng = np.random.default_rng(0)
n = 100
a = rng.normal(size=n)
b = a + rng.normal(0, 0.1, n)          # almost a copy of a
c = rng.normal(size=n)
y = 2 * a + 1 * c + rng.normal(0, 1, n)

def vif(features):
    out = []
    for j in range(features.shape[1]):
        others = np.delete(features, j, axis=1)
        Xo = np.column_stack([np.ones(len(features)), others])
        beta = np.linalg.lstsq(Xo, features[:, j], rcond=None)[0]
        resid = features[:, j] - Xo @ beta
        r2 = 1 - resid @ resid / ((features[:, j] - features[:, j].mean()) ** 2).sum()
        out.append(1 / (1 - r2))
    return np.round(out, 1)

F = np.column_stack([a, b, c])
print("VIF for a, b, c:", vif(F))
table, *_ = ols(np.column_stack([np.ones(n), a, b, c]), y, ["const", "a", "b", "c"])
print(table[["coef", "std err"]])
```

OUTPUT

```
VIF for a, b, c: [103.9 103.9  1. ]
      coef  std err
const -0.038    0.095
a      3.095    0.989
b     -1.148    0.979
c      1.021    0.084
```

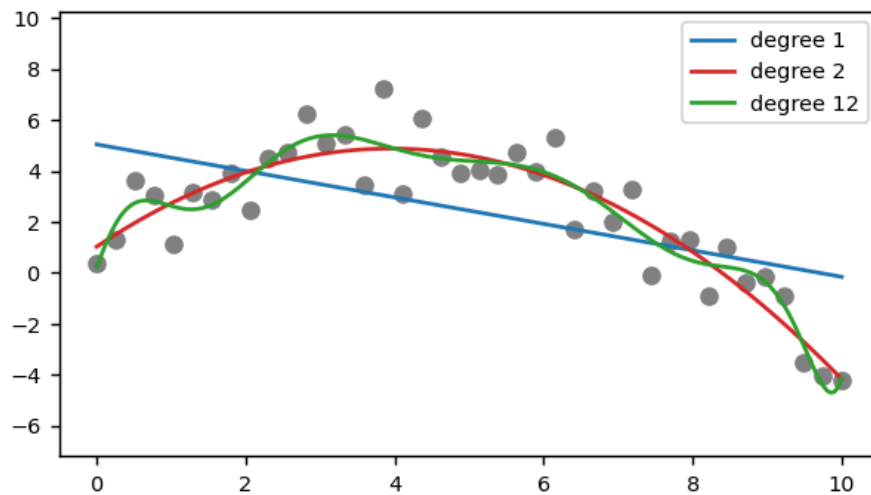
Remedies: drop or combine redundant predictors, use PCA (Chapter 19) or regularisation (below).

16.9 Polynomial regression and non-linear terms

A model is **linear regression** if it is linear in the *coefficients*, even when x is squared or logged. To model a curve add powers of x .

```
rng = np.random.default_rng(4)
x = np.linspace(0, 10, 40)
y = 1 + 2 * x - 0.25 * x ** 2 + rng.normal(0, 1, 40)

fig, ax = plt.subplots(figsize=(5.5, 3))
ax.scatter(x, y, color="grey")
xs = np.linspace(0, 10, 200)
for deg, col in [(1, "tab:blue"), (2, "tab:red"), (12, "tab:green")]:
    coefs = np.polyfit(x, y, deg)
    ax.plot(xs, np.polyval(coefs, xs), color=col, label=f"degree {deg}")
ax.set_ylim(y.min() - 3, y.max() + 3)
ax.legend(); plt.show()
```



Degree 1 is too simple (**underfitting**); degree 12 wiggles through the noise (**overfitting**): great on these points, poor on new ones. Chapter 18 shows how to choose the complexity with cross-validation.

Other useful transformations: $\log(y)$ for multiplicative effects (coefficient then means "percent change per unit"), $\log(x)$ for diminishing returns, $\sqrt{x}$ for counts.

16.10 Regularisation: Ridge and Lasso

When there are many predictors or strong collinearity, OLS overfits. **Regularisation** adds a penalty that shrinks the coefficients:

$$\text{Ridge: minimise } SSR + \lambda \sum \beta_j^2 \quad \text{Lasso: minimise } SSR + \lambda \sum |\beta_j|$$

- **Ridge** (L2 penalty) shrinks all coefficients smoothly towards zero.
- **Lasso** (L1 penalty) can shrink some coefficients **exactly to zero**, performing feature selection.
- λ (called **alpha** in scikit-learn) controls the strength: $\lambda = 0$ is plain OLS; larger λ means simpler models.

Always **standardise** predictors first, so the penalty treats them fairly.

```
from sklearn.linear_model import Ridge, Lasso
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(10)
n, p = 80, 10
Xr = rng.normal(size=(n, p))
true_beta = np.array([3, -2, 1.5, 0, 0, 0, 0, 0, 0, 0]) # only three features matter
yr = Xr @ true_beta + rng.normal(0, 1, n)
Xs = StandardScaler().fit_transform(Xr)

ols_c = LinearRegression().fit(Xs, yr).coef_
ridge_c = Ridge(alpha=10).fit(Xs, yr).coef_
lasso_c = Lasso(alpha=0.3).fit(Xs, yr).coef_
cmp = pd.DataFrame({"true": true_beta, "OLS": ols_c, "Ridge": ridge_c, "Lasso": lasso_c})
print(cmp.round(2))
```

OUTPUT

	true	OLS	Ridge	Lasso
0	3.0	3.16	2.77	2.84
1	-2.0	-1.83	-1.58	-1.48
2	1.5	1.13	0.99	0.78
3	0.0	0.08	0.11	0.00
4	0.0	-0.01	-0.02	-0.00

```

5  0.0 -0.07 -0.07 -0.00
6  0.0 -0.14 -0.13 -0.00
7  0.0 -0.23 -0.11 -0.00
8  0.0 -0.06 -0.08 -0.00
9  0.0 -0.10 -0.17 -0.00

```

Notice that Lasso sets the irrelevant coefficients to exactly 0.

16.11 Logistic regression: predicting yes or no

For a **binary outcome** (survived or died, detected or not) a straight line would predict impossible probabilities below 0 or above 1. **Logistic regression** models the **probability** with an S-shaped curve.

The steps:

1. A linear score: $z = \beta_0 + \beta_1 x_1 + \dots$ (can be any number).
2. The **sigmoid** squashes it into a probability: $p = 1 / (1 + e^{-z})$.
3. Equivalently, the **log-odds** (logit) is linear: $\ln(p / (1 - p)) = \beta_0 + \beta_1 x_1 + \dots$

The **odds** are $p/(1-p)$; a probability of 0.75 means odds of 3 to 1. Each coefficient β_j changes the log-odds; e^{β} is the **odds ratio**: the multiplicative change in odds per unit of x_j . The coefficients are found by maximum likelihood (Chapter 14), minimising the **log-loss**:

$$-\sum [y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i)]$$

```

from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(21)
dose = rng.uniform(0, 10, 200)
true_p = 1 / (1 + np.exp(-(dose - 5) * 1.2))
responded = (rng.uniform(size=200) < true_p).astype(int)

clf = LogisticRegression().fit(dose.reshape(-1, 1), responded)
b0, b1 = clf.intercept_[0], clf.coef_[0][0]
print(f"log-odds = {b0:.2f} + {b1:.2f} * dose")
print(f"odds ratio per unit of dose: {np.exp(b1):.2f}")
print("P(response) at dose 3, 5, 8:", clf.predict_proba([[3], [5], [8]])[:, 1].round(2))
print("dose where P = 0.5 (ED50):", round(-b0 / b1, 2))

xs = np.linspace(0, 10, 100)
fig, ax = plt.subplots(figsize=(5.5, 3))
ax.scatter(dose, responded + np.random.default_rng(0).normal(0, 0.02, 200), s=10, alpha=0.5)
ax.plot(xs, clf.predict_proba(xs.reshape(-1, 1))[:, 1], "r-")
ax.set_xlabel("dose"); ax.set_ylabel("P(response)")
plt.show()

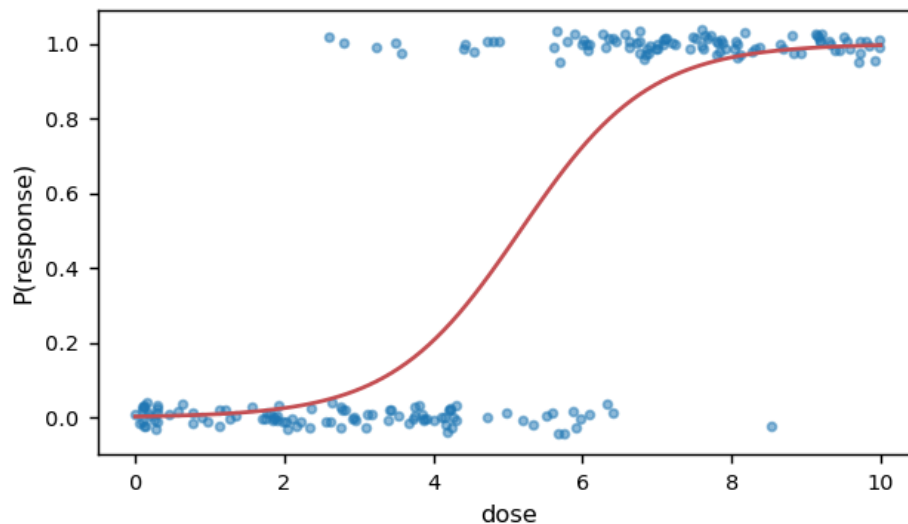
```

OUTPUT

```

log-odds = -5.98 + 1.16 * dose
odds ratio per unit of dose: 3.18
P(response) at dose 3, 5, 8: [0.08 0.45 0.96]
dose where P = 0.5 (ED50): 5.17

```



The dose where the probability is 0.5 (**ED50** in pharmacology) is $-\beta_0/\beta_1$. Classification metrics (accuracy, precision, recall, ROC) are covered in Chapter 18.

16.12 Choosing and comparing models

- Prefer the **simplest model that explains the data** (parsimony).
- Compare with **adjusted R^2** , AIC/BIC (lower is better), or (best) **cross-validated error** on held-out data.
- Never choose variables only because their p-values are small: fitting many candidates and keeping the "significant" ones inflates false discoveries.
- Always report coefficients **with confidence intervals** and describe the residual checks you performed.

16.13 Chapter summary

CHAPTER SUMMARY

- Simple linear regression fits $y = \beta_0 + \beta_1 x$ by minimising the sum of squared residuals; the slope equals $r \times s_y/s_x$ and the line passes through $(\bar{x}, \bar{y})$.
- $R^2 = 1 - \text{SSR}/\text{SST}$ is the fraction of variance explained; RMSE is the typical error in y units.
- Inference: SE, t-test and CI for each coefficient; confidence bands versus wider prediction bands.
- Check LINE assumptions with residual plots; watch outliers and leverage.
- Multiple regression: $\beta = (X^T X)^{-1} X^T y$; coefficients are effects "holding others fixed". Use dummies for categories, interaction terms for dependence, VIF for collinearity.
- Polynomials capture curvature but can overfit; Ridge and Lasso regularise; logistic regression models probabilities through log-odds.

16.14 Practice exercises

1. Fit a line to $x = [1, 2, 3, 4, 5]$, $y = [2.1, 3.9, 6.2, 7.8, 10.1]$ by hand and verify with `np.polyfit`.
2. Using the spring data, compute the 99% confidence interval for the slope and the predicted extension with its prediction interval for a 275 g mass.
3. Generate data with $y = 3x^2$ and fit a straight line; inspect the residual plot. Then add an x^2 term.

4. Create two nearly identical predictors, fit OLS, then Ridge. Compare the coefficient stability by repeating with different seeds.
5. Fit a logistic regression to predict cultivar 0 versus the rest in the wine dataset using `proline` only. What odds ratio do you get?

PART IV

Advanced Scientific Analysis

Prepare data for models, learn machine learning, discover structure, analyse time series, and use scientific computing tools with confidence.

CHAPTER 17

17. Preprocessing and Feature Engineering

IN THIS CHAPTER YOU WILL LEARN

- Why models need prepared data, and the golden rule: avoid data leakage
- Scaling (standardisation, min-max, robust) and transformations (log, Box-Cox)
- Encoding categories, binning, date features, interaction and polynomial features
- Imputing missing values the model-friendly way
- Feature selection and pipelines with `ColumnTransformer`

Raw tables rarely go straight into a model. **Preprocessing** cleans and reshapes the inputs; **feature engineering** creates new informative variables. Good features often matter more than a fancier algorithm.

17.1 The golden rule: fit on training data only

Preprocessing steps such as "subtract the mean" **learn** numbers (the mean) from data. If you learn them from the **whole** dataset, information from the test set leaks into training, and your performance estimate becomes too optimistic. This is called **data leakage**.

Rule: split first, then fit every preprocessing step on the training part only, and only *apply* it to the test part.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
X = rng.normal(50, 10, size=(100, 3))
y = rng.integers(0, 2, 100)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)    # learns mean and std from TRAIN only
X_test_s = scaler.transform(X_test)         # applies the same numbers to TEST
print("train mean ~0:", X_train_s.mean(axis=0).round(2))
print("test mean (not exactly 0, and that is fine):", X_test_s.mean(axis=0).round(2))
```

OUTPUT

```
train mean ~0: [-0. -0.  0.]
test mean (not exactly 0, and that is fine): [-0.13  0.71  0.11]
```

17.2 Scaling numeric features

Many methods (k-nearest neighbours, SVM, PCA, regularised regression, neural nets, gradient descent) compare or add features, so a variable measured in thousands would dominate one measured in fractions. **Scaling** puts features on comparable ranges.

Scaler	Formula	Use when
Standardisation (<code>StandardScaler</code>)	$z = (x - \text{mean}) / \text{std}$	default choice; roughly normal data

Scaler	Formula	Use when
Min-max (MinMaxScaler)	$(x - \min) / (\max - \min)$, range 0 to 1	bounded inputs, images
Robust (RobustScaler)	$(x - \text{median}) / \text{IQR}$	data with outliers
Max-abs	$x / \max(\text{abs}(x))$	sparse data

```
from sklearn.preprocessing import MinMaxScaler, RobustScaler

vals = np.array([[1.0], [2.0], [3.0], [4.0], [100.0]]) # one huge outlier
print("standard:", StandardScaler().fit_transform(vals).ravel().round(2))
print("min-max :", MinMaxScaler().fit_transform(vals).ravel().round(2))
print("robust  :", RobustScaler().fit_transform(vals).ravel().round(2))
```

OUTPUT
standard: [-0.54 -0.51 -0.49 -0.46 2.]
min-max : [0. 0.01 0.02 0.03 1.]
robust : [-1. -0.5 0. 0.5 48.5]

Notice how the outlier squashes the other values under standard and min-max scaling, while robust scaling keeps them spread.

NOTE

Tree-based models (decision trees, random forests, gradient boosting) do **not** need scaling: they only use the order of values.

17.3 Transforming skewed data

Right-skewed variables (incomes, concentrations, counts) often become more symmetric and better behaved after a transformation:

Transform	When
$\log(x)$ or $\log_{1p}(x) = \log(1 + x)$	strictly positive, strong right skew; multiplicative effects
$\text{sqrt}(x)$	counts, milder skew
Box-Cox ($x > 0$) and Yeo-Johnson (any x)	let the data choose the power λ
reciprocal $1/x$	rates; strong skew

Box-Cox applies $y = (x^\lambda - 1)/\lambda$, with $\lambda = 0$ meaning log; λ is estimated by maximum likelihood.

```
from scipy import stats
from sklearn.preprocessing import PowerTransformer

rng = np.random.default_rng(1)
x = rng.lognormal(mean=1.0, sigma=0.8, size=1000)
print("skew before:", round(stats.skew(x), 2))
print("skew after log:", round(stats.skew(np.log(x)), 2))
print("skew after sqrt:", round(stats.skew(np.sqrt(x)), 2))

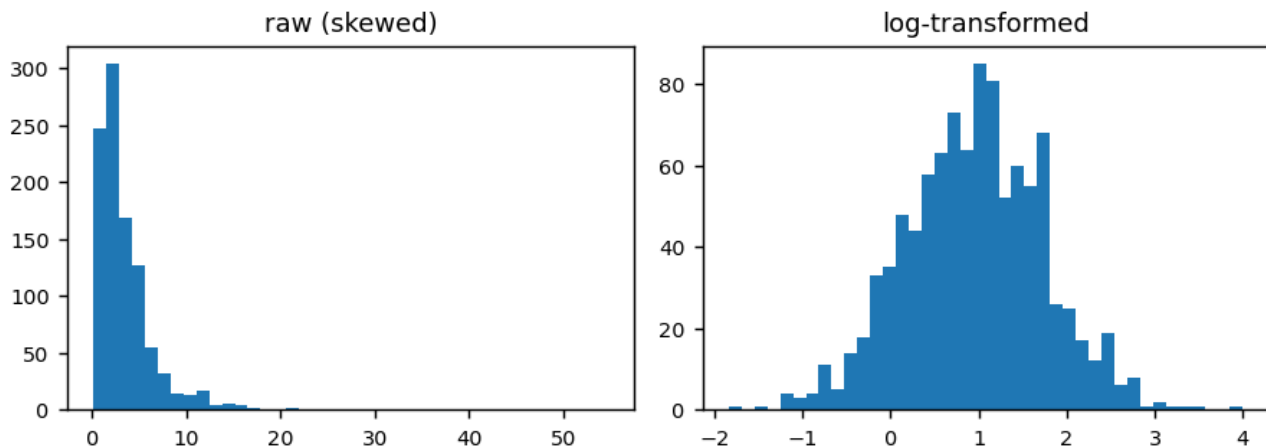
bc, lam = stats.boxcox(x)
print("Box-Cox lambda:", round(lam, 3), " skew:", round(stats.skew(bc), 2))

pt = PowerTransformer(method="yeo-johnson")
print("Yeo-Johnson skew:", round(stats.skew(pt.fit_transform(x.reshape(-1, 1)).ravel()), 2))

fig, axes = plt.subplots(1, 2, figsize=(7, 2.6))
axes[0].hist(x, bins=40); axes[0].set_title("raw (skewed)")
axes[1].hist(np.log(x), bins=40); axes[1].set_title("log-transformed")
plt.tight_layout(); plt.show()
```

OUTPUT
skew before: 5.05
skew after log: 0.0

```
skew after sqrt: 1.56
Box-Cox lambda: -0.001  skew: -0.0
Yeo-Johnson skew: 0.03
```



17.4 Encoding categorical variables

Models need numbers. The right encoding depends on the kind of category.

Type	Example	Encoding
Nominal (no order)	species, country	one-hot (dummy) columns
Ordinal (ordered)	low < medium < high	integer codes that respect the order
High cardinality (many levels)	postcodes, gene names	target/frequency encoding, hashing, or grouping rare levels

```
df = pd.DataFrame({
    "species": ["cat", "dog", "bird", "dog", "cat"],
    "size": ["small", "large", "small", "medium", "medium"],
})
print(pd.get_dummies(df["species"], prefix="sp", dtype=int))

order = {"small": 0, "medium": 1, "large": 2}
df["size_code"] = df["size"].map(order)
print(df)

from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
print(enc.fit_transform(df[["species"]]))
print(enc.categories_)
```

```
OUTPUT
  sp_bird  sp_cat  sp_dog
0        0        1        0
1        0        0        1
2        1        0        0
3        0        0        1
4        0        1        0
  species  size  size_code
0    cat  small          0
1    dog  large          2
2    bird  small          0
3    dog  medium          1
4    cat  medium          1
[[0.  1.  0.]
 [0.  0.  1.]
 [1.  0.  0.]
 [0.  0.  1.]
 [0.  1.  0.]]
[array(['bird', 'cat', 'dog'], dtype=object)]
```

TIP

`handle_unknown="ignore"` prevents a crash when new data contains a category not seen in training. Label-encoding nominal categories as 0, 1, 2 misleads linear models into thinking one category is "bigger" than another.

17.5 Binning and discretising

```
from sklearn.preprocessing import KBinsDiscretizer

ages = np.array([[5], [17], [25], [33], [48], [61], [79]])
kb = KBinsDiscretizer(n_bins=3, encode="ordinal", strategy="quantile")
print(kb.fit_transform(ages).ravel(), kb.bin_edges_[0].round(1))
```

OUTPUT

```
[0. 0. 1. 1. 2. 2. 2.] [ 5. 25. 48. 79.]
```

Binning loses information; use it for interpretation or when the relationship is truly step-like.

17.6 Creating new features

Date and time features

```
t = pd.DataFrame({"when": pd.date_range("2024-06-28 22:00", periods=5, freq="9h")})
t["hour"] = t["when"].dt.hour
t["dayofweek"] = t["when"].dt.dayofweek
t["is_weekend"] = t["dayofweek"] >= 5
t["month"] = t["when"].dt.month
# cyclic encoding: hour 23 and hour 0 are neighbours!
t["hour_sin"] = np.sin(2 * np.pi * t["hour"] / 24).round(3)
t["hour_cos"] = np.cos(2 * np.pi * t["hour"] / 24).round(3)
print(t)
```

OUTPUT

	when	hour	dayofweek	is_weekend	month	hour_sin	hour_cos
0	2024-06-28 22:00:00	22	4	False	6	-0.500	0.866
1	2024-06-29 07:00:00	7	5	True	6	0.966	-0.259
2	2024-06-29 16:00:00	16	5	True	6	-0.866	-0.500
3	2024-06-30 01:00:00	1	6	True	6	0.259	0.966
4	2024-06-30 10:00:00	10	6	True	6	0.500	-0.866

Periodic quantities (hour, month, wind direction) are best represented by a sine and cosine pair, so that 23:00 and 00:00 end up close together.

Ratios, differences, interactions and polynomials

```
from sklearn.preprocessing import PolynomialFeatures

d = pd.DataFrame({"mass_kg": [60, 72, 85], "height_m": [1.65, 1.78, 1.90]})
d["bmi"] = d["mass_kg"] / d["height_m"] ** 2 # domain knowledge feature
print(d.round(2))

pf = PolynomialFeatures(degree=2, include_bias=False)
print(pf.fit_transform(np.array([[2, 3]]))) # x1, x2, x1^2, x1*x2, x2^2
print(pf.get_feature_names_out(["a", "b"]))
```

OUTPUT

	mass_kg	height_m	bmi
0	60	1.65	22.04
1	72	1.78	22.72
2	85	1.90	23.55

```
[[2. 3. 4. 6. 9.]]
['a' 'b' 'a^2' 'a b' 'b^2']
```

Domain knowledge beats automation. A physicist builds dimensionless numbers, a biologist computes growth rates, a chemist uses log concentrations. Ask: what quantity does the science say should matter?

17.7 Handling missing values for models

`SimpleImputer` learns the fill value from the training data (avoiding leakage), and `KNNImputer` fills gaps from similar rows.

```
from sklearn.impute import SimpleImputer, KNNImputer

A = np.array([[1.0, 2.0], [np.nan, 3.0], [7.0, np.nan], [4.0, 5.0]])
print(SimpleImputer(strategy="median").fit_transform(A))
print(KNNImputer(n_neighbors=2).fit_transform(A).round(2))
```

OUTPUT

```
[[1. 2.]
 [4. 3.]
 [7. 3.]
 [4. 5.]]
[[1. 2. ]
 [2.5 3. ]
 [7. 3.5]
 [4. 5. ]]
```

For a *missingness indicator* add `add_indicator=True`; sometimes the fact that a value is missing is itself informative.

17.8 Feature selection

More features are not always better: they add noise, slow training and invite overfitting. Approaches:

1. **Filter:** rank features by a simple statistic (correlation, ANOVA F-test, mutual information) and keep the top k.
2. **Wrapper:** search subsets by repeatedly fitting a model (for example recursive feature elimination).
3. **Embedded:** the model selects while training (Lasso, tree importances).

```
from sklearn.datasets import load_wine
from sklearn.feature_selection import SelectKBest, f_classif, mutual_info_classif, RFE
from sklearn.linear_model import LogisticRegression

wine = load_wine(as_frame=True)
Xw, yw = wine.data, wine.target

skb = SelectKBest(f_classif, k=4).fit(Xw, yw)
print("ANOVA F top 4:", list(Xw.columns[skb.get_support()]))

mi = pd.Series(mutual_info_classif(Xw, yw, random_state=0), index=Xw.columns)
print("mutual information top 4:", mi.sort_values(ascending=False).head(4).round(2).to_dict())

Xw_scaled = StandardScaler().fit_transform(Xw)
rfe = RFE(LogisticRegression(max_iter=5000), n_features_to_select=4).fit(Xw_scaled, yw)
print("RFE picks:", list(Xw.columns[rfe.support_]))
```

OUTPUT

```
ANOVA F top 4: ['alcohol', 'flavanoids', 'od280/od315_of_diluted_wines', 'proline']
mutual information top 4: {'flavanoids': 0.67, 'proline': 0.57, 'color_intensity': 0.55, 'od280/od
315_of_diluted_wines': 0.51}
RFE picks: ['alcohol', 'flavanoids', 'color_intensity', 'proline']
```

```
# remove near-constant and highly correlated features
from sklearn.feature_selection import VarianceThreshold
```

```
corr = Xw.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape, dtype=bool), k=1))
drop = [c for c in upper.columns if (upper[c] > 0.8).any()]
print("highly correlated (> 0.8) to drop:", drop)
vt = VarianceThreshold(0.01).fit(Xw)
print("low variance columns:", list(Xw.columns[~vt.get_support()])))
```

OUTPUT

```
highly correlated (> 0.8) to drop: ['flavanoids']
low variance columns: []
```

17.9 Pipelines: one object for the whole workflow

A **Pipeline** chains preprocessing and a model into a single estimator. It guarantees that scaling and imputation are learned only from training data each time (even inside cross-validation) and makes your work reproducible and easy to deploy. **ColumnTransformer** applies different steps to different columns.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier

rng = np.random.default_rng(5)
n = 300
data = pd.DataFrame({
    "age": rng.normal(45, 12, n),
    "income": rng.lognormal(10, 0.5, n),
    "city": rng.choice(["Pune", "Delhi", "Mumbai"], n),
    "plan": rng.choice(["basic", "pro"], n),
})
data.loc[rng.choice(n, 20, replace=False), "age"] = np.nan # some missing ages
li = np.log(data["income"])
z_income = (li - li.mean()) / li.std()
logit = 3.0 * z_income + 2.0 * (data["plan"] == "pro") - 1.0
data["bought"] = (rng.random(n) < 1 / (1 + np.exp(-logit))).astype(int)

num_cols = ["age", "income"]
cat_cols = ["city", "plan"]
pre = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                     ("scale", StandardScaler())]), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])
model = Pipeline([("pre", pre), ("clf", LogisticRegression(max_iter=1000))])

Xtr, Xte, ytr, yte = train_test_split(data[num_cols + cat_cols], data["bought"],
                                     test_size=0.25, random_state=0, stratify=data["bought"])
model.fit(Xtr, ytr)
print("test accuracy:", round(model.score(Xte, yte), 3))
print("pipeline steps:", list(model.named_steps))
print("feature names out:", list(model.named_steps["pre"].get_feature_names_out())[:6])
```

OUTPUT

```
test accuracy: 0.84
pipeline steps: ['pre', 'clf']
feature names out: ['num__age', 'num__income', 'cat__city_Delhi', 'cat__city_Mumbai', 'cat__city_P
une', 'cat__plan_basic']
```

17.10 A preprocessing checklist

1. Split data first (and stratify by class for classification).
2. Handle missing values with imputers inside the pipeline.
3. Encode categories; scale numeric features if the model needs it.

4. Transform heavy skew; engineer features from domain knowledge.
5. Select features, using only the training data.
6. Wrap everything in a pipeline and evaluate with cross-validation (Chapter 18).

17.11 Chapter summary

CHAPTER SUMMARY

- Split first; fit every preprocessing step on training data only to avoid leakage.
- Standardise, min-max scale or robust-scale numeric features (trees do not need it).
- Transform right-skewed data with log, sqrt, Box-Cox or Yeo-Johnson.
- Encode categories (one-hot for nominal, integer codes for ordinal); use sine/cosine for cyclic features.
- Impute with `SimpleImputer` or `KNNImputer`; select features by filter, wrapper or embedded methods.
- Pipelines with `ColumnTransformer` make preprocessing safe, reusable and reproducible.

17.12 Practice exercises

1. Show that fitting a scaler on the full dataset before splitting changes the test mean compared with fitting on training only.
2. Take a skewed variable and find which of log, sqrt and Box-Cox gives skewness closest to 0.
3. Encode wind direction in degrees (0 to 360) with sine and cosine and check that 359 and 1 are close.
4. Build a pipeline with imputation, scaling and a Ridge regression for the diabetes dataset and report the test R^2 .
5. Use `SelectKBest` with `f_regression` to choose the 3 best features of the diabetes dataset.

CHAPTER 18

18. Machine Learning for Scientists

IN THIS CHAPTER YOU WILL LEARN

- The vocabulary: supervised vs unsupervised, training, testing, generalisation
- Overfitting, underfitting and the bias-variance trade-off
- Train/test splits, cross-validation, hyperparameter tuning
- Classification and regression metrics (accuracy, precision, recall, F1, ROC-AUC, RMSE, R^2)
- Key algorithms: k-NN, decision trees, random forests, gradient boosting, SVM, naive Bayes
- Interpreting models: feature importance, permutation importance

Machine learning (ML) builds models that learn patterns from data in order to predict. Scientists use it to classify cells, predict material properties, detect signals and more. The crucial difference from classical statistics is the emphasis: statistics asks "what is the effect and how certain are we?", ML asks "how well can we predict **new** data?"

18.1 Vocabulary

Term	Meaning
Feature (X)	an input variable
Target / label (y)	what we want to predict
Supervised learning	learning from examples that have labels: classification (categories) and regression (numbers)
Unsupervised learning	finding structure without labels: clustering, dimensionality reduction (Chapter 19)
Training set	data used to fit the model
Test set	unseen data used once to estimate final performance
Hyperparameter	a setting chosen <i>before</i> training (number of neighbours, tree depth)
Parameter	learned during training (coefficients)

18.2 Overfitting and the bias-variance trade-off

A model can be **too simple** (high **bias**: misses the pattern, *underfits*) or **too flexible** (high **variance**: memorises noise, *overfits*). Training error keeps dropping with complexity; error on new data is U-shaped.

$$\text{expected prediction error} = \text{bias}^2 + \text{variance} + \text{irreducible noise}$$

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
```

```

from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(0)
x = rng.uniform(0, 10, 60)
y = np.sin(x) * 3 + rng.normal(0, 0.8, 60)
xtr, xte, ytr, yte = train_test_split(x.reshape(-1, 1), y, test_size=0.4, random_state=1)

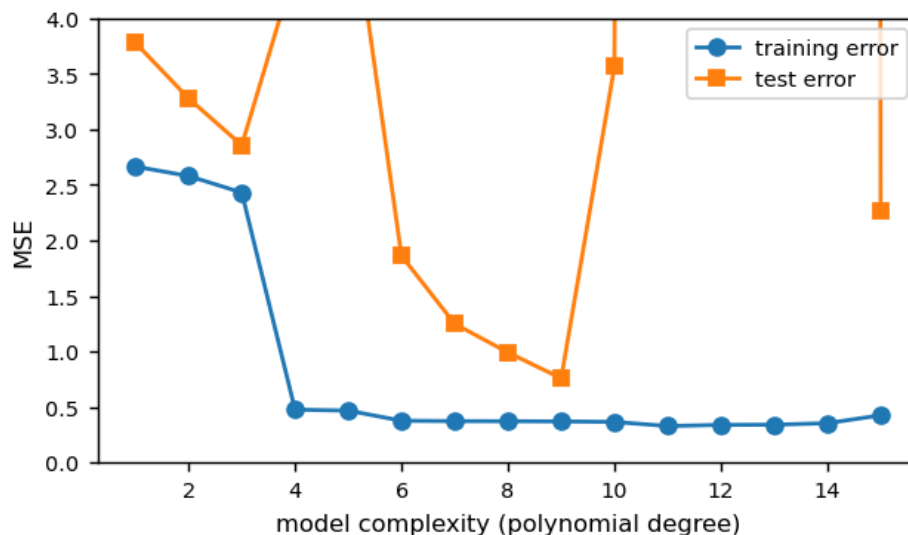
degrees = range(1, 16)
train_err, test_err = [], []
for d in degrees:
    m = make_pipeline(PolynomialFeatures(d), LinearRegression()).fit(xtr, ytr)
    train_err.append(mean_squared_error(ytr, m.predict(xtr)))
    test_err.append(mean_squared_error(yte, m.predict(xte)))

fig, ax = plt.subplots(figsize=(5.5, 3))
ax.plot(degrees, train_err, "o-", label="training error")
ax.plot(degrees, test_err, "s-", label="test error")
ax.set_ylim(0, 4); ax.set_xlabel("model complexity (polynomial degree)"); ax.set_ylabel("MSE")
ax.legend(); plt.show()
print("best degree on test data:", list(degrees)[int(np.argmin(test_err))])

```

OUTPUT

best degree on test data: 9



The training error falls forever, but the test error bottoms out and rises: that gap is overfitting. **The goal is to find the complexity where test error is lowest.**

18.3 Train, validation, test and cross-validation

- Use the **training set** to fit the model.
- Use a **validation set** (or cross-validation) to choose hyperparameters and compare models.
- Touch the **test set once**, at the very end, for an honest final estimate.

k-fold cross-validation splits the training data into k parts (folds). Each fold takes a turn as the validation set while the model trains on the other k – 1 folds; the k scores are averaged. It uses the data efficiently and shows the **variability** of performance.

```

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

Xb, yb = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(Xb, yb, test_size=0.25, random_state=0, stratify=yb)
print("train/test sizes:", Xtr.shape[0], Xte.shape[0], " class balance:", np.bincount(yb))

```

```
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
scores = cross_val_score(pipe, Xtr, ytr, cv=cv, scoring="accuracy")
print("fold accuracies:", scores.round(3))
print(f"mean = {scores.mean():.3f} +/- {scores.std():.3f}")
```

OUTPUT

```
train/test sizes: 426 143   class balance: [212 357]
fold accuracies: [1.    0.976 0.976 0.988 0.965]
mean = 0.981 +/- 0.012
```

WATCH OUT

For **time series**, random splits leak the future into the past. Use `TimeSeriesSplit` (always train on earlier, validate on later). For **grouped** data (several measurements per patient), use `GroupKFold` so one subject never appears in both training and validation.

18.4 Classification metrics

For two classes, call one "positive" (for example disease). Every prediction falls in a cell of the **confusion matrix**:

	Predicted positive	Predicted negative
Actually positive	true positive (TP)	false negative (FN)
Actually negative	false positive (FP)	true negative (TN)

Metric	Formula	Meaning
Accuracy	$(TP + TN) / \text{all}$	fraction correct (misleading if classes are imbalanced)
Precision	$TP / (TP + FP)$	of the predicted positives, how many are real?
Recall (sensitivity)	$TP / (TP + FN)$	of the real positives, how many did we find?
Specificity	$TN / (TN + FP)$	of the real negatives, how many did we clear?
F1 score	$2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$	balance of precision and recall

THE MATH BEHIND IT

Accuracy can lie. If 99% of samples are healthy, a model that always says "healthy" is 99% accurate yet useless. Use recall when missing a positive is costly (disease screening) and precision when false alarms are costly (spam filter that deletes real mail).

Many models output a **probability**; the **threshold** (default 0.5) turns it into a class. Moving it trades precision against recall. The **ROC curve** plots the true-positive rate against the false-positive rate over all thresholds; the **AUC** (area under it) is 0.5 for a coin flip and 1.0 for a perfect ranker.

```
from sklearn.metrics import (confusion_matrix, classification_report, roc_auc_score,
                             roc_curve, ConfusionMatrixDisplay, precision_recall_curve)

pipe.fit(Xtr, ytr)
pred = pipe.predict(Xte)
proba = pipe.predict_proba(Xte)[: , 1]

print(confusion_matrix(yte, pred))
print(classification_report(yte, pred, target_names=["malignant", "benign"], digits=3))
print("ROC AUC:", round(roc_auc_score(yte, proba), 4))

fpr, tpr, thr = roc_curve(yte, proba)
```

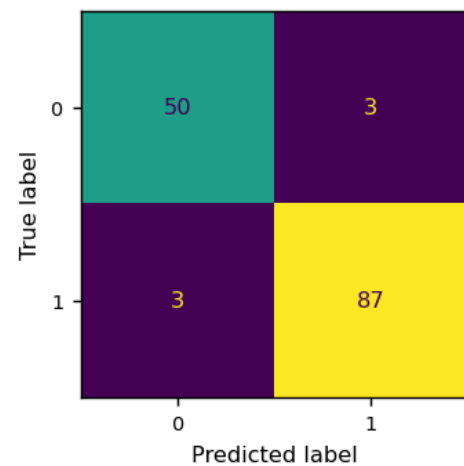
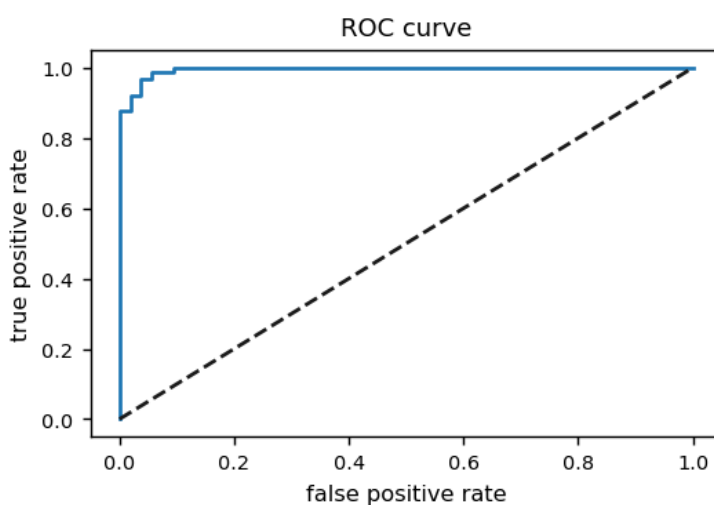
```
fig, axes = plt.subplots(1, 2, figsize=(8, 3))
axes[0].plot(fpr, tpr); axes[0].plot([0, 1], [0, 1], "k--")
axes[0].set_xlabel("false positive rate")
axes[0].set_ylabel("true positive rate"); axes[0].set_title("ROC curve")
ConfusionMatrixDisplay.from_predictions(yte, pred, ax=axes[1], colorbar=False)
plt.tight_layout(); plt.show()
```

OUTPUT

```
[[50  3]
 [ 3 87]]
```

	precision	recall	f1-score	support
malignant	0.943	0.943	0.943	53
benign	0.967	0.967	0.967	90
accuracy			0.958	143
macro avg	0.955	0.955	0.955	143
weighted avg	0.958	0.958	0.958	143

ROC AUC: 0.9952



18.5 Regression metrics

Metric	Formula	Notes
MAE	mean of $\text{abs}(y - \hat{y})$	typical error, same units as y , robust
MSE / RMSE	mean of $(y - \hat{y})^2$ / its square root	punishes large errors
R^2	$1 - \text{SSR}/\text{SST}$	fraction of variance explained (can be negative on test data!)
MAPE	mean of $\text{abs}((y - \hat{y})/y)$	relative error; breaks near $y = 0$

```
from sklearn.datasets import load_diabetes
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

Xd, yd = load_diabetes(return_X_y=True)
Xtr2, Xte2, ytr2, yte2 = train_test_split(Xd, yd, test_size=0.25, random_state=0)
reg = LinearRegression().fit(Xtr2, ytr2)
p = reg.predict(Xte2)
print("MAE :", round(mean_absolute_error(yte2, p), 1))
print("RMSE:", round(np.sqrt(mean_squared_error(yte2, p)), 1))
print("R2  :", round(r2_score(yte2, p), 3))
mean_pred = np.full(len(yte2), ytr2.mean())
print("baseline (predict the training mean) RMSE:",
      round(np.sqrt(mean_squared_error(yte2, mean_pred)), 1))
```

OUTPUT

MAE : 45.1

```
RMSE: 56.4
R2 : 0.359
baseline (predict the training mean) RMSE: 70.5
```

Always compare with a **baseline** (predict the mean, or the majority class). A model that cannot beat the baseline has learned nothing.

18.6 Key algorithms

k-Nearest Neighbours (k-NN)

To classify a new point, find the **k** training points **closest** to it (usually by Euclidean distance) and let them vote. It has no training phase, is easy to understand and needs scaled features. Small **k** gives jagged, high-variance boundaries; large **k** gives smooth, high-bias ones.

$$\text{distance}(a, b) = \sqrt{(\sum (a_j - b_j)^2)}$$

```
from sklearn.neighbors import KNeighborsClassifier

for k in [1, 5, 15, 51]:
    knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    s = cross_val_score(knn, Xtr, ytr, cv=cv).mean()
    print(f"k = {k:>2}: CV accuracy = {s:.3f}")
```

OUTPUT

```
k = 1: CV accuracy = 0.965
k = 5: CV accuracy = 0.967
k = 15: CV accuracy = 0.960
k = 51: CV accuracy = 0.946
```

Decision trees

A tree asks a sequence of yes/no questions ("is proline > 755?") that split the data into ever purer groups. At each node it picks the split that most reduces **impurity**, measured by the **Gini index** $1 - \sum p_k^2$ (0 = all one class) or entropy. Trees are interpretable and need no scaling, but a deep tree overfits.

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.datasets import load_wine

wine = load_wine(as_frame=True)
Xw, yw = wine.data, wine.target
tree = DecisionTreeClassifier(max_depth=2, random_state=0).fit(Xw, yw)
print(export_text(tree, feature_names=list(Xw.columns)))
```

OUTPUT

```
|--- proline <= 755.00
|   |--- od280/od315_of_diluted_wines <= 2.11
|   |   |--- class: 2
|   |   |--- od280/od315_of_diluted_wines > 2.11
|   |   |--- class: 1
|--- proline > 755.00
|   |--- flavanoids <= 2.17
|   |   |--- class: 2
|   |   |--- flavanoids > 2.17
|   |   |--- class: 0
```

Random forests and gradient boosting

An **ensemble** combines many models. A **random forest** trains hundreds of trees on random resamples of the data and random subsets of features, then averages their votes: the errors of individual trees cancel out, giving strong performance with little tuning. **Gradient boosting** builds trees one after another, each correcting the errors of the previous ones; it is often the most accurate method on tables.

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

models = {
    "logistic": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
    "k-NN (k=5)": make_pipeline(StandardScaler(), KNeighborsClassifier(5)),
    "tree (depth 3)": DecisionTreeClassifier(max_depth=3, random_state=0),
    "random forest": RandomForestClassifier(n_estimators=200, random_state=0),
    "gradient boosting": GradientBoostingClassifier(random_state=0),
}
for name, m in models.items():
    s = cross_val_score(m, Xtr, ytr, cv=cv)
    print(f"{name:18s} accuracy = {s.mean():.3f} +/- {s.std():.3f}")
```

OUTPUT

logistic	accuracy = 0.981 +/- 0.012
k-NN (k=5)	accuracy = 0.967 +/- 0.023
tree (depth 3)	accuracy = 0.923 +/- 0.022
random forest	accuracy = 0.962 +/- 0.009
gradient boosting	accuracy = 0.960 +/- 0.012

Support vector machines and naive Bayes

A **support vector machine (SVM)** finds the boundary with the widest **margin** between classes; with a *kernel* it can draw curved boundaries. **Naive Bayes** applies Bayes' rule with the simplifying ("naive") assumption that features are independent given the class; it is fast and works well on small or text data.

```
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB

for name, m in [("SVM (rbf)", make_pipeline(StandardScaler(), SVC())),
                ("naive Bayes", GaussianNB())]:
    print(f"{name:12s} accuracy = {cross_val_score(m, Xtr, ytr, cv=cv).mean():.3f}")
```

OUTPUT

SVM (rbf)	accuracy = 0.981
naive Bayes	accuracy = 0.939

18.7 Hyperparameter tuning

Search over settings using cross-validation with **GridSearchCV** (try every combination) or **RandomizedSearchCV** (sample combinations; better for many settings).

```
from sklearn.model_selection import GridSearchCV

param_grid = {"max_depth": [2, 4, 8, None], "min_samples_leaf": [1, 3, 10]}
gs = GridSearchCV(RandomForestClassifier(n_estimators=100, random_state=0),
                  param_grid, cv=cv, scoring="accuracy")
gs.fit(Xtr, ytr)
print("best parameters:", gs.best_params_)
print("best CV accuracy:", round(gs.best_score_, 3))
print("final test accuracy:", round(gs.score(Xte, yte), 3)) # used once, at the end
```

OUTPUT

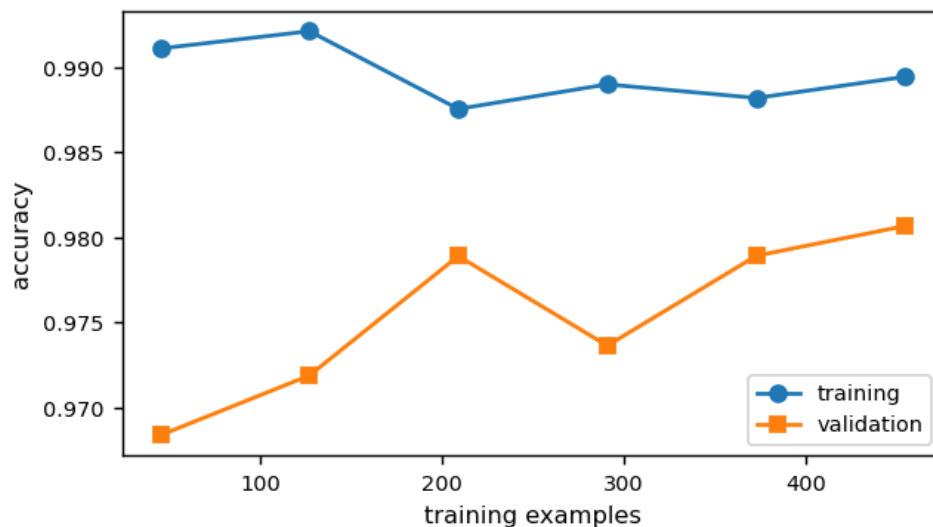
```
best parameters: {'max_depth': 8, 'min_samples_leaf': 1}
best CV accuracy: 0.967
```

```
final test accuracy: 0.944
```

18.8 Learning curves: do I need more data?

```
from sklearn.model_selection import learning_curve

lc_model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
sizes, tr, va = learning_curve(lc_model, Xb, yb, cv=5, shuffle=True, random_state=0,
                              train_sizes=np.linspace(0.1, 1.0, 6))
fig, ax = plt.subplots(figsize=(5.5, 3))
ax.plot(sizes, tr.mean(axis=1), "o-", label="training")
ax.plot(sizes, va.mean(axis=1), "s-", label="validation")
ax.set_xlabel("training examples"); ax.set_ylabel("accuracy"); ax.legend()
plt.show()
```



If the two curves have converged at a low score the model is too simple (get better features or a richer model). If there is a wide gap, the model overfits and more data or regularisation will help.

18.9 Interpreting models

Scientists need to know **why** a model predicts what it does.

- **Coefficients** of linear models (on standardised features) show direction and size of effects.
- **Impurity-based importance** in forests is fast but favours high-cardinality features.
- **Permutation importance** shuffles one feature at a time on held-out data and measures the drop in score; it is model-agnostic and more trustworthy.

```
from sklearn.inspection import permutation_importance

rf = RandomForestClassifier(n_estimators=200, random_state=0).fit(Xtr, ytr)
pi = permutation_importance(rf, Xte, yte, n_repeats=20, random_state=0)
cols = load_breast_cancer().feature_names
top = pd.Series(pi.importances_mean, index=cols).sort_values(ascending=False).head(5)
print(top.round(4))
```

```
OUTPUT
worst area                0.0087
worst concave points      0.0063
worst texture             0.0063
worst symmetry            0.0042
mean concave points       0.0042
dtype: float64
```

WATCH OUT

Feature importance shows what the **model** uses, not what **causes** the outcome. Correlated features share importance, and a model may rely on a confounder. Predictive accuracy does not establish causation.

18.10 Good ML practice for science

1. Define the prediction task and the cost of each error before modelling.
2. Split data in a way that mirrors deployment (by time, by subject, by site).
3. Start with a baseline and a simple model; add complexity only if it helps under cross-validation.
4. Put preprocessing inside pipelines; never tune on the test set.
5. Report uncertainty (cross-validation spread, confidence intervals), not a single number.
6. Check failure cases and subgroups; examine residuals or misclassified examples.
7. Save the model with `joblib.dump(model, "model.joblib")` and record library versions for reproducibility.

```
import joblib
joblib.dump(pipe, "cancer_model.joblib")
reloaded = joblib.load("cancer_model.joblib")
print("reloaded model accuracy:", round(reloaded.score(Xte, yte), 3))
```

OUTPUT

```
reloaded model accuracy: 0.958
```

18.11 Chapter summary

CHAPTER SUMMARY

- ML optimises prediction on **new** data. Split into train / validation / test and touch the test set once.
- Underfitting = bias, overfitting = variance; cross-validation finds the sweet spot.
- Choose metrics that match the problem: precision, recall, F1, ROC-AUC for classification; MAE, RMSE, R^2 for regression; always compare with a baseline.
- k-NN, trees, random forests, gradient boosting, SVM, logistic regression and naive Bayes cover most tabular problems.
- Tune hyperparameters with cross-validated search; diagnose with learning curves; interpret with permutation importance.

18.12 Practice exercises

1. On the wine data, compare three classifiers with 5-fold cross-validation and report mean $\pm$ std accuracy.
2. Build a model on an imbalanced dataset (simulate 95% vs 5%) and compare accuracy with recall for the rare class. Try `class_weight="balanced"`.
3. Tune `n_neighbors` for k-NN on the breast cancer data using `GridSearchCV`.
4. Plot a validation curve of a decision tree's `max_depth` from 1 to 15 for train and validation accuracy.
5. Compute permutation importances for a random forest on the diabetes regression task.

CHAPTER 19

19. Unsupervised Learning: Clustering and Dimensionality Reduction

IN THIS CHAPTER YOU WILL LEARN

- Finding groups with k-means, hierarchical clustering, DBSCAN and Gaussian mixtures
- Choosing the number of clusters (elbow, silhouette)
- PCA from scratch: covariance, eigenvectors, explained variance
- Visualising high-dimensional data (PCA, t-SNE)
- Detecting anomalies

In **unsupervised learning** there is no target column. We ask the data: *are there natural groups? Can the many variables be summarised by a few? Which points are odd?*

19.1 Distance: the basis of clustering

Clustering puts **similar** points together, so we need a definition of "similar". The usual choice is **Euclidean distance**, the straight-line distance:

$$d(a, b) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots}$$

Other choices: **Manhattan** (sum of absolute differences, like city blocks), **cosine** (angle between vectors, good for text), **correlation distance** (good for profiles such as gene expression). Because distance adds up differences, **features must be scaled** first.

19.2 k-means clustering

k-means divides data into k groups. The algorithm is wonderfully simple:

1. Choose k and place k **centroids** (cluster centres) at random.
2. **Assign** every point to its nearest centroid.
3. **Update** each centroid to the mean of the points assigned to it.
4. Repeat steps 2 and 3 until nothing changes.

It minimises the **inertia** (within-cluster sum of squares): $\sum$ over all points of the squared distance to their centroid. To see exactly how it works, here it is from scratch in a dozen lines:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

X, true_labels = make_blobs(n_samples=300, centers=3, cluster_std=1.1, random_state=7)

def kmeans(X, k, n_iter=20, seed=0):
    rng = np.random.default_rng(seed)
    centroids = X[rng.choice(len(X), k, replace=False)] # random starting centres
```

```

for _ in range(n_iter):
    dist = np.linalg.norm(X[:, None, :] - centroids[None, :, :], axis=2) # (n, k) distances
    labels = dist.argmax(axis=1) # nearest centroid
    new = np.array([X[labels == j].mean(axis=0) for j in range(k)]) # recompute means
    if np.allclose(new, centroids):
        break
    centroids = new
inertia = ((X - centroids[labels]) ** 2).sum()
return labels, centroids, inertia

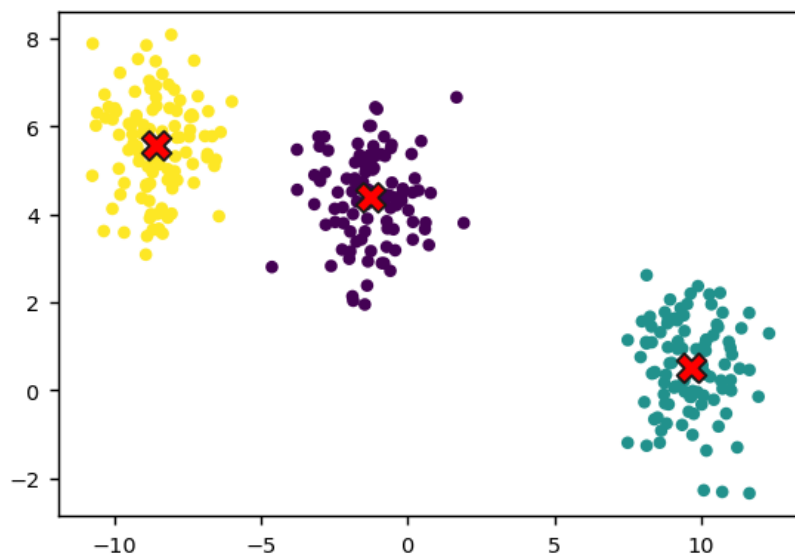
labels, centroids, inertia = kmeans(X, 3)
print("cluster sizes:", np.bincount(labels), " inertia:", round(inertia, 1))

fig, ax = plt.subplots(figsize=(5, 3.4))
ax.scatter(X[:, 0], X[:, 1], c=labels, cmap="viridis", s=14)
ax.scatter(centroids[:, 0], centroids[:, 1], c="red", marker="X", s=120, edgecolor="k")
plt.show()

```

OUTPUT

```
cluster sizes: [100 100 100] inertia: 693.8
```



The same with scikit-learn (which adds smarter starting points, **k-means++**, and several restarts):

```

from sklearn.cluster import KMeans

km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(X)
print("inertia:", round(km.inertia_, 1))
print("centres:\n", km.cluster_centers_.round(2))
print(km.predict([[0, 0], [-8, -7]])) # assign new points to a cluster

```

OUTPUT

```

inertia: 693.8
centres:
[[-8.56  5.57]
 [ 9.64  0.52]
 [-1.26  4.38]]
[2 0]

```

How many clusters?

k-means needs k in advance. Two common guides:

- **Elbow method:** plot inertia against k; choose where the curve bends (extra clusters stop helping much).
- **Silhouette score:** for each point compare the average distance to its own cluster (a) with the average distance to the nearest other cluster (b): $s = (b - a) / \max(a, b)$. It ranges from -1 to 1;

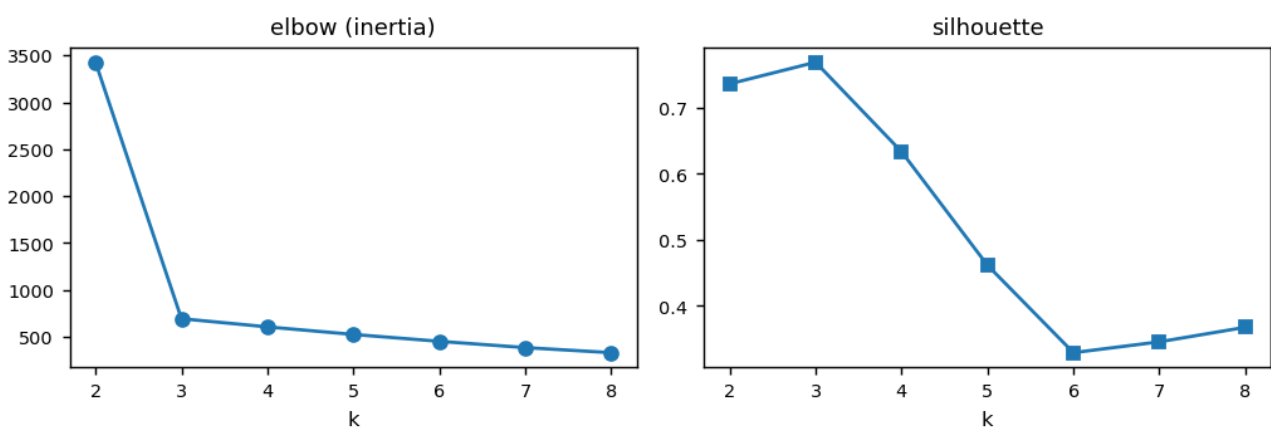
higher is better, and the mean over all points rates the clustering.

```
from sklearn.metrics import silhouette_score

ks = range(2, 9)
inertias, sils = [], []
for k in ks:
    m = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    inertias.append(m.inertia_)
    sils.append(silhouette_score(X, m.labels_))

fig, axes = plt.subplots(1, 2, figsize=(8, 2.8))
axes[0].plot(ks, inertias, "o-"); axes[0].set_xlabel("k"); axes[0].set_title("elbow (inertia)")
axes[1].plot(ks, sils, "s-"); axes[1].set_xlabel("k"); axes[1].set_title("silhouette")
plt.tight_layout(); plt.show()
print("best k by silhouette:", list(ks)[int(np.argmax(sils))])
```

OUTPUT
best k by silhouette: 3



WATCH OUT

k-means assumes roughly **round, similarly sized** clusters and is sensitive to scale and outliers. On crescent or ring shapes it fails; try DBSCAN or a Gaussian mixture. Cluster numbers (0, 1, 2) are arbitrary labels and can change between runs.

19.3 Hierarchical clustering

Hierarchical (agglomerative) clustering starts with every point as its own cluster and repeatedly **merges the two closest clusters** until one remains. The merge history is a tree called a **dendrogram**: cut it at a chosen height to get clusters. It needs no k in advance and suits small datasets and biological data.

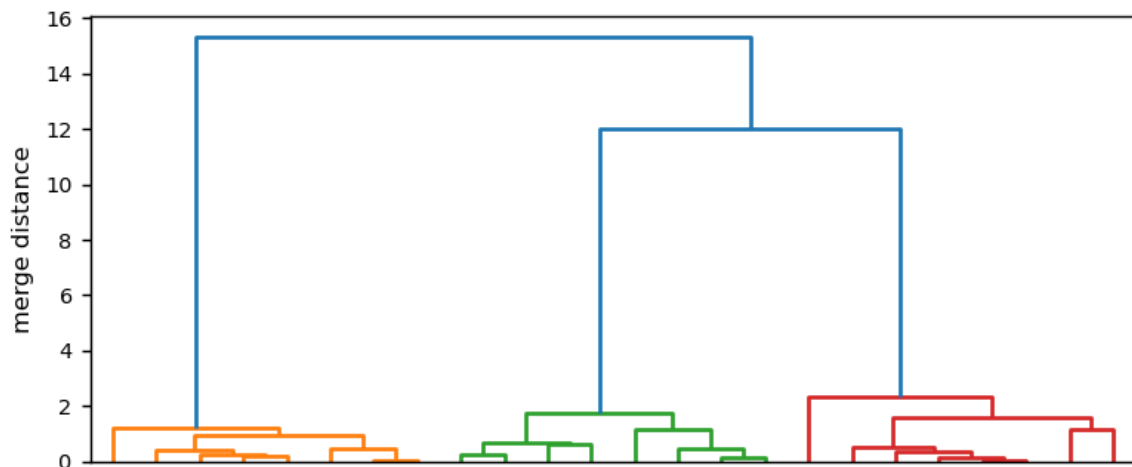
The distance between two clusters is defined by the **linkage**: **single** (closest members), **complete** (farthest members), **average**, or **ward** (merge that increases within-cluster variance the least; usually the best default).

```
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

rng = np.random.default_rng(1)
small = np.vstack([rng.normal(0, 0.5, (8, 2)),
                  rng.normal(4, 0.5, (8, 2)),
                  rng.normal([0, 5], 0.5, (8, 2))])
Z = linkage(small, method="ward")
fig, ax = plt.subplots(figsize=(7, 3))
dendrogram(Z, ax=ax, no_labels=True)
ax.set_ylabel("merge distance")
plt.show()
```

```
groups = fcluster(Z, t=3, criterion="maxclust")    # cut the tree into 3 clusters
print(np.bincount(groups)[1:])
```

OUTPUT
[8 8 8]



Tall vertical lines in the dendrogram show merges of very different groups; cut just below the tallest jump.

19.4 DBSCAN: density-based clustering

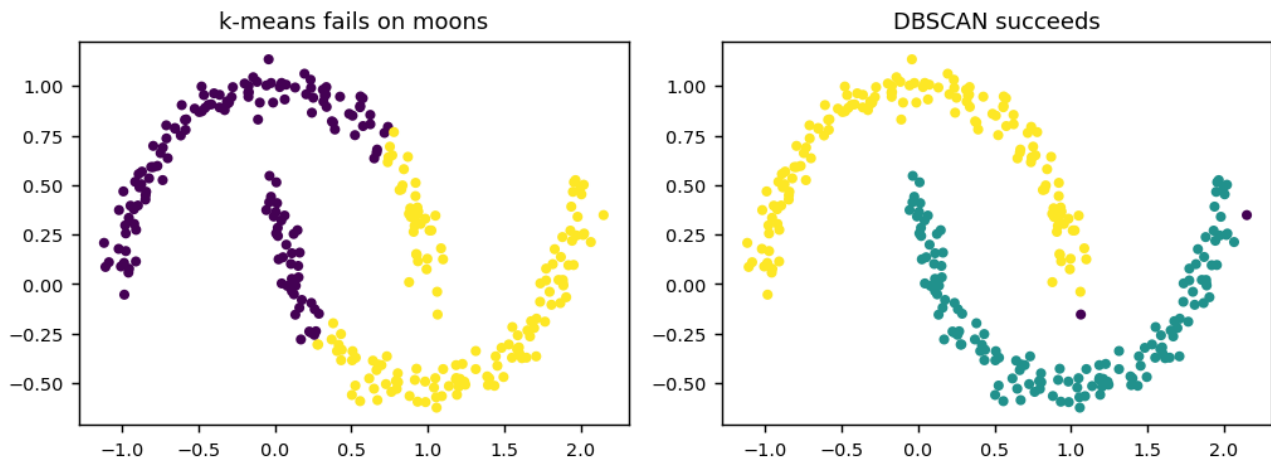
DBSCAN finds clusters as dense regions separated by sparse areas. You give two numbers: `eps` (neighbourhood radius) and `min_samples` (how many neighbours make a point "dense"). Points that belong to no dense region are labelled **noise** (-1), so DBSCAN handles outliers and arbitrary shapes and needs no `k`.

```
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN

M, _ = make_moons(n_samples=300, noise=0.06, random_state=0)
km_labels = KMeans(2, n_init=10, random_state=0).fit_predict(M)
db_labels = DBSCAN(eps=0.15, min_samples=5).fit_predict(M)
print("DBSCAN clusters:", sorted(set(db_labels) - {-1}),
      " noise points:", int((db_labels == -1).sum()))

fig, axes = plt.subplots(1, 2, figsize=(8, 3))
axes[0].scatter(M[:, 0], M[:, 1], c=km_labels, s=12); axes[0].set_title("k-means fails on moons")
axes[1].scatter(M[:, 0], M[:, 1], c=db_labels, s=12); axes[1].set_title("DBSCAN succeeds")
plt.tight_layout(); plt.show()
```

OUTPUT
DBSCAN clusters: [np.int64(0), np.int64(1)] noise points: 2



19.5 Gaussian mixture models

A **Gaussian mixture** assumes the data is a blend of several bell-shaped (normal) clusters, each with its own mean and covariance, and gives each point a **probability** of belonging to each cluster ("soft clustering"). It copes with elongated clusters; you can choose the number of components with the BIC criterion.

```
from sklearn.mixture import GaussianMixture

stretch = X @ np.array([[1.0, 0.8], [0.0, 0.6]]) # make clusters elongated
for k in range(1, 6):
    gm = GaussianMixture(k, random_state=0, n_init=3).fit(stretch)
    print(f"components = {k}: BIC = {gm.bic(stretch):.1f}")
gm = GaussianMixture(3, random_state=0, n_init=3).fit(stretch)
print("membership probabilities of first point:", gm.predict_proba(stretch[:1]).round(3))
```

OUTPUT

```
components = 1: BIC = 2732.1
components = 2: BIC = 2429.0
components = 3: BIC = 2237.0
components = 4: BIC = 2260.5
components = 5: BIC = 2289.6
membership probabilities of first point: [[1. 0. 0.]]
```

The lowest BIC marks the best balance between fit and complexity.

19.6 Principal Component Analysis (PCA)

With 50 measured variables we cannot picture the data, and many variables carry overlapping information. **PCA** finds a new set of axes, the **principal components**, that are **uncorrelated** and ordered so that the first captures the most variance, the second the most of what remains, and so on. Keeping only the first few gives a compact summary with little loss of information.

The idea in a picture

Imagine a cloud of points stretched along a diagonal. The first principal component is the **direction of greatest spread** (the long axis); the second is perpendicular to it.

The math, step by step

1. **Centre** each variable (subtract its mean). Usually also **scale** to unit variance.
2. Compute the **covariance matrix** C : its entry C_{jk} is the covariance of variables j and k (Chapter 12).

3. Find the **eigenvectors** and **eigenvalues** of C (Chapter 5). Each eigenvector is a principal component direction; its eigenvalue λ is the variance of the data along it.
4. Sort by eigenvalue. The **explained variance ratio** of component i is $\lambda_i / \sum \lambda$.
5. **Project** the data: scores = (centred data) $\times$ (eigenvectors).

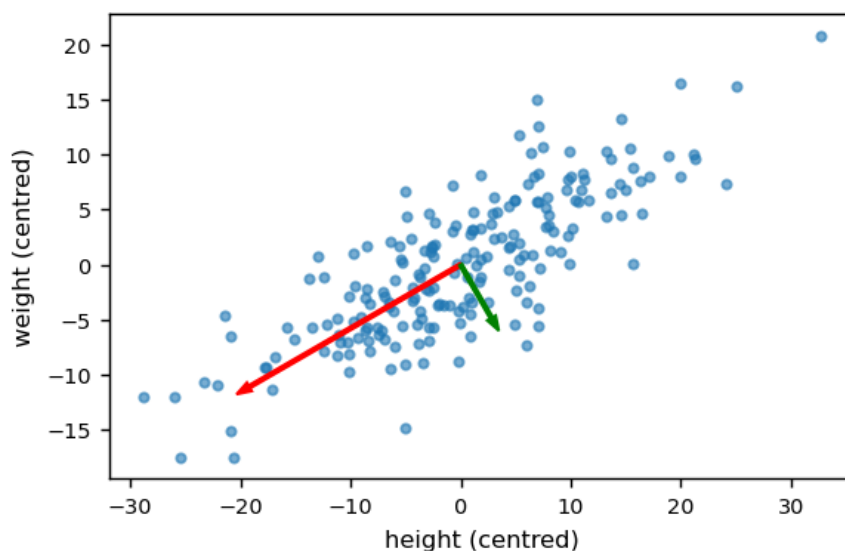
```
rng = np.random.default_rng(3)
n = 200
h = rng.normal(170, 10, n) # height
w = 0.5 * (h - 170) + rng.normal(0, 4, n) + 70 # weight, correlated with height
data = np.column_stack([h, w])

centred = data - data.mean(axis=0)
C = np.cov(centred.T)
eigvals, eigvecs = np.linalg.eigh(C) # eigh: for symmetric matrices, ascending order
order = np.argsort(eigvals)[::-1]
eigvals, eigvecs = eigvals[order], eigvecs[:, order]
print("covariance matrix:\n", C.round(1))
print("eigenvalues (variance along each PC):", eigvals.round(1))
print("explained variance ratio:", (eigvals / eigvals.sum()).round(3))
print("first PC direction:", eigvecs[:, 0].round(3))

fig, ax = plt.subplots(figsize=(5, 3.6))
ax.scatter(centred[:, 0], centred[:, 1], s=12, alpha=0.6)
for i, col in enumerate(["red", "green"]):
    v = eigvecs[:, i] * 2 * np.sqrt(eigvals[i])
    ax.arrow(0, 0, v[0], v[1], color=col, width=0.3, length_includes_head=True)
ax.set_aspect("equal"); ax.set_xlabel("height (centred)"); ax.set_ylabel("weight (centred)")
plt.show()
```

OUTPUT

```
covariance matrix:
[[106.6  54.7]
 [ 54.7  43.6]]
eigenvalues (variance along each PC): [138.2  12. ]
explained variance ratio: [0.92 0.08]
first PC direction: [-0.866 -0.5 ]
```



PCA with scikit-learn

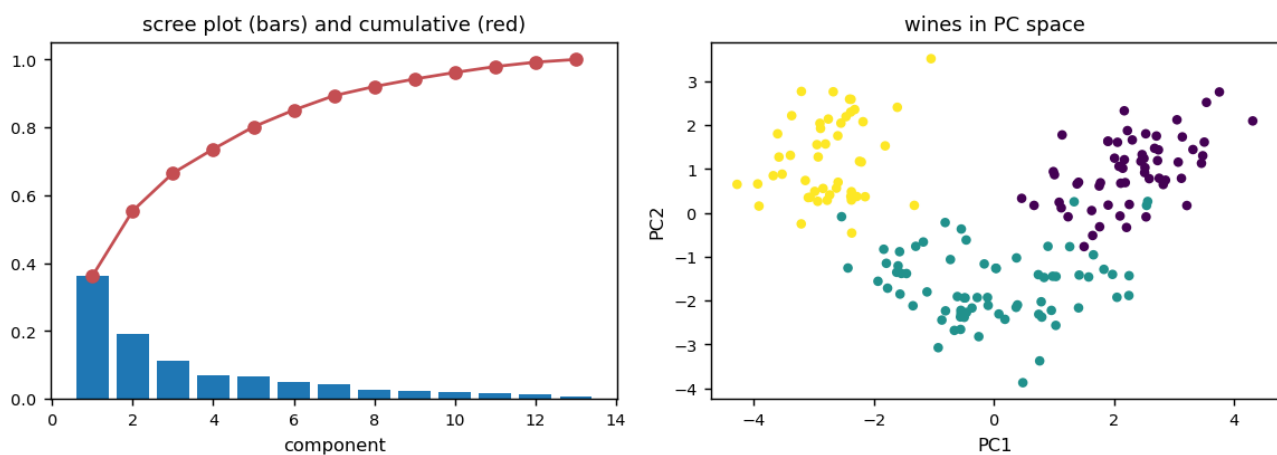
```
from sklearn.decomposition import PCA
from sklearn.datasets import load_wine

wine = load_wine(as_frame=True)
Xw = StandardScaler().fit_transform(wine.data)
pca = PCA().fit(Xw)
ratio = pca.explained_variance_ratio_
print("explained variance per PC:", ratio.round(3)[:5])
print("cumulative:", ratio.cumsum().round(3)[:5])

scores = pca.transform(Xw)
fig, axes = plt.subplots(1, 2, figsize=(9, 3.3))
axes[0].bar(range(1, 14), ratio); axes[0].plot(range(1, 14), ratio.cumsum(), "ro-")
axes[0].set_xlabel("component"); axes[0].set_title("scree plot (bars) and cumulative (red)")
axes[1].scatter(scores[:, 0], scores[:, 1], c=wine.target, cmap="viridis", s=14)
axes[1].set_xlabel("PC1"); axes[1].set_ylabel("PC2"); axes[1].set_title("wines in PC space")
plt.tight_layout(); plt.show()
```

OUTPUT

```
explained variance per PC: [0.362 0.192 0.111 0.071 0.066]
cumulative: [0.362 0.554 0.665 0.736 0.802]
```



The 13 chemical measurements collapse into two components that already separate the three cultivars. Common rule: keep enough components to explain 80 to 95% of the variance, or look for the elbow in the scree plot.

Loadings: what does each PC mean?

The **loadings** (the eigenvector entries) say how much each original variable contributes to a component.

```
loadings = pd.DataFrame(pca.components_[:2].T, index=wine.data.columns, columns=["PC1", "PC2"])
print(loadings.round(2).sort_values("PC1").iloc[[0, 1, 2, -3, -2, -1]])
```

OUTPUT

	PC1	PC2
nonflavanoid_phenols	-0.30	0.03
malic_acid	-0.25	0.22
alcalinity_of_ash	-0.24	-0.01
od280/od315_of_diluted_wines	0.38	-0.16
total_phenols	0.39	0.07
flavonoids	0.42	-0.00

TIP

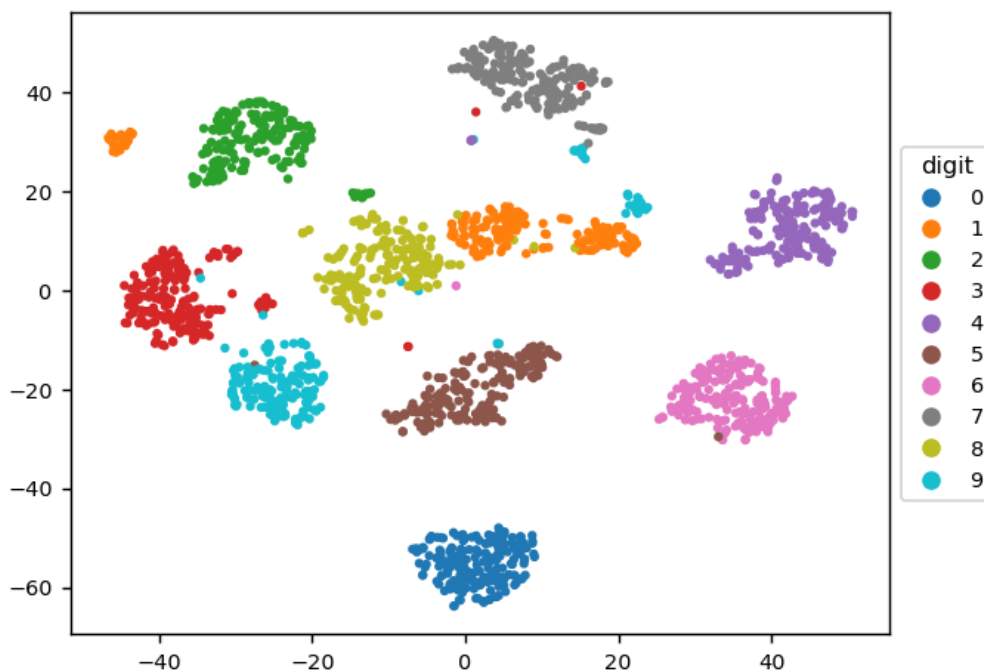
PCA is also a great preprocessing step: it removes correlation between predictors (curing multicollinearity, Chapter 16) and reduces noise. Always standardise before PCA unless the variables share units and scale truly matters.

19.7 t-SNE and UMAP for visualisation

PCA is linear. **t-SNE** (and the similar **UMAP**, `pip install umap-learn`) squeezes high-dimensional data into 2D while keeping **neighbours close**, revealing clusters that PCA misses. It is for **visualisation only**: distances between far-apart clusters and cluster sizes are not meaningful, and results depend on the **perplexity** setting (roughly the number of neighbours considered).

```
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE

digits = load_digits() # 1,797 images of handwritten digits, 64 pixels each
emb = TSNE(n_components=2, perplexity=30, random_state=0, init="pca").fit_transform(digits.data)
fig, ax = plt.subplots(figsize=(5.5, 4.2))
sc = ax.scatter(emb[:, 0], emb[:, 1], c=digits.target, cmap="tab10", s=6)
ax.legend(*sc.legend_elements(), loc="center left", bbox_to_anchor=(1, 0.5), title="digit")
plt.show()
```



19.8 Anomaly detection

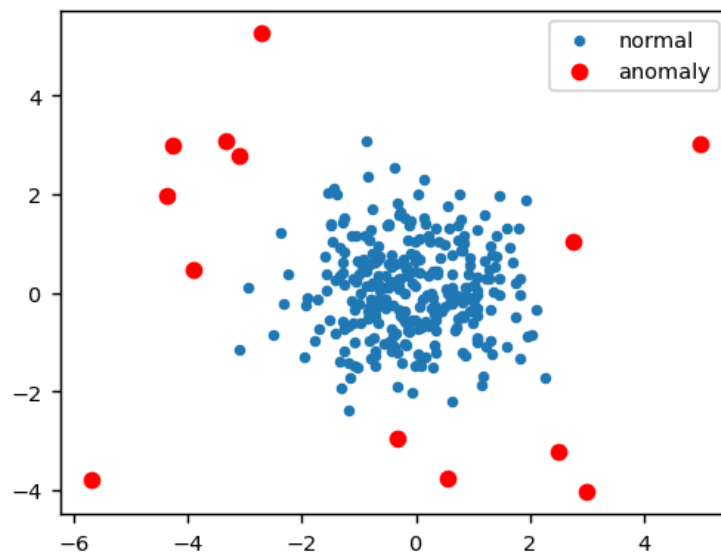
Anomalies (outliers) can be errors or the most interesting discoveries. Approaches: statistical rules (z-score, IQR; Chapter 7), distance-based (far from all neighbours), and model-based methods such as **Isolation Forest**, which randomly splits the data: anomalies are isolated in fewer splits.

```
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(0)
normal = rng.normal(0, 1, (300, 2))
odd = rng.uniform(-6, 6, (12, 2))
pts = np.vstack([normal, odd])

iso = IsolationForest(contamination=0.04, random_state=0).fit(pts)
flag = iso.predict(pts) # -1 = anomaly, 1 = normal
print("flagged anomalies:", int((flag == -1).sum()))
fig, ax = plt.subplots(figsize=(4.5, 3.4))
ax.scatter(pts[flag == 1, 0], pts[flag == 1, 1], s=10, label="normal")
ax.scatter(pts[flag == -1, 0], pts[flag == -1, 1], s=30, c="red", label="anomaly")
ax.legend(); plt.show()
```

OUTPUT
flagged anomalies: 13



19.9 Evaluating clusters

With no true labels, use internal measures: silhouette, inertia, stability across random restarts. If you do have reference labels (to test the method), use the **adjusted Rand index** (1 = perfect, about 0 = random).

```
from sklearn.metrics import adjusted_rand_score

km3 = KMeans(3, n_init=10, random_state=0).fit(X)
print("ARI vs true blobs:", round(adjusted_rand_score(true_labels, km3.labels_), 3))
print("ARI of DBSCAN on moons vs truth:",
      round(adjusted_rand_score(make_moons(300, noise=0.06, random_state=0)[1], db_labels), 3))
```

OUTPUT
ARI vs true blobs: 1.0
ARI of DBSCAN on moons vs truth: 0.987

Finally, **interpret** clusters: compute the mean of each original variable per cluster and ask whether the groups make scientific sense.

19.10 Chapter summary

CHAPTER SUMMARY

- Scale first; distance defines similarity.
- k-means: assign to nearest centroid, update the centroids; choose k with the elbow and silhouette. Good for round clusters.
- Hierarchical clustering gives a dendrogram; DBSCAN finds dense shapes and noise; Gaussian mixtures give soft, elliptical clusters.
- PCA = eigenvectors of the covariance matrix. Explained variance ratios and the scree plot decide how many components to keep; loadings interpret them.
- t-SNE/UMAP are for visualising structure, not for measuring distances.
- Isolation Forest and statistical rules flag anomalies.

19.11 Practice exercises

1. Cluster the iris data (without its labels) with k-means for $k = 2$ to 6 and pick k by silhouette. Compare with the true species using ARI.
2. Perform PCA on the breast cancer data; how many components explain 90% of the variance?
3. Run DBSCAN on a two-circles dataset (`make_circles`) and tune `eps`.
4. Reproduce PCA by hand (eigen-decomposition) on the diabetes data and confirm it equals `sklearn`'s result up to sign.
5. Add 5 extreme points to a dataset and compare how k-means and DBSCAN react.

CHAPTER 20

20. Time Series Analysis

IN THIS CHAPTER YOU WILL LEARN

- Time-indexed data: trend, seasonality, cycles, noise
- Moving averages, decomposition, differencing
- Stationarity and autocorrelation (ACF, PACF)
- AR models, exponential smoothing, ARIMA ideas
- Forecasting properly: baselines, walk-forward validation

A **time series** is a sequence of measurements in time order: daily temperature, a patient's heart rate, yearly CO₂, a telescope's light curve. Time order creates a special feature: neighbouring values are **correlated** (autocorrelated), which breaks the independence assumption of most classical methods, so we need specialised tools.

20.1 A synthetic monthly series

We build a series with known parts (trend + yearly season + noise) so we can see whether our methods recover them.

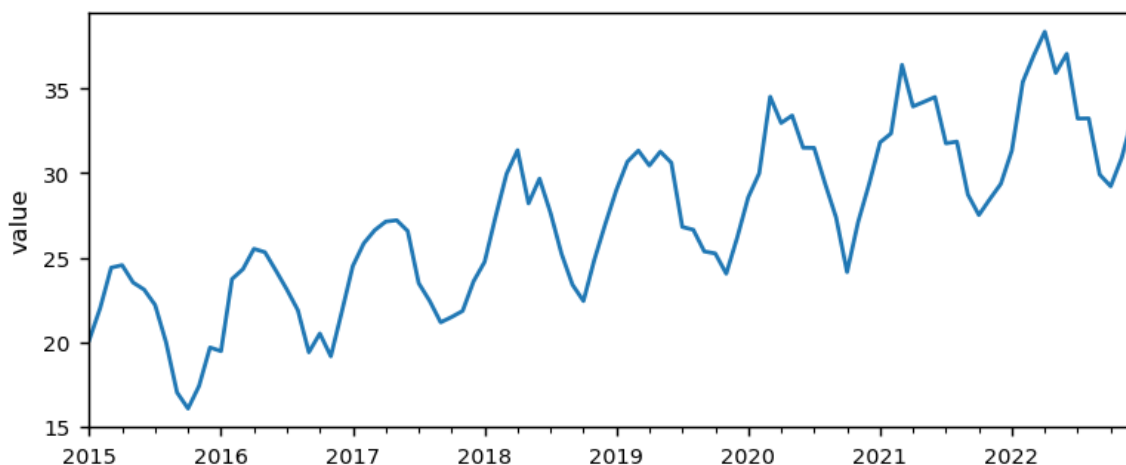
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
idx = pd.date_range("2015-01-01", periods=96, freq="MS") # 8 years, month starts
t = np.arange(96)
trend = 0.15 * t + 20
season = 4 * np.sin(2 * np.pi * t / 12)
noise = rng.normal(0, 1.0, 96)
ts = pd.Series(trend + season + noise, index=idx, name="temperature")

print(ts.head(3).round(2))
fig, ax = plt.subplots(figsize=(7, 2.8))
ts.plot(ax=ax); ax.set_ylabel("value")
plt.show()
```

OUTPUT

```
2015-01-01    20.13
2015-02-01    22.02
2015-03-01    24.40
Freq: MS, Name: temperature, dtype: float64
```



20.2 Components of a time series

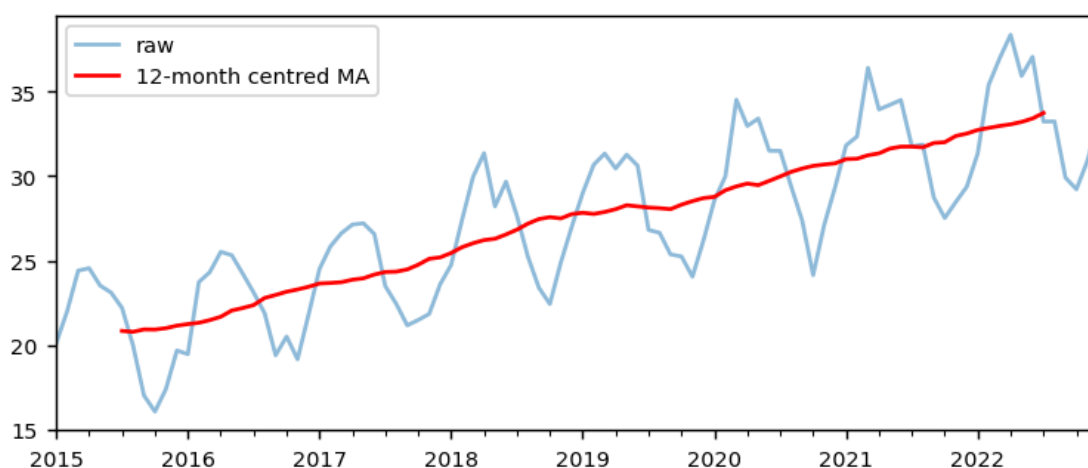
Component	Meaning	Example
Trend	long-term increase or decrease	global warming
Seasonality	fixed-period repetition	yearly cycle, daily tide
Cycle	irregular rises and falls, no fixed period	economic booms
Noise (residual)	random variation	measurement error

- **Additive** model: $y = \text{trend} + \text{season} + \text{noise}$ (seasonal swings have constant size).
- **Multiplicative** model: $y = \text{trend} \times \text{season} \times \text{noise}$ (swings grow with the level); a log transform makes it additive.

20.3 Smoothing with moving averages

A **moving average** averages each point with its neighbours, suppressing noise and revealing the trend. A window equal to the seasonal period (12 months) cancels the seasonal pattern.

```
ma12 = ts.rolling(window=12, center=True).mean()
fig, ax = plt.subplots(figsize=(7, 2.8))
ts.plot(ax=ax, alpha=0.5, label="raw")
ma12.plot(ax=ax, color="red", label="12-month centred MA")
ax.legend(); plt.show()
```



20.4 Decomposition

Classical decomposition estimates the parts step by step: (1) trend by a centred moving average; (2) detrend (subtract the trend); (3) seasonal component = average detrended value for each month; (4) residual = what is left.

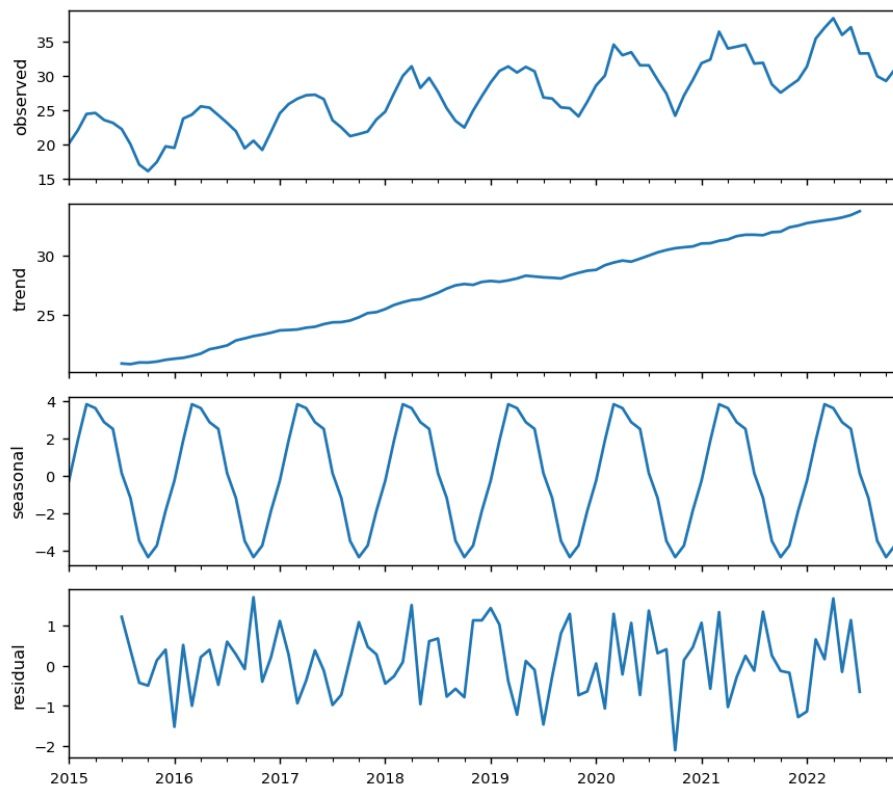
```
trend_est = ts.rolling(window=12, center=True).mean()
detrended = ts - trend_est
seasonal_est = detrended.groupby(detrended.index.month).mean()
seasonal_est = seasonal_est - seasonal_est.mean() # centre at zero
season_series = pd.Series(seasonal_est.loc[ts.index.month].values, index=ts.index)
resid = ts - trend_est - season_series

print("estimated seasonal pattern (by month):")
print(seasonal_est.round(2).to_dict())
print("residual std:", round(resid.std(), 2), " (true noise std = 1.0)")

fig, axes = plt.subplots(4, 1, figsize=(7, 6), sharex=True)
for ax, s, name in zip(axes, [ts, trend_est, season_series, resid],
                          ["observed", "trend", "seasonal", "residual"]):
    s.plot(ax=ax); ax.set_ylabel(name)
plt.tight_layout(); plt.show()
```

OUTPUT

```
estimated seasonal pattern (by month):
{1: -0.26, 2: 1.89, 3: 3.83, 4: 3.62, 5: 2.87, 6: 2.51, 7: 0.14, 8: -1.18, 9: -3.48, 10: -4.35, 11: -3.73, 12: -1.87}
residual std: 0.84 (true noise std = 1.0)
```



With `statsmodels` the same thing is one call:

```
from statsmodels.tsa.seasonal import seasonal_decompose, STL
seasonal_decompose(ts, model="additive", period=12).plot()
STL(ts, period=12).fit().plot() # STL: robust to outliers, allows changing seasonality
```

20.5 Stationarity

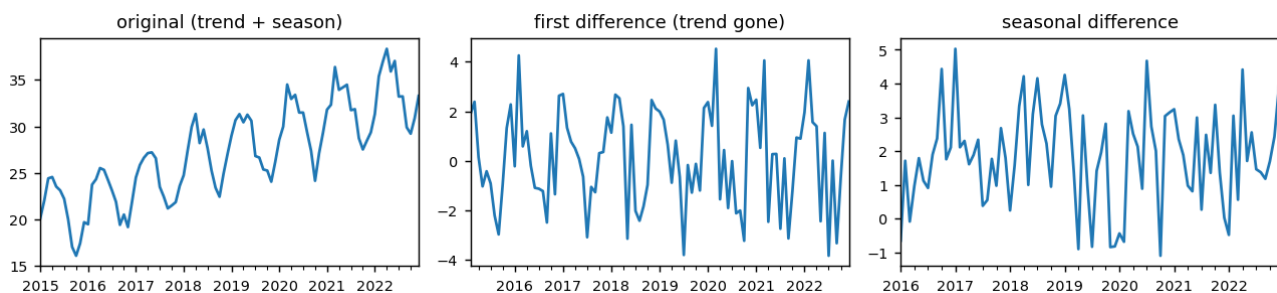
Most forecasting models need a **stationary** series: its mean, variance and autocorrelation structure do not change over time. A trend or a growing seasonal swing breaks stationarity. Trends are removed by **differencing**: replace y_t by $y_t - y_{t-1}$. Seasonal patterns by **seasonal differencing**: $y_t - y_{t-12}$.

```
diff1 = ts.diff().dropna()
diff12 = ts.diff(12).dropna()

fig, axes = plt.subplots(1, 3, figsize=(10, 2.4))
ts.plot(ax=axes[0], title="original (trend + season)")
diff1.plot(ax=axes[1], title="first difference (trend gone)")
diff12.plot(ax=axes[2], title="seasonal difference")
plt.tight_layout(); plt.show()
print("mean of first half / second half of original:",
      ts[:48].mean().round(1), ts[48:].mean().round(1))
print("mean of first half / second half of differenced:",
      diff12[:45].mean().round(2), diff12[45:].mean().round(2))
```

OUTPUT

```
mean of first half / second half of original: 23.6 30.9
mean of first half / second half of differenced: 1.92 1.75
```



The **Augmented Dickey-Fuller (ADF) test** formally tests H_0 : "the series has a unit root (non-stationary)"; a small p-value supports stationarity (`statsmodels.tsa.stattools.adfuller`). The KPSS test has the opposite null.

20.6 Autocorrelation

The **autocorrelation at lag k** is the correlation between the series and itself shifted k steps:

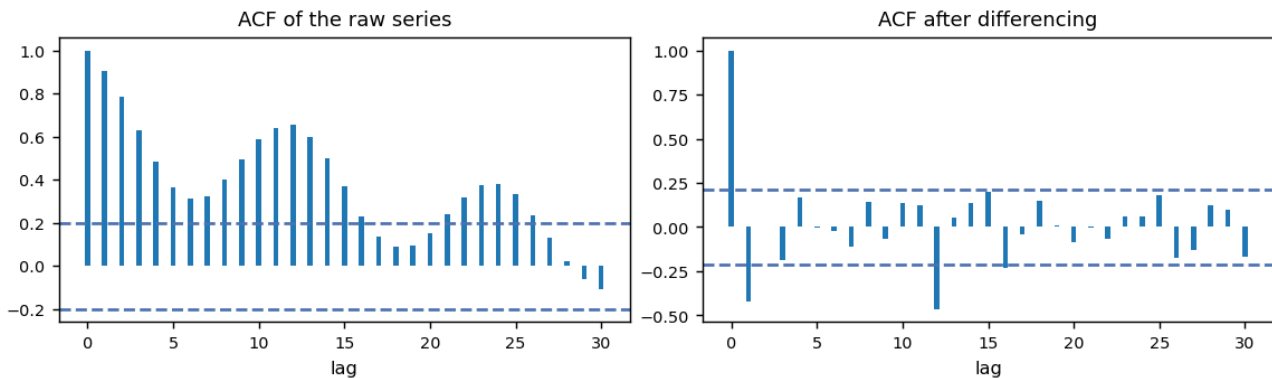
$$r_k = \frac{\sum (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum (y_t - \bar{y})^2}$$

The plot of r_k against k is the **ACF**. A slowly decaying ACF indicates a trend; spikes at lag 12, 24 indicate yearly seasonality; values inside the blue band $\pm 1.96/\sqrt{n}$ are not significantly different from zero. The **PACF** (partial autocorrelation) removes the influence of the shorter lags, isolating each lag's direct effect, and is used to choose the order of AR models.

```
def acf(x, nlags):
    x = np.asarray(x) - np.mean(x)
    denom = (x ** 2).sum()
    return np.array([(x[k:] * x[:len(x) - k]).sum() / denom for k in range(nlags + 1)])

fig, axes = plt.subplots(1, 2, figsize=(9, 2.8))
for ax, series, title in [(axes[0], ts, "ACF of the raw series"),
                          (axes[1], ts.diff().diff(12).dropna(), "ACF after differencing")]:
    r = acf(series, 30)
    ax.bar(range(31), r, width=0.3)
    band = 1.96 / np.sqrt(len(series))
    ax.axhline(band, color="b", linestyle="--"); ax.axhline(-band, color="b", linestyle="--")
    ax.set_title(title); ax.set_xlabel("lag")
```

```
plt.tight_layout(); plt.show()
```



20.7 Autoregressive models

An **AR(p)** model predicts the next value from the last p values:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p} + \varepsilon_t$$

For AR(1), ϕ is the "memory": $\phi = 0.9$ means today is 90% like yesterday (slow wandering); $\phi = 0$ is pure noise; ϕ close to 1 approaches a **random walk**. An AR model is just a regression of the series on its own lags, so we can fit it with least squares (Chapter 16).

```
rng = np.random.default_rng(5)
phi_true, n = 0.8, 500
y = np.zeros(n)
for i in range(1, n):
    y[i] = phi_true * y[i - 1] + rng.normal()

# fit AR(1) by regressing y_t on y_{t-1}
X = np.column_stack([np.ones(n - 1), y[:-1]])
coef, *_ = np.linalg.lstsq(X, y[1:], rcond=None)
print("estimated phi:", coef[1].round(3), " (true 0.8)")

r = acf(y, 5)
print("ACF lags 1-4:", r[1:5].round(2), " theory phi^k:", (0.8 ** np.arange(1, 5)).round(2))
```

OUTPUT

```
estimated phi: 0.825 (true 0.8)
ACF lags 1-4: [0.82 0.67 0.57 0.46] theory phi^k: [0.8 0.64 0.51 0.41]
```

For an AR(1) process the ACF decays geometrically as ϕ^k , a fingerprint used to recognise the model.

ARIMA and SARIMA

ARIMA(p, d, q) combines three ideas: **AR(p)** terms for the series' own past, **I(d)**: differencing d times to remove trend, and **MA(q)**: moving-average terms for past forecast errors. **SARIMA** adds seasonal terms, written (P, D, Q, s). Choose orders from ACF/PACF plots, or by comparing AIC across candidates.

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

model = SARIMAX(ts, order=(1, 1, 1), seasonal_order=(0, 1, 1, 12)).fit(dis= False)
print(model.summary())
forecast = model.get_forecast(steps=12)
fc_mean = forecast.predicted_mean
fc_ci = forecast.conf_int() # prediction intervals widen as we look further ahead
```

`pmdarima.auto_arima` (installable with `!pip install pmdarima`) searches orders automatically.

20.8 Exponential smoothing

Simple exponential smoothing forecasts with a weighted average of past values where weights decay exponentially, giving recent data more influence:

$$level_t = \alpha \cdot y_t + (1 - \alpha) \cdot level_{t-1}$$

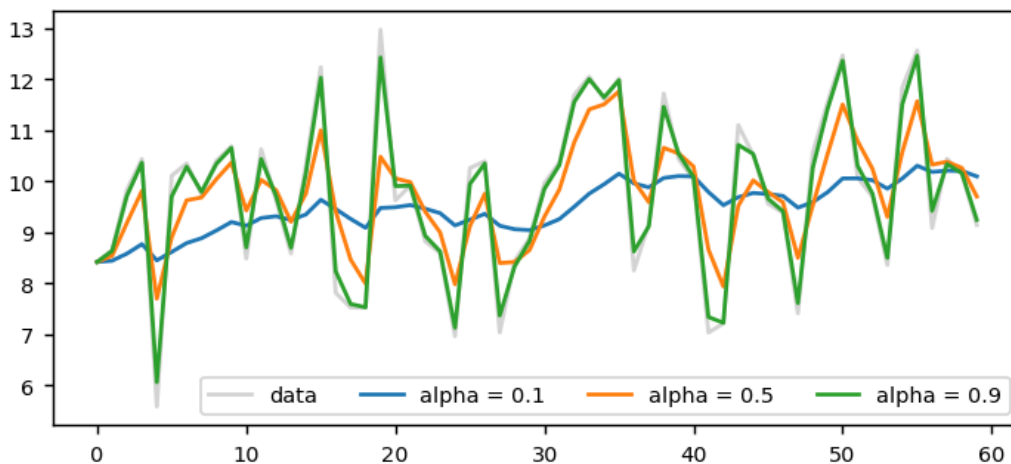
Here α (between 0 and 1) is the smoothing parameter: near 1 the forecast follows the latest value; near 0 it changes slowly. **Holt's method** adds a trend term, and **Holt-Winters** adds seasonality.

```
def simple_exp_smooth(y, alpha):
    level = np.zeros(len(y))
    level[0] = y[0]
    for i in range(1, len(y)):
        level[i] = alpha * y[i] + (1 - alpha) * level[i - 1]
    return level

noisy = pd.Series(10 + rng.normal(0, 1.5, 60))
fig, ax = plt.subplots(figsize=(6.5, 2.8))
ax.plot(noisy, color="lightgrey", label="data")
for a in [0.1, 0.5, 0.9]:
    ax.plot(simple_exp_smooth(noisy.values, a), label=f"alpha = {a}")
ax.legend(ncol=4); plt.show()
print("pandas version:", noisy.ewm(alpha=0.5, adjust=False).mean().iloc[-1].round(4),
      "ours:", simple_exp_smooth(noisy.values, 0.5)[-1].round(4))
```

OUTPUT

pandas version: 9.6984 ours: 9.6984



20.9 Forecasting and validating properly

Baselines first

Before any fancy model, compute simple benchmarks:

- **Naive:** tomorrow = today.
- **Seasonal naive:** this month = same month last year.
- **Mean** or **drift** (extend the average trend).

```
train, test = ts[:-12], ts[-12:] # hold out the last 12 months
naive = pd.Series(train.iloc[-1], index=test.index)
seasonal_naive = pd.Series(train.iloc[-12:].values, index=test.index)

# a trend + season regression model (harmonic regression)
def design(tt):
```

```

return np.column_stack([np.ones(len(tt)), tt,
                        np.sin(2 * np.pi * tt / 12), np.cos(2 * np.pi * tt / 12)])
tt_train, tt_test = np.arange(len(train)), np.arange(len(train), len(ts))
beta, *_ = np.linalg.lstsq(design(tt_train), train.values, rcond=None)
regression = pd.Series(design(tt_test) @ beta, index=test.index)

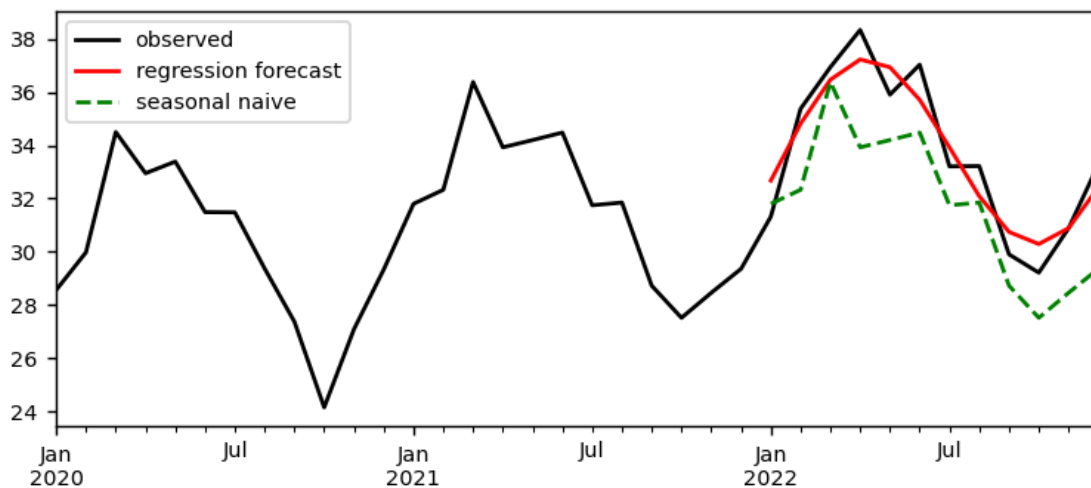
def mae(a, b): return np.mean(np.abs(a - b))
forecasts = [("naive", naive), ("seasonal naive", seasonal_naive),
             ("trend+season regression", regression)]
for name, f in forecasts:
    print(f"{name:24s} MAE = {mae(test, f):.2f}")

fig, ax = plt.subplots(figsize=(7, 2.8))
ts[-36:].plot(ax=ax, label="observed", color="black")
regression.plot(ax=ax, color="red", label="regression forecast")
seasonal_naive.plot(ax=ax, color="green", linestyle="--", label="seasonal naive")
ax.legend(); plt.show()

```

OUTPUT

naive	MAE = 4.39
seasonal naive	MAE = 2.07
trend+season regression	MAE = 0.87



Walk-forward (rolling origin) validation

Never shuffle time series. Train on the past, test on the future, then move the window forward.

`TimeSeriesSplit` does this:

```

from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=4, test_size=12)
for fold, (tr, te) in enumerate(tscv.split(ts)):
    beta, *_ = np.linalg.lstsq(design(tr.astype(float)), ts.values[tr], rcond=None)
    pred = design(te.astype(float)) @ beta
    last = ts.index[tr[-1]].date()
    print(f"fold {fold}: train up to {last}, MAE = {mae(ts.values[te], pred):.2f}")

```

OUTPUT

fold 0:	train up to 2018-12-01,	MAE = 0.90
fold 1:	train up to 2019-12-01,	MAE = 0.88
fold 2:	train up to 2020-12-01,	MAE = 0.98
fold 3:	train up to 2021-12-01,	MAE = 0.87

Forecast error measures

Measure	Formula	Note
MAE	mean $\text{abs}(y - \hat{y})$	same units as the data
RMSE	$\sqrt{\text{mean}(y - \hat{y})^2}$	penalises big misses
MAPE	mean $\text{abs}((y - \hat{y})/y)$	percent; breaks near zero
MASE	MAE / MAE of naive forecast	below 1 means better than naive

20.10 Practical tasks

Resampling, gaps and interpolation

```
hourly = pd.Series(np.sin(np.linspace(0, 6, 48)),
                  index=pd.date_range("2024-05-01", periods=48, freq="h"))
hourly.iloc[[5, 6, 7, 20]] = np.nan  # sensor dropouts
print("missing:", hourly.isna().sum())
filled = hourly.interpolate(method="time")  # straight-line in time
daily = filled.resample("D").agg(["mean", "max"])
print(daily.round(3))
```

OUTPUT

```
missing: 4
          mean    max
2024-05-01  0.647  0.999
2024-05-02 -0.642  0.078
```

Lagged features for machine learning

Turn forecasting into ordinary regression by giving the model the recent past as features:

```
feat = pd.DataFrame({"y": ts})
for lag in [1, 2, 12]:
    feat[f"lag{lag}"] = feat["y"].shift(lag)
feat["rolling3"] = feat["y"].shift(1).rolling(3).mean()  # shift first: never use the future!
feat["month"] = feat.index.month
feat = feat.dropna()
print(feat.head(3).round(2))
```

OUTPUT

```
          y  lag1  lag2  lag12  rolling3  month
2016-01-01 19.47 19.69 17.41 20.13    17.73     1
2016-02-01 23.73 19.47 19.69 22.02    18.86     2
2016-03-01 24.32 23.73 19.47 24.40    20.97     3
```

WATCH OUT

Leakage in time series is easy: a rolling mean that includes the current value, or scaling using the whole series, lets the future into the past. Always **shift** features so they only use information available at prediction time.

Detecting periodicity with the spectrum

The **periodogram** (based on the Fourier transform, Chapter 22) shows which periods dominate.

```
detr = ts.values - np.polyval(np.polyfit(np.arange(96), ts.values, 1), np.arange(96))
spec = np.abs(np.fft.rfft(detr)) ** 2
freq = np.fft.rfftfreq(96, d=1.0)  # cycles per month
peak = freq[np.argmax(spec[1:])] + 1
print("dominant period (months):", round(1 / peak, 1))
```

OUTPUT

```
dominant period (months): 12.0
```

20.11 Chapter summary

CHAPTER SUMMARY

- Time series have trend, seasonality, cycles and noise; decompose them with moving averages or STL.
- Stationarity (constant mean and variance) is needed by many models; difference to remove trend or seasonality; check with the ADF test.
- The ACF reveals memory and seasonality; PACF helps pick AR order. AR(p) is regression on past values; ARIMA/SARIMA add differencing and moving-average terms.
- Exponential smoothing weights recent data more (α controls this); Holt-Winters adds trend and seasonality.
- Always beat naive and seasonal-naive baselines, validate by walk-forward splits and avoid using future information in features.

20.12 Practice exercises

1. Decompose a series of your own (simulate a daily series with a weekly pattern) and recover the weekly seasonal component.
2. Simulate AR(1) with $\phi = 0.3$ and $\phi = 0.95$ and compare the plots and ACFs.
3. Implement Holt's linear trend method and forecast 12 steps ahead.
4. Create lag features for a series and fit a random forest; compare with seasonal naive using walk-forward validation.
5. Explain why randomly splitting a time series into train and test sets gives over-optimistic results.

CHAPTER 21

21. Scientific Computing with SciPy

IN THIS CHAPTER YOU WILL LEARN

- Curve fitting with parameter uncertainties (`curve_fit`) and least squares
- Optimisation, root finding, interpolation
- Numerical integration and solving differential equations (ODEs)
- Linear algebra tools: solve, eigenvalues, SVD
- Maximum-likelihood fitting with `minimize`

SciPy (Scientific Python) builds on NumPy and supplies the algorithms of computational science. You have already used `scipy.stats`. This chapter covers the workhorse numerical modules: `optimize`, `interpolate`, `integrate` and `linalg`.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import optimize, interpolate, integrate, linalg, constants
```

21.1 Physical constants and units

```
print("speed of light:", constants.c, "m/s")
print("Boltzmann constant:", constants.k, "J/K")
print("electron mass:", constants.m_e, "kg")
print("1 inch in metres:", constants.inch, "  1 hour in seconds:", constants.hour)
print(constants.physical_constants["Avogadro constant"])
```

OUTPUT

```
speed of light: 299792458.0 m/s
Boltzmann constant: 1.380649e-23 J/K
electron mass: 9.1093837139e-31 kg
1 inch in metres: 0.0254  1 hour in seconds: 3600.0
(6.02214076e+23, 'mol^-1', 0.0)
```

21.2 Non-linear curve fitting

Most scientific laws are **non-linear** in their parameters: exponential decay $N(t) = N_0 e^{-\lambda t}$, Michaelis-Menten enzyme kinetics, Gaussian peaks. `curve_fit` finds parameter values that make the model match the data by **non-linear least squares**: it minimises $\sum (y - \text{model}(x; \theta))^2$ by iteratively improving a starting guess, using calculus on the error surface.

You provide: (1) a model function `f(x, parameters...)`, (2) the data, (3) an initial guess `p0`. It returns the best parameters and their **covariance matrix**; the square roots of its diagonal are the parameter standard errors.

```
rng = np.random.default_rng(1)
t = np.linspace(0, 10, 25) # time (hours)
true_N0, true_lam = 1000, 0.45
counts = true_N0 * np.exp(-true_lam * t) + rng.normal(0, 25, t.size)

def decay(t, N0, lam):
```

```

return N0 * np.exp(-lam * t)

popt, pcov = optimize.curve_fit(decay, t, counts, p0=[800, 0.3])
perr = np.sqrt(np.diag(pcov))
print(f"N0 = {popt[0]:.1f} +/- {perr[0]:.1f} (true {true_N0})")
print(f"lam = {popt[1]:.4f} +/- {perr[1]:.4f} (true {true_lam})")
print(f"half-life = ln(2)/lam = {np.log(2) / popt[1]:.3f} h")

fig, ax = plt.subplots(figsize=(5.5, 3))
ax.scatter(t, counts, label="data")
tt = np.linspace(0, 10, 200)
ax.plot(tt, decay(tt, *popt), "r-", label="fit")
ax.set_xlabel("time (h)"); ax.set_ylabel("counts"); ax.legend()
plt.show()

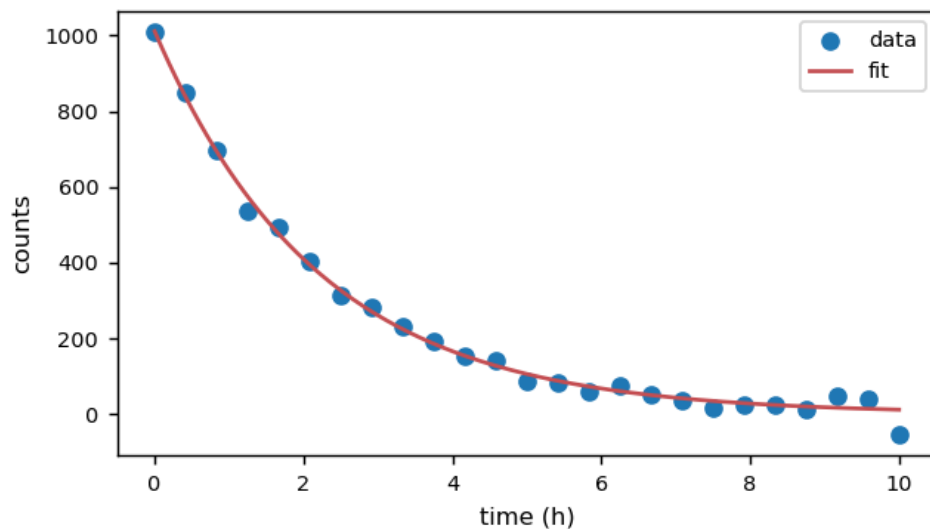
```

OUTPUT

```

N0 = 1011.6 +/- 15.4 (true 1000)
lam = 0.4527 +/- 0.0107 (true 0.45)
half-life = ln(2)/lam = 1.531 h

```



Weights: using measurement uncertainties

If each point has its own measurement error σ_i , give `sigma=` so precise points count more. With `absolute_sigma=True` the covariance reflects the real sizes of your errors. The fit then minimises the **chi-square**:

$$\chi^2 = \sum ((y_i - f(x_i)) / \sigma_i)^2$$

A good fit has **reduced chi-square** $\chi^2/(N - p)$ near 1 (N data points, p parameters). Much larger than 1 means a poor model or underestimated errors; much smaller than 1 suggests overestimated errors.

```

sigma = 25 * np.ones_like(t) # the known measurement error of each point
popt, pcov = optimize.curve_fit(decay, t, counts, p0=[800, 0.3], sigma=sigma, absolute_sigma=True)
resid = (counts - decay(t, *popt)) / sigma
chi2 = (resid ** 2).sum()
dof = len(t) - len(popt)
print(f"chi2 = {chi2:.1f}, dof = {dof}, reduced chi2 = {chi2 / dof:.2f}")

from scipy import stats
print("p-value of the fit:", round(stats.chi2.sf(chi2, dof), 3))

```

OUTPUT

```

chi2 = 16.3, dof = 23, reduced chi2 = 0.71
p-value of the fit: 0.841

```

Fit quality and model comparison

```
def gaussian(x, a, mu, s, c):
    return a * np.exp(-(x - mu) ** 2 / (2 * s ** 2)) + c

rng = np.random.default_rng(2)
x = np.linspace(-5, 5, 80)
y = gaussian(x, 4.0, 0.7, 1.1, 0.5) + rng.normal(0, 0.25, x.size)

popt, pcov = optimize.curve_fit(gaussian, x, y, p0=[3, 0, 1, 0])
names = ["amplitude", "centre", "width", "baseline"]
for n_, v, e in zip(names, popt, np.sqrt(np.diag(pcov))):
    print(f"{n_:10s} = {v:6.3f} +/- {e:.3f}")
ss_res = ((y - gaussian(x, *popt)) ** 2).sum()
print("R^2 =", round(1 - ss_res / ((y - y.mean()) ** 2).sum(), 4))
print("FWHM = 2.355 * width =", round(2.355 * abs(popt[2]), 3))
```

OUTPUT

```
amplitude = 4.039 +/- 0.085
centre    = 0.714 +/- 0.025
width     = 1.114 +/- 0.031
baseline  = 0.486 +/- 0.044
R^2 = 0.9689
FWHM = 2.355 * width = 2.623
```

TIP

Non-linear fits are sensitive to the **starting guess** `p0` and can get stuck in a wrong solution. Choose starting values from the plot (peak height, rough centre, etc.) and supply **bounds** with `bounds=([lower...], [upper...])` to keep parameters physical (for instance a positive rate).

21.3 General optimisation: minimise

`scipy.optimize.minimize` finds the input that minimises any function of one or several variables. The function can be a cost, an energy, a negative log-likelihood...

```
def rosen(p):
    # the famous banana-shaped test function, minimum at (1, 1)
    x, y = p
    return (1 - x) ** 2 + 100 * (y - x ** 2) ** 2

res = optimize.minimize(rosen, x0=[-1.5, 2.0], method="BFGS")
print(res.x.round(4), "function value:", float(res.fun))
print("iterations:", res.nit, " success:", res.success)

# with bounds and a constraint: minimise x^2 + y^2 subject to x + y >= 1, 0 <= x,y <= 2
cons = {"type": "ineq", "fun": lambda p: p[0] + p[1] - 1}
res = optimize.minimize(lambda p: p[0] ** 2 + p[1] ** 2, x0=[2, 2], bounds=[(0, 2), (0, 2)],
                        constraints=[cons], method="SLSQP")
print(res.x.round(3))
```

OUTPUT

```
[1. 1.] function value: 1.576242325275791e-11
iterations: 36 success: False
[0.5 0.5]
```

Methods: **BFGS** and **L-BFGS-B** use gradients (fast for smooth functions); **Nelder-Mead** needs no derivatives (robust for noisy functions); **SLSQP** handles constraints. For functions with many local minima use `optimize.differential_evolution` (global search):

```
def bumpy(p):
    x, y = p
    return x ** 2 + y ** 2 + 10 * np.sin(3 * x) ** 2 + 10 * np.sin(3 * y) ** 2

local = optimize.minimize(bumpy, [3, 3])
best = optimize.differential_evolution(bumpy, bounds=[(-5, 5), (-5, 5)], seed=0)
print("local minimiser ends at :", local.x.round(3), round(local.fun, 3))
```

```
print("global search ends at :", best.x.round(3), round(best.fun, 3))
```

OUTPUT

```
local minimiser ends at : [3.107 3.107] 19.522
global search ends at : [0. 0.] 0.0
```

Maximum likelihood with minimize

In Chapter 14 we met maximum likelihood. Real problems have no formula, so we minimise the **negative log-likelihood** numerically. Example: fit a gamma distribution.

```
from scipy import stats
rng = np.random.default_rng(3)
data = rng.gamma(shape=3.0, scale=2.0, size=400)

def neg_loglik(params):
    shape, scale = params
    if shape <= 0 or scale <= 0:
        return np.inf
    return -stats.gamma.logpdf(data, a=shape, scale=scale).sum()

res = optimize.minimize(neg_loglik, x0=[1.0, 1.0], method="Nelder-Mead")
print("MLE shape, scale:", res.x.round(3), " (true 3, 2)")
print("scipy built-in fit:", np.round(stats.gamma.fit(data, floc=0)[:2], 3))
```

OUTPUT

```
MLE shape, scale: [3.265 1.942] (true 3, 2)
scipy built-in fit: [3.265 1.942]
```

21.4 Root finding

A **root** is where $f(x) = 0$. To solve an equation such as $\cos(x) = x$, rearrange to $f(x) = \cos(x) - x = 0$.

```
f = lambda x: np.cos(x) - x
root = optimize.brentq(f, 0, 1) # needs a bracket [a, b] where the sign changes
print("root:", round(root, 6), " check f(root) =", f(root))

# several equations: x^2 + y^2 = 4 and x = y -> fsolve
sol = optimize.fsolve(lambda v: [v[0]**2 + v[1]**2 - 4, v[0] - v[1]], [1, 1])
print("solution of the system:", sol.round(4))
```

OUTPUT

```
root: 0.739085 check f(root) = 7.993605777301127e-15
solution of the system: [1.4142 1.4142]
```

21.5 Interpolation

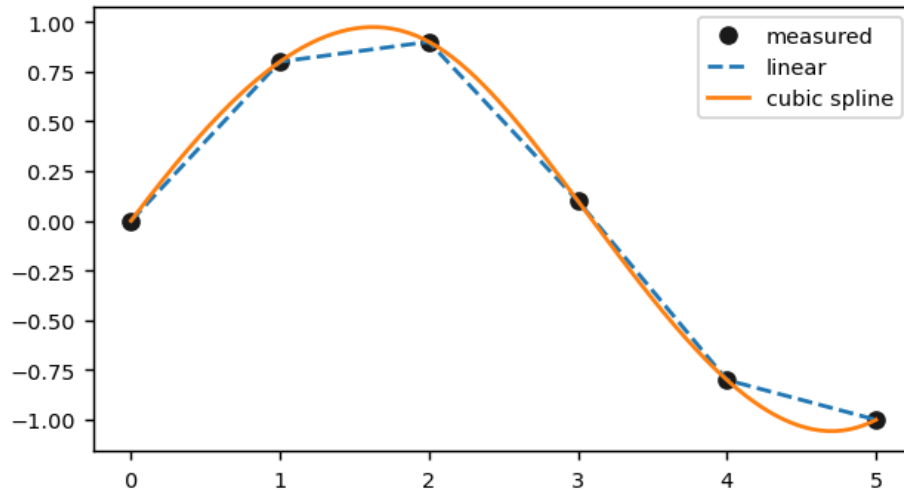
Interpolation estimates values *between* measured points. `interp1d` (or `np.interp` for the linear case) and `CubicSpline` build functions from data.

```
xk = np.array([0, 1, 2, 3, 4, 5.0])
yk = np.array([0, 0.8, 0.9, 0.1, -0.8, -1.0])

linear = interpolate.interp1d(xk, yk, kind="linear")
cubic = interpolate.CubicSpline(xk, yk)
xf = np.linspace(0, 5, 200)
fig, ax = plt.subplots(figsize=(5.5, 3))
ax.plot(xk, yk, "ko", label="measured")
ax.plot(xf, linear(xf), "--", label="linear")
ax.plot(xf, cubic(xf), label="cubic spline")
ax.legend(); plt.show()
print("value at x = 2.5:", float(linear(2.5)), float(cubic(2.5)))
```

OUTPUT

value at x = 2.5: 0.5 0.575



WATCH OUT

Interpolation is **not** extrapolation. Outside the measured range results are unreliable, and high-degree polynomial interpolation through many points oscillates wildly. Splines are the safe choice. For noisy data do not interpolate; **fit** a smooth model or smooth first.

21.6 Numerical integration

```
# area under a curve: quad returns (value, estimated error)
val, err = integrate.quad(lambda x: np.exp(-x ** 2), -np.inf, np.inf)
print("integral of exp(-x^2):", round(val, 6), " vs sqrt(pi) =", round(np.sqrt(np.pi), 6))

val, err = integrate.quad(np.sin, 0, np.pi)
print("integral of sin from 0 to pi:", round(val, 6))

# integrating measured data points (no formula): trapezoid and Simpson
x = np.linspace(0, np.pi, 11)
y = np.sin(x)
trap = float(np.trapezoid(y, x))
simp = float(integrate.simpson(y, x=x))
print("trapezoid:", round(trap, 5), " Simpson:", round(simp, 5))

# a double integral: volume under x*y over the unit square
vol, _ = integrate.dblquad(lambda y, x: x * y, 0, 1, 0, 1)
print("double integral:", round(vol, 4))
```

OUTPUT

```
integral of exp(-x^2): 1.772454 vs sqrt(pi) = 1.772454
integral of sin from 0 to pi: 2.0
trapezoid: 1.98352 Simpson: 2.00011
double integral: 0.25
```

Use `integrate.quad` when you have a function; `trapezoid` or `simpson` when you only have measured samples (for example integrating a velocity curve to get distance, or a spectrum to get total energy).

21.7 Ordinary differential equations (ODEs)

Many systems are described by **how fast they change**: $dy/dt = f(t, y)$. Examples: radioactive decay, chemical kinetics, population growth, epidemics, planetary motion. `solve_ivp` integrates an ODE forward in time from an initial condition.

Example 1: exponential decay

$dy/dt = -k y$ has the known solution $y = y_0 e^{-kt}$, so we can test the solver.

```
k = 0.7
sol = integrate.solve_ivp(lambda t, y: -k * y, t_span=(0, 6), y0=[10.0],
                           t_eval=np.linspace(0, 6, 7))
print(np.round(sol.y[0], 4))
print(np.round(10 * np.exp(-k * sol.t), 4))
```

OUTPUT

```
[10.      4.9661  2.4654  1.2247  0.6086  0.3025  0.1502]
[10.      4.9659  2.466   1.2246  0.6081  0.302   0.15   ]
```

Example 2: a predator-prey system (Lotka-Volterra)

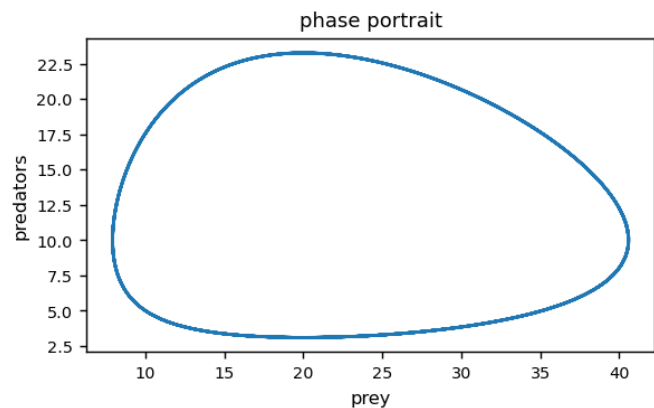
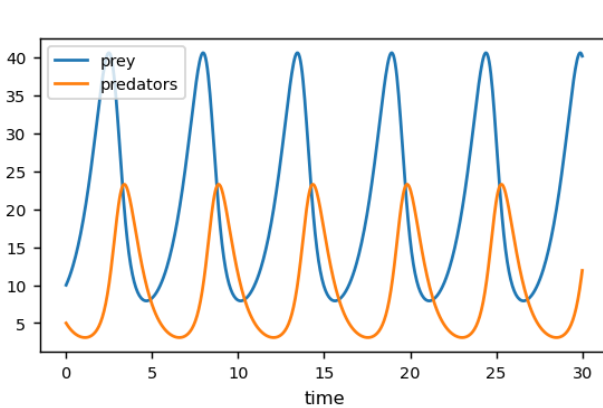
Two coupled equations for prey x and predators y :

$$dx/dt = a x - b x y \quad dy/dt = -c y + d x y$$

Prey grow ($a x$) and are eaten in proportion to meetings ($b x y$); predators starve without prey ($-c y$) and multiply by eating ($d x y$).

```
def lotka(t, z, a=1.0, b=0.1, c=1.5, d=0.075):
    x, y = z
    return [a * x - b * x * y, -c * y + d * x * y]

sol = integrate.solve_ivp(lotka, (0, 30), [10, 5], t_eval=np.linspace(0, 30, 600), rtol=1e-8)
fig, axes = plt.subplots(1, 2, figsize=(9, 3))
axes[0].plot(sol.t, sol.y[0], label="prey"); axes[0].plot(sol.t, sol.y[1], label="predators")
axes[0].set_xlabel("time"); axes[0].legend()
axes[1].plot(sol.y[0], sol.y[1]); axes[1].set_xlabel("prey"); axes[1].set_ylabel("predators")
axes[1].set_title("phase portrait")
plt.tight_layout(); plt.show()
```



Example 3: an epidemic (SIR model) fitted to data

Susceptible (S), infected (I) and recovered (R) people: $dS/dt = -\beta S I / N$, $dI/dt = \beta S I / N - \gamma I$, $dR/dt = \gamma I$. The ratio $R_0 = \beta/\gamma$ is the basic reproduction number.

```
def sir(t, z, beta, gamma, N=1000):
    S, I, R = z
    return [-beta * S * I / N, beta * S * I / N - gamma * I, gamma * I]

days = np.arange(0, 60)
true = integrate.solve_ivp(sir, (0, 59), [990, 10, 0], t_eval=days, args=(0.35, 0.1))
rng = np.random.default_rng(0)
observed = true.y[1] + rng.normal(0, 8, len(days)) # noisy daily infected counts
```

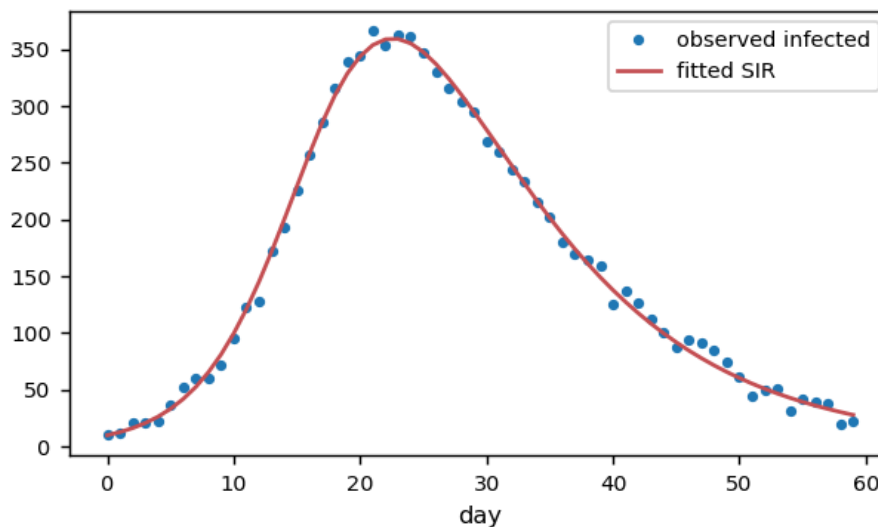
```
def model_I(days, beta, gamma):
    s = integrate.solve_ivp(sir, (0, 59), [990, 10, 0], t_eval=days, args=(beta, gamma))
    return s.y[1]

popt, pcov = optimize.curve_fit(model_I, days, observed, p0=[0.2, 0.05], bounds=(0, 2))
print("beta, gamma =", popt.round(3), " true (0.35, 0.10)")
print("R0 = beta/gamma =", round(popt[0] / popt[1], 2))

fig, ax = plt.subplots(figsize=(5.5, 3))
ax.plot(days, observed, ".", label="observed infected")
ax.plot(days, model_I(days, *popt), "r-", label="fitted SIR")
ax.set_xlabel("day"); ax.legend(); plt.show()
```

OUTPUT

```
beta, gamma = [0.348 0.1 ] true (0.35, 0.10)
R0 = beta/gamma = 3.5
```



This recipe (simulate the ODE inside a function, then `curve_fit` it to data) fits **any** dynamical model to measurements.

21.8 Linear algebra with `scipy.linalg`

```
A = np.array([[4.0, 1.0, 0.0],
              [1.0, 3.0, 1.0],
              [0.0, 1.0, 2.0]])
b = np.array([1.0, 2.0, 3.0])
print("solve:", linalg.solve(A, b).round(4))
print("eigenvalues:", linalg.eigvalsh(A).round(4))          # symmetric matrix
print("determinant:", round(linalg.det(A), 3))

# least squares for an over-determined system (more equations than unknowns)
rng = np.random.default_rng(0)
M = rng.normal(size=(50, 3))
truth = np.array([1.0, -2.0, 0.5])
rhs = M @ truth + rng.normal(0, 0.1, 50)
print("least squares solution:", linalg.lstsq(M, rhs)[0].round(3))
```

OUTPUT

```
solve: [0.2222 0.1111 1.4444]
eigenvalues: [1.2679 3.      4.7321]
determinant: 18.0
least squares solution: [ 1.018 -2.016  0.502]
```

The singular value decomposition (SVD)

Any matrix X can be factored as $X = U S V^T$, where S holds the **singular values** (how important each "direction" is). SVD underlies PCA, image compression, noise removal, pseudo-inverses and recommender systems. Keeping only the largest singular values gives the best low-rank approximation.

```
rng = np.random.default_rng(1)
signal = np.outer(np.sin(np.linspace(0, 6, 40)), np.cos(np.linspace(0, 4, 30))) # rank 1 pattern
noisy = signal + rng.normal(0, 0.05, signal.shape)

U, s, Vt = linalg.svd(noisy, full_matrices=False)
print("largest singular values:", s[:4].round(2))
rank1 = s[0] * np.outer(U[:, 0], Vt[0])
print("error of noisy vs truth :", round(np.linalg.norm(noisy - signal), 3))
print("error of rank-1 vs truth:", round(np.linalg.norm(rank1 - signal), 3))
```

OUTPUT

```
largest singular values: [18.68  0.56  0.52  0.49]
error of noisy vs truth : 1.725
error of rank-1 vs truth: 0.435
```

The first singular value dominates; the rest are noise. Dropping them *denoises* the matrix.

21.9 Special functions and sparse matrices

```
from scipy import special
print("gamma(5) = 4! =", special.gamma(5), " erf(1) =", round(special.erf(1), 4))
print("binomial coefficient C(10,3):", special.comb(10, 3))
print("Bessel J0(2.4):", round(special.j0(2.4), 4))

from scipy import sparse
big = sparse.random(1000, 1000, density=0.01, random_state=0, format="csr")
print("sparse nonzeros:", big.nnz, " of", 1000 * 1000, "entries")
```

OUTPUT

```
gamma(5) = 4! = 24.0   erf(1) = 0.8427
binomial coefficient C(10,3): 120.0
Bessel J0(2.4): 0.0025
sparse nonzeros: 10000  of 1000000 entries
```

Sparse matrices store only the non-zero entries: essential for huge networks, text data and finite-element grids.

21.10 A general workflow for model fitting

1. **Plot** the data and think about the science: what functional form is expected?
2. **Choose** a model function and sensible starting values (and bounds).
3. **Fit** with `curve_fit` (or `minimize` on a custom cost).
4. **Check** residuals (random scatter, no pattern), reduced χ^2 and parameter uncertainties.
5. **Compare** alternative models (AIC: lower is better, penalising extra parameters).
6. **Report** parameters with uncertainties and units.

```
def aic(resid, k):
    n = len(resid)
    return n * np.log((resid ** 2).sum() / n) + 2 * k

lin = lambda x, a, b: a * x + b
quad = lambda x, a, b, c: a * x ** 2 + b * x + c
rng = np.random.default_rng(6)
x = np.linspace(0, 5, 30)
y = 1 + 0.5 * x + 0.3 * x ** 2 + rng.normal(0, 0.4, 30)
```

```
for name, f, p0 in [("linear", lin, [1, 1]), ("quadratic", quad, [1, 1, 1])]:
    p, _ = optimize.curve_fit(f, x, y, p0=p0)
    print(f"{name:10s} AIC = {aic(y - f(x, *p), len(p)):.1f}")
```

OUTPUT

```
linear      AIC = -14.9
quadratic   AIC = -44.7
```

21.11 Chapter summary

CHAPTER SUMMARY

- `curve_fit` does non-linear least squares; parameter errors come from the covariance matrix; include `sigma` for weighted fits and check reduced $\chi^2 \approx 1$.
- `minimize` (BFGS, Nelder-Mead, SLSQP) optimises any function; `differential_evolution` searches globally; minimise the negative log-likelihood for MLE.
- Root finding: `brentq` and `fsolve`; interpolation: `interp1d` and `CubicSpline` (never extrapolate).
- Integrate functions with `quad` and data with `trapezoid` or `simpson`; `solve_ivp` solves ODEs and can be wrapped inside `curve_fit` to fit dynamical models.
- `scipy.linalg` provides solve, eigen and SVD tools; SVD truncation denoises and compresses.

21.12 Practice exercises

1. Fit the Michaelis-Menten curve $v = V_{\max} \cdot S / (K_m + S)$ to data you simulate, and report $V_{\max}$ and K_m with errors.
2. Find all solutions of $\sin(x) = x/3$ using `brentq` on suitable brackets.
3. Integrate a noisy velocity record with `trapezoid` to obtain distance, and compare with the exact integral.
4. Solve the damped oscillator $y'' + 0.3 y' + 4y = 0$ as a first-order system and plot $y(t)$.
5. Reduce a 100×100 noisy matrix to rank 3 with SVD and compare the error with the noisy version.

CHAPTER 22

22. Signals and Images

IN THIS CHAPTER YOU WILL LEARN

- Sampling, frequency, and the Nyquist limit
- The Fourier transform (FFT), power spectra, spectrograms
- Filtering: smoothing, low-pass, high-pass, band-pass; peak finding
- Convolution and correlation
- Images as arrays: channels, thresholds, filters, edges, labelling objects

Instruments produce **signals** (voltage, sound, spectra, ECG) and **images** (microscopy, satellite, X-ray). Both are just arrays, so everything you know about NumPy applies; this chapter adds frequency thinking and filtering.

22.1 Sampling

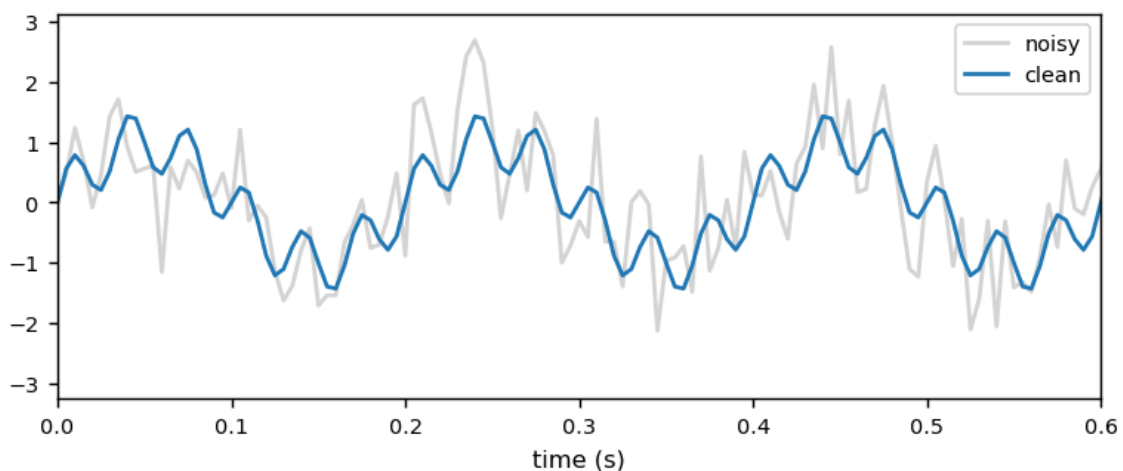
A continuous signal is recorded as numbers at a **sampling rate** f_s (samples per second, Hz). The time between samples is $1/f_s$.

Nyquist limit: to capture a frequency f you must sample at more than **2f**. The highest frequency you can capture is $f_s/2$. Sampling too slowly causes **aliasing**: fast oscillations masquerade as slow ones.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal, ndimage

fs = 200                                # sampling rate (Hz)
t = np.arange(0, 2, 1 / fs)             # 2 seconds
clean = np.sin(2 * np.pi * 5 * t) + 0.5 * np.sin(2 * np.pi * 30 * t)  # 5 Hz + 30 Hz
rng = np.random.default_rng(0)
noisy = clean + rng.normal(0, 0.7, t.size)

fig, ax = plt.subplots(figsize=(7, 2.6))
ax.plot(t, noisy, color="lightgrey", label="noisy"); ax.plot(t, clean, label="clean")
ax.set_xlim(0, 0.6); ax.set_xlabel("time (s)"); ax.legend()
plt.show()
```



22.2 The Fourier transform: from time to frequency

Fourier's idea: any signal can be written as a sum of sine waves of different frequencies, amplitudes and phases. The **Fourier transform** finds the recipe; the **FFT** (Fast Fourier Transform) computes it quickly. The result tells **how much of each frequency** the signal contains.

THE MATH BEHIND IT

For N samples x_n , the discrete Fourier transform is $X_k = \sum x_n e^{-2\pi i k n / N}$. Each X_k is a complex number whose **magnitude** $|X_k|$ measures the strength of the frequency $f_k = k f_s / N$, and whose angle is the phase. Intuitively it **correlates the signal with a sine and a cosine of each frequency**: if the signal contains that frequency, the correlation is large. The frequency resolution is $f_s / N = 1/(\text{duration})$, so longer recordings separate close frequencies better.

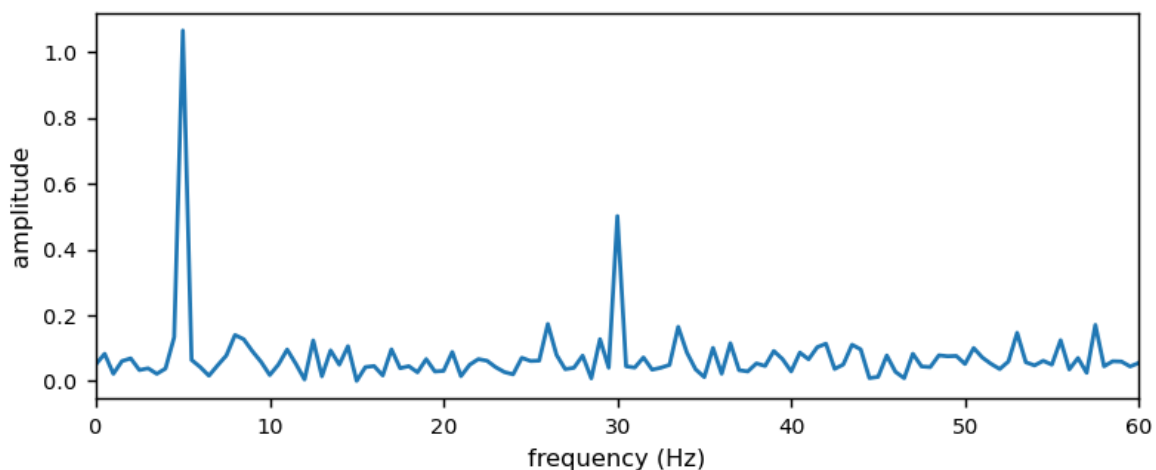
```
N = len(noisy)
spectrum = np.fft.rfft(noisy)                # rfft: for real signals, positive frequencies only
freqs = np.fft.rfftfreq(N, d=1 / fs)         # the frequency of each bin (Hz)
amplitude = 2 * np.abs(spectrum) / N         # scale so a sine of amplitude A shows as A

peaks, _ = signal.find_peaks(amplitude, height=0.3)
print("frequency resolution:", fs / N, "Hz")
print("peaks found at (Hz):", freqs[peaks], " amplitudes:", amplitude[peaks].round(2))

fig, ax = plt.subplots(figsize=(7, 2.6))
ax.plot(freqs, amplitude)
ax.set_xlim(0, 60); ax.set_xlabel("frequency (Hz)"); ax.set_ylabel("amplitude")
plt.show()
```

OUTPUT

```
frequency resolution: 0.5 Hz
peaks found at (Hz): [ 5. 30.]  amplitudes: [1.07 0.5 ]
```



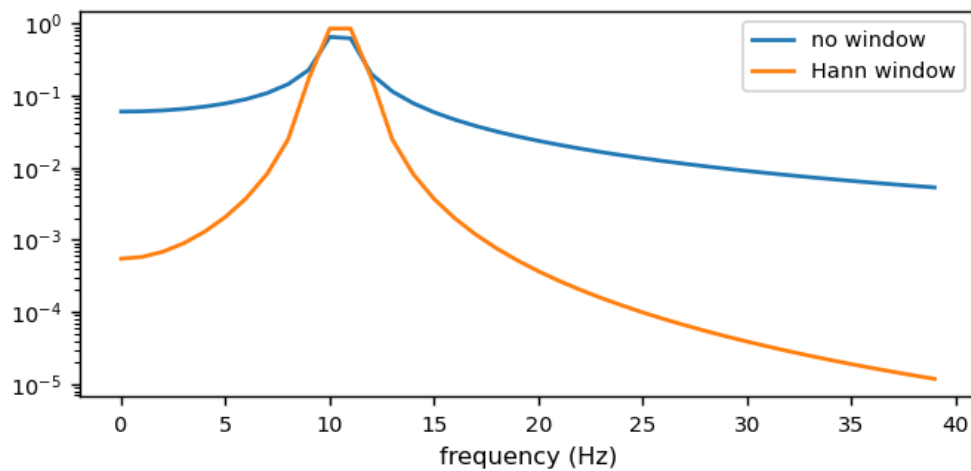
Both hidden oscillations (5 Hz and 30 Hz) stand out clearly even though they are hard to see in the noisy time trace. **The FFT is the standard way to find periodicities.**

Windowing and leakage

The FFT assumes the recording repeats forever. If the window does not contain a whole number of cycles, the abrupt jump at the ends **leaks** energy into neighbouring bins. Multiplying the signal by a smooth **window** (Hann, Hamming) that fades to zero at the ends reduces leakage.

```
tt = np.arange(0, 1, 1 / fs)
odd = np.sin(2 * np.pi * 10.5 * tt)          # 10.5 cycles: not a whole number
raw = 2 * np.abs(np.fft.rfft(odd)) / len(tt)
win = 2 * np.abs(np.fft.rfft(odd * np.hanning(len(tt)))) / np.sum(np.hanning(len(tt)))
f = np.fft.rfftfreq(len(tt), 1 / fs)
```

```
fig, ax = plt.subplots(figsize=(6, 2.6))
ax.semilogy(f[:40], raw[:40] + 1e-6, label="no window")
ax.semilogy(f[:40], win[:40] + 1e-6, label="Hann window")
ax.set_xlabel("frequency (Hz)"); ax.legend()
plt.show()
```

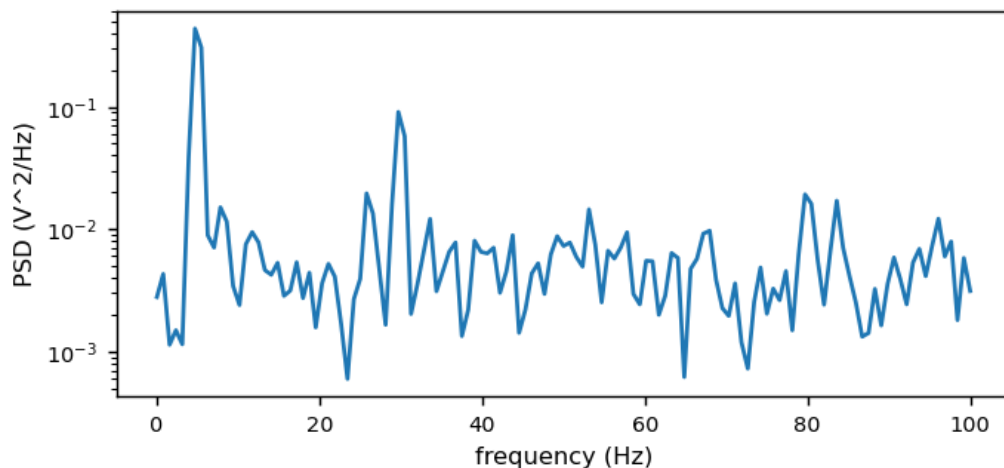


The power spectral density

For noisy signals the periodogram is itself noisy. **Welch's method** averages the spectra of overlapping segments, trading resolution for lower variance: the standard estimate of the **power spectral density (PSD)**.

```
f, Pxx = signal.welch(noisy, fs=fs, nperseg=256)
print("strongest frequency (Hz):", f[np.argmax(Pxx)])
fig, ax = plt.subplots(figsize=(6, 2.6))
ax.semilogy(f, Pxx); ax.set_xlabel("frequency (Hz)"); ax.set_ylabel("PSD (V^2/Hz)")
plt.show()
```

OUTPUT
strongest frequency (Hz): 4.6875

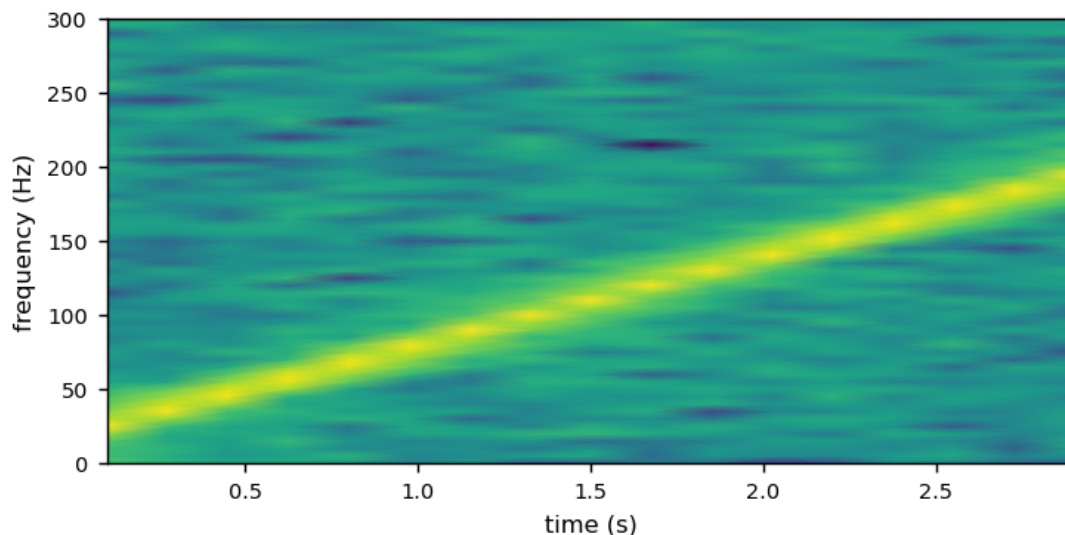


22.3 Spectrograms: frequency changing over time

When the frequency content **changes** (a bird song, an earthquake, a chirp) one spectrum is not enough. A **spectrogram** computes spectra in short sliding windows and shows them as an image: time across, frequency up, colour = power.

```
fs2 = 1000
```

```
t2 = np.arange(0, 3, 1 / fs2)
chirp = signal.chirp(t2, f0=20, f1=200, t1=3, method="linear") # frequency rises from 20 to 200 Hz
chirp_noisy = chirp + 0.2 * np.random.default_rng(1).normal(size=t2.size)
f, tm, Sxx = signal.spectrogram(chirp_noisy, fs=fs2, nperseg=200)
fig, ax = plt.subplots(figsize=(6.5, 3))
ax.pcolormesh(tm, f, 10 * np.log10(Sxx + 1e-12), shading="gouraud", cmap="viridis")
ax.set_ylim(0, 300); ax.set_xlabel("time (s)"); ax.set_ylabel("frequency (Hz)")
plt.show()
```



22.4 Filtering

A **filter** removes or keeps chosen frequencies.

Filter	Keeps	Use
Low-pass	frequencies below a cutoff	remove high-frequency noise
High-pass	frequencies above a cutoff	remove slow drift or baseline
Band-pass	a range of frequencies	isolate a signal of known frequency
Band-stop (notch)	all except a range	remove 50/60 Hz mains hum

Butterworth filters

The **Butterworth** filter has a maximally flat pass-band. `signal.butter` designs it; `filtfilt` applies it forwards *and* backwards so the result has **zero phase shift** (peaks stay where they were).

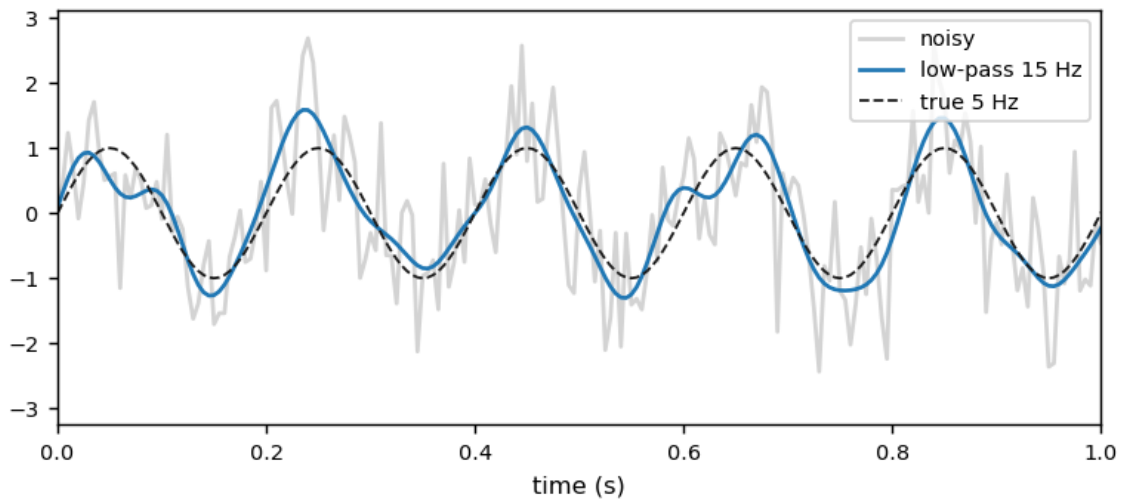
```
cutoff = 15 # Hz: keeps the 5 Hz wave, removes 30 Hz and noise
b, a = signal.butter(N=4, Wn=cutoff, btype="low", fs=fs)
filtered = signal.filtfilt(b, a, noisy)

fig, ax = plt.subplots(figsize=(7, 2.8))
ax.plot(t, noisy, color="lightgrey", label="noisy")
ax.plot(t, filtered, label="low-pass 15 Hz")
ax.plot(t, np.sin(2 * np.pi * 5 * t), "k--", linewidth=1, label="true 5 Hz")
ax.set_xlim(0, 1); ax.legend(loc="upper right"); ax.set_xlabel("time (s)")
plt.show()

bb, ab = signal.butter(4, [25, 35], btype="band", fs=fs) # isolate the 30 Hz component
band = signal.filtfilt(bb, ab, noisy)
print("std of the 30 Hz band-passed part:", band.std().round(3), " expected 0.5/sqrt(2) = 0.354")
```

OUTPUT

```
std of the 30 Hz band-passed part: 0.417 expected 0.5/sqrt(2) = 0.354
```



Smoothing in the time domain

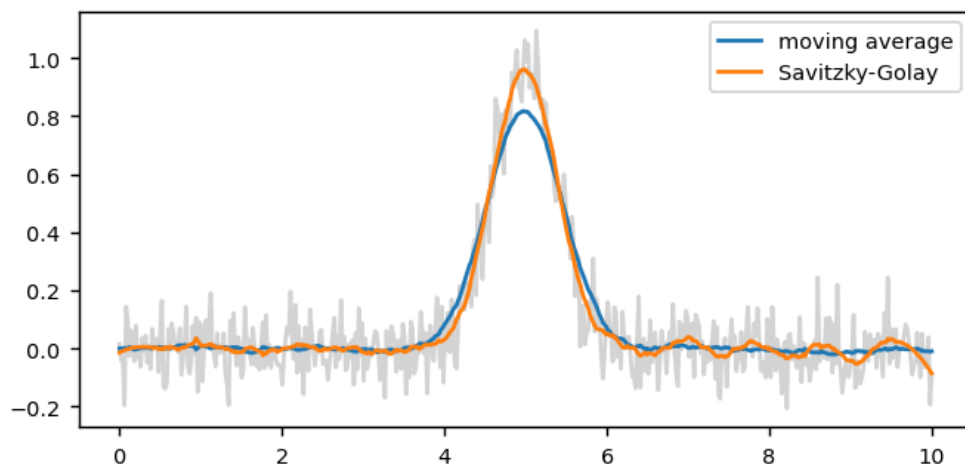
- **Moving average:** simple, but blurs peaks.
- **Savitzky-Golay filter:** fits a small polynomial in a sliding window; smooths noise while preserving peak heights and widths: a favourite in spectroscopy.
- **Median filter:** removes spikes (outliers) while keeping edges sharp.

```
x = np.linspace(0, 10, 500)
peak_clean = np.exp(-(x - 5) ** 2 / 0.3)
peak_noisy = peak_clean + np.random.default_rng(2).normal(0, 0.08, x.size)
ma = np.convolve(peak_noisy, np.ones(41) / 41, mode="same")
sg = signal.savgol_filter(peak_noisy, window_length=41, polyorder=3)
print("max of true peak:", peak_clean.max())
print("moving avg max   :", ma.max().round(3), " Savitzky-Golay max:", sg.max().round(3))

fig, ax = plt.subplots(figsize=(6, 2.8))
ax.plot(x, peak_noisy, color="lightgrey"); ax.plot(x, ma, label="moving average")
ax.plot(x, sg, label="Savitzky-Golay"); ax.legend(); plt.show()
```

OUTPUT

```
max of true peak: 0.9996653853186241
moving avg max   : 0.818 Savitzky-Golay max: 0.961
```



22.5 Peak finding

`find_peaks` locates local maxima and measures them. Parameters: **height** (minimum value), **distance** (minimum spacing in samples), **prominence** (how much a peak stands out from its surroundings; the most robust criterion).

```
xs = np.linspace(0, 20, 1000)
spec = (np.exp(-(xs - 5) ** 2 / 0.1) + 0.6 * np.exp(-(xs - 9) ** 2 / 0.2) +
        0.3 * np.exp(-(xs - 14) ** 2 / 0.15) + np.random.default_rng(3).normal(0, 0.03, xs.size))
smooth = signal.savgol_filter(spec, 21, 3)
pk, props = signal.find_peaks(smooth, prominence=0.15)
widths = signal.peak_widths(smooth, pk, rel_height=0.5)[0] * (xs[1] - xs[0])      # FWHM
print("peak positions:", xs[pk].round(2))
print("peak heights  :", smooth[pk].round(2))
print("FWHM          :", widths.round(2))
```

OUTPUT

```
peak positions: [ 5.01  9.01 13.99]
peak heights  : [1.01 0.6  0.3 ]
FWHM          : [0.53 0.77 0.67]
```

22.6 Convolution and correlation

Convolution slides a small pattern (a **kernel**) along a signal, multiplying and adding at each position: $y[n] = \sum x[k] h[n-k]$. Smoothing, derivative estimates and blurring are all convolutions, and the convolution theorem says convolution in time equals multiplication in frequency (which makes it fast with FFT). **Cross-correlation** is convolution without flipping the kernel and **finds where a pattern appears** in a signal or the **time delay** between two signals.

```
sig = np.zeros(100); sig[30:35] = [1, 2, 3, 2, 1]      # a pulse hidden at position 30
template = np.array([1, 2, 3, 2, 1.0])
corr = signal.correlate(sig + np.random.default_rng(0).normal(0, 0.2, 100), template, mode="valid")
print("pattern found at index:", np.argmax(corr))

# time delay between two recordings of the same event
rng = np.random.default_rng(1)
event = rng.normal(size=500)
a_ = event[50:400]
b_ = event[62:412] + rng.normal(0, 0.3, 350)          # b is 12 samples "earlier"
lags = signal.correlation_lags(len(a_), len(b_))
cc = signal.correlate(a_, b_, mode="full")
print("estimated lag (samples):", lags[np.argmax(cc)])
```

OUTPUT

```
pattern found at index: 30
estimated lag (samples): 12
```

22.7 Images as arrays

A digital image is a NumPy array: **height × width** for greyscale (each number a brightness, often 0 to 255) and **height × width × 3** for colour (red, green, blue channels).

```
from sklearn.datasets import load_sample_image

img = load_sample_image("china.jpg")
print(img.shape, img.dtype, img.min(), img.max())
gray = img.mean(axis=2)      # simple greyscale: average the channels
crop = img[100:250, 200:400] # slicing = cropping
red = img[:, :, 0]

fig, axes = plt.subplots(1, 4, figsize=(11, 2.6))
for ax, im, title, cmap in [(axes[0], img, "colour", None), (axes[1], gray, "greyscale", "gray"),
                            (axes[2], red, "red channel", "Reds"), (axes[3], crop, "cropped", None)]:
```

```
ax.imshow(im, cmap=cmap); ax.set_title(title); ax.axis("off")
plt.tight_layout(); plt.show()
```

OUTPUT

```
(427, 640, 3) uint8 0 255
```



Histograms and thresholding

A brightness **histogram** shows the distribution of pixel values. **Thresholding** separates objects from background by turning the image into a mask: $\text{gray} > T$. **Otsu's method** picks T automatically to best separate two peaks in the histogram.

```
def otsu(image):
    hist, edges = np.histogram(image.ravel(), bins=256, range=(0, 256))
    p = hist / hist.sum()
    centers = (edges[:-1] + edges[1:]) / 2
    w0 = np.cumsum(p); w1 = 1 - w0
    m0 = np.cumsum(p * centers) / np.maximum(w0, 1e-12)
    m1 = (p * centers).sum()
    m1 = (m1 - np.cumsum(p * centers)) / np.maximum(w1, 1e-12)
    between = w0 * w1 * (m0 - m1) ** 2 # between-class variance
    return centers[np.argmax(between)]

T = otsu(gray)
mask = gray > T
print("Otsu threshold:", round(T, 1), " fraction of bright pixels:", mask.mean().round(3))
```

OUTPUT

```
Otsu threshold: 138.5  fraction of bright pixels: 0.53
```

22.8 Filtering images

Image filters are 2D convolutions with small kernels:

- **Gaussian blur**: averages with bell-shaped weights; removes noise.
- **Median filter**: removes salt-and-pepper noise, preserves edges.
- **Sobel filter**: estimates the gradient (rate of brightness change), highlighting **edges**.

```
blur = ndimage.gaussian_filter(gray, sigma=3)
med = ndimage.median_filter(gray, size=5)
sx = ndimage.sobel(gray, axis=1)
sy = ndimage.sobel(gray, axis=0)
edges = np.hypot(sx, sy) # gradient magnitude

fig, axes = plt.subplots(1, 3, figsize=(10, 2.6))
panels = [(axes[0], blur, "Gaussian blur"), (axes[1], med, "median 5x5"), (axes[2], edges, "Sobel edges")]
for ax, im, title in panels:
    ax.imshow(im, cmap="gray"); ax.set_title(title); ax.axis("off")
plt.tight_layout(); plt.show()
```



22.9 Counting objects: labelling

A standard microscopy task: count cells or particles. Steps: smooth, threshold, label **connected components**, measure each object. We simulate an image with bright blobs on a noisy background.

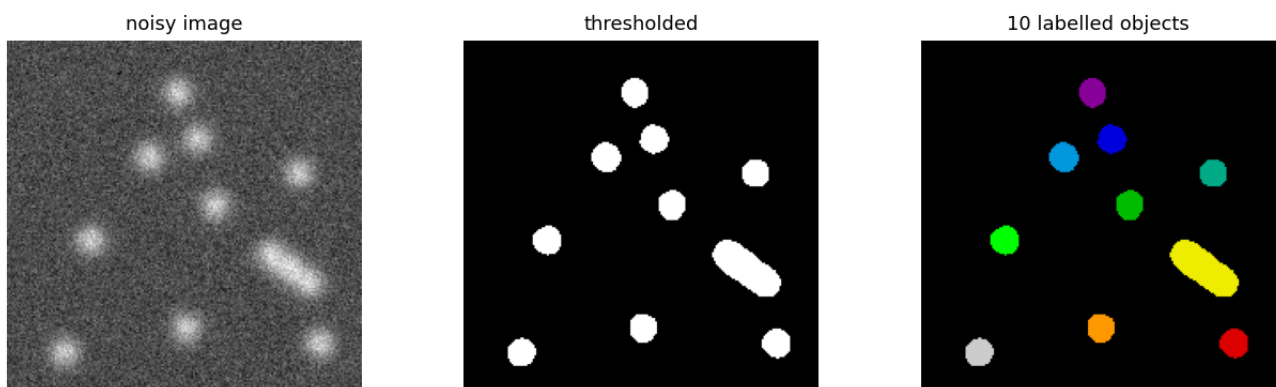
```
rng = np.random.default_rng(4)
yy, xx = np.mgrid[0:200, 0:200]
cells = np.zeros((200, 200))
centres = rng.integers(20, 180, size=(12, 2))
for cy, cx in centres:
    cells += np.exp(-((yy - cy) ** 2 + (xx - cx) ** 2) / (2 * 6 ** 2))
image = cells + rng.normal(0, 0.15, cells.shape)

smooth = ndimage.gaussian_filter(image, 2)
binary = smooth > 0.4
labels, n = ndimage.label(binary)
sizes = ndimage.sum(binary, labels, index=range(1, n + 1))
coms = ndimage.center_of_mass(binary, labels, range(1, n + 1))
print("objects found:", n, " (12 were drawn; touching ones may merge)")
print("area of first 5 objects (pixels):", sizes[:5].astype(int))

fig, axes = plt.subplots(1, 3, figsize=(10, 3))
axes[0].imshow(image, cmap="gray"); axes[0].set_title("noisy image")
axes[1].imshow(binary, cmap="gray"); axes[1].set_title("thresholded")
axes[2].imshow(labels, cmap="nipy_spectral"); axes[2].set_title(f"{n} labelled objects")
for a in axes: a.axis("off")
plt.tight_layout(); plt.show()
```

OUTPUT

```
objects found: 10 (12 were drawn; touching ones may merge)
area of first 5 objects (pixels): [200 206 219 193 208]
```



Morphology: cleaning up masks

Erosion shrinks objects (removing thin specks), **dilation** grows them (closing small gaps). **Opening** (erode then dilate) removes small noise; **closing** (dilate then erode) fills small holes.

```
noisy_mask = binary.copy()
noisy_mask[rng.integers(0, 200, 60), rng.integers(0, 200, 60)] = True # add speckle
```

```
opened = ndimage.binary_opening(noisy_mask, structure=np.ones((3, 3)))
print("objects before opening:", ndimage.label(noisy_mask)[1], " after:", ndimage.label(opened)[1])
```

OUTPUT

```
objects before opening: 65  after: 10
```

22.10 Image data for machine learning

Flatten each image into a row of pixel values, then use the methods of Chapters 17 to 19. The digits dataset (8×8 images) classified with a small model:

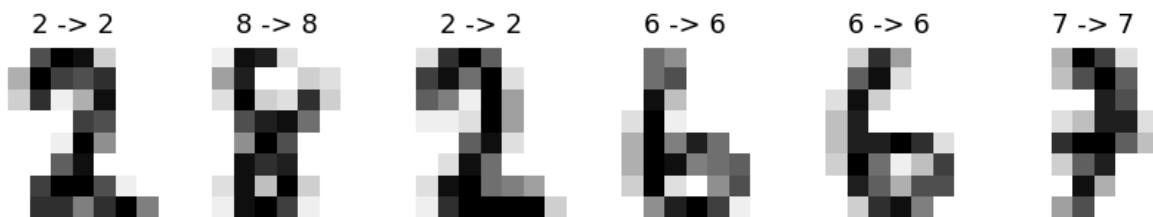
```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

d = load_digits()
Xtr, Xte, ytr, yte = train_test_split(d.data, d.target, test_size=0.25, random_state=0)
clf = SVC(gamma=0.001).fit(Xtr, ytr)
print("test accuracy on handwritten digits:", round(clf.score(Xte, yte), 3))

fig, axes = plt.subplots(1, 6, figsize=(8, 1.6))
for ax, im, lab, pr in zip(axes, Xte[:6].reshape(-1, 8, 8), yte[:6], clf.predict(Xte[:6])):
    ax.imshow(im, cmap="gray_r"); ax.set_title(f"{lab} -> {pr}"); ax.axis("off")
plt.show()
```

OUTPUT

```
test accuracy on handwritten digits: 0.996
```



For serious image work use `scikit-image`, `OpenCV` (`import cv2`) and deep-learning frameworks (PyTorch, TensorFlow), all available in Colab.

22.11 Chapter summary

CHAPTER SUMMARY

- Sample at more than twice the highest frequency (Nyquist) to avoid aliasing.
- The FFT converts a signal into its frequency content; use `rfft`, `rfftfreq`, windows, Welch's PSD and spectrograms for changing frequencies.
- Filter with Butterworth (`butter + filtfilt`), Savitzky-Golay (keeps peaks) or median filters (removes spikes); locate peaks with `find_peaks`.
- Convolution smooths or differentiates; cross-correlation finds patterns and time delays.
- Images are arrays: crop by slicing, threshold (Otsu), filter with `ndimage`, label and count objects, clean masks with morphology.

22.12 Practice exercises

1. Create a signal with frequencies 3 Hz and 12 Hz sampled at 100 Hz, add noise, and recover both frequencies with the FFT.

2. Remove 50 Hz hum from a signal using a notch (`signal.iirnotch`) or a band-stop Butterworth filter.
3. Sample a 40 Hz sine at 50 Hz and show aliasing in the FFT. Which frequency appears?
4. Load a sample image, compute the histogram of its green channel and apply a threshold.
5. Count objects in the simulated cell image for several values of the threshold. How sensitive is the count?

CHAPTER 23

23. Uncertainty, Error Analysis and Bayesian Thinking

IN THIS CHAPTER YOU WILL LEARN

- Random versus systematic error; accuracy versus precision
- Propagation of uncertainty: formulas and Monte Carlo
- Weighted averages and combining measurements
- Bayes' theorem for data: priors, likelihoods and posteriors
- Grid approximation, conjugate models and a first look at MCMC

In science a number without an uncertainty is almost meaningless. "The mass is 5.2 g" is useless; "5.2 $\pm$ 0.1 g" is a scientific statement. This chapter shows how to quantify, combine and reason about uncertainty.

23.1 Types of error

	Random error	Systematic error
Cause	unpredictable fluctuations (noise, reading variation)	a consistent bias (miscalibrated instrument, zero offset)
Effect	scatter around the true value	all values shifted the same way
Reduced by	averaging more measurements	calibration, better method; not by averaging
Quantified by	standard deviation, standard error	comparing with standards, estimating the possible bias

Precision is how closely repeated measurements agree (low random error). **Accuracy** is how close they are to the truth (low systematic error). A scale that always reads 2 g too high is precise but not accurate.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(0)
true_value = 10.0
precise_biased = rng.normal(10.8, 0.1, 30)      # precise but inaccurate
accurate_noisy = rng.normal(10.0, 0.8, 30)      # accurate on average but imprecise
for name, d in [("precise but biased", precise_biased), ("accurate but noisy", accurate_noisy)]:
    print(f"{name:20s} mean = {d.mean():.2f}, std = {d.std(ddof=1):.2f}, "
          f"SEM = {stats.sem(d):.3f}, error of mean = {d.mean() - true_value:+.2f}")
```

OUTPUT

```
precise but biased    mean = 10.79, std = 0.08, SEM = 0.015, error of mean = +0.79
accurate but noisy    mean = 10.22, std = 0.76, SEM = 0.139, error of mean = +0.22
```

Averaging many readings shrinks the SEM, but the biased instrument converges confidently to the **wrong** value.

23.2 Reporting a measurement

The standard form is **value ± uncertainty (unit)**, with the uncertainty quoted to 1 or 2 significant figures and the value rounded to the same decimal place.

- Correct: $9.81 \pm 0.03 \text{ m/s}^2$.
- Wrong: $9.8123456 \pm 0.0342 \text{ m/s}^2$ (false precision).

State what the $\pm$ means: one standard deviation, standard error or 95% interval.

```
def report(value, unc, unit=""):
    """Round to 2 significant figures of the uncertainty."""
    digits = 1 - int(np.floor(np.log10(abs(unc))))
    return f"{round(value, digits):.{max(digits, 0)}f} +/- {round(unc, digits):.{max(digits, 0)}f} {unit}"

print(report(9.8123456, 0.0342, "m/s^2"))
print(report(152340, 870, "J"))
```

OUTPUT
 9.812 +/- 0.034 m/s^2
 152340 +/- 870 J

23.3 Propagation of uncertainty

Often the quantity of interest is **calculated** from measured ones (density = mass / volume). How do their uncertainties combine?

The general formula

For a result $f(x, y, \dots)$ computed from **independent** measurements with uncertainties $\sigma_x, \sigma_y, \dots$:

$$\sigma_f^2 = \left(\frac{\partial f}{\partial x}\right)^2 \sigma_x^2 + \left(\frac{\partial f}{\partial y}\right)^2 \sigma_y^2 + \dots$$

Here $\partial f / \partial x$ (the *partial derivative*) says how much f changes per unit change of x : a sensitive input contributes more uncertainty. The special cases worth memorising:

Operation	Rule for the uncertainty σ_f
$f = x + y$ or $x - y$	$\sigma_f = \sqrt{\sigma_x^2 + \sigma_y^2}$ (add in quadrature)
$f = x \times y$ or x / y	relative errors add in quadrature: $\sigma_f/f = \sqrt{(\sigma_x/x)^2 + (\sigma_y/y)^2}$
$f = x^n$	$\sigma_f/f = n \times \sigma_x/x$
$f = \ln(x)$	$\sigma_f = \sigma_x/x$
$f = c \cdot x$ (constant c)	$\sigma_f = c \sigma_x$

```
# density of a cylinder: rho = m / (pi r^2 h)
m, sm = 25.3, 0.1          # g
r, sr = 1.20, 0.02         # cm
h, sh = 5.50, 0.05         # cm

rho = m / (np.pi * r ** 2 * h)
rel = np.sqrt((sm / m) ** 2 + (2 * sr / r) ** 2 + (sh / h) ** 2)  # r enters squared -> factor 2
print(f"rho = {rho:.3f} g/cm^3, relative uncertainty = {rel:.1%}, sigma = {rho * rel:.3f}")
print("largest contribution is from the radius:",
      {k: round(v, 4) for k, v in {"mass": sm / m, "radius (x2)": 2 * sr / r, "height": sh / h}.items()})
```

OUTPUT
 rho = 1.017 g/cm^3, relative uncertainty = 3.5%, sigma = 0.035
 largest contribution is from the radius: {'mass': 0.004, 'radius (x2)': 0.0333, 'height': 0.0091}

The radius dominates the budget: to improve the result, measure the **radius** more precisely, not the mass. An **uncertainty budget** like this tells you where to spend effort.

Monte Carlo propagation

Formulas get ugly for complicated functions or large uncertainties. The **Monte Carlo** method is general and needs no calculus:

1. Draw many random values of each input from its distribution (for instance normal with the measured mean and uncertainty).
2. Compute the result for each draw.
3. The mean and standard deviation (or percentiles) of the results give the answer and its uncertainty.

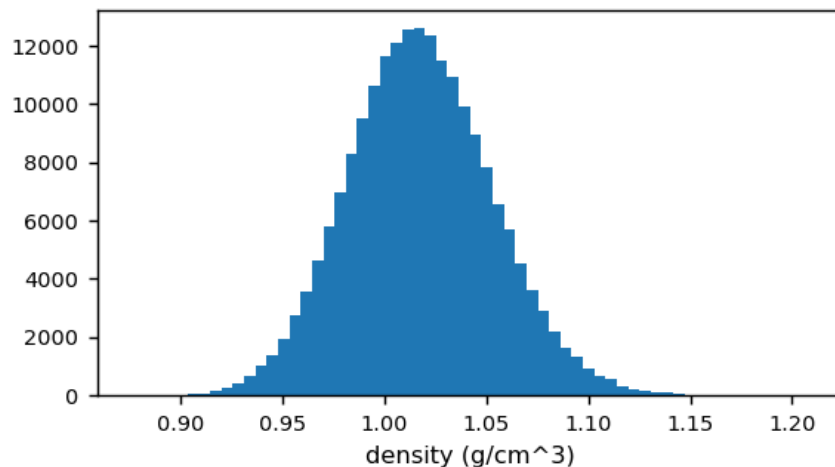
```
rng = np.random.default_rng(1)
N = 200000
m_s = rng.normal(m, sm, N)
r_s = rng.normal(r, sr, N)
h_s = rng.normal(h, sh, N)
rho_s = m_s / (np.pi * r_s ** 2 * h_s)

print(f"Monte Carlo: rho = {rho_s.mean():.3f} +/- {rho_s.std():.3f}")
print("95% interval:", np.percentile(rho_s, [2.5, 97.5]).round(3))

fig, ax = plt.subplots(figsize=(5, 2.6))
ax.hist(rho_s, bins=60, color="tab:blue"); ax.set_xlabel("density (g/cm^3)")
plt.show()
```

OUTPUT

```
Monte Carlo: rho = 1.018 +/- 0.035
95% interval: [0.951 1.09 ]
```



The Monte Carlo result agrees with the formula, and also reveals any skewness of the output distribution. Correlated inputs can be sampled from a multivariate normal using

```
rng.multivariate_normal(mean, cov, N).
```

A nonlinear case where the formula fails

```
x0, sx = 1.0, 0.8 # a large relative uncertainty
xs = rng.normal(x0, sx, 200000)
f = 1 / xs
print("first-order formula: sigma_f =", round(sx / x0 ** 2, 2))
print("Monte Carlo (clipped to |x|>0.2): sigma_f =", round(np.std(f[np.abs(xs) > 0.2]), 2))
```

OUTPUT

```
first-order formula: sigma_f = 0.8
Monte Carlo (clipped to |x|>0.2): sigma_f = 1.26
```

When the uncertainty is not small compared with the value, linear error propagation is unreliable, and simulation should be used.

23.4 Combining measurements: the weighted mean

Suppose several labs measure the same quantity with different precisions. The best combined estimate gives each measurement a weight $w_i = 1/\sigma_i^2$ (**inverse-variance weighting**):

$$\bar{x}_w = \sum w_i x_i / \sum w_i \quad \sigma_{\bar{x}} = 1 / \sqrt{(\sum w_i)}$$

```
vals = np.array([9.78, 9.85, 9.81, 9.70])
errs = np.array([0.05, 0.10, 0.03, 0.20])
w = 1 / errs ** 2
mean_w = (w * vals).sum() / w.sum()
err_w = 1 / np.sqrt(w.sum())
chi2 = (w * (vals - mean_w) ** 2).sum()
print(f"weighted mean = {mean_w:.4f} +/- {err_w:.4f}")
print(f"chi2 = {chi2:.2f} for {len(vals) - 1} degrees of freedom, p = {stats.chi2.sf(chi2, 3):.2f}")
print("unweighted mean:", vals.mean().round(4))
```

OUTPUT

```
weighted mean = 9.8034 +/- 0.0247
chi2 = 0.75 for 3 degrees of freedom, p = 0.86
unweighted mean: 9.785
```

The chi-square value checks consistency: a value far above its degrees of freedom says the measurements disagree more than their stated errors allow (suspect hidden systematic errors).

23.5 Bayesian inference

Classical (frequentist) statistics treats parameters as fixed unknowns and asks how data would vary. **Bayesian** statistics treats parameters as uncertain quantities described by probability distributions, updated by data using **Bayes' theorem** (Chapter 3):

$$P(\theta | \text{data}) = P(\text{data} | \theta) \times P(\theta) / P(\text{data})$$

Term	Name	Meaning
$P(\theta)$	prior	what we believe about θ before seeing the data
$P(\text{data} \theta)$	likelihood	how probable the observed data is for each θ
$P(\theta \text{data})$	posterior	updated belief after seeing the data
$P(\text{data})$	evidence	a normalising constant making the posterior sum to 1

In words: **posterior** $\propto$ **likelihood** $\times$ **prior**. The result is a full distribution, from which we read the best value, a **credible interval** (a range containing θ with, say, 95% probability, a direct statement people often wrongly attribute to the confidence interval) and any probability we want.

Example: the fraction of defective items by grid approximation

Test 20 chips and find 3 defective. What is the defect rate p ? Evaluate the posterior on a fine grid of p -values:

```
p_grid = np.linspace(0, 1, 1001)
prior = np.ones_like(p_grid)
likelihood = stats.binom.pmf(3, 20, p_grid)
posterior = prior * likelihood
posterior /= posterior.sum()

# flat prior: all rates equally plausible
# P(3 of 20 | p)
# normalise
```

```

mean_post = (p_grid * posterior).sum()
cdf = np.cumsum(posterior)
lo, hi = p_grid[np.searchsorted(cdf, 0.025)], p_grid[np.searchsorted(cdf, 0.975)]
print(f"posterior mean = {mean_post:.3f}, MAP = {p_grid[np.argmax(posterior)]:.3f}")
print(f"95% credible interval: ({lo:.3f}, {hi:.3f})")
print("P(defect rate > 20%) =", round(posterior[p_grid > 0.2].sum(), 3))

fig, ax = plt.subplots(figsize=(5.5, 2.8))
ax.plot(p_grid, posterior / posterior.max(), label="posterior (flat prior)")
ax.set_xlabel("defect rate p"); ax.set_xlim(0, 0.6); ax.legend()
plt.show()

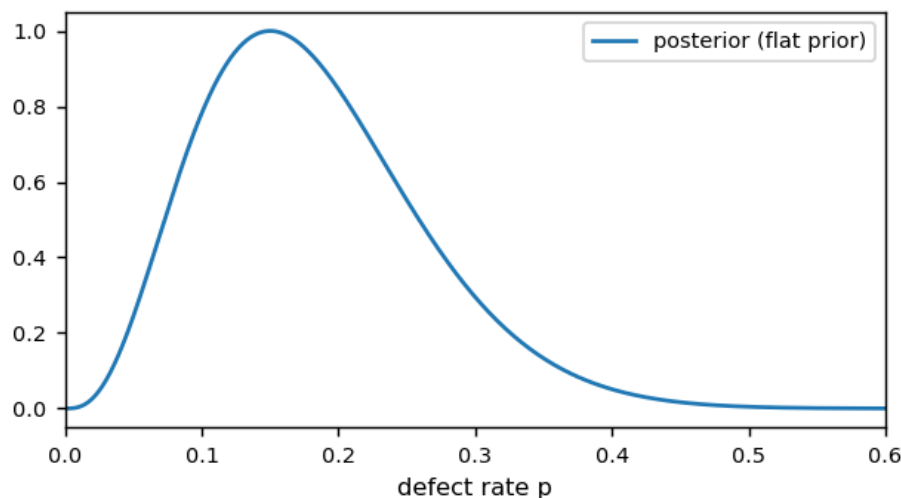
```

OUTPUT

```

posterior mean = 0.182, MAP = 0.150
95% credible interval: (0.054, 0.363)
P(defect rate > 20%) = 0.368

```



The influence of the prior

With little data the prior matters; with a lot of data the prior is overwhelmed ("the data speak"). We compare an **informative** prior (an engineer believes defects are rare, around 2%) with a flat one.

```

informative = stats.beta.pdf(p_grid, 2, 60) # centred near 3%
def posterior_for(prior, k, n):
    post = prior * stats.binom.pmf(k, n, p_grid)
    return post / post.sum()

for k, n in [(3, 20), (30, 200), (300, 2000)]:
    flat_mean = (p_grid * posterior_for(np.ones_like(p_grid), k, n)).sum()
    inf_mean = (p_grid * posterior_for(informative, k, n)).sum()
    print(f"data {k}>3/{n}<4: flat-prior mean {flat_mean:.3f} informative-prior mean {inf_mean:.3f}")

```

OUTPUT

```

data 3/20 : flat-prior mean 0.182 informative-prior mean 0.061
data 30/200 : flat-prior mean 0.153 informative-prior mean 0.122
data 300/2000: flat-prior mean 0.150 informative-prior mean 0.146

```

As the data grows, both priors give the same answer.

Conjugate priors: exact answers without a grid

For some model families the posterior has the same form as the prior ("conjugate"). For a rate p estimated from binomial data, a **Beta(a, b)** prior and k successes in n trials give a **Beta(a + k, b + n - k)** posterior. Read a and b as "prior successes and failures".

```

a0, b0 = 1, 1 # flat prior
k, n = 3, 20
post = stats.beta(a0 + k, b0 + n - k)

```

```
print("posterior mean:", round(post.mean(), 4), " 95% credible interval:", np.round(post.interval(0.95), 3))

# sequential updating: today's posterior is tomorrow's prior
a, b = 1, 1
for batch_k, batch_n in [(1, 10), (2, 10)]:
    a, b = a + batch_k, b + batch_n - batch_k
    print(f"after batch ({batch_k}/{batch_n}): Beta({a}, {b}), mean = {a / (a + b):.3f}")
```

OUTPUT

```
posterior mean: 0.1818 95% credible interval: [0.054 0.363]
after batch (1/10): Beta(2, 10), mean = 0.167
after batch (2/10): Beta(4, 18), mean = 0.182
```

Similarly: Gamma prior with Poisson data (rates), Normal prior with Normal data (means).

Bayesian estimation of a mean

For normal data with known noise σ and a Normal prior $N(\mu_0, \tau^2)$, the posterior is again normal, with precision (1/variance) equal to the **sum of the prior and data precisions**: the same inverse-variance weighting as above.

```
data = np.array([4.9, 5.4, 5.1, 5.6, 5.0, 5.3])
sigma = 0.4 # known measurement noise
mu0, tau = 4.0, 1.0 # prior: mean 4, sd 1
n = len(data)
prec_post = 1 / tau ** 2 + n / sigma ** 2
mu_post = (mu0 / tau ** 2 + data.sum() / sigma ** 2) / prec_post
print(f"sample mean = {data.mean():.3f}; posterior mean = {mu_post:.3f}, sd = {prec_post ** -0.5:.3f}")
```

OUTPUT

```
sample mean = 5.217; posterior mean = 5.185, sd = 0.161
```

The posterior mean sits between the prior mean and the sample mean, pulled towards whichever is more precise: **shrinkage**.

23.6 MCMC: when there is no closed form

For models with many parameters a grid is impossible. **Markov chain Monte Carlo** (MCMC) draws samples **from the posterior** by a random walk that visits regions in proportion to their posterior probability. The classic **Metropolis algorithm**:

1. Start at some parameter value θ .
2. Propose a nearby value θ' (a random step).
3. Compute the ratio $r = \text{posterior}(\theta') / \text{posterior}(\theta)$.
4. If $r \geq 1$ move to θ' ; otherwise move with probability r . If you do not move, repeat the current value.
5. Record θ and repeat thousands of times.

A histogram of the recorded values approximates the posterior. We estimate both the mean μ and noise σ of a normal sample:

```
rng = np.random.default_rng(42)
obs = rng.normal(12.0, 2.0, 40)

def log_post(mu, sigma):
    if sigma <= 0:
        return -np.inf
    log_prior = stats.norm.logpdf(mu, 0, 100) + stats.halfnorm.logpdf(sigma, scale=10)
    return log_prior + stats.norm.logpdf(obs, mu, sigma).sum()

steps = 20000
chain = np.zeros((steps, 2))
cur = np.array([5.0, 5.0])
```

```

cur_lp = log_post(*cur)
accepted = 0
for i in range(steps):
    prop = cur + rng.normal(0, 0.4, 2)
    prop_lp = log_post(*prop)
    if np.log(rng.uniform()) < prop_lp - cur_lp:
        cur, cur_lp = prop, prop_lp
        accepted += 1
    chain[i] = cur

burn = chain[2000:] # discard the warm-up ("burn-in")
print("acceptance rate:", round(accepted / steps, 2))
print("mu    posterior mean/sd:", burn[:, 0].mean().round(3), burn[:, 0].std().round(3))
print("sigma posterior mean/sd:", burn[:, 1].mean().round(3), burn[:, 1].std().round(3))
print("sample mean and std:", obs.mean().round(3), obs.std(ddof=1).round(3))

fig, axes = plt.subplots(1, 3, figsize=(10, 2.6))
axes[0].plot(chain[:500, 0]); axes[0].set_title("trace of mu (first 500)")
axes[1].hist(burn[:, 0], bins=40); axes[1].set_title("posterior of mu")
axes[2].hist(burn[:, 1], bins=40); axes[2].set_title("posterior of sigma")
plt.tight_layout(); plt.show()

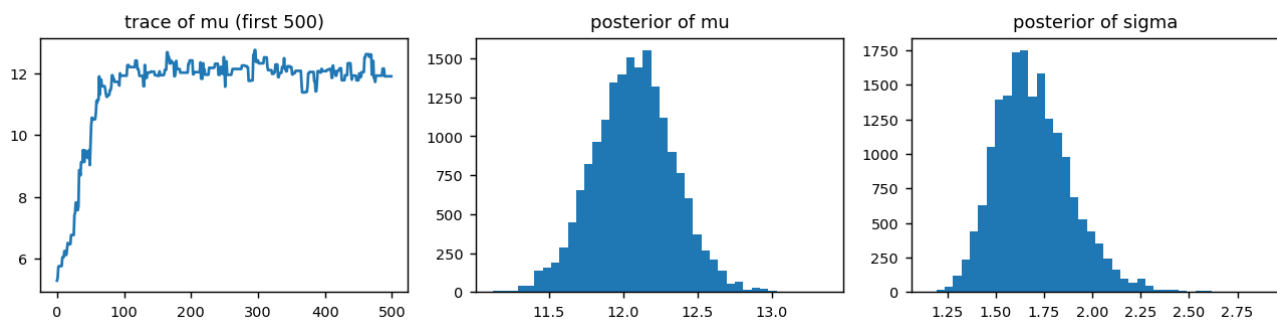
```

OUTPUT

```

acceptance rate: 0.33
mu    posterior mean/sd: 12.07 0.271
sigma posterior mean/sd: 1.696 0.196
sample mean and std: 12.077 1.652

```

**NOTE**

In practice use a library: **PyMC** (!`pip install pymc`) or **Stan/CmdStanPy** run advanced samplers (NUTS) with automatic diagnostics. Check **convergence** by plotting traces, running several chains, and the R-hat statistic (should be near 1.00).

```

import pymc as pm
with pm.Model():
    mu = pm.Normal("mu", 0, 100)
    sigma = pm.HalfNormal("sigma", 10)
    pm.Normal("y", mu, sigma, observed=obs)
    trace = pm.sample(2000, tune=1000, chains=4)
import arviz as az
az.summary(trace)

```

23.7 Frequentist or Bayesian?

	Frequentist	Bayesian
Parameters	fixed, unknown	random variables with distributions
Result	estimate, CI, p-value	posterior distribution, credible interval
Prior knowledge	informal	built in through the prior (must be stated)
Interpretation	long-run behaviour of procedures	direct probability statements about parameters
Strength	simple, standard, no prior needed	natural with small data, hierarchical models, uncertainty propagation

Both are valid tools; with plenty of data and weak priors they give similar numbers.

23.8 Chapter summary

CHAPTER SUMMARY

- Random error averages out; systematic error does not. Report value $\pm$ uncertainty with sensible digits and say what $\pm$ means.
- Propagate uncertainty with partial-derivative formulas (quadrature for sums, relative errors for products) or, more generally, with Monte Carlo simulation. Build an uncertainty budget.
- Combine measurements by inverse-variance weighting and check χ^2 .
- Bayesian inference: posterior $\propto$ likelihood $\times$ prior. Use grids, conjugate formulas or MCMC; report credible intervals and test sensitivity to the prior.

23.9 Practice exercises

1. A pendulum of length 1.000 ± 0.002 m has period 2.007 ± 0.004 s. Compute $g = 4\pi^2 L/T^2$ and its uncertainty by formula and by Monte Carlo.
2. Five labs report 3.2 ± 0.1 , 3.4 ± 0.2 , 3.1 ± 0.1 , 3.6 ± 0.3 , 3.3 ± 0.1 . Find the weighted mean, its error and test consistency.
3. Using the Beta-binomial model, update a flat prior with 7 successes in 10 trials and then 70 in 100. Compare the credible interval widths.
4. Modify the Metropolis sampler step size to 0.05 and 5. What happens to the acceptance rate and the trace?
5. Explain in your own words the difference between a 95% confidence interval and a 95% credible interval.

CHAPTER 24

24. Special Topics: Survival, Experiments, Text, Networks and Maps

IN THIS CHAPTER YOU WILL LEARN

- Survival analysis: censoring, Kaplan-Meier curves, the log-rank test
- Design of experiments: randomisation, blocking, factorial designs
- Text as data, network analysis and geospatial basics
- Other tools worth knowing: Dask, xarray, Polars

Scientific datasets come in many shapes. This chapter gives a working introduction to the most common special cases so you recognise them and know where to start.

24.1 Survival analysis: time until an event

Many experiments measure **time until something happens**: death, relapse, machine failure, seed germination. The challenge is **censoring**: for some subjects the event has *not* yet happened when the study ends (or they dropped out). We only know the time exceeded some value. Ignoring censored subjects, or treating their last time as an event, biases the results.

The **survival function** $S(t) = P(\text{survive beyond time } t)$. The **Kaplan-Meier estimator** builds it step by step: at each event time, multiply by the fraction of those still at risk who survived:

$$S(t) = \prod_{t_i \leq t} (1 - d_i / n_i)$$

where d_i is the number of events at time t_i and n_i the number still at risk just before it. We implement it from scratch.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(0)
n = 80
group = np.repeat(["control", "treated"], n // 2)
true_scale = np.where(group == "control", 10.0, 16.0) # treatment lengthens survival
event_time = rng.exponential(true_scale)
censor_time = rng.uniform(5, 30, n) # study ends/dropout at random times
time = np.minimum(event_time, censor_time)
event = (event_time <= censor_time).astype(int) # 1 = event seen, 0 = censored
df = pd.DataFrame({"group": group, "time": time.round(2), "event": event})
print(df.groupby("group")["event"].agg(["count", "sum"]).rename(columns={"sum": "events"}))

def kaplan_meier(time, event):
    order = np.argsort(time)
    time, event = np.asarray(time)[order], np.asarray(event)[order]
    surv, times, s = [], [], 1.0
    for t in np.unique(time[event == 1]):
        at_risk = (time >= t).sum()
        d = ((time == t) & (event == 1)).sum()
        s *= 1 - d / at_risk
```

```

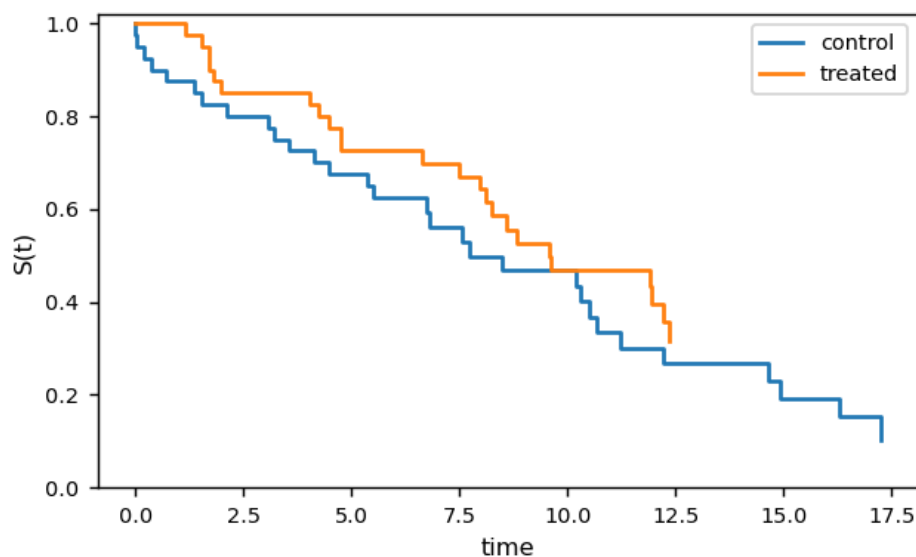
        times.append(t); surv.append(s)
    return np.array(times), np.array(surv)

fig, ax = plt.subplots(figsize=(5.5, 3.2))
for g in ["control", "treated"]:
    sub = df[df["group"] == g]
    tt, ss = kaplan_meier(sub["time"], sub["event"])
    ax.step(np.append(0, tt), np.append(1, ss), where="post", label=g)
    median_idx = np.where(ss <= 0.5)[0]
    print(g, "median survival:", tt[median_idx[0]] if len(median_idx) else "not reached")
ax.set_xlabel("time"); ax.set_ylabel("S(t)"); ax.set_ylim(0, 1.02); ax.legend()
plt.show()

```

OUTPUT

	count	events
group		
control	40	30
treated	40	24
control median survival:	7.75	
treated median survival:	9.58	



Comparing groups: the log-rank test

The **log-rank test** compares survival curves. At each event time it compares the observed events in a group with the number expected if both groups had the same risk, and accumulates the difference.

```

def logrank(time, event, group_is_a):
    time, event, a = np.asarray(time), np.asarray(event), np.asarray(group_is_a)
    obs_minus_exp, var = 0.0, 0.0
    for t in np.unique(time[event == 1]):
        at_risk = time >= t
        n, n_a = at_risk.sum(), (at_risk & a).sum()
        d = ((time == t) & (event == 1)).sum()
        d_a = ((time == t) & (event == 1) & a).sum()
        exp_a = d * n_a / n
        obs_minus_exp += d_a - exp_a
        if n > 1:
            var += d * (n_a / n) * (1 - n_a / n) * (n - d) / (n - 1)
    chi2 = obs_minus_exp ** 2 / var
    return chi2, stats.chi2.sf(chi2, 1)

chi2, p = logrank(df["time"], df["event"], df["group"] == "treated")
print(f"log-rank chi2 = {chi2:.2f}, p = {p:.4f}")

```

OUTPUT

log-rank chi2 = 2.30, p = 0.1294

With the `lifelines` library (`!pip install lifelines`) this is `KaplanMeierFitter` and `logrank_test`; **Cox proportional hazards** regression (`CoxPHFitter`) estimates how covariates change the **hazard** (instantaneous event rate): a hazard ratio of 0.6 means a 40% lower event rate.

```
from lifelines import KaplanMeierFitter, CoxPHFitter
kmf = KaplanMeierFitter().fit(df["time"], df["event"], label="all")
kmf.plot_survival_function()
dd = df.assign(treated=(df.group == "treated").astype(int))[["time", "event", "treated"]]
cox = CoxPHFitter().fit(dd, "time", "event")
cox.print_summary()
```

24.2 Design of experiments (DOE)

Good analysis cannot rescue a badly designed experiment. Core principles (R. A. Fisher):

1. **Randomisation:** assign treatments to units at random so unknown factors average out between groups.
2. **Replication:** repeat each condition to measure the noise level and gain power.
3. **Blocking:** group similar units (same day, same batch, same patient) and compare treatments *within* blocks to remove nuisance variation.
4. **Control:** include a baseline or placebo; use blinding to prevent expectations from biasing results.

```
rng = np.random.default_rng(1)
units = pd.DataFrame({"plot": range(12)})
units["treatment"] = rng.permutation(np.repeat(["A", "B", "C"], 4)) # completely randomised
print(units.T)
```

OUTPUT

	0	1	2	3	4	5	6	7	8	9	10	11
plot	0	1	2	3	4	5	6	7	8	9	10	11
treatment	C	C	B	B	B	A	A	C	A	C	B	A

Factorial designs

A **factorial** experiment varies several factors **together**, so you estimate each factor's **main effect** and their **interaction**, using fewer runs than testing one factor at a time. A 2×2 design (temperature low/high × catalyst no/yes) needs 4 conditions.

```
import itertools
rng = np.random.default_rng(2)
rows = []
for temp, cat, rep in itertools.product([0, 1], [0, 1], range(5)): # 5 replicates each
    yield_ = 50 + 6 * temp + 4 * cat + 5 * temp * cat + rng.normal(0, 2) # true interaction = 5
    rows.append((temp, cat, yield_))
d = pd.DataFrame(rows, columns=["temp", "catalyst", "yield"])
means = d.pivot_table(index="temp", columns="catalyst", values="yield", aggfunc="mean").round(2)
print(means)

m = means.values
main_temp = (m[1].mean() - m[0].mean())
main_cat = (m[:, 1].mean() - m[:, 0].mean())
interaction = (m[1, 1] - m[1, 0]) - (m[0, 1] - m[0, 0])
print(f"main effect temp = {main_temp:.2f}, catalyst = {main_cat:.2f}, interaction = {interaction:.2f}")
```

OUTPUT

catalyst	0	1
temp		
0	49.44	54.53
1	56.00	64.63

main effect temp = 8.33, catalyst = 6.86, interaction = 3.54

If the effect of catalyst is much larger at high temperature than at low temperature, there is a positive interaction: factors cannot be understood in isolation.

Sample size and power in practice

Before collecting data, use effect size, noise level, α and desired power to compute the sample size (Chapter 15). Many underpowered studies miss real effects, and those that find "significant" effects exaggerate them.

24.3 Text as data

Simple text analysis converts documents to numbers. The **bag-of-words** representation counts how often each word appears, ignoring order. **TF-IDF** weights words by how distinctive they are: frequent in one document but rare overall.

$$tf-idf(\text{word}, \text{doc}) = (\text{count of word in doc}) \times \log(\text{number of docs} / \text{number of docs containing the word})$$

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = [
    "the enzyme binds the substrate and catalyses the reaction",
    "the telescope observed the distant galaxy and its spectrum",
    "enzyme kinetics measure the reaction rate of the enzyme",
    "galaxy spectrum shows redshift of distant galaxy",
]

cv = CountVectorizer(stop_words="english")
counts = cv.fit_transform(docs)
print(pd.DataFrame(counts.toarray(), columns=cv.get_feature_names_out()).iloc[:, :8])

tfidf = TfidfVectorizer(stop_words="english").fit_transform(docs)
print(cosine_similarity(tfidf).round(2))
```

OUTPUT

```
   binds  catalyses  distant  enzyme  galaxy  kinetics  measure  observed
0      1          1        0        1        0          0          0          0
1      0          0        1        0        1          0          0          1
2      0          0        0        2        0          1          1          0
3      0          0        1        0        2          0          0          0
[[1.  0.  0.37 0.  ]
 [0.  1.  0.  0.53]
 [0.37 0.  1.  0.  ]
 [0.  0.53 0.  1.  ]]
```

The similarity matrix shows documents 1 and 3 (enzymes) are similar to each other, and 2 and 4 (astronomy) likewise. The same vectors can be fed to clustering or classifiers. For modern language tasks use pretrained transformer models via Hugging Face.

24.4 Networks and graphs

Many scientific systems are **networks**: proteins that interact, neurons, food webs, collaboration or transport networks. A graph has **nodes** (vertices) and **edges** (links). Key measures:

Measure	Meaning
Degree	number of connections of a node (a "hub" has high degree)
Betweenness centrality	how often a node lies on shortest paths between others
Clustering coefficient	how often a node's neighbours are also linked to each other

Measure	Meaning
Shortest path	fewest edges between two nodes
Connected components / communities	groups of nodes densely linked internally

```
import networkx as nx

G = nx.karate_club_graph() # 34 members of a real club, classic dataset
print("nodes, edges:", G.number_of_nodes(), G.number_of_edges())
deg = dict(G.degree())
top = sorted(deg, key=deg.get, reverse=True)[:3]
print("highest-degree members:", [(k, deg[k]) for k in top])
bet = nx.betweenness_centrality(G)
print("highest betweenness:", sorted(bet, key=bet.get, reverse=True)[:3])
print("average clustering:", round(nx.average_clustering(G), 3))
print("shortest path 16 -> 24:", nx.shortest_path(G, 16, 24))

communities = nx.community.greedy_modularity_communities(G)
print("communities found:", [len(c) for c in communities])

fig, ax = plt.subplots(figsize=(4.8, 3.8))
pos = nx.spring_layout(G, seed=3)
colors = {n: i for i, c in enumerate(communities) for n in c}
nx.draw(G, pos, node_color=[colors[n] for n in G], cmap="Set2", node_size=[30 + 25 * deg[n] for n in G],
        edge_color="lightgrey", ax=ax)
plt.show()
```

OUTPUT

```
nodes, edges: 34 78
highest-degree members: [(33, 17), (0, 16), (32, 12)]
highest betweenness: [0, 33, 32]
average clustering: 0.571
shortest path 16 -> 24: [16, 5, 0, 31, 24]
communities found: [17, 9, 8]
```



24.5 Geospatial data

Spatial data has coordinates: stations, samples, satellite pixels. A basic need is the **distance between two points on the Earth**, given by the **haversine formula**:

$$d = 2 R \arcsin(\sqrt{(\sin^2(\Delta\phi/2) + \cos\phi_1 \cos\phi_2 \sin^2(\Delta\lambda/2))})$$

where ϕ is latitude, λ is longitude (in radians), Δ the differences and $R \approx 6371$ km the Earth's radius.

```
def haversine(lat1, lon1, lat2, lon2, R=6371.0):
    lat1, lon1, lat2, lon2 = map(np.radians, (lat1, lon1, lat2, lon2))
    a = np.sin((lat2 - lat1) / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin((lon2 - lon1) / 2) ** 2
    return 2 * R * np.arcsin(np.sqrt(a))

print("Pune -> Delhi (km):", round(float(haversine(18.52, 73.86, 28.61, 77.21))))
print("Mumbai -> Chennai (km):", round(float(haversine(19.08, 72.88, 13.08, 80.27))))
```

OUTPUT

```
Pune -> Delhi (km): 1173
Mumbai -> Chennai (km): 1033
```

For real work use **GeoPandas** (vector shapes such as boundaries and points; `!pip install geopandas`), **rasterio** (satellite grids), **folium** (interactive maps) and **cartopy**. Remember **coordinate reference systems** (CRS): latitude/longitude (EPSG:4326) are angles, so convert to a projected CRS (metres) before computing areas or buffers.

Spatial interpolation example: inverse distance weighting

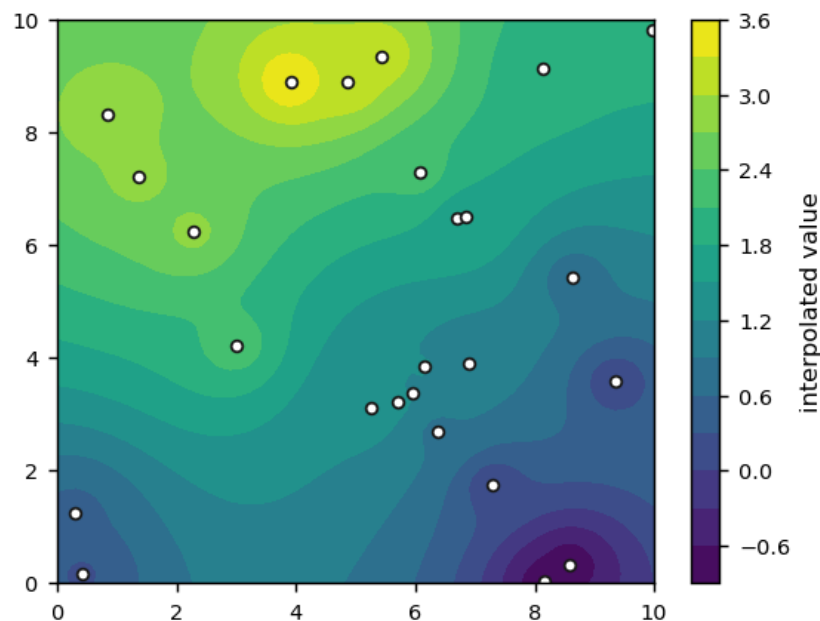
Estimate a value at an unsampled location as a distance-weighted average of nearby samples.

```
rng = np.random.default_rng(0)
pts = rng.uniform(0, 10, (25, 2))
vals = np.sin(pts[:, 0] / 2) + 0.3 * pts[:, 1]

# sampling stations
# measured values

def idw(x, y, pts, vals, power=2):
    d = np.hypot(pts[:, 0] - x, pts[:, 1] - y)
    if d.min() < 1e-9:
        return vals[d.argmin()]
    w = 1 / d ** power
    return (w * vals).sum() / w.sum()

gx, gy = np.meshgrid(np.linspace(0, 10, 60), np.linspace(0, 10, 60))
grid = np.vectorize(lambda x, y: idw(x, y, pts, vals))(gx, gy)
fig, ax = plt.subplots(figsize=(5, 3.8))
c = ax.contourf(gx, gy, grid, 15, cmap="viridis")
ax.scatter(pts[:, 0], pts[:, 1], c="white", edgecolor="k", s=18)
fig.colorbar(c, label="interpolated value"); plt.show()
```



Kriging (Gaussian-process interpolation) additionally gives an **uncertainty map**.

24.6 Scaling up: bigger data

Problem	Tool
Table does not fit in memory	Dask or Polars (parallel, lazy), chunked pandas reading, Parquet files
Labelled multi-dimensional arrays (climate, imaging)	xarray with NetCDF/Zarr
Slow Python loops over numbers	Numba (@ jit compiles functions) or vectorise with NumPy
Heavy model training	GPU runtime in Colab (Runtime, Change runtime type), PyTorch/JAX
Very large databases	SQL, DuckDB, BigQuery

```
# Numba-free illustration of why vectorising matters: a loop vs NumPy for a pairwise distance sum
import time
rng = np.random.default_rng(0)
P = rng.random((600, 3))
t0 = time.perf_counter()
total_loop = 0.0
for i in range(len(P)):
    for j in range(len(P)):
        total_loop += np.sqrt((P[i] - P[j]) ** 2).sum())
t1 = time.perf_counter()
diff = P[:, None, :] - P[None, :, :]
total_vec = np.sqrt((diff ** 2).sum(axis=2)).sum()
t2 = time.perf_counter()
print("same answer:", np.isclose(total_loop, total_vec))
print(f"loop: {t1 - t0:.2f} s    vectorised: {t2 - t1:.3f} s")
```

OUTPUT

```
same answer: True
loop: 0.84 s    vectorised: 0.014 s
```

24.7 Chapter summary

CHAPTER SUMMARY

- Survival data has **censoring**; use Kaplan-Meier curves, log-rank tests and Cox regression (hazard ratios) instead of ordinary averages.
- Good experiments use randomisation, replication, blocking and controls; factorial designs reveal main effects and interactions efficiently.
- Text becomes numbers via bag-of-words and TF-IDF; networks are analysed with degree, centrality, clustering and communities; geospatial work needs correct distance formulas and coordinate systems.
- For large data use Parquet, Dask/Polars, xarray, Numba or GPUs; vectorise first.

24.8 Practice exercises

- Simulate survival data where the treatment effect is absent. How often does the log-rank test reject at $\alpha = 0.05$ over 500 simulations?
- Design a randomised block experiment with 4 treatments and 5 blocks; print the allocation table.
- Extend the factorial example to three factors and estimate all main effects.
- Build TF-IDF vectors for five short abstracts you write and find the most similar pair.
- Compute the shortest path and betweenness for a small graph that you define as an edge list.

PART V

Putting It All Together

Professional habits for reproducible science, and three complete projects that combine everything you have learned.

CHAPTER 25

25. Reproducible Analysis and Good Scientific Practice

IN THIS CHAPTER YOU WILL LEARN

- Structuring a notebook and a project so others (and future you) can repeat it
- Writing clean, reusable functions; testing with `assert`
- Reproducibility: seeds, versions, data provenance
- Avoiding common statistical sins; reporting honestly
- Communicating results: figures, tables and a short report

A result that cannot be reproduced is not yet science. Reproducibility is mostly **habit**, not advanced technology.

25.1 The structure of a good notebook

1. **Title and purpose**: one paragraph stating the question.
2. **Setup cell**: imports, random seed, settings, version printout.
3. **Data loading**: where the data came from, date downloaded, units.
4. **Cleaning**: every change documented (and counted: "removed 3 duplicates").
5. **Exploration, analysis, results**, each with a text cell explaining the *why*.
6. **Conclusions and limitations** at the end.

Restart the runtime and **Run all** before sharing: if it fails from the top, it is not reproducible.

```
import sys, platform
import numpy as np
import pandas as pd
import scipy
import matplotlib

SEED = 20240601
rng = np.random.default_rng(SEED)          # one generator, created once, passed to functions

print("Python  :", sys.version.split()[0])
print("NumPy   :", np.__version__)
print("pandas  :", pd.__version__)
print("SciPy   :", scipy.__version__)
print("Matplotlib:", matplotlib.__version__)
```

OUTPUT

```
Python   : 3.12.3
NumPy    : 2.4.4
pandas   : 3.0.2
SciPy    : 1.17.1
Matplotlib: 3.10.8
```

Record versions in your notebook (or run `!pip freeze > requirements.txt` in Colab) so that someone can rebuild the same environment.

25.2 Randomness and seeds

Any analysis using random numbers (train/test splits, bootstrap, simulation, k-means) must set a seed so results can be repeated. Pass `random_state=` to scikit-learn functions and use `np.random.default_rng(seed)`.

```
def noisy_mean(seed):
    r = np.random.default_rng(seed)
    return r.normal(10, 2, 50).mean()

print(noisy_mean(1), noisy_mean(1), noisy_mean(2))
```

OUTPUT
9.927843845143384 9.927843845143384 10.043909194730066

NOTE

Fixing the seed makes results *repeatable*, but do not hunt for a lucky seed. A conclusion that depends on the seed is not robust. Repeat the analysis with several seeds and report the variability.

25.3 Write functions, not copy-pasted code

When you paste the same ten lines three times you will eventually fix a bug in only two of them. Turn repeated steps into **functions** with clear names, inputs and outputs, and add a **docstring** that states the units.

```
def standard_error(x):
    """Standard error of the mean of a 1-D array-like (ignores NaN)."""
    x = np.asarray(x, dtype=float)
    x = x[~np.isnan(x)]
    return x.std(ddof=1) / np.sqrt(len(x))

def mean_ci(x, level=0.95):
    """Return (mean, lower, upper) using the t distribution."""
    from scipy import stats
    x = np.asarray(x, dtype=float)
    x = x[~np.isnan(x)]
    m, se = x.mean(), standard_error(x)
    t = stats.t.ppf(0.5 + level / 2, len(x) - 1)
    return m, m - t * se, m + t * se

print(mean_ci([4.1, 4.5, 3.9, np.nan, 4.3, 4.6]))
```

OUTPUT
(np.float64(4.2799999999999999), np.float64(3.9244415410601237), np.float64(4.635558458939875))

Test your functions

A simple safeguard: check your code against cases with known answers using `assert`.

```
def test_mean_ci():
    m, lo, hi = mean_ci([1, 2, 3, 4, 5])
    assert abs(m - 3) < 1e-12
    assert lo < m < hi
    m2, _, _ = mean_ci([1, 2, np.nan, 3]) # NaN must be ignored
    assert abs(m2 - 2) < 1e-12

test_mean_ci()
print("tests passed")
```

OUTPUT
tests passed

For larger projects use `pytest`. Also compare a function's output with an independent method (for example your own t-interval against `scipy.stats`).

25.4 Organise your project files

```
my_project/
|-- README.md          what is this? how to run it?
|-- data/
|   |-- raw/           original files, NEVER edited
|   |-- processed/     cleaned data produced by code
|-- notebooks/         01_clean.ipynb, 02_explore.ipynb, 03_model.ipynb
|-- src/               reusable functions (.py files)
|-- figures/           figures exactly as used in the paper
|-- results/           tables, fitted models
|-- requirements.txt    package versions
```

Use **Git** and **GitHub** to track changes (Colab: File, Save a copy in GitHub). Number notebooks in the order they should be run. Keep **raw data read-only** and record its source, download date and licence.

25.5 Data provenance and a data dictionary

A **data dictionary** documents each column: meaning, unit, allowed range, how missing values are coded. Create it before analysing.

```
dictionary = pd.DataFrame([
    ("site", "text", "-", "monitoring station code (N, S, E, W)"),
    ("temp_c", "float", "degrees C", "water temperature; 999 = sensor error"),
    ("ph", "float", "pH units", "acidity; valid range 0 to 14"),
    ("date", "date", "YYYY-MM-DD", "day of sampling"),
], columns=["column", "type", "unit", "description"])
print(dictionary.to_string(index=False))
```

OUTPUT

column	type	unit	description
site	text	-	monitoring station code (N, S, E, W)
temp_c	float	degrees C	water temperature; 999 = sensor error
ph	float	pH units	acidity; valid range 0 to 14
date	date	YYYY-MM-DD	day of sampling

25.6 Automating a whole analysis: a pipeline function

```
from scipy import stats

def analyse_two_groups(a, b, alpha=0.05, n_boot=5000, seed=0):
    """Complete comparison of two independent samples: summary, Welch test, effect size, bootstrap CI."""
    a, b = np.asarray(a, float), np.asarray(b, float)
    rng = np.random.default_rng(seed)
    welch = stats.ttest_ind(a, b, equal_var=False)
    pooled = np.sqrt(((len(a) - 1) * a.var(ddof=1) + (len(b) - 1) * b.var(ddof=1)) / (len(a) + len(b) - 2))
    boots = [rng.choice(a, len(a)).mean() - rng.choice(b, len(b)).mean() for _ in range(n_boot)]
    return {
        "n": (len(a), len(b)),
        "mean": (round(a.mean(), 3), round(b.mean(), 3)),
        "difference": round(a.mean() - b.mean(), 3),
        "welch_p": round(welch.pvalue, 4),
        "cohens_d": round((a.mean() - b.mean()) / pooled, 2),
        "boot_95ci": tuple(np.percentile(boots, [2.5, 97.5]).round(2)),
        "significant": bool(welch.pvalue < alpha),
    }

rng = np.random.default_rng(1)
result = analyse_two_groups(rng.normal(10.5, 2, 40), rng.normal(9.5, 2, 40))
for k, v in result.items():
    print(f"{k:12s} {v}")
```

OUTPUT

```
n          (40, 40)
mean       (np.float64(10.506), np.float64(9.208))
difference  1.298
welch_p    0.001
cohens_d   0.76
boot_95ci  (np.float64(0.57), np.float64(2.02))
significant True
```

25.7 Common statistical mistakes (and fixes)

Mistake	Why it is a problem	What to do
p-hacking: trying many analyses until $p < 0.05$	inflates false positives	decide the analysis before looking; report everything tried
HARKing: inventing the hypothesis after seeing results	the data cannot test a hypothesis it suggested	label such findings <i>exploratory</i> and confirm on new data
Ignoring multiple comparisons	many tests guarantee false hits	Bonferroni/Holm/FDR (Chapter 15)
Treating "not significant" as "no effect"	low power can hide real effects	report the CI and power
Pseudoreplication: counting repeated measures from one subject as independent	falsely tiny standard errors	analyse per subject or use mixed models
Data leakage in ML	optimistic performance	split first; pipelines (Chapters 17, 18)
Dropping outliers because they spoil the result	biased conclusions	justify exclusion rules in advance
Overfitting with many predictors	works only on this dataset	cross-validation, regularisation, hold-out test
Confusing correlation with causation	confounders, reverse causation	experiments, causal designs, careful language
Reporting p without effect size	trivial effects can be "significant"	report estimate, CI and effect size

Pseudoreplication demonstrated

```
from scipy import stats
rng = np.random.default_rng(0)
false_pos_naive, false_pos_correct = 0, 0
for _ in range(1000):
    # 3 animals per group, 10 repeated measures each; NO true group difference
    animal_effects = rng.normal(0, 2, size=(2, 3))
    a = animal_effects[0][:, None] + rng.normal(0, 0.5, (3, 10))
    b = animal_effects[1][:, None] + rng.normal(0, 0.5, (3, 10))
    naive = stats.ttest_ind(a.ravel(), b.ravel()) # 30 vs 30 "independent" points
    correct = stats.ttest_ind(a.mean(axis=1), b.mean(axis=1)) # 3 vs 3 animals
    false_pos_naive += naive.pvalue < 0.05
    false_pos_correct += correct.pvalue < 0.05
print("false-positive rate, treating measurements as independent:", false_pos_naive / 1000)
print("false-positive rate, analysing animal averages          :", false_pos_correct / 1000)
```

OUTPUT

```
false-positive rate, treating measurements as independent: 0.582
false-positive rate, analysing animal averages          : 0.053
```

Treating 30 repeated measurements as 30 independent samples produces a false-positive rate many times the intended 5%.

25.8 Communicating results

Figures that persuade

- One figure, one message; a clear caption that says what is plotted and what the error bars mean.
- Show data points when n is small; label axes with units; use colour-blind-safe colours.
- Use consistent scales and fonts; export at 300 dpi or as PDF/SVG.

Tables

```
summary = pd.DataFrame({
    "group": ["control", "treated"],
    "n": [40, 40],
    "mean": [9.5, 10.5],
    "sd": [2.0, 2.0],
}).set_index("group")
summary["sem"] = summary["sd"] / np.sqrt(summary["n"])
print(summary.round(2).to_markdown())
print()
print(summary.round(2).to_latex()[:140], "...")
```

OUTPUT

group	n	mean	sd	sem
control	40	9.5	2	0.32
treated	40	10.5	2	0.32

```
\begin{tabular}{lrrrr}
\toprule
& n & mean & sd & sem \\
group & & & & \\
\midrule
control & 40 & 9.500000 & 2.000000 & 0.320000 \\
tre ...
```

`to_markdown()` and `to_latex()` produce tables ready for text cells or papers.

A short report template

1. **Question:** what you wanted to know and why.
2. **Data:** source, size, collection method, cleaning decisions.

3. **Methods:** tests and models used, and why they were appropriate; assumptions checked.
4. **Results:** estimates with uncertainty, effect sizes, key figures.
5. **Limitations:** sample, bias, confounders, what the analysis cannot show.
6. **Conclusion:** the answer in plain language, and next steps.

TIP

Write the conclusion for a smart colleague from another field. If you cannot explain your result in two plain sentences, you may not yet understand it.

25.9 Ethics and responsibility

- **Privacy:** remove identifiers from human data; follow consent and ethics-board rules.
- **Honesty:** report negative results and failed checks; do not discard inconvenient data.
- **Fairness:** examine whether models perform differently across groups.
- **Credit and licences:** cite data sources and code; respect licences.
- **Humility about causality and generalisation:** say where your results may not apply.

25.10 Practising the workflow: a checklist before sharing

```
checklist = [
    "Notebook runs top to bottom after Runtime > Restart session",
    "Seeds set; package versions printed",
    "Raw data untouched; cleaning steps documented with counts",
    "Units on every axis and column",
    "Assumptions of each test checked and mentioned",
    "Effect sizes and confidence intervals reported with p-values",
    "Figures labelled; captions explain error bars",
    "Limitations stated; code and data location given",
]
for i, item in enumerate(checklist, 1):
    print(f"[ ] {i}. {item}")
```

OUTPUT

```
[ ] 1. Notebook runs top to bottom after Runtime > Restart session
[ ] 2. Seeds set; package versions printed
[ ] 3. Raw data untouched; cleaning steps documented with counts
[ ] 4. Units on every axis and column
[ ] 5. Assumptions of each test checked and mentioned
[ ] 6. Effect sizes and confidence intervals reported with p-values
[ ] 7. Figures labelled; captions explain error bars
[ ] 8. Limitations stated; code and data location given
```

25.11 Chapter summary

CHAPTER SUMMARY

- Structure notebooks and projects clearly, keep raw data read-only, document data with a dictionary.
- Seed random generators, print package versions, and re-run from a clean start.
- Use functions and simple tests (`assert`) to avoid copy-paste errors.
- Avoid p-hacking, pseudoreplication, leakage and uncorrected multiple testing; report estimates with uncertainty.
- Communicate with clear figures, tables and a structured report; state limitations honestly.

25.12 Practice exercises

1. Turn one of your earlier analyses into a function that takes a `DataFrame` and returns a results dictionary; write two `assert` tests.
2. Simulate p-hacking: test 20 random outcomes against a random group variable and see how often at least one $p < 0.05$ appears.
3. Create a data dictionary for a dataset you use.
4. Write a one-page report (in text cells) using the template above for any analysis from Chapters 15 or 16.
5. Re-run the pseudoreplication experiment with 10 animals per group; what happens to the correct method's power?

CHAPTER 26

26. Capstone Projects

IN THIS CHAPTER YOU WILL LEARN

- Three complete analyses, each from raw data to conclusion
- Practising the full workflow: load, clean, explore, model, test, communicate
- Combining pandas, statistics, SciPy fitting and machine learning

Each project below generates its own data (so it runs anywhere) but is written as a real analysis. Read the question, try it yourself first, then compare with the solution. At the end of each project there are extension tasks.

26.1 Project 1: Climate station analysis

Question. A weather station has recorded daily temperature for 10 years. Is there a warming trend after accounting for the seasonal cycle, and is the trend statistically significant?

Step 1: load and inspect

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(100)
days = pd.date_range("2014-01-01", "2023-12-31", freq="D")
t = np.arange(len(days)) / 365.25
temp = (22 + 0.06 * t
        + 8 * np.sin(2 * np.pi * (t - 0.28))
        + rng.normal(0, 2.5, len(days)))
df = pd.DataFrame({"date": days, "temp_c": temp.round(1)})
df.loc[rng.choice(len(df), 40, replace=False), "temp_c"] = np.nan
df.loc[rng.choice(len(df), 3, replace=False), "temp_c"] = 99.9
df.to_csv("station.csv", index=False)

raw = pd.read_csv("station.csv", parse_dates=["date"])
print(raw.shape)
print(raw["temp_c"].describe().round(2))
print("missing:", raw["temp_c"].isna().sum())
```

OUTPUT

```
(3652, 2)
count    3612.00
mean      22.43
std       6.65
min       7.50
25%      17.00
50%      22.40
75%      27.70
max       99.90
Name: temp_c, dtype: float64
missing: 40
```

Step 2: clean

```
clean = raw.copy()
bad = clean["temp_c"] > 60
print("sensor errors flagged:", int(bad.sum()))
clean.loc[bad, "temp_c"] = np.nan
clean = clean.set_index("date")
clean["temp_c"] = clean["temp_c"].interpolate(method="time", limit=3) # fill short gaps only
print("remaining missing:", int(clean["temp_c"].isna().sum()))
```

OUTPUT

```
sensor errors flagged: 3
remaining missing: 0
```

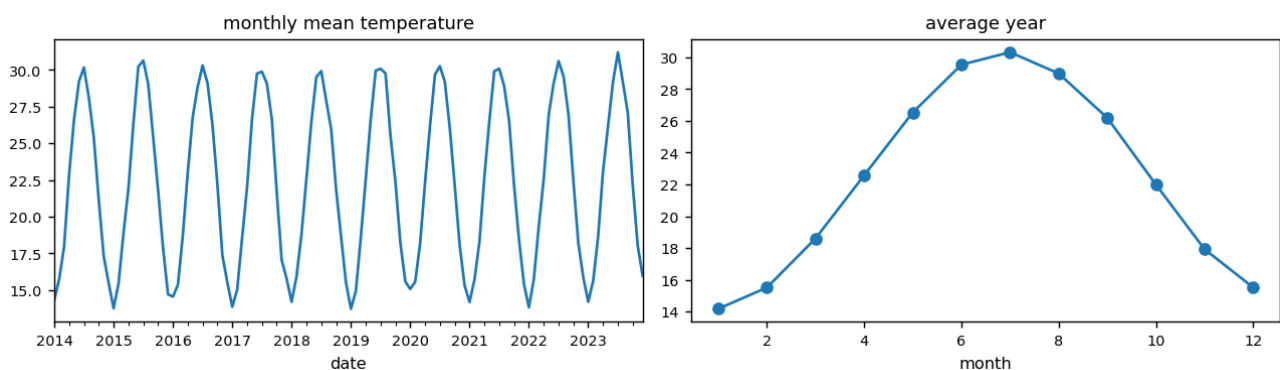
Step 3: explore

```
monthly = clean["temp_c"].resample("MS").mean()
annual = clean["temp_c"].resample("YS").mean()
print(annual.round(2).to_string())

fig, axes = plt.subplots(1, 2, figsize=(10, 3))
monthly.plot(ax=axes[0], title="monthly mean temperature")
clean.groupby(clean.index.month)["temp_c"].mean().plot(ax=axes[1], marker="o", title="average year")
axes[1].set_xlabel("month")
plt.tight_layout(); plt.show()
```

OUTPUT

```
date
2014-01-01    22.06
2015-01-01    22.21
2016-01-01    22.36
2017-01-01    22.21
2018-01-01    22.31
2019-01-01    22.37
2020-01-01    22.36
2021-01-01    22.44
2022-01-01    22.63
2023-01-01    22.58
Freq: YS-JAN
```



The seasonal cycle dominates; a trend of a few hundredths of a degree per year is hidden underneath. Plotting annual means alone is noisy.

Step 4: model trend + seasonality with regression

We fit $\text{temperature} = a + b \cdot t + c \cdot \sin(2\pi t) + d \cdot \cos(2\pi t)$. The slope b is the warming rate. Using a sine and cosine pair lets the model find the seasonal phase automatically (Chapter 17).

```
d = clean.dropna().copy()
tt = (d.index - d.index[0]).days.values / 365.25
X = np.column_stack([np.ones(len(d)), tt, np.sin(2 * np.pi * tt), np.cos(2 * np.pi * tt)])
y = d["temp_c"].values

beta, *_ = np.linalg.lstsq(X, y, rcond=None)
```

```

resid = y - X @ beta
n, k = X.shape
s2 = resid @ resid / (n - k)
cov = s2 * np.linalg.inv(X.T @ X)
se = np.sqrt(np.diag(cov))
print(f"trend = {beta[1]:.4f} +/- {se[1]:.4f} C per year")
print(f"amplitude of seasonal cycle = {np.hypot(beta[2], beta[3]):.2f} C")
print(f"R^2 = {1 - resid @ resid / ((y - y.mean()) @ (y - y.mean())):.3f}")

```

OUTPUT

```

trend = 0.0529 +/- 0.0143 C per year
amplitude of seasonal cycle = 8.12 C
R^2 = 0.842

```

Step 5: check the assumptions: autocorrelation

Daily temperatures are autocorrelated (a warm day is followed by another warm day), which makes the naive standard error too small. We check the residual autocorrelation and correct the standard error with an **effective sample size**.

```

def acf(x, nlags):
    x = x - x.mean()
    return np.array([(x[k:] * x[:len(x) - k]).sum() / (x ** 2).sum() for k in range(nlags + 1)])

r1 = acf(resid, 3)
print("residual autocorrelation (lags 1-3):", r1[1:].round(3))
n_eff = n * (1 - r1[1]) / (1 + r1[1]) # AR(1) effective sample size
se_trend_adj = se[1] * np.sqrt(n / n_eff)
t_stat = beta[1] / se_trend_adj
p_val = 2 * stats.t.sf(abs(t_stat), df=n_eff - k)
print(f"effective n = {n_eff:.0f} of {n}")
print(f"trend = {beta[1]:.3f} +/- {se_trend_adj:.3f} C/yr, t = {t_stat:.1f}, p = {p_val:.2g}")

```

OUTPUT

```

residual autocorrelation (lags 1-3): [-0.012 -0.011 -0.008]
effective n = 3743 of 3652
trend = 0.053 +/- 0.014 C/yr, t = 3.7, p = 0.00019

```

Here the residuals show little autocorrelation, but with real data the adjustment can matter a lot. Another, assumption-light check is the non-parametric **Theil-Sen** slope on annual means:

```

yrs = np.arange(len(annual))
ts = stats.theilslopes(annual.values, yrs)
mk = stats.kendalltau(yrs, annual.values)
print(f"Theil-Sen slope on annual means: {ts[0]:.3f} C/yr (95% CI {ts[2]:.3f} to {ts[3]:.3f})")
print(f"Kendall tau = {mk[0]:.2f}, p = {mk[1]:.3f}")

```

OUTPUT

```

Theil-Sen slope on annual means: 0.053 C/yr (95% CI 0.032 to 0.075)
Kendall tau = 0.78, p = 0.001

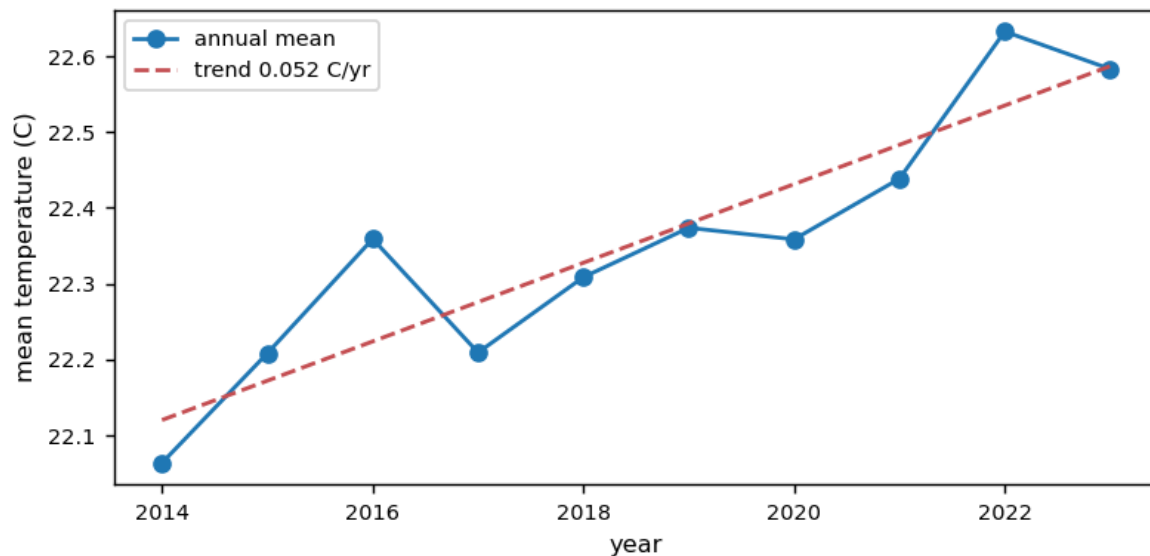
```

Step 6: communicate

```

fig, ax = plt.subplots(figsize=(7, 3.2))
ax.plot(annual.index.year, annual.values, "o-", label="annual mean")
fit = np.polyval(np.polyfit(annual.index.year, annual.values, 1), annual.index.year)
slope = np.polyfit(annual.index.year, annual.values, 1)[0]
ax.plot(annual.index.year, fit, "r--", label=f"trend {slope:.3f} C/yr")
ax.set_xlabel("year"); ax.set_ylabel("mean temperature (C)"); ax.legend()
plt.show()

```



Conclusion (example write-up): After removing the seasonal cycle, the station shows a warming trend of about 0.05 °C per year (standard error 0.014, so a 95% interval of roughly 0.025 to 0.08), statistically distinguishable from zero ($p < 0.001$). Over a decade this is about 0.5 °C, small compared with the 16 °C peak-to-peak seasonal swing, which is why a plot of raw data hides it. (The data were simulated with a true trend of 0.06, inside this interval.) *Limitations:* single station, urban effects not controlled, linear trend assumed.

Extensions: (1) Add a quadratic term for acceleration. (2) Compare the first and last three years with a t-test. (3) Look at trend per season. (4) Detect heatwaves as days above the 95th percentile of the same calendar-day distribution.

26.2 Project 2: Dose-response experiment

Question. A drug is tested at eight doses on cell cultures. What is the **EC50** (the dose giving half the maximal response), how uncertain is it, and does a second drug differ?

Step 1: data

```
rng = np.random.default_rng(7)
doses = np.array([0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30]) # micromolar
reps = 4

def hill(dose, top, ec50, slope, bottom=0.0):
    return bottom + (top - bottom) / (1 + (ec50 / dose) ** slope)

rows = []
for drug, ec50, top in [("A", 0.8, 100), ("B", 2.5, 95)]:
    for d_ in doses:
        for r_ in range(reps):
            rows.append((drug, d_, hill(d_, top, ec50, 1.3) + rng.normal(0, 5)))
data = pd.DataFrame(rows, columns=["drug", "dose", "response"])
print(data.groupby(["drug", "dose"])["response"].agg(["mean", "std"]).round(1).head(8))
```

OUTPUT

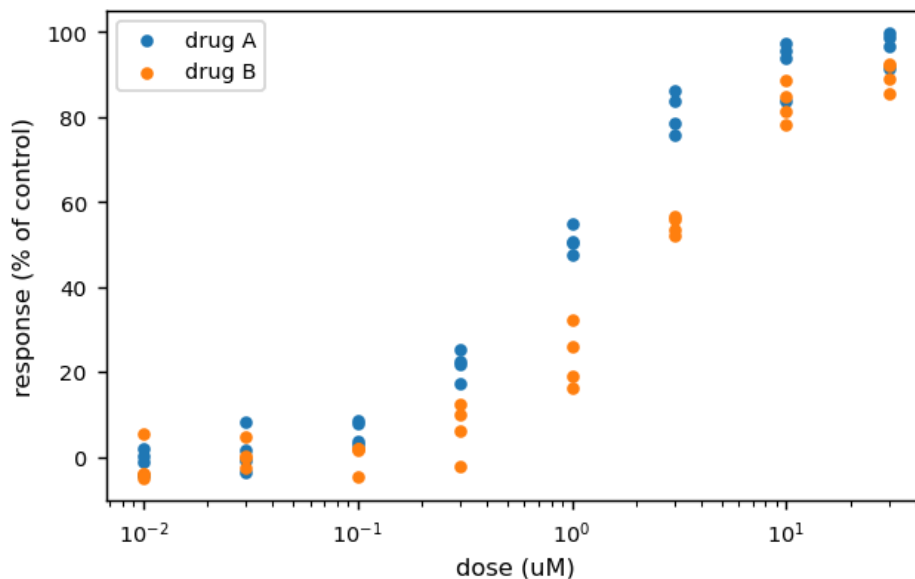
drug	dose	mean	std
A	0.01	-0.7	2.5
	0.03	1.3	5.0
	0.10	5.9	2.9
	0.30	21.6	3.4
	1.00	51.0	3.0
	3.00	80.9	4.8
	10.00	92.5	6.0

```
30.00  96.7  3.7
```

Step 2: explore on a log scale

Dose-response data is analysed against **log(dose)**: biological effects follow ratios of concentration, not differences.

```
fig, ax = plt.subplots(figsize=(5.5, 3.3))
for drug, g in data.groupby("drug"):
    ax.scatter(g["dose"], g["response"], s=14, label=f"drug {drug}")
ax.set_xscale("log"); ax.set_xlabel("dose (uM)"); ax.set_ylabel("response (% of control)"); ax.legend()
plt.show()
```



Step 3: fit the four-parameter Hill (logistic) model

$$response = bottom + (top - bottom) / (1 + (EC50 / dose)^{slope})$$

top and **bottom** are the plateaus, **EC50** the half-maximal dose and **slope** the steepness. We fit by non-linear least squares (Chapter 21) and get EC50 with a confidence interval.

```
from scipy import optimize

def hill4(dose, top, ec50, slope):
    return hill(dose, top, ec50, slope)

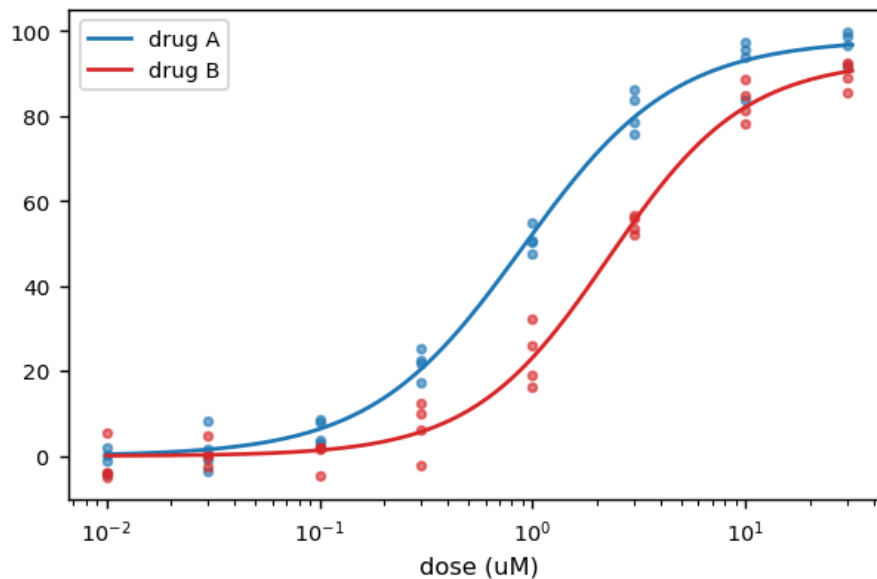
results = {}
for drug, g in data.groupby("drug"):
    popt, pcov = optimize.curve_fit(hill4, g["dose"].values, g["response"].values,
                                   p0=[100, 1.0, 1.0], bounds=([0, 1e-3, 0.2], [200, 100, 5]))
    perr = np.sqrt(np.diag(pcov))
    dof = len(g) - 3
    tcrit = stats.t.ppf(0.975, dof)
    results[drug] = (popt, perr)
    lo, hi = popt[1] - tcrit * perr[1], popt[1] + tcrit * perr[1]
    print(f"drug {drug}: top = {popt[0]:.1f} +/- {perr[0]:.1f}, EC50 = {popt[1]:.2f} "
          f"(95% CI {lo:.2f} to {hi:.2f}), slope = {popt[2]:.2f}")

fig, ax = plt.subplots(figsize=(5.5, 3.3))
xs = np.logspace(-2, 1.5, 200)
for (drug, g), c in zip(data.groupby("drug"), ["tab:blue", "tab:red"]):
    ax.scatter(g["dose"], g["response"], s=10, color=c, alpha=0.6)
    ax.plot(xs, hill4(xs, *results[drug][0]), color=c, label=f"drug {drug}")
```

```
ax.set_xscale("log"); ax.set_xlabel("dose (uM)"); ax.legend(); plt.show()
```

OUTPUT

drug A: top = 98.0 +/- 1.8, EC50 = 0.90 (95% CI 0.78 to 1.01), slope = 1.22
 drug B: top = 93.2 +/- 2.7, EC50 = 2.27 (95% CI 1.89 to 2.65), slope = 1.35



Step 4: compare the two drugs

Are the EC50 values different? Compare the log-EC50 estimates with a z-test using their standard errors, (the standard error of log EC50 is about SE/EC50 by the delta method).

```
(pa, ea), (pb, eb) = results["A"], results["B"]
diff = np.log(pb[1]) - np.log(pa[1])
se_diff = np.sqrt((ea[1] / pa[1]) ** 2 + (eb[1] / pb[1]) ** 2)
z = diff / se_diff
p_z = 2 * stats.norm.sf(abs(z))
print(f"log EC50 difference = {diff:.2f} +/- {se_diff:.2f}, z = {z:.1f}, p = {p_z:.4f}")
print(f"drug B is {np.exp(diff):.1f}x less potent than A (EC50 ratio)")
```

OUTPUT

log EC50 difference = 0.93 +/- 0.10, z = 8.9, p = 0.0000
 drug B is 2.5x less potent than A (EC50 ratio)

Step 5: bootstrap as a cross-check

```
g = data[data["drug"] == "A"].reset_index(drop=True)
ec50s = []
for _ in range(500):
    idx = rng.integers(0, len(g), len(g))
    try:
        p, _ = optimize.curve_fit(hill4, g["dose"].values[idx], g["response"].values[idx],
                                  p0=[100, 1, 1], bounds=([0, 1e-3, 0.2], [200, 100, 5]))
        ec50s.append(p[1])
    except RuntimeError:
        pass
print("bootstrap 95% CI for EC50 of drug A:", np.percentile(ec50s, [2.5, 97.5]).round(2))
```

OUTPUT

bootstrap 95% CI for EC50 of drug A: [0.8 1.]

Conclusion (example): Drug A has EC50 ≈ 0.9 μM (95% CI 0.78 to 1.01) and drug B ≈ 2.3 μM (95% CI 1.89 to 2.65), so B is about 2.5 times less potent, a difference far beyond the fit uncertainty. Both reach a similar maximal response (about 93 to 98%). *Limitations:* four replicates per dose; plateaus must be well covered by the dose range.

Extensions: (1) Fit with `sigma` weights if variance grows with the response. (2) Compare a 3-parameter model (bottom = 0) with the 4-parameter model using AIC. (3) Plan a new experiment: how many replicates are needed to halve the EC50 uncertainty?

26.3 Project 3: Predicting a measured property with machine learning

Question. Using chemical measurements of 442 diabetes patients, can we predict disease progression one year later better than a baseline, and which measurements matter most?

Step 1: load, explore

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error, r2_score
```

```
bunch = load_diabetes(as_frame=True)
X, y = bunch.data, bunch.target
print(X.shape, "target mean/std:", y.mean().round(1), y.std().round(1))
print(X.describe().loc[["mean", "std"]].round(3))
print("strongest correlations with the target:")
print(X.corrwith(y).sort_values(key=abs, ascending=False).head(4).round(2))
```

OUTPUT

```
(442, 10) target mean/std: 152.1 77.1
      age  sex  bmi  bp  s1  s2  s3  s4  s5  s6
mean -0.000 0.000 -0.000 -0.000 -0.000 0.000 -0.000 -0.000 0.000 0.000
std   0.048 0.048 0.048 0.048 0.048 0.048 0.048 0.048 0.048 0.048
strongest correlations with the target:
bmi    0.59
s5     0.57
bp     0.44
s4     0.43
dtype: float64
```

The features are already centred and scaled. The target is a disease-progression score, so a typical error of 50 would be large relative to its spread (about 77).

Step 2: split and set up a fair comparison

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
cv = KFold(5, shuffle=True, random_state=0)

models = {
    "baseline (mean)": DummyRegressor(strategy="mean"),
    "linear": make_pipeline(StandardScaler(), LinearRegression()),
    "ridge": make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-3, 3, 30))),
    "lasso": make_pipeline(StandardScaler(), LassoCV(cv=5, random_state=0)),
    "random forest": RandomForestRegressor(n_estimators=300, min_samples_leaf=5, random_state=0),
    "gradient boosting": GradientBoostingRegressor(n_estimators=150, max_depth=2,
                                                    learning_rate=0.05, random_state=0),
}

rows = []
for name, m in models.items():
    r2 = cross_val_score(m, X_train, y_train, cv=cv, scoring="r2")
    mae = -cross_val_score(m, X_train, y_train, cv=cv, scoring="neg_mean_absolute_error")
    rows.append((name, r2.mean(), r2.std(), mae.mean()))
table = pd.DataFrame(rows, columns=["model", "CV R2", "R2 std", "CV MAE"]).set_index("model")
print(table.round(3))
```

OUTPUT

```
CV R2  R2 std  CV MAE
```

model			
baseline (mean)	-0.030	0.028	67.766
linear	0.506	0.101	43.935
ridge	0.509	0.100	43.875
lasso	0.507	0.103	44.005
random forest	0.427	0.121	47.423
gradient boosting	0.448	0.100	46.716

Step 3: choose a model and test once

```
best = models["ridge"].fit(X_train, y_train)
pred = best.predict(X_test)
print(f"test R2 = {r2_score(y_test, pred):.3f}, test MAE = {mean_absolute_error(y_test, pred):.1f}")
base_mae = mean_absolute_error(y_test, np.full(len(y_test), y_train.mean()))
print(f"baseline MAE = {base_mae:.1f}")

# uncertainty of the test MAE by bootstrap over test cases
errs = np.abs(y_test.values - pred)
boot = [rng.choice(errs, len(errs)).mean() for _ in range(2000)]
print("95% CI of test MAE:", np.percentile(boot, [2.5, 97.5]).round(1))
```

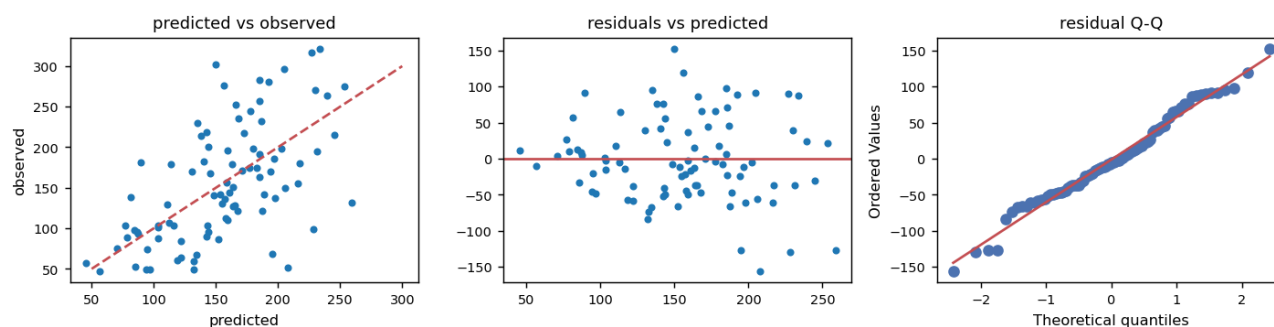
OUTPUT
test R2 = 0.336, test MAE = 45.9
baseline MAE = 59.2
95% CI of test MAE: [39. 53.9]

Step 4: examine residuals and importance

```
resid = y_test.values - pred
fig, axes = plt.subplots(1, 3, figsize=(11, 2.9))
axes[0].scatter(pred, y_test, s=12); axes[0].plot([50, 300], [50, 300], "r--")
axes[0].set_xlabel("predicted"); axes[0].set_ylabel("observed"); axes[0].set_title("predicted vs observed")
axes[1].scatter(pred, resid, s=12); axes[1].axhline(0, color="r")
axes[1].set_title("residuals vs predicted")
stats.probplot(resid, plot=axes[2]); axes[2].set_title("residual Q-Q")
plt.tight_layout(); plt.show()

from sklearn.inspection import permutation_importance
pi = permutation_importance(best, X_test, y_test, n_repeats=30, random_state=0)
print(pd.Series(pi.importances_mean, index=X.columns).sort_values(ascending=False).round(3).head(5))
coefs = pd.Series(best[-1].coef_, index=X.columns).sort_values()
print("ridge coefficients (per SD of each feature):")
print(coefs.round(1).to_string())
```

OUTPUT
s5 0.157
bmi 0.134
bp 0.104
sex 0.042
s3 0.016
dtype: float64
ridge coefficients (per SD of each feature):
sex -10.6
s3 -9.7
s1 -5.6
s2 -4.4
age -1.2
s6 2.9
s4 5.5
bp 13.7
s5 24.1
bmi 26.5



Step 5: interpret honestly

Conclusion (example): In cross-validation the linear models explain about 50% of the variance ($R^2 \approx 0.51$) and cut the mean absolute error from about 68 (baseline) to 44. Body mass index (`bmi`) and the serum measurement `s5` (related to serum triglycerides) are the strongest predictors, with blood pressure (`bp`) also positive. The simple linear and ridge models beat random forests and boosting (CV R^2 of 0.43 and 0.45), so extra complexity brings no benefit here. On the single held-out test split the R^2 was lower (0.34), a reminder that one small test set gives a noisy estimate; the bootstrap interval for the test error is wide (39 to 54). *Limitations:* the model is predictive, not causal; residual scatter is still large, so individual predictions are uncertain; the patients may not represent other populations.

Extensions: (1) Build a classifier for "above-median progression" and report ROC-AUC. (2) Add interaction features and use cross-validation to see if they help. (3) Use `GridSearchCV` to tune gradient boosting. (4) Produce prediction intervals by quantile regression (`GradientBoostingRegressor(loss="quantile")`).

26.4 What to do next

You now have a complete toolkit. To keep improving:

1. **Analyse your own data.** Nothing teaches more than a real question you care about. Start with the workflow of Chapter 1.
2. **Read other people's notebooks** on Kaggle and GitHub; reproduce a published figure from its data.
3. **Learn the next layer** as needed: statsmodels (mixed models, GLMs), scikit-learn pipelines at scale, PyMC (Bayesian models), PyTorch (deep learning), xarray/Dask (large arrays), SQL.
4. **Statistics is the foundation.** Revisit Chapters 12 to 16 regularly; clear statistical thinking matters more than any library.
5. **Share your work** with a clean notebook and a short write-up. Explaining it to others is the best test of understanding.

26.5 Chapter summary

CHAPTER SUMMARY

- A complete analysis flows: question, load, clean (documenting each step), explore, model with checked assumptions, quantify uncertainty, and communicate with limitations.
- Time series trend: regress on time plus seasonal terms and correct for autocorrelation; confirm with a non-parametric test.
- Dose-response: fit the Hill model on a log-dose scale, report EC50 with a confidence interval, compare drugs through log-EC50.

- ML for prediction: baseline first, cross-validated comparison, one final test, bootstrap uncertainty, residual checks, permutation importance.

26.6 Practice exercises

1. Redo Project 1 with a station that has a real change point (a jump of 1 °C in 2019). How does a linear trend model misbehave, and how can you detect the jump?
2. In Project 2, simulate data where the dose range misses the top plateau. Show how the EC50 uncertainty explodes.
3. In Project 3, repeat with a different random split and show how much the test R^2 varies. Then use repeated cross-validation for a stable estimate.
4. Pick a topic from the book and create your own mini-project with a question, data (simulated or real), analysis and one-page report.
5. Make a list of the three things you found hardest in this book and plan an exercise for each.

PART VI

Appendices

Quick-reference cheat sheets, formulas, a glossary, Colab troubleshooting and worked solutions to selected exercises.

APPENDIX A

A. Cheat Sheets

A.1 Python and Colab essentials

Task	Code
Print with formatting	<code>print(f"{x:.2f}")</code>
List comprehension	<code>[v**2 for v in data if v > 0]</code>
Dictionary lookup with default	<code>d.get("key", 0)</code>
Function	<code>def f(x, k=1): return k * x</code>
Install a package	<code>!pip install package</code>
Mount Drive	<code>from google.colab import drive; drive.mount('/content/drive')</code>
Upload a file	<code>from google.colab import files; files.upload()</code>
Download a file	<code>files.download("out.csv")</code>
Time a cell	<code>%%time</code> on the first line of the cell
Run a shell command	<code>!ls -lh</code>
Help on an object	<code>help(obj)</code> or <code>obj?</code>

A.2 NumPy

Task	Code
Create	<code>np.array([..]), zeros, ones, arange(a,b,s), linspace(a,b,n), eye(n)</code>
Random	<code>rng = np.random.default_rng(0); rng.normal(mu, sd, n), rng.integers(a,b,n), rng.choice(x, n)</code>
Shape	<code>a.shape, a.reshape(r, c), a.T, a.ravel(), a[:, None]</code>
Index	<code>a[i, j], a[:, 1], a[a > 0], a[[0, 2]]</code>
Maths	<code>np.sqrt, exp, log, sin, abs, round, clip(a, lo, hi)</code>
Aggregate	<code>a.sum(axis=0), mean, std(ddof=1), min, max, argmax, cumsum, percentile</code>
Missing	<code>np.isnan(a), np.nanmean(a)</code>
Combine	<code>np.concatenate, vstack, hstack, column_stack</code>
Condition	<code>np.where(cond, x, y), np.select(conds, choices)</code>
Linear algebra	<code>A @ B, np.linalg.solve(A, b), inv, eig, norm, lstsq</code>
Sort / unique	<code>np.sort, np.argsort, np.unique(a, return_counts=True)</code>
Save / load	<code>np.save("a.npy", a), np.load("a.npy")</code>

A.3 pandas

Task	Code
Read / write	<code>pd.read_csv(path), df.to_csv(path, index=False), read_excel, read_json</code>
Look	<code>df.head(), shape, info(), describe(), dtypes, columns</code>
Select	<code>df["a"], df[["a", "b"]], df.loc[rows, cols], df.iloc[0:3, 0:2]</code>
Filter	<code>df[df["a"] > 5], df.query("a > 5 and b == 'x'"), df["a"].isin([1,2])</code>
New column	<code>df["c"] = df["a"] / df["b"], df.assign(c=lambda d: d.a * 2)</code>
Missing	<code>df.isna().sum(), dropna(), fillna(v), interpolate()</code>
Duplicates	<code>df.duplicated(), drop_duplicates()</code>
Types	<code>astype(float), pd.to_numeric(s, errors="coerce"), pd.to_datetime(s)</code>
Text	<code>s.str.strip(), lower(), replace(), contains(), split(), extract()</code>
Dates	<code>s.dt.year, .dt.month, .dt.day_name(), pd.date_range()</code>
Sort	<code>df.sort_values("a", ascending=False)</code>
Group	<code>df.groupby("g")["x"].agg(["mean", "std", "count"]), .transform("mean")</code>
Reshape	<code>pivot_table, melt, pivot, stack, unstack, crosstab</code>
Combine	<code>pd.concat([a, b]), a.merge(b, on="key", how="left")</code>
Time series	<code>resample("W").mean(), rolling(7).mean(), shift(1), diff(), pct_change()</code>
Counts	<code>s.value_counts(normalize=True), s.nunique()</code>
Correlation	<code>df.corr(), df.corr(method="spearman")</code>
Bins	<code>pd.cut(s, bins, labels), pd.qcut(s, q)</code>

A.4 Matplotlib and Seaborn

Task	Code
Figure and axes	<code>fig, ax = plt.subplots(figsize=(6, 3)); several: plt.subplots(2, 2)</code>
Plot types	<code>ax.plot, scatter, bar, hist, boxplot, errorbar, imshow, contourf</code>
Labels	<code>ax.set_xlabel("time (s)"), set_ylabel, set_title, legend()</code>
Limits / scales	<code>set_xlim(a, b), set_xscale("log")</code>
Save	<code>fig.savefig("f.png", dpi=300, bbox_inches="tight")</code>
Seaborn	<code>sns.histplot, boxplot, violinplot, scatterplot, regplot, heatmap, pairplot</code>

A.5 scipy.stats quick reference

Task	Code
Distribution	<code>d = stats.norm(mu, sd); d.pdf(x), d.cdf(x), d.ppf(q), d.rvs(n)</code>
One-sample t	<code>stats.ttest_1samp(x, mu0)</code>
Two-sample Welch t	<code>stats.ttest_ind(a, b, equal_var=False)</code>
Paired t	<code>stats.ttest_rel(a, b)</code>
Non-parametric	<code>mannwhitneyu, wilcoxon, kruskal</code>
ANOVA, post-hoc	<code>f_oneway(a, b, c), tukey_hsd(a, b, c)</code>
Chi-square	<code>chi2_contingency(table), chisquare(obs), fisher_exact(table2x2)</code>
Normality	<code>shapiro(x), probplot(x, plot=ax)</code>
Correlation	<code>pearsonr, spearmanr, kendalltau</code>
Regression	<code>linregress(x, y)</code>
Intervals	<code>stats.t.interval(0.95, df, loc=m, scale=se)</code>
Bootstrap / permutation	<code>stats.bootstrap, stats.permutation_test</code>

A.6 scikit-learn pattern

Step	Code
Split	<code>train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)</code>
Pipeline	<code>make_pipeline(StandardScaler(), LogisticRegression())</code>
Fit / predict	<code>m.fit(Xtr, ytr); m.predict(Xte); m.predict_proba(Xte)</code>
Cross-validate	<code>cross_val_score(m, X, y, cv=5, scoring="accuracy")</code>
Tune	<code>GridSearchCV(m, grid, cv=5).fit(X, y)</code>
Metrics	<code>accuracy_score, classification_report, roc_auc_score, r2_score, mean_absolute_error</code>
Unsupervised	<code>KMeans(k), DBSCAN(eps, min_samples), PCA(n_components)</code>

APPENDIX B

B. Formula Sheet and Symbols

B.1 Descriptive statistics

Quantity	Formula
Mean	$\bar{x} = \sum x_i / n$
Sample variance	$s^2 = \sum (x_i - \bar{x})^2 / (n - 1)$
Standard deviation	$s = \sqrt{s^2}$
Standard error of the mean	$SE = s / \sqrt{n}$
z-score	$z = (x - \bar{x}) / s$
IQR outlier fences	$Q1 - 1.5 \cdot IQR$ and $Q3 + 1.5 \cdot IQR$
Covariance	$cov(x, y) = \sum (x_i - \bar{x})(y_i - \bar{y}) / (n - 1)$
Pearson correlation	$r = cov(x, y) / (s_x s_y)$
Coefficient of variation	$CV = s / \bar{x}$

B.2 Probability

Quantity	Formula
Complement	$P(\text{not } A) = 1 - P(A)$
Independent AND	$P(A \text{ and } B) = P(A) P(B)$
Conditional	$P(A \text{ given } B) = P(A \text{ and } B) / P(B)$
Bayes' rule	$P(A \text{ given } B) = P(B \text{ given } A) P(A) / P(B)$
Binomial	$P(X = k) = C(n, k) p^k (1 - p)^{n-k}$; mean np ; variance $np(1 - p)$
Poisson	$P(X = k) = \lambda^k e^{-\lambda} / k!$; mean = variance = λ
Normal	$f(x) = \exp(-(x - \mu)^2 / 2\sigma^2) / (\sigma\sqrt{2\pi})$
Exponential	$f(x) = \lambda e^{-\lambda x}$; mean $1/\lambda$
Central limit theorem	sample means $\approx$ Normal(μ , $\sigma/\sqrt{n}$)

B.3 Inference

Quantity	Formula
CI for a mean	$\bar{x} \pm t^* \cdot s / \sqrt{n}$
CI for a proportion (Wald)	$\hat{p} \pm z^* \cdot \sqrt{(\hat{p}(1 - \hat{p}))/n}$
One-sample t	$t = (\bar{x} - \mu_0) / (s/\sqrt{n})$, df = $n - 1$
Welch t	$t = (\bar{x}_1 - \bar{x}_2) / \sqrt{(s_1^2/n_1 + s_2^2/n_2)}$
Cohen's d	$d = (\bar{x}_1 - \bar{x}_2) / s_{\text{pooled}}$
Chi-square	$\chi^2 = \sum (O - E)^2 / E$
ANOVA F	$F = (\text{between-group variance}) / (\text{within-group variance})$
Sample size for a mean	$n \approx (z^* \sigma / E)^2$
Bonferroni	use α / m for m tests

B.4 Regression

Quantity	Formula		
Line	$y = \beta_0 + \beta_1 x + \varepsilon$		
Slope and intercept	$\beta_1 = r \cdot s_y / s_x; \beta_0 = \bar{y} - \beta_1 \bar{x}$		
Matrix solution	$\beta = (X^T X)^{-1} X^T y$		
R^2	$1 - SSR / SST$		
Adjusted R^2	$1 - (1 - R^2)(n - 1)/(n - k)$		
RMSE	$\sqrt{(SSR / n)}$		
VIF	$1 / (1 - R_j^2)$		
Ridge / Lasso	minimise $SSR + \lambda \sum \beta^2 / SSR + \lambda \sum \beta $	$\beta \backslash$	
Logistic	$p = 1 / (1 + e^{-z}), z = \beta_0 + \beta_1 x_1 + \dots$		

B.5 Machine learning metrics

Metric	Formula
Accuracy	$(TP + TN) / \text{total}$
Precision	$TP / (TP + FP)$
Recall	$TP / (TP + FN)$
F1	$2PR / (P + R)$
Gini impurity	$1 - \sum p_k^2$
Silhouette	$(b - a) / \max(a, b)$
PCA	eigenvectors of the covariance matrix; variance ratio $\lambda_i / \sum \lambda$

B.6 Error propagation

Operation	Uncertainty				
$f = x \pm y$	$\sigma = \sqrt{(\sigma_x^2 + \sigma_y^2)}$				
$f = x \cdot y$ or x / y	$\sigma/f = \sqrt{((\sigma_x/x)^2 + (\sigma_y/y)^2)}$				
$f = x^n$	$\sigma/f = n $	$n \backslash$	$\sigma_x / \backslash$	$x \backslash$	
Weighted mean	$\bar{x} = \sum w_i x_i / \sum w_i, w_i = 1/\sigma_i^2, \sigma = 1/\sqrt{(\sum w_i)}$				
Reduced chi-square	$\chi^2 / (N - p)$, should be about 1				

B.7 Greek letters and notation

Symbol	Meaning
Σ, Π	sum, product
μ, σ, σ^2	population mean, standard deviation, variance
$\bar{x}, s, s^2$	sample mean, standard deviation, variance
α	significance level (false-positive rate)
β	regression coefficient (also type II error rate in testing)
ε	error term
ρ, r	population, sample correlation
λ	rate parameter; regularisation strength; eigenvalue
θ	a generic parameter
χ^2	chi-square statistic
∂	partial derivative
$\int$	integral
$\approx$	approximately equal
$\propto$	proportional to
$\hat{y}$	predicted value of y

APPENDIX C

C. Glossary

Term	Meaning
Aliasing	false low frequencies produced by sampling too slowly
ANOVA	test comparing the means of three or more groups
Autocorrelation	correlation of a series with a shifted copy of itself
Bias	systematic difference between an estimate's average and the truth
Bootstrap	resampling the data with replacement to estimate uncertainty
Broadcasting	NumPy rule that stretches smaller arrays to match larger ones
Censoring	event time only known to exceed a value (survival data)
Central limit theorem	sample means tend to a normal distribution as n grows
Confidence interval	range computed so that a stated fraction of such intervals contain the true value
Confounder	a variable that influences both predictor and outcome
Credible interval	Bayesian interval holding a stated posterior probability
Cross-validation	repeatedly training on part of the data and validating on the rest
DataFrame	pandas table of labelled columns
Data leakage	information from outside the training data influencing the model
Degrees of freedom	number of independent pieces of information left after estimating parameters
Dtype	the data type of an array or column
Effect size	magnitude of a difference or relationship, independent of sample size
EDA	exploratory data analysis
Eigenvector / eigenvalue	direction a matrix only stretches, and the stretch factor
Feature	an input variable of a model
FFT	fast algorithm for the discrete Fourier transform
Heteroscedasticity	non-constant error variance
Hyperparameter	a setting chosen before training a model
Imputation	filling missing values with estimates
Interaction	effect of one variable depends on another
IQR	interquartile range, $Q3 - Q1$
Likelihood	probability of the observed data as a function of the parameters
Log-odds	$\ln(p / (1 - p))$
Maximum likelihood	choosing parameters that make the data most probable
MCMC	sampling from a posterior by a random walk
Monte Carlo	solving problems with repeated random simulation
Multicollinearity	strongly correlated predictors
Nyquist frequency	half the sampling rate, the highest frequency recoverable
Outlier	observation far from the rest
Overfitting	model captures noise and fails on new data

Term	Meaning
p-value	probability, under the null hypothesis, of a result at least as extreme as observed
PCA	principal component analysis
Power	probability of detecting a real effect
Prior / posterior	belief before / after seeing the data
Pseudoreplication	treating non-independent measurements as independent
Quantile	value below which a stated fraction of data lies
Regularisation	penalty that shrinks model coefficients
Residual	observed minus predicted value
Series	pandas one-dimensional labelled array
Skewness	asymmetry of a distribution
Standard error	standard deviation of an estimate
Stationarity	statistical properties constant over time
Tidy data	one variable per column, one observation per row
Type I / II error	false positive / false negative
Variance	average squared deviation from the mean
Vectorisation	applying operations to whole arrays without loops
z-score	distance from the mean in standard deviations

APPENDIX D

D. Google Colab Tips and Troubleshooting

D.1 Everyday tips

- **Use a GPU only when you need one** (deep learning): Runtime, Change runtime type. Ordinary data analysis runs perfectly on the default CPU.
- **Keep data in Drive**, code in notebooks, and results saved back to Drive regularly.
- **Table of contents**: use headings (`#`, `##`) in text cells; the left sidebar shows an outline.
- **Forms**: `#@param` creates widgets, for example `n = 100 #@param {type:"integer"}`.
- **Check resources**: the RAM and disk indicator at the top right; click it for details.
- **Display more rows**: `pd.set_option("display.max_rows", 200)`; show all columns with `pd.set_option("display.max_columns", None)`.
- **Magic commands**: `%matplotlib inline` (default), `%timeit expression` (benchmark), `%who` (list variables), `%writefile file.py` (save a cell as a file).
- **Secrets**: store API keys in the key icon (Secrets) in the left sidebar, not in code.

D.2 Library versions

Every example in this book was run with recent releases (Python 3.12, NumPy 2.x, pandas 3.x, SciPy 1.1x, Matplotlib 3.10, scikit-learn 1.8). Colab updates its libraries regularly and works with pandas 2.x too. If a function is reported as missing or an argument is not recognised, check your versions with `print(lib.__version__)` and upgrade with `!pip install -U library`, then restart the session. Two differences you may notice: older pandas prints text columns as `object` instead of `str`, and older NumPy prints scalars without the `np.float64(...)` wrapper.

D.3 Common errors and what to do

Message	Likely cause	Fix
<code>NameError: name 'df' is not defined</code>	cell not run, or runtime was reset	run the earlier cells (Runtime, Run all)
<code>ModuleNotFoundError: No module named 'x'</code>	package not installed	<code>!pip install x</code> , then restart session if needed
<code>FileNotFoundError</code>	wrong path or file lost after reset	check <code>os.listdir()</code> ; upload again or mount Drive
<code>KeyError: 'col'</code>	misspelled column or hidden spaces	<code>print(df.columns.tolist());</code> <code>df.columns =</code> <code>df.columns.str.strip()</code>
<code>ValueError: could not convert string to float</code>	text in a numeric column	<code>pd.to_numeric(col,</code> <code>errors="coerce")</code>
<code>ValueError: operands could not be broadcast together</code>	array shapes incompatible	print <code>.shape</code> of each array
<code>SettingWithCopyWarning</code>	assigning to a slice	use <code>df.loc[mask, "col"] = value</code> or <code>.copy()</code>
<code>ValueError: Input contains NaN</code>	missing values given to scikit-learn	impute (Chapter 17)
<code>LinAlgError: Singular matrix</code>	perfectly collinear columns	remove duplicates or use Ridge
<code>ConvergenceWarning</code>	solver did not converge	scale features; raise <code>max_iter</code>
<code>MemoryError</code> or the session crashes	data too big for RAM	read fewer columns or chunks, use <code>float32</code> , categories, Parquet
Plot does not appear	missing <code>plt.show()</code> in a script, or backend issue	add <code>plt.show()</code> ; restart the session

D.4 A debugging routine that works

1. **Read the last line** of the error: what kind of error, and what does the message say?
2. **Read upwards** to find the line of your code that triggered it.
3. **Print** the types, shapes and first few values of every object on that line.
4. **Shrink** the problem: try the code on a tiny example.
5. **Search** the exact error message, or ask for help with a minimal reproducible example.
6. Take a break. Many bugs are obvious after a walk.

APPENDIX E

E. Solutions to Selected Exercises

These worked solutions use the same style as the chapters. Try each exercise yourself first. All the code below was run to produce the output shown.

E.1 Chapter 2: Python essentials

Exercise 1. Kelvin conversion with a list comprehension.

```
def to_kelvin(c):
    return c + 273.15

print([to_kelvin(c) for c in [0, 37, 100]])
```

OUTPUT
[273.15, 310.15, 373.15]

Exercise 2. Maximum with a loop.

```
data = [3, 8, 1, 9, 4]
biggest = data[0]
for v in data:
    if v > biggest:
        biggest = v
print(biggest)
```

OUTPUT
9

Exercise 5. Safe conversion.

```
def to_num(text):
    try:
        return float(text)
    except ValueError:
        return 0

print([to_num(t) for t in ["1", "two", "3"]])
```

OUTPUT
[1.0, 0, 3.0]

E.2 Chapter 3: Math toolkit

Exercise 3. Doubling time at 5% growth: solve $1.05^t = 2$, so $t = \ln 2 / \ln 1.05$.

```
import numpy as np
print(np.log(2) / np.log(1.05))
```

OUTPUT
14.206699082890463

Exercise 5. Bayes with sensitivity 99%, false positives 2%, prevalence 0.5%.

```
prev, sens, fpr = 0.005, 0.99, 0.02
p_pos = sens * prev + fpr * (1 - prev)
print(round(sens * prev / p_pos, 3))
```

OUTPUT
0.199

Only about a 20% chance despite a positive test, because the condition is rare.

E.3 Chapter 4 and 5: NumPy

Chapter 4, exercise 2. The middle 2 x 2 block of a 4 x 4 array.

```
a = np.arange(1, 17).reshape(4, 4)
print(a[1:3, 1:3])
```

```
OUTPUT
[[ 6  7]
 [10 11]]
```

Chapter 5, exercise 3. Subtract each row's mean using `keepdims`.

```
m = np.array([[1., 2, 3], [4, 5, 9]])
print(m - m.mean(axis=1, keepdims=True))
```

```
OUTPUT
[[-1.  0.  1.]
 [-2. -1.  3.]]
```

Chapter 5, exercise 5. Probability that two dice sum to 7.

```
rng = np.random.default_rng(0)
s = rng.integers(1, 7, 100000) + rng.integers(1, 7, 100000)
print((s == 7).mean(), 1 / 6)
```

```
OUTPUT
0.16672 0.16666666666666666
```

E.4 Chapter 12 and 13: statistics and distributions

Chapter 12, exercise 1. Statistics of [4, 8, 15, 16, 23, 42].

```
x = np.array([4, 8, 15, 16, 23, 42])
print(x.mean(), np.median(x), x.var(ddof=1).round(2), x.std(ddof=1).round(2))
```

```
OUTPUT
18.0 15.5 182.0 13.49
```

Chapter 13, exercise 1. Coin flips.

```
from scipy import stats
print(round(stats.binom.pmf(10, 20, 0.5), 4), round(stats.binom.sf(14, 20, 0.5), 4))
```

```
OUTPUT
0.1762 0.0207
```

Chapter 13, exercise 3. The height marking the tallest 10%.

```
print(round(stats.norm(170, 8).ppf(0.90), 1))
```

```
OUTPUT
180.3
```

E.5 Chapter 14 and 15: intervals and tests

Chapter 14, exercise 1. Confidence intervals of increasing level widen.

```
x = np.array([12.1, 11.8, 12.5, 12.0, 12.3, 11.9])
se = stats.sem(x)
```

```
for level in [0.90, 0.95, 0.99]:
    lo, hi = stats.t.interval(level, len(x) - 1, loc=x.mean(), scale=se)
    print(level, round(lo, 3), round(hi, 3), "width", round(hi - lo, 3))
```

OUTPUT

```
0.9 11.885 12.315 width 0.429
0.95 11.826 12.374 width 0.547
0.99 11.671 12.529 width 0.859
```

Chapter 14, exercise 4. Sample size to estimate a mean to within 0.5 when sigma is about 4.

```
z = stats.norm.ppf(0.975)
print(int(np.ceil((z * 4 / 0.5) ** 2)))
```

OUTPUT

```
246
```

Chapter 15, exercise 1. Battery lifetime test.

```
rng = np.random.default_rng(0)
batt = rng.normal(490, 20, 15)
res = stats.ttest_lsamp(batt, 500)
print(round(res.statistic, 2), round(res.pvalue, 4))
```

OUTPUT

```
-3.06 0.0085
```

Chapter 15, exercise 4. Chi-square test of association for a 2 x 3 table.

```
table = np.array([[20, 15, 30], [25, 25, 15]])
chi2, p, dof, _ = stats.chi2_contingency(table)
print(round(chi2, 2), dof, round(p, 4))
```

OUTPUT

```
8.06 2 0.0178
```

E.6 Chapter 16: regression

Exercise 1. Fit a line by hand and by NumPy.

```
x = np.array([1, 2, 3, 4, 5.0])
y = np.array([2.1, 3.9, 6.2, 7.8, 10.1])
b1 = ((x - x.mean()) * (y - y.mean())).sum() / ((x - x.mean()) ** 2).sum()
b0 = y.mean() - b1 * x.mean()
print(round(b1, 3), round(b0, 3), np.polyfit(x, y, 1).round(3))
```

OUTPUT

```
1.99 0.05 [1.99 0.05]
```

E.7 Chapter 18: machine learning

Exercise 1. Compare three classifiers on the wine data.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier

X, y = load_wine(return_X_y=True)
for name, m in [("logistic", make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))),
                ("k-NN", make_pipeline(StandardScaler(), KNeighborsClassifier(5))),
                ("forest", RandomForestClassifier(random_state=0))]:
```

```
s = cross_val_score(m, X, y, cv=5)
print(f"{name:9s} {s.mean():.3f} +/- {s.std():.3f}")
```

OUTPUT

```
logistic  0.983 +/- 0.014
k-NN      0.949 +/- 0.038
forest    0.983 +/- 0.022
```

E.8 Chapter 19: PCA

Exercise 2. How many components explain 90% of the breast cancer data?

```
from sklearn.datasets import load_breast_cancer
from sklearn.decomposition import PCA
Xb = StandardScaler().fit_transform(load_breast_cancer().data)
cum = PCA().fit(Xb).explained_variance_ratio_.cumsum()
print(int(np.argmax(cum >= 0.90)) + 1, "components")
```

OUTPUT

```
7 components
```

E.9 Chapter 21: curve fitting

Exercise 2. All solutions of $\sin(x) = x/3$ between -4 and 4.

```
from scipy import optimize
f = lambda x: np.sin(x) - x / 3
brackets = [(-3, -2), (-0.1, 0.1), (2, 3)]
roots = [optimize.brentq(f, lo, hi) for lo, hi in brackets]
print(np.round(roots, 4))
```

OUTPUT

```
[-2.2789  0.      2.2789]
```

Note the root at $x = 0$ was found with a tiny bracket. A plot of $f(x)$ is the best way to choose brackets.

E.10 Chapter 22: signals

Exercise 3. Aliasing of a 40 Hz sine sampled at 50 Hz.

```
fs = 50
t = np.arange(0, 2, 1 / fs)
x = np.sin(2 * np.pi * 40 * t)
freqs = np.fft.rfftfreq(len(t), 1 / fs)
print("apparent frequency (Hz):", freqs[np.argmax(np.abs(np.fft.rfft(x)))])
```

OUTPUT

```
apparent frequency (Hz): 10.0
```

The 40 Hz wave appears at $|40 - 50| = 10$ Hz, an alias.

E.11 Chapter 23: uncertainty

Exercise 1. Pendulum estimate of g .

```
L, sL, T, sT = 1.000, 0.002, 2.007, 0.004
g = 4 * np.pi ** 2 * L / T ** 2
rel = np.sqrt((sL / L) ** 2 + (2 * sT / T) ** 2)
print(f"formula : g = {g:.3f} +/- {g * rel:.3f}")
rng = np.random.default_rng(0)
gs = 4 * np.pi ** 2 * rng.normal(L, sL, 200000) / rng.normal(T, sT, 200000) ** 2
print(f"Monte Carlo: g = {gs.mean():.3f} +/- {gs.std():.3f}")
```

OUTPUT

```
formula      : g = 9.801 +/- 0.044  
Monte Carlo: g = 9.801 +/- 0.044
```

APPENDIX F

F. A Learning Roadmap and Further Reading

F.1 If you want to go deeper

Goal	Next steps
Stronger statistics	Linear and generalised linear models (statsmodels), mixed-effects models, nonparametric methods, causal inference
Bayesian modelling	PyMC or Stan; hierarchical models; model comparison (LOO, WAIC)
Predictive modelling	gradient boosting libraries (XGBoost, LightGBM), model calibration, interpretability (SHAP)
Deep learning	PyTorch or TensorFlow/Keras; convolutional networks for images; transformers for text and sequences
Time series	statsmodels, sktime, Prophet; state-space models and Kalman filters
Big or labelled array data	Dask, Polars, xarray, Zarr, DuckDB
Geospatial	GeoPandas, rasterio, cartopy, Google Earth Engine
Images	scikit-image, OpenCV, napari
Bioinformatics	Biopython, scanpy, scikit-bio
Astronomy	astropy, lightkurve
Chemistry and materials	RDKit, pymatgen
Reproducible workflows	Git/GitHub, Conda environments, Snakemake or Nextflow, Quarto and Jupyter Book

F.2 Suggested books and resources (search by title)

- *Python Data Science Handbook* by Jake VanderPlas (NumPy, pandas, Matplotlib, scikit-learn; free online).
- *Python for Data Analysis* by Wes McKinney (pandas by its creator).
- *Statistical Rethinking* by Richard McElreath (Bayesian thinking, very readable).
- *An Introduction to Statistical Learning* by James, Witten, Hastie and Tibshirani (free PDF; machine learning with statistics).
- *Think Stats* and *Think Bayes* by Allen Downey (free, Python-based).
- *Data Reduction and Error Analysis for the Physical Sciences* by Philip Bevington and Keith Robinson.
- *The Elements of Statistical Learning* by Hastie, Tibshirani and Friedman (advanced reference).
- Official documentation: numpy.org, pandas.pydata.org, matplotlib.org, scipy.org, scikit-learn.org.

F.3 Final advice

Learning to analyse data is a long game. Keep a notebook of problems you solved, reuse your own functions, and always return to the questions that matter most: *What did I measure? How certain am I? Could I be fooling myself?* If you keep asking those, the code becomes the easy part.